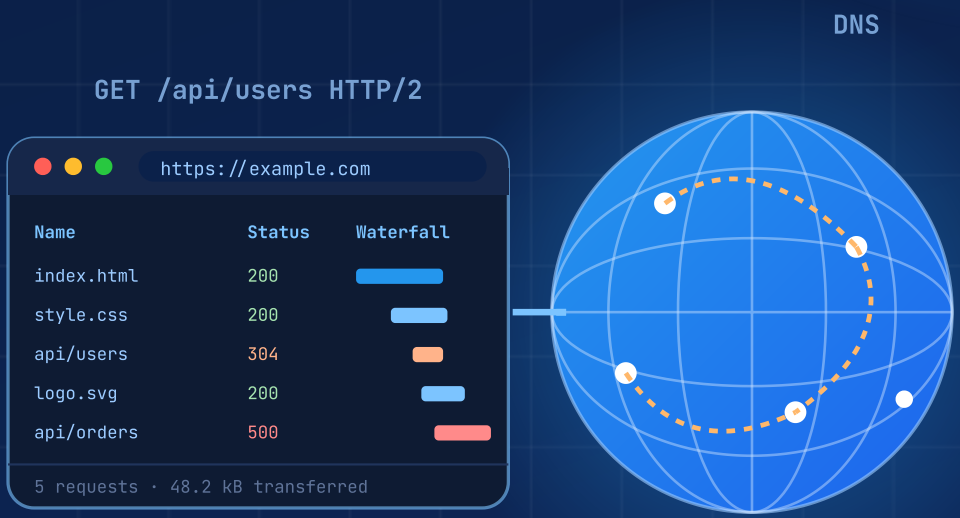
كتاب مذاكرة عملي · بالعامية المصري

الإنترنت والويب

إزاي الصفحة بتوصلك فعلاً... وإزاي تمسك أي مشكلة بالـ DevTools

الحكاية من أول الكابل لحد ما الصفحة ترسم: IP والـ Ports و DNS و TCP، و HTTP بطلباته وأكواده وهيدراته، و HTTPS والكوكيز والـ CORS والكاش. وبعدين نفتح DevTools حتة حتة ونتعلم نعمل Debugging لأي طلب. كل أمر وكل سطر متشرّح.

DevTools والـ Debugging على الـ Mac



Cmd + Option + I

200 OK

TLS 1.3

`GET /api HTTP/2` `dig +short example.com` `curl -v https://example.com`

`Cache-Control` `Access-Control-Allow-Origin` `Set-Cookie` `200 OK`

`Cmd + Option + I`

كتابك للمذاكرة

من الصفر خطوة بخطوة

23 فصل + معامل Terminal ورسومات في كل فصل

+ Cheat Sheet وقاموس ومصادر حديثة

المحتويات

الكتاب ده ماشي معاك على 6 مراحل. هنبدأ بالحكاية: الإنترنت اتعمل ليه، والصفحة بتوصلك إزاي. وبعدين نفهم العناوين والتوصيل، ولغة الويب HTTP، والأمان والكوكيز والـ CORS، والسرعة والكاش. وفي الآخر هنفتح DevTools ونتعلم نمسك بيها أي مشكلة. كل حاجة هنجربها بإيدنا من الـ Terminal على الـ Mac ومن المتصفح.

إزاي تستخدم الكتاب ده؟ دليل الألوان

- **معلومة مهمة**: حاجة لازم تفضل فاكرها
- **خلي بالك**: غلطة شائعة أو فخ
- **حاجة مشتركة**: فكرة أو Flag بيتكرر في أكتر من مكان
- **إحالة**: رقم الجزء والصفحة اللي فيها الشرح الأصلي
- **مشكلة شائعة**: العَرَض والسبب والحل
- **ملخص + صح ولا غلط + أسئلة**: آخر كل فصل
- **Best Practice**: الطريقة الصح في الشغل الحقيقي
- **مثال عملي**: جرّبه بإيدك
- **لو حابب تتعمق**: تفاصيل زيادة تقدر تعدّيها أول مرة
- **تشريح**: كل حتة في الأمر أو الهيدر عليها رقم وشرح
- **تحديات الفصل**: سهل / متوسط / صعب، والحلول في الملحق

01 الحكاية من الأول: الإنترنت اتولد ليه؟ 8
1.1 قبل الإنترنت: كل جهاز جزيرة 8
1.2 الفكرة العبقرية: قطّع الرسالة حتت 8
1.3 من ARPANET لـ TCP/IP: لغة واحدة لكل الشبكات 9
1.4 الإنترنت مش هو الويب 10
1.5 مين بيطلب ومين بيرد: الـ Client والـ Server 10
1.6 مين بيدير الإنترنت؟ ومين بيكتب القواعد؟ 11
1.7 طب وأنا كمبرمج JavaScript، ده يفرق معايا في إيه؟ 12
02 رحلة طلب واحد: من Enter لحد ما الصفحة تظهر 14
2.1 السؤال اللي بيتسأل في كل Interview 14
2.2 المحطة 1 و 2: المتصفح يفهم انت عاوز إيه 15
2.3 المحطة 3: DNS، دليل التليفونات 15
2.4 المحطة 4 و 5: نفتح خط، ونأمّنه 16
2.5 المحطة 6 و 7 و 8: الطلب والرد 16
2.6 المحطة 9 و 10: المتصفح يرسم، ويطلب تاني 16
2.7 شوف الرحلة بعينك دلوقتي 17
03 العناوين: IP والـ Ports و localhost 20
3.1 المشكلة: الـ Packet رايحة فين؟ 20
3.2 IPv4: أربع أرقام بنقط 20
3.3 الحل الأول: Private IP والـ NAT 21
3.4 الحل الجذري: IPv6 22
3.5 اعرف عناوينك على الـ Mac 22
3.6 الـ Port: رقم الشقة 23
3.7 localhost: الجهاز بيكلم نفسه 25
3.8 معمل: شغّل سيرفر واعرف مين ماسك الـ Port 25
04 الطبقات و TCP و UDP: إزاي الكلام يوصل سليم 28
4.1 المشكلة: الـ IP مبيضمنش حاجة 28
4.2 الطبقات: ظرف جوه ظرف 28
4.3 الـ TCP Handshake: "سامعني؟" 29
4.4 UDP: ابعت وخلاص 31
4.5 ping: "انت عايش؟" 31
4.6 traceroute: السكة فيها كام محطة؟ 32
4.7 nc: سكينة الشبكة السويسري 33

05 DNS: دليل التليفونات بتاع الإنترنت 37
5.1 قبل DNS: ملف واحد للدنيا كلها 37
5.2 تشريح الدومين: بيتقرا من اليمين 37
5.3 رحلة السؤال: example.com فين؟ 38
5.4 أنواع السجلات: مش أرقام بس 39
5.5 dig: اسأل الـ DNS بنفسك 40
5.6 /etc/hosts: اكذب على جهازك 42
5.7 لما الـ DNS يبوظ: إزاي تعرف؟ 43

06 الـ URL متشرّح: كل حرف في العنوان ليه شغلانة 46
6.1 المشكلة: عنوان واحد يقول كل حاجة 46
6.2 التشريح الكامل 46
6.3 الـ Query string: بيانات في العنوان 47
6.4 الـ Percent-encoding: ليه المسافة بقت %20؟ 48
6.5 العناوين النسبية: من غير ما تكتب كل حاجة 49
6.6 الـ Origin: أهم مفهوم في أمان الويب 49
6.7 URLs في الـ Terminal: الـ Quotes إجبارية 50

07 HTTP: الطلب والرد سطر سطر 53
7.1 الحكاية: من سطر واحد لـ لغة كاملة 53
7.2 رسالة الطلب: أربع حتت 53
7.3 رسالة الرد: نفس الشكل تقريباً 54
7.4 الـ Methods: انت عاوز تعمل إيه؟ 55
7.5 الـ Body والـ Content-Type: البيانات شكلها إيه؟ 56
7.6 نفس الكلام في JavaScript 58

08 الـ Status Codes: السيرفر بيقولك إيه بالأرقام 61
8.1 المشكلة: البرنامج مش بيقرا كلام 61
8.2 2xx: نجح، بس بأنهي طريقة؟ 62
8.3 3xx: "مش هنا، روح هناك" 62
8.4 4xx: المشكلة في طلبك 63
8.5 5xx: السيرفر هو اللي وقع 64
8.6 1xx: الأكواد اللي مش هتشوفها كتير 65
8.7 معمل: اطلب أي كود بإيدك 65

09 الـ Headers: الكلام اللي على الظرف 69
9.1 الفكرة: معلومات عن الرسالة، مش الرسالة 69
9.2 مين انت؟ Host و User-Agent 69
9.3 عاوز الرد شكله إيه؟ الـ Content negotiation 70
9.4 Content-Type: إيه اللي جوه؟ 71
9.5 الضغط: نفس المحتوى بعُشر الحجم 72
9.6 Authorization: الكارت بتاعك 73
9.7 جاي منين؟ Referer و Origin 73
9.8 Headers تانية هتشوفها في كل رد 74
9.9 معمل: ابعت Headers وشوف وصلت إزاي 75

10 معمل curl: متصفح في سطر واحد 77
10.1 ليه curl لما عندي متصفح؟ 77
10.2 تقرا الرد: -i و -I و -v 77
10.3 CRUD كامل: اقرا واعمل وعدّل وامسح 78
10.4 فورمز وملفات و Query 80
10.5 التحويلات والتحميل: -L و -o و -O 81
10.6 الـ Scripts: اسكت، بس افشل بصوت عالي 81
10.7 قيس الرحلة بالملي ثانية 82
10.8 حاجات تانية هتحتاجها 83

11 HTTP/1.1 و 2 و 3: نفس اللغة، طرق أسرع 86
11.1 المشكلة: صفحة واحدة بقت 100 طلب 86
11.2 HTTP/2: اتصال واحد، طلبات كتير مع بعض 87
11.3 HTTP/3: المشكلة كانت في TCP نفسه 88
11.4 تشوف النسخة بعينك 89
11.5 النهارده: مين بيستخدم إيه؟ 89

12 HTTPS و TLS: القفل اللي جنب العنوان 92
12.1 المشكلة: كلامك ماشي في الشارع مكشوف 92
12.2 فكرة المفتاحين: قفل أي حد يقفله، ومفتاح مع واحد بس 93
12.3 TLS 1.3 Handshake خطوة بخطوة 93
12.4 الشهادة: مين يضمن إن ده bank.com بجد؟ 94
12.5 Let's Encrypt والشهادات اللي عمرها بيقصر 96
12.6 HSTS: متجيش هنا من غير قفل تاني 96
12.7 تقرا TLS بعينك 97
12.8 أخطاء الشهادات: الصفحة الحمرا 98

13 الكوكيز والـ Sessions: إزاي السيرفر بيفتكرك 101
13.1 قبل الكوكيز: عربية تسوق بتفضى كل صفحة 101
13.2 Set-Cookie: كل كلمة ليها معنى 102
13.3 SameSite: الحماية من الطلبات اللي جاية من بره 103
13.4 First-party و Third-party: وإعلان الشوز اللي بيمشي وراك 103
13.5 Session على السيرفر ولا Token معاك؟ 104
13.6 أماكن تخزين تانية في المتصفح 104
13.7 تتابع الكوكيز في curl 105

14 CORS: أشهر خطأ أحمر في الـ Console 106
14.1 ليه المتصفح بيمنع أصلاً؟ 109
14.2 أبسط حالة: Access-Control-Allow-Origin 109
14.3 الـ Preflight: "ممكن أبعت؟" قبل ما تبعت 110
14.4 تقرا الخطأ صح 110
14.5 ليه curl مبيتأثرش؟ 112
14.6 تحلها إزاي؟ 113

15 الكاش والـ CDN: أسرع طلب هو اللي مبيتبعتش 114
15.1 ليه نجيب نفس الحاجة كل مرة؟ 117
15.2 Cache-Control: السيرفر بيقول للكاش يعمل إيه 117
15.3 "لسه زي ما هي؟" : ETag و 304 118
15.4 الحيلة اللي بتحل كل حاجة: اسم الملف فيه بصمته 119
15.5 الكاش في DevTools 120
15.6 الـ CDN: نسخة في كل مدينة 121

16 من HTML لبيكسلات: المتصفح بيرسم إزاي 122
16.1 الرحلة من النص للشاشة 124
16.2 مين بيوقف مين؟ 124
16.3 الخطين الأزرق والأحمر 125

17 DevTools: الشبابيك اللي جوه المتصفح 126
17.1 قبل DevTools: alert وربنا يستر 128
17.2 تفتحه إزاي 128
17.3 خريطة الـ Panels 129
17.4 الـ Drawer والـ Command Menu: اتنين هيغيروا حياتك 129
17.5 ظبّطه مرة واحدة 130
17.6 أدوات الـ AI جوه DevTools 131

18 Network Panel (1): تقرا قايمة الطلبات 133
18.1 أول قاعدة: افتحه الأول، وبعدين حمّل 133
18.2 الشاشة حتة حتة 134
18.3 شريط الأدوات 134
18.4 الفلاتر: تلاقي الإبرة في 200 طلب 135
18.5 الأعمدة: كل واحد بيقولك إيه 136
18.6 قيم Status الخاصة 137
18.7 الـ Waterfall و Timing: الرحلة بالملي ثانية 138
18.8 Throttling: جرّب نت وحش من غير ما تنزل الشارع 139
18.9 شريط الملخص تحت 139

19 Network Panel (2): جوه الطلب الواحد 141
19.1 تابة Headers: الحوار كله 141
19.2 تابة Payload: اللي انت بعته 142
19.3 Preview و Response 143

19.4 تابة Initiator: مين طلب ده؟ 143
19.5 تابة Cookies: اتبعتت ولا اتمنعت؟ 143
19.6 انسخ الطلب: cURL و fetch 144
19.7 Request conditions: امنع طلب وشوف إيه اللي هيحصل 145
19.8 Overrides: غيّر الرد من غير ما تلمس السيرفر 145
19.9 HAR: الجلسة كلها في ملف 146

20 الـ Console: تقرا الأخطاء وتكلم الصفحة 148
20.1 وشين في مكان واحد 148
20.2 المستويات والفلاتر 148
20.3 تقرا الخطأ والـ Stack trace 150
20.4 كتالوج رسايل الشبكة 151
20.5 fetch من الـ Console 153
20.6 أدوات الـ Console الصغيرة 153
20.7 الـ Issues panel و Understand this error 154

21 Application Panel و Safari: الكوكيز والتخزين بإيدك 156
21.1 فين كل حاجة 156
21.2 جدول الكوكيز 156
21.3 localStorage و sessionStorage 157
21.4 Clear site data: ابدأ من الصفر 158
21.5 Service Workers باختصار 158
21.6 Safari Web Inspector 159

22 الـ Debugging بطريقة: كودي ولا السيرفر؟ 161
22.1 الطريقة قبل الأدوات 161
22.2 الخريطة: 5 أسئلة بالترتيب 162
22.3 7 حالات هتقابلها 162
22.4 لما المشكلة في كودك: Breakpoints 164
22.5 تسأل حد إزاي 165

23 المشروع النهائي: صفحة بايظة وانت المحقق 167
23.1 جهّز المعمل 167
23.2 قواعد التحقيق 169
23.3 المهام 169
23.4 مهام إضافية (اختياري) 171
23.5 إمتى تعرف إنك خلصت؟ 171

أ Cheat Sheet وجدول الـ Flags وجدول الرموز 173
أ.1 جدول الـ Flags 175
أ.2 جدول الرموز 176

ب قاموس المصطلحات 178

ج مصادر للاستزادة 182
ج.1 ابدأ من هنا: الصورة الكبيرة 182
ج.2 العناوين والأسماء: IP و DNS و URL 182
ج.3 HTTP: اللغة نفسها 183
ج.4 curl والأدوات 183
ج.5 الأمان: TLS والكوكيز و CORS 184
ج.6 السرعة والكاش 185
ج.7 Chrome DevTools (دوكيومنتيشن جوجل الرسمية) 185
ج.8 Safari على الـ Mac 186

د الإجابات والحلول 187

خريطة الكتاب

بص على الطريق كله مرة واحدة قبل ما تبدأ. كل مرحلة مبنية على اللي قبلها: مش هتفهم الـ CORS كويس من غير ما تفهم الـ Origin، ومش هتعرف تقرا تابة الـ Network من غير ما تعرف HTTP. فمتستعجلش على مرحلة الـ DevTools، هي هتبقى سهلة جداً لما توصلها. الأرقام اللي جوه الدوايز أرقام المراحل، وفوقها أرقام الفصول.



الطريق من "يعني إيه إنترنت أصلاً" لحد ما تفتح DevTools وتعرف المشكلة فين في دقيقة.

1 · الصورة الكبيرة

الإنترنت اتولد إزاي وليه، والفرق بينه وبين الويب، ويعني إيه Client و Server. وبعدين رحلة طلب واحد من أول ما تدوس Enter لحد ما الصفحة ترسم.

2 · العناوين والتوصيل

عناوين الـ IP والـ Ports و localhost (اللي هتقابله كل يوم وانت بتشغّل مشاريعك)، والطبقات TCP و UDP، و DNS اللي بيترجم الأسماء لأرقام.

3 · لغة الويب: HTTP

الـ URL متشرّح، والطلب والرد سطر سطر، والـ Methods والـ Status codes والـ Headers، ومعمل curl كامل، ونسخ HTTP من 1.1 لـ 3.

4 · الأمان والحالة

HTTPS و TLS والشهادات، والكوكيز والـ Sessions والـ Storage، والـ CORS اللي هيقابلك أول ما تكلم أي API.

5 · السرعة والعرض

الكاش بكل أنواعه والـ CDN، وإزاي المتصفح بياخد HTML و CSS و JS ويحولهم لصفحة قدامك.

6 · Debugging والـ DevTools

تابة الـ Network بالتفصيل الممل، والـ Console ورسايل الأخطاء، والـ Application، وخطة تمشي عليها لأي مشكلة، ومشروع نهائي تصلّح فيه صفحة بايظة.

Best Practice

متقراش الكتاب وبس. افتح الـ Terminal جنبك وجرّب كل أمر بإيدك، وافتح Chrome والـ DevTools وبص على طلبات أي موقع بتستخدمه كل يوم. الإنترنت حاجة بتحصل في الخفا، والطريقة الوحيدة إنك تشوفها إنك تبص عليها بالأدوات. وخلي الـ **Cheat Sheet** و**قاموس المصطلحات** جنبك وانت بتذاكر.

01

المرحلة 1 · الصورة الكبيرة

الفصل الأول

الحكاية من الأول: الإنترنت اتولد ليه؟

قبل ما نتكلم عن IP و HTTP و DevTools، لازم نعرف الحكاية. مين احتاج يوصّل كمبيوتر بكمبيوتر أصلاً؟ وإيه المشكلة اللي خلّت الناس تقطّع الرسايل لحتت صغيرة؟ وليه الإنترنت والويب حاجتين مختلفتين؟ الفصل ده هيديك الخريطة اللي كل الفصول الجاية هتتعلق عليها.

بعد الفصل ده هتكون عارف

- المشكلة اللي اتعمل الإنترنت عشان يحلها، وفكرة الـ Packet switching
- الفرق بين الإنترنت (الشبكة) والويب (خدمة فوقها)
- يعني إيه Client ويعني إيه Server بالظبط
- يعني إيه Protocol وليه كل حاجة في الكتاب ده Protocol
- مين بيدير الإنترنت ومين بيكتب القواعد (RFCs)
- ليه الموضوع ده مهم ليك كمبرمج JavaScript

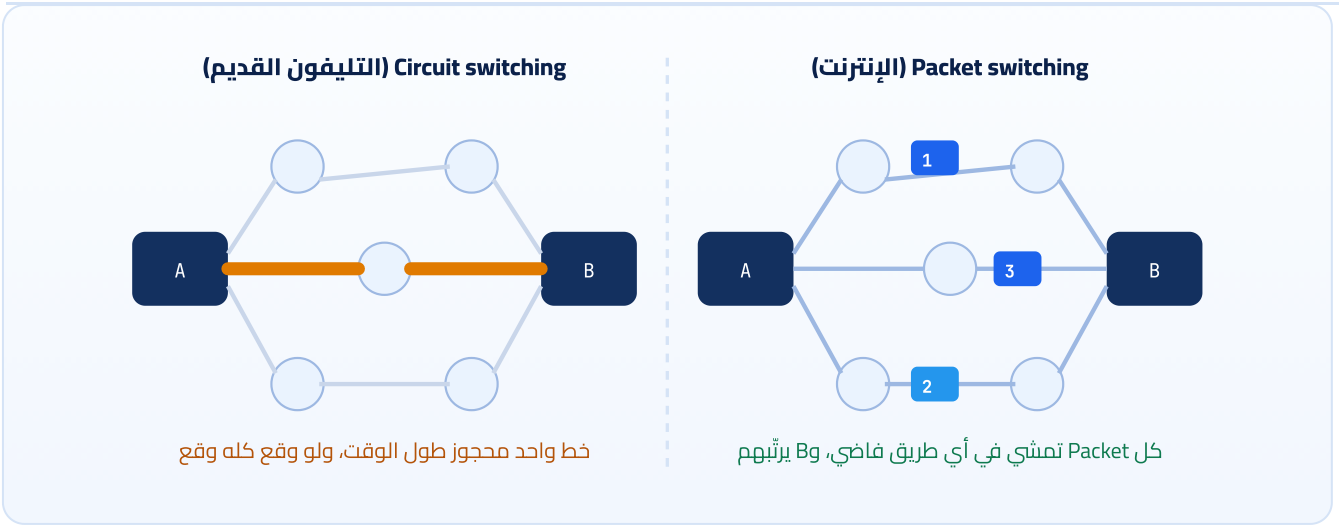
1.1 قبل الإنترنت: كل جهاز جزيرة

خلينا نرجع لستينيات القرن اللي فات. الكمبيوتر وقتها كان أوضة كاملة، وغالي جداً، وكل جامعة أو مركز أبحاث عنده واحد أو اتنين بالكتير. لو انت باحث في جامعة في كاليفورنيا ومحتاج برنامج أو بيانات موجودة على كمبيوتر في جامعة تانية في ولاية تانية، الحل كان بسيط ومرهق: تسافر، أو تستنى حد يبعتلك شريط بالبريد. كل كمبيوتر كان **جزيرة** لوحده، والمعلومة اللي عليه محبوسة جواه.

والحل الأولاني اللي يجي في دماغ أي حد: طب نوصّل الكمبيوترات ببعض بسلك، زي التليفون. وهنا كانت المشكلة. شبكة التليفون وقتها كانت شغالة بطريقة اسمها **Circuit switching**: لما تتصل بحد، السنترال بيحجزلك خط كامل من أولك لآخرك طول المكالمة، حتى لو انتو الاتنين ساكتين. الطريقة دي معقولة للكلام، بس للكمبيوترات مضيعة كبيرة: الكمبيوتر بيبعت حتة بيانات، ويسكت شوية، ويبعت تاني، والخط محجوز على الفاضي. وفيه مشكلة أخطر: لو سنترال واحد في النص وقع، المكالمة كلها بتقع.

1.2 الفكرة العبقرية: قطّع الرسالة حتت

في نفس الفترة تقريباً، كذا باحث وصلوا لنفس الفكرة كل واحد لوحده (منهم Paul Baran في أمريكا و Donald Davies في إنجلترا): بدل ما نحجز خط كامل، **نقطّع الرسالة لحتت صغيرة**، كل حتة عليها عنوان الرايح والجاي ورقمها، ونرميها في الشبكة. كل حتة تمشي لوحدها في أي طريق فاضي، وممكن حتتين من نفس الرسالة يمشوا في طريقين مختلفين، وفي الآخر الجهاز اللي مستلم يرتّبهم بالأرقام ويركّب الرسالة تاني. الحتة دي اسمها **Packet**، والطريقة دي اسمها **Packet switching**.



شكل 1.1 الفرق بين حجز خط كامل وتقطيع الرسالة لـ Packets بتمشي في أي طريق.

أحلى حاجة في الفكرة دي إنها حلّت المشكلتين مرة واحدة. الخطوط بقت متشاركة بين كل الناس، فمفيش خط محجوز على الفاضي. والشبكة بقت **بتعالج نفسها**: لو حتة من الشبكة وقعت، الـ Packets تلف من طريق تاني وخلاص. والفكرة دي لسه هي أساس كل حاجة لحد النهارده: الصورة اللي بتفتحها على Instagram دلوقتي واصلالك مقطّعة لمئات الـ Packets، وكل واحدة فيهم ممكن تكون مشيت في طريق مختلف.

1.3 من ARPANET لـ TCP/IP: لغة واحدة لكل الشبكات

سنة 1969 اتعملت أول شبكة حقيقية بالفكرة دي، اسمها **ARPANET**، ووصّلت في الأول 4 جامعات بس في أمريكا. أول رسالة اتبعتت عليها كانت المفروض كلمة `LOGIN`، بس الشبكة وقعت بعد أول حرفين، فأول رسالة في تاريخ الإنترنت كانت `LO`. ومن هنا بدأت الشبكة تكبر.

بس ظهرت مشكلة جديدة. في السبعينيات بقى فيه شبكات كتير: شبكة جامعات، وشبكة بالراديو، وشبكة بالأقمار الصناعية. كل شبكة ليها طريقتها وقواعدها، ومفيش شبكة تعرف تكلم التانية. زي ما يكون كل بلد بيتكلم لغة، ومفيش مترجم. فاتطور حل: اتنين باحثين اسمهم **Vint Cerf** و **Bob Kahn** صمموا مجموعة قواعد واحدة أي شبكة تقدر تتكلمها فوق طريقتها الخاصة، وسموها **TCP/IP**. ويوم 1 يناير 1983 كل ARPANET اتحولت ليها في يوم واحد. اليوم ده بيعتبره ناس كتير "عيد ميلاد الإنترنت".

وكلمة **Internet** نفسها جاية من هنا: inter-network، يعني "شبكة بين الشبكات". الإنترنت مش شبكة واحدة كبيرة، ده آلاف الشبكات (شبكة شركة الإنترنت بتاعتك، وشبكة Google، وشبكة الجامعة) متوصلة ببعض وكلها بتتكلم نفس اللغة.

> **يعني إيه Protocol؟**
>
> الكلمة دي هتتكرر في الكتاب كله: TCP و IP و HTTP و TLS و DNS، كلهم **Protocols**. الـ Protocol ببساطة **اتفاق على شكل الكلام**: مين يبدأ، ويقول إيه، والرد يبقى شكله إيه، ولو حصلت مشكلة نعمل إيه. زي لما تكلم حد في التليفون: "ألو" الأول، وبعدين الكلام، وبعدين "مع السلامة". لو واحد اتكلم بطريقة والتاني مستني طريقة تانية، مفيش تفاهم. وعشان كده الكتاب ده كله في الحقيقة كتاب عن اتفاقيات: كل فصل بيشرح اتفاق، وليه اتعمل كده.

1.4 الإنترنت مش هو الويب

لحد آخر الثمانينيات، الإنترنت كان موجود وشغال، بس كان مكان للباحثين والمهندسين. تبعت إيميل، وتنقل ملفات بأوامر معقدة، وتدخل على كمبيوتر تاني من بعيد. مفيش "مواقع"، ومفيش حاجة تدوس عليها.

سنة 1989، مهندس في مركز أبحاث CERN في سويسرا اسمه **Tim Berners-Lee** كانت عنده مشكلة: آلاف الباحثين بيكتبوا أوراق ومستندات على أجهزة مختلفة، ومحدش عارف يلاقي حاجة. ففكر: طب ليه كل مستند ميبقاش فيه **روابط** لمستندات تانية، تدوس عليها توصلك للمستند التاني حتى لو على جهاز في آخر الدنيا؟ وعشان الفكرة دي تشتغل، اخترع تلات حاجات مع بعض، وهي التلات حاجات اللي الكتاب ده هيشرحهم بالتفصيل:

HTML

لغة لكتابة المستند نفسه، وفيها الروابط. ده اللي بتكتبه في كورس JavaScript.

URL

عنوان فريد لكل مستند في الدنيا، عشان الرابط يعرف يروح فين (الفصل 6).

HTTP

الـ Protocol اللي المتصفح يطلب بيه المستند، والسيرفر يرد بيه (الفصول 7 لـ 11).

وعمل كمان أول متصفح وأول سيرفر ويب في الدنيا. والاختراع ده اسمه **World Wide Web**، أو الويب. فخلي الفرق ده في دماغك عشان هيفرق معاك: **الإنترنت هو الطرق والكباري**، الشبكة اللي بتنقل الـ Packets من أي جهاز لأي جهاز. و**الويب خدمة من الخدمات اللي ماشية فوقه**، زي الإيميل وزي ألعاب الأونلاين وزي مكالمات الفيديو. الويب ركب على الإنترنت، مش هو الإنترنت.



شكل 1.2 الويب واحد من خدمات كتير ماشية فوق الإنترنت.

1.5 مين بيطلب ومين بيرد: الـ Client والـ Server

أهم فكرة في الويب كله إن فيه طرفين في أي كلام. طرف **بيطلب** حاجة، اسمه **Client**. وطرف **بيستنى الطلبات ويرد عليها**، اسمه **Server**. زي المطعم بالظبط: انت الزبون بتطلب من الجرسون، والمطبخ بيجهّز الطلب ويرجعهولك. انت مبتدخلش المطبخ، والمطبخ مبيجيش يسألك جعان ولا لأ.

وفيه حاجة بتلخبط ناس كتير في الأول: **الـ Server مش نوع جهاز خاص**. هو مجرد برنامج شغال على أي كمبيوتر، قاعد مستني حد يكلمه. وأحلى دليل على كده إنك هتعمل Server على اللابتوب بتاعك قريب جداً: أول ما تشغّل مشروع React بـ `npm run dev` أو Node بـ `node server.js`، اللابتوب بتاعك بقى Server، والمتصفح اللي على نفس اللابتوب هو الـ Client. ونفس الجهاز ممكن يبقى الاتنين في نفس الوقت: السيرفر بتاعك وهو بيرد على المتصفح، ممكن هو نفسه يبقى Client لسيرفر تاني (زي API بتاع الطقس).



شكل 1.3 الـ Client بيبدأ دايماً بطلب (Request)، والـ Server بيرد (Response).

ولاحظ حاجة مهمة في الرسمة: **الـ Client هو اللي بيبدأ دايماً**. السيرفر في HTTP العادي مبيبعتش حاجة لحد من نفسه، هو بيرد بس. والمعلومة دي هتفسرلك حاجات كتير بعدين، زي ليه الشات محتاج تقنيات زي WebSockets عشان السيرفر يقدر يبعت رسالة جديدة من غير ما تسأله.

> **حاجة مشتركة: نفس الفكرة في كل مكان**
>
> فكرة "واحد بيطلب وواحد بيرد" مش للويب بس. الـ Terminal نفسه بيعمل كده لما تكتب `git push` : الـ Git اللي على جهازك Client، و GitHub هو الـ Server. ولما تعمل `docker pull` ، الـ Docker Client بيكلم Registry هو Server. وفي كتاب Docker، الـ Docker CLI نفسه Client بيكلم الـ Docker daemon اللي هو Server على نفس الجهاز. أول ما تفهم النمط ده هتلاقيه في كل حتة.

1.6 مين بيدير الإنترنت؟ ومين بيكتب القواعد؟

سؤال منطقي: لو الإنترنت آلاف الشبكات، مين صاحبه؟ الإجابة: **محدش لوحده**. كل شبكة ليها صاحب (شركة الاتصالات بتاعتك عندها شبكتها، و Google عندها شبكتها)، والشبكات دي بتتفق مع بعض إنها تنقل لبعض. بس فيه كام جهة مهمة لازم تعرف أساميها عشان هتقابلها في المصادر:

الجهة	بتعمل إيه
IETF	بتكتب قواعد الإنترنت نفسه (TCP و IP و HTTP و TLS و DNS) في مستندات اسمها **RFC**. مثلاً HTTP مكتوب في `RFC 9110` .
W3C و WHATWG	بيكتبوا قواعد الويب في المتصفح: HTML و CSS والـ APIs زي `fetch` .
ICANN و IANA	بيديروا أسماء الدومينات وتوزيع عناوين الـ IP وأرقام الـ Ports المشهورة.
ISP	شركة الإنترنت بتاعتك (WE أو Vodafone أو Orange مثلاً): الشبكة اللي بتوصّلك بباقي الإنترنت.

جدول 1.1 الجهات اللي هتقابل أساميها

لو حابب تتعمق: يعني إيه RFC؟ تقدر تعدّيه في أول قراية

الـ RFC اختصار Request for Comments، يعني "طلب آراء". الاسم ده من أيام ARPANET لما الباحثين كانوا بيكتبوا أفكارهم ويقولوا للباقيين "إيه رأيكم؟". النهارده الـ RFC مستند رسمي بيوصف Protocol بالتفصيل، وكل واحد ليه رقم مبيتغيرش. ولو اتعدل الـ Protocol بيطلع RFC جديد برقم جديد. مش محتاج تقرا RFCs دلوقتي، بس لما تشوف `RFC 9110` في MDN، تعرف إنه "المرجع الرسمي اللي كل المتصفحات والسيرفرات ماشية عليه". وفي ملحق المصادر هتلاقي روابط أهمهم.

1.7 طب وأنا كمبرمج JavaScript، ده يفرق معايا في إيه؟

سؤال في محله. انت دلوقتي في كورس Jonas، وكتير من الكلام ده ممكن يبان بعيد عن `let` و`function`. بس استنى لحد ما توصل لجزء الـ **Asynchronous JavaScript** و`fetch`. ساعتها هتكتب سطر زي `fetch('https://restcountries.com/v3.1/name/egypt')`، وحاجات كتير ممكن تبوظ: الـ URL غلط، أو السيرفر رد بـ 404، أو المتصفح منع الرد بسبب CORS، أو الرد جه بس شكله مش زي ما انت متوقع. ومن غير الكتاب ده، كل المشاكل دي هتبان ليك "الكود مش شغال" وخلاص. ومع الكتاب ده، هتفتح تابة الـ Network، وتبص على الطلب، وتعرف في ثانية المشكلة فين.

في كورس JavaScript

`fetch` و`async/await` والـ Promises هيبقوا منطقيين، لأنك عارف إيه اللي بيحصل ورا السطر ده على الشبكة.

في Node بعد كده

هتبقى انت اللي بتكتب السيرفر: هتختار الـ Status code وتحط الـ Headers وتظبط الـ CORS والكوكيز بنفسك.

في React

كل صفحة بتجيب بيانات من API. ولما الصفحة تفضل فاضية، هتعرف تفرّق بين مشكلة في الـ Component ومشكلة في الطلب.

في شغلك كـ Content Engineer

الـ SEO والكاش والـ CDN والـ Redirects وسرعة الصفحة، كلها مفاهيم من الكتاب ده، وهتتكلم فيها مع المبرمجين بلغتهم.

ملخص الفصل

- الإنترنت اتعمل عشان يوصّل كمبيوترات كانت جزر منفصلة، والفكرة الأساسية **Packet switching**: الرسالة بتتقطع حتت كل واحدة تمشي لوحدها.
- **TCP/IP** هو اللغة الواحدة اللي خلت كل الشبكات تكلم بعض، والإنترنت = شبكة الشبكات.
- الويب (HTML و URL و HTTP) خدمة فوق الإنترنت، مش هو الإنترنت.
- الـ Client بيبدأ بطلب، والـ Server برنامج مستني يرد. واللابتوب بتاعك هيبقى Server أول ما تشغّل مشروع.
- الـ Protocol = اتفاق على شكل الكلام، وقواعد الإنترنت مكتوبة في RFCs.

صح ولا غلط؟

1. الإنترنت والويب اسمين لنفس الحاجة. صح / غلط
2. في Packet switching، كل الـ Packets بتاعة رسالة واحدة لازم تمشي في نفس الطريق. صح / غلط
3. الـ Server لازم يبقى جهاز خاص غالي في Data center. صح / غلط

؟ أسئلة مراجعة

1. إيه المشكلتين اللي في Circuit switching، و Packet switching حلّهم إزاي؟
2. Tim Berners-Lee اخترع تلات حاجات عشان الويب يشتغل. إيه هما، وكل واحدة بتعمل إيه؟
3. اشرح بمثال من عندك ليه نفس الجهاز ممكن يبقى Client و Server في نفس الوقت.
4. يعني إيه Protocol؟ واذكر 3 Protocols هتتعلمهم في الكتاب.

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اكتب في ورقة 5 خدمات بتستخدمها كل يوم، وقسّمهم: أنهي واحدة "ويب" (بتفتحها في متصفح) وأنهي واحدة خدمة تانية فوق الإنترنت.
2. متوسط افتح `https://info.cern.ch` في المتصفح. دي نسخة من أول موقع في التاريخ. لاحظ إن كله HTML وروابط بس، وفكّر: إيه اللي ناقص عن مواقع النهارده؟
3. صعب افتح `https://www.rfc-editor.org/rfc/rfc9110` واقرا الـ Abstract (أول فقرة). حاول تلخصها بالعامي في سطرين.

02

المرحلة 1 · الصورة الكبيرة

الفصل التاني

رحلة طلب واحد: من Enter لحد ما الصفحة تظهر

بتكتب example.com وتدوس Enter، وفي أقل من ثانية الصفحة قدامك. في الثانية دي بيحصل أكتر من عشر خطوات، كل خطوة فيهم ليها فصل كامل في الكتاب ده. الفصل ده هو الخريطة: هنمشي الرحلة كلها مرة واحدة بسرعة، عشان لما ندخل في تفاصيل كل محطة تبقى عارف هي فين من الصورة الكبيرة.

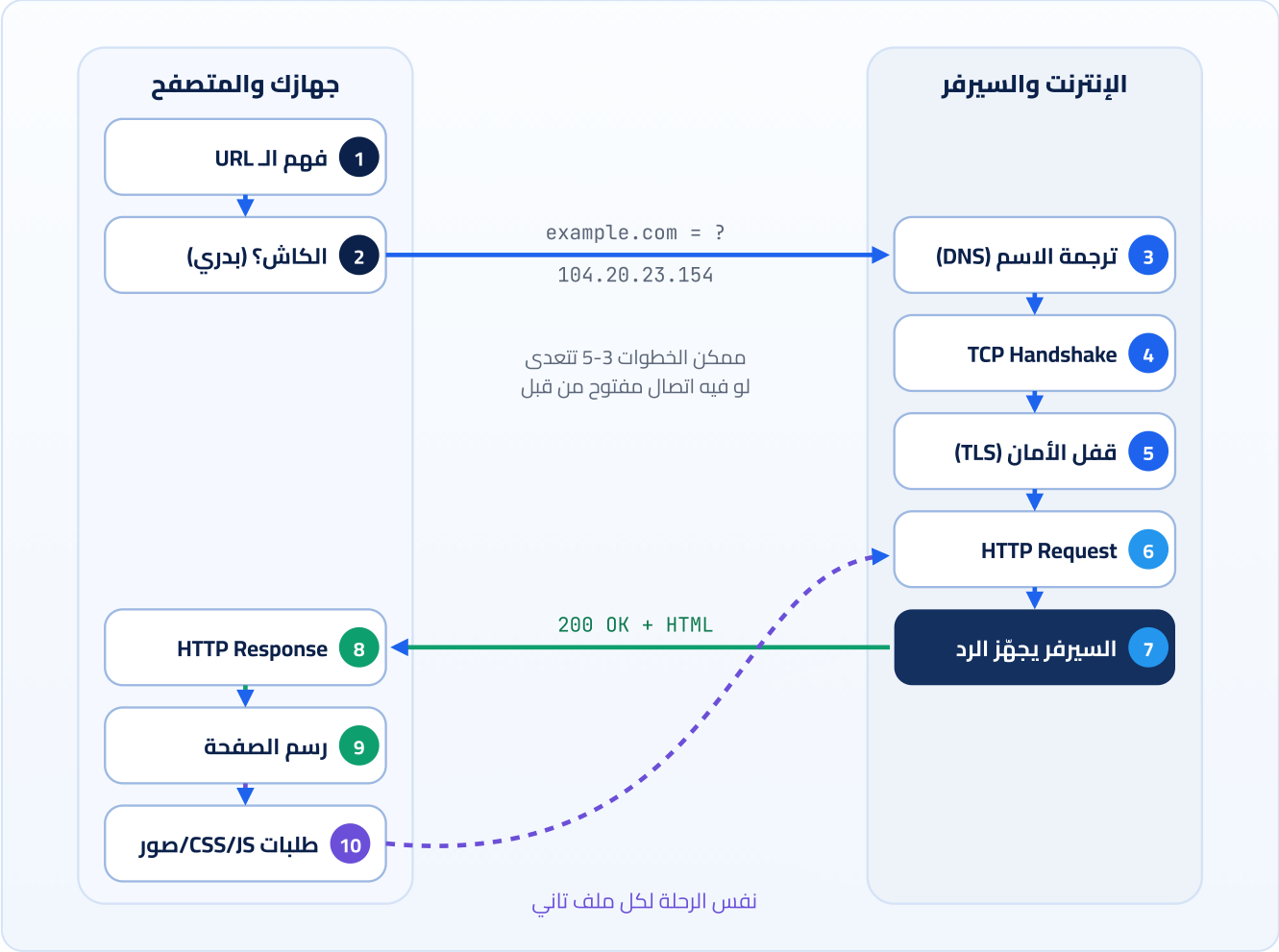
بعد الفصل ده هتكون عارف

- [x] الخطوات الكاملة من ما تدوس Enter لحد ما الصفحة تترسم
- [x] ليه الصفحة الواحدة بتعمل عشرات الطلبات مش طلب واحد
- [x] كل خطوة بتحل مشكلة إيه، وفين تفاصيلها في الكتاب
- [x] أول تجربة ليك تشوف الرحلة دي بعينك في الـ Terminal وفي DevTools

2.1 السؤال اللي بيتسأل في كل Interview

فيه سؤال مشهور جداً في مقابلات الشغل لأي مبرمج ويب: "إيه اللي بيحصل لما تكتب `google.com` في المتصفح وتدوس Enter؟". السؤال ده مشهور لأنه بيكشف في دقيقتين إذا كنت فاهم الويب ولا حافظ كود وخلاص. والحلو إن الإجابة عليه هي الكتاب ده كله بالظبط. فخلينا نجاوبه دلوقتي مرة بسرعة، وبعدين كل فصل هيفتح محطة من المحطات دي ويشرحها بالتفصيل.

تخيل إنك قاعد على اللابتوب، فتحت Chrome، وكتبت `https://example.com/products`، ودوست Enter. من وجهة نظرك الموضوع خطوة واحدة. من وجهة نظر المتصفح، هو لسه عنده مشكلة كبيرة: هو عارف **اسم** الموقع بس، ومحتاج يوصل لجهاز معين في مكان ما في الدنيا، ويكلمه بأمان، ويطلب منه الصفحة، ويفهم الرد، ويرسمه. يلا نمشي معاه.



شكل 2.1 الرحلة كلها في رسمة واحدة. كل رقم هنا ليه فصل كامل في الكتاب.

2.2 المحطة 1 و 2: المتصفح يفهم انت عاوز إيه

أول حاجة المتصفح بيعملها إنه يقرا اللي كتبته. هل ده **عنوان** ولا **كلمة بحث**؟ لو كتبت `pizza near me` هيبعتها لمحرك البحث. لو كتبت `example.com` هيفهم إنه عنوان، وهيكمّله لوحده لـ `https://example.com/`. وبعدين يفصّص العنوان لحتت: البروتوكول `https`، واسم الجهاز `example.com`، والمسار `/products`. التفصيص ده مهم جداً عشان كل حتة هتستخدم في خطوة مختلفة، وده موضوع **الفصل 6**.

بعدين يسأل نفسه: "هل أنا جبت الصفحة دي قبل كده ولسه معايا نسخة صالحة؟". لو أيوه، ممكن يعرضها على طول من غير ما يكلم حد خالص. ده **الكاش**، وهو سبب إن المواقع اللي بتفتحها كل يوم بتفتح بسرعة. وده موضوع **الفصل 15**.

2.3 المحطة 3: DNS، دليل التليفونات

هنا أول مشكلة حقيقية. الأجهزة على الإنترنت مبتعرفش بعض بالأسماء، بتعرف بعض بالأرقام، اسمها **IP addresses**، زي `104.20.23.154`. والبشر مش هيحفظوا أرقام لكل موقع. فاتعمل نظام اسمه **DNS** شغلته يترجم الاسم لرقم، زي دليل التليفونات بالظبط: انت عارف اسم المطعم، والدليل يديك رقمه. المتصفح بيسأل: "`example.com` رقمه كام؟"، والسؤال ده بيلف على كذا سيرفر لحد ما يلاقي الإجابة. الرحلة دي هنرسمها خطوة خطوة في **الفصل 5**، والـ IP نفسه موضوع **الفصل 3**.

2.4 المحطة 4 و 5: نفتح خط، ونأمّنه

دلوقتي المتصفح معاه الرقم. بس مينفعش يرمي الطلب كده في الشبكة ويتمنى إنه يوصل. محتاج الأول **يفتح اتصال** مع السيرفر ويتأكد إن فيه حد على الطرف التاني بيسمع. ده بيحصل بحاجة اسمها **TCP Handshake**، تلات رسايل صغيرة زي "سامعني؟" و"سامعك، وانت سامعني؟" و"أيوه". ومعاها رقم مهم اسمه **Port** بيحدد أنهي برنامج على السيرفر هيرد (443 للـ HTTPS). ده **الفصل 4**.

وبعدين، عشان العنوان فيه `https` بـ `s`، لازم الخط يبقى مقفول بحيث محدش في النص يقدر يقرا أو يعدّل الكلام (لا شركة الإنترنت ولا اللي قاعد معاك على نفس الواي فاي في الكافيه). ده الـ **TLS Handshake**: السيرفر بيثبت إنه فعلاً `example.com` بشهادة، والطرفين بيتفقوا على مفتاح تشفير. وده **الفصل 12**.

2.5 المحطة 6 و 7 و 8: الطلب والرد

أخيراً الخط جاهز ومتأمّن. المتصفح بيكتب رسالة بلغة **HTTP**، شكلها تقريباً كده:

الطلب (مبسّط)

```
GET /products HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) ...
Accept: text/html
```

يعني: "هاتلي `/products` من `example.com`، وأنا متصفح على Mac، وعاوز HTML". السيرفر بيستلم الرسالة، وبيشغّل الكود بتاعه (ممكن يجيب بيانات من Database، أو يقرا ملف)، وبيرجّع رد:

الرد (مبسّط)

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1256
Cache-Control: max-age=600

<!doctype html><html>...
```

أول سطر فيه رقم `200` معناه "تمام، أهو". وبعده سطور اسمها **Headers** بتوصف الرد، وبعد سطر فاضي جسم الرد نفسه (الـ HTML). كل سطر من دول هنفصّصه حتة حتة في الفصول 7 و 8 و 9. ومش هتصدق قد إيه هتستفيد لما تعرف تقرا السطور دي، لأنها أول حاجة هتبص عليها في DevTools لما حاجة تبوظ.

2.6 المحطة 9 و 10: المتصفح يرسم، ويطلب تاني

المتصفح يبدأ يقرا الـ HTML من فوق لتحت ويبني منه شجرة اسمها **DOM** (اللي هتتعامل معاها في JavaScript بـ `document.querySelector`). وأثناء القراية بيلاقي سطور زي `<link href="style.css">` و`<script src="app.js">` و`<img src="logo.png">`. كل سطر من دول معناه **طلب HTTP جديد**، وكل طلب ممكن يعدّي على نفس الرحلة (أو جزء منها لو الاتصال لسه مفتوح). وعشان كده لما تفتح تابة الـ Network هتلاقي الصفحة الواحدة عملت 30 أو 80 أو حتى 200 طلب، مش طلب واحد. وفي الآخر، لما يتجمع الـ HTML والـ CSS، المتصفح بيحسب مكان كل حاجة ويلوّنها على الشاشة. ده **الفصل 16**.

الرحلة دي بتاخد قد إيه؟

على اتصال كويس، الرحلة كلها لموقع قريب ممكن تاخد أقل من 100 ملي ثانية. والجزء الأكبر من الوقت غالباً مش في السيرفر، في **المسافة**: كل رسالة لازم تروح وترجع، والضوء نفسه في الألياف الضوئية محتاج حوالي 100 ملي ثانية عشان يروح من القاهرة لكاليفورنيا ويرجع. وده سبب ظهور حاجات زي الـ CDN (سيرفرات قريبة منك) و HTTP/3 (رسايل handshake أقل). هتشوف الأوقات دي بعينك في تابة Timing في الفصل 19.

2.7 شوف الرحلة بعينك دلوقتي

كفاية كلام. افتح الـ Terminal على الـ Mac وجرّب الأمر ده. متقلقش لو الناتج مش مفهوم كله دلوقتي، هنرجعله في الفصل 10 ونفصّصه سطر سطر. المهم دلوقتي إنك تشوف إن المحطات اللي اتكلمنا عنها حقيقية:

تشريح الأمر

```
curl -v https://example.com -o /dev/null
```

1. `curl` — برنامج بيعمل طلبات HTTP من الـ Terminal. زي متصفح من غير شاشة. موجود على الـ Mac من غير تسطيب.
2. `-v` — Verbose: "احكيلي كل حاجة بتعملها". هيطبع الـ DNS والاتصال والـ TLS والطلب والرد.
3. `https://example.com` — العنوان اللي هنطلبه. `example.com` دومين محجوز رسمياً للأمثلة، فآمن تجرب عليه.
4. `-o /dev/null` — Output = "اكتب جسم الرد في الملف ده". و `/dev/null` ملف خاص في أي نظام Unix بيبلع أي حاجة تتكتب فيه (زي البلاعة)، فالـ HTML مش هيتطبع ويزحم الشاشة.

Terminal

```
user@MacBook ~ % curl -v https://example.com -o /dev/null
* Host example.com:443 was resolved.
* IPv4: 104.20.23.154
*   Trying 104.20.23.154:443...
* Connected to example.com (104.20.23.154) port 443
* SSL connection using TLSv1.3 / AEAD-AES256-GCM-SHA384
* Server certificate:
*  subject: CN=example.com
> GET / HTTP/2
> Host: example.com
> User-Agent: curl/8.7.1
> Accept: */*
>
< HTTP/2 200
< content-type: text/html; charset=utf-8
< server: cloudflare
< cf-cache-status: HIT
<
```

curl -v المحطات في ناتج

1 المحطة 3: الـ DNS رجّع الرقم. السطور اللي بتبدأ بـ `*` معلومات من curl نفسه.	`* Host ... was resolved / IPv4:`
2 المحطة 4: فتح اتصال TCP على Port `443`.	`* Trying ...:443 / Connected`
3 المحطة 5: الـ TLS خلص، والخط متأمّن. والشهادة باسم السيرفر.	`* SSL connection using TLSv1.3`
4 المحطة 6: الطلب اللي curl بعته. السطور اللي بتبدأ بـ `>` = رايح للسيرفر.	`> GET / HTTP/2 ...`
5 المحطة 8: الرد. السطور اللي بتبدأ بـ `<` = جاي من السيرفر.	`< HTTP/2 200 ...`

الأرقام اللي عندك (الـ IP ونسخة curl) ممكن تختلف، ده طبيعي. و example.com نفسه دلوقتي ورا Cloudflare، فالـ IP اللي هيطلعلك ممكن يتغير من وقت للتاني.

مثال عملي: نفس الرحلة في المتصفح

افتح Chrome، ودوس `Cmd+Option+I` عشان تفتح DevTools، واختار تابة **Network**، وبعدين افتح `https://example.com`. هتلاقي سطر للطلب. دوس عليه وبص على تابة **Timing**: هتلاقي نفس المحطات بأساميها: **DNS Lookup** و**Initial connection** و**SSL** و**Waiting for server response** و**Content Download**. ده بالظبط اللي هنتعلم نقراه في المرحلة الأخيرة من الكتاب.

حاجة مشتركة: الرموز > و < و *

الرموز دي هتشوفها كتير مع `curl -v` في الكتاب كله: `>` = حاجة رايحة منك للسيرفر، و`<` = حاجة جاية من السيرفر ليك، و`*` = curl بيحكيلك بيعمل إيه. وخلي بالك إن معناهم هنا **مختلف** عن معنى `>` في الـ Terminal نفسه (اللي بيحوّل الناتج لملف زي `ls > files.txt`). هنا هي مجرد علامة مطبوعة في الناتج، مش أمر.

ملخص الفصل

- الرحلة: فهم الـ URL، الكاش، DNS، TCP، TLS، HTTP Request، السيرفر، HTTP Response، الرسم، وطلبات للملفات التانية.
- كل محطة بتحل مشكلة: الاسم لرقم (DNS)، خط مضمون (TCP)، خط متأمّن (TLS)، لغة مشتركة (HTTP).
- الصفحة الواحدة = عشرات الطلبات، كل ملف CSS و JS وصورة طلب لوحده.
- `curl -v` وتابة Timing في DevTools بيوروك المحطات دي بعينك.

صح ولا غلط؟

1. المتصفح بيعمل طلب HTTP واحد بس لكل صفحة. صح / غلط
2. الـ DNS بيحصل بعد ما المتصفح يبعت طلب الـ HTTP. صح / غلط
3. في ناتج `curl -v`، السطور اللي بتبدأ بـ `<` جاية من السيرفر. صح / غلط

أسئلة مراجعة

1. رتّب المحطات دي: TLS، DNS، HTTP Request، TCP، الرسم.
2. ليه المتصفح محتاج DNS أصلاً؟ ما هو عارف اسم الموقع.
3. إيه فايدة `-o /dev/null` في أمر الـ curl اللي جربناه؟
4. اذكر سببين يخلّوا الموقع يفتح أسرع تاني مرة تفتحه.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل جرّب `curl -v https://example.com -o /dev/null` وعلّم بقلم على كل محطة في الناتج بتاعك.
2. متوسط افتح تابة Network في DevTools وافتح موقع بتستخدمه كل يوم. شوف في الشريط اللي تحت عدد الطلبات (requests). كام طلب؟
3. صعب اكتب بإيدك إجابة سؤال الـ Interview ("إيه اللي بيحصل لما تكتب google.com؟") في 10 سطور من غير ما تبص في الكتاب. احتفظ بيها وقارنها بعد ما تخلص الكتاب.

03

المرحلة 2 · العناوين والتوصيل

الفصل التالت

العناوين: IP والـ Ports و localhost

عشان Packet توصل لجهاز معين في وسط مليارات الأجهزة، لازم يبقى لكل جهاز عنوان. وعشان توصل لبرنامج معين جوه الجهاز ده، لازم يبقى فيه رقم شقة كمان. الفصل ده عن العنوان (IP) ورقم الشقة (Port)، وعن أشهر عنوان هتقابله في حياتك كمبرمج: localhost:3000.

بعد الفصل ده هتكون عارف

- ☑ IPv4 شكله إيه وليه العناوين خلصت
- ☑ الـ Private IP والـ Public IP والـ Router والـ NAT
- ☑ IPv6 وليه موجود
- ☑ يعني إيه Port، وأشهر الأرقام اللي هتقابلها
- ☑ localhost و 127.0.0.1 و 0.0.0.0 والفرق بينهم
- ☑ تعرف IP جهازك على الـ Mac، وتشغّل سيرفر، وتعرف مين ماسك Port بـ lsof

3.1 المشكلة: الـ Packet رايحة فين؟

في الفصل الأول قلنا إن الرسالة بتتقطع لـ Packets، وكل Packet بتمشي لوحدها. طب إزاي كل جهاز في النص (اسمه **Router**) يعرف يوديها فين؟ نفس المشكلة اللي عند البوسطة: الجواب لازم يبقى عليه **عنوان** واضح، وكل مكتب بريد يبص على العنوان ويقرر يبعته للمكتب اللي بعده. فاتطور حل: كل جهاز متوصل بالإنترنت ياخد رقم فريد اسمه **IP address** (الـ IP هو الـ Protocol اللي اتكلمنا عنه، Internet Protocol). وكل Packet بيبقى مكتوب عليها IP المرسل و IP المستلم، زي الجواب بالظبط.

3.2 IPv4: أربع أرقام بنقط

الشكل اللي هتشوفه أغلب الوقت اسمه **IPv4**، وشكله كده: `192.168.1.15`. أربع أرقام، بينهم نقط، وكل رقم من `0` لـ `255`.

تشريح العنوان

1 2 3 4
192.168.1.15

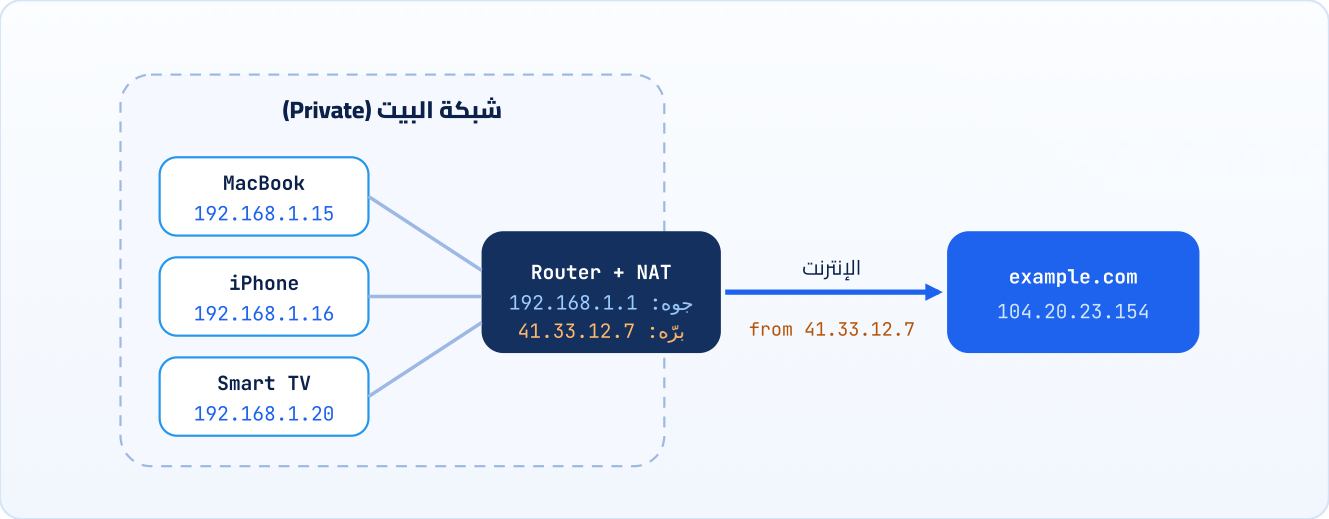
1	`192`	أول "Octet". كل رقم من الأربعة اسمه Octet لأنه 8 Bits. و 8 Bits تقدر تعدّ من 0 لـ 255، عشان كده مفيش رقم أكبر من 255.
2	`.`	النقطة مجرد فاصل للقراية. الكمبيوتر شايف العنوان رقم واحد طويل من 32 Bit.
3	`168.1`	الـ Octet التاني والتالت. في الشبكات البيتي، الجزء الأول ده (`192.168.1`) غالباً بيبقى "اسم الشبكة" نفسها.
4	`15`	آخر Octet: غالباً ده رقم جهازك جوه الشبكة. موبايلك ممكن يبقى `.16` والتلفزيون `.20`.

32 Bit يعني حوالي 4.3 مليار عنوان. رقم كان يبان مهول في السبعينيات، لما الإنترنت كان كام ألف جهاز. بس النهارده كل بني آدم معاه موبايل ولابتوب وساعة وتلفزيون ذكي. وهنا كانت المشكلة: **العناوين خلصت**. المنظمات اللي بتوزع العناوين أعلنت إن مخزون IPv4

الجديد خلص فعلاً من سنين. والناس طلعت بحلّين، واحد "ترقيع" شغال من زمان، وواحد حل جذري لسه بينتشر.

3.3 الحل الأول: Private IP والـ NAT

الفكرة: مش لازم كل جهاز في البيت ياخد عنوان عام على الإنترنت. البيت كله ياخد **عنوان عام واحد** (اسمه **Public IP**) والـ Router بتاعك هو اللي ماسكه. وجوه البيت، كل جهاز ياخد عنوان **خاص** (اسمه **Private IP**) من نطاقات محجوزة مخصوص للشبكات الداخلية، ومحدش برّه بيشوفها. وعشان كده ممكن تلاقي جهازك `192.168.1.15` وجهاز صاحبك في بيته برضه `192.168.1.15`، ومفيش أي مشكلة، زي ما ممكن يبقى فيه "شقة 5" في كل عمارة في الشارع.



شكل 3.1 الـ Router بيبدّل العنوان الخاص بالعنوان العام وهو خارج، وبيفتكر يرجّع الرد لمين. ده الـ NAT. (الأرقام توضيحية)

طب السيرفر هيرد على مين، لو كل الأجهزة طالعة باسم الـ Router؟ هنا شغل **NAT** (Network Address Translation): الـ Router وهو بيطلّع الـ Packet بيشيل عنوانك الخاص ويحط عنوانه العام، ويكتب في جدول عنده "الطلب ده بتاع الـ MacBook". ولما الرد يرجع، يبص في الجدول ويوديه للجهاز الصح. عشان كده موقع زي `example.com` عمره ما بيشوف `192.168.1.15`، بيشوف عنوان الـ Router بس.

النطاق	يعني إيه	هتقابله فين
`192.168.0.0 → 192.168.255.255`	Private	أغلب راوترات البيوت
`10.0.0.0 → 10.255.255.255`	Private	شبكات الشركات، والـ VPN، والـ Cloud
`172.16.0.0 → 172.31.255.255`	Private	شبكات Docker الداخلية (`172.17.0.x`)
`127.0.0.0 → 127.255.255.255`	Loopback: الجهاز نفسه	localhost = `127.0.0.1`
`169.254.x.x`	عنوان طوارئ: الجهاز ملقاش Router يديله عنوان	لو شفته على جهازك يبقى الواي فاي فيه مشكلة

جدول 3.1 النطاقات المحجوزة اللي هتقابلها

لو حابب تتعمق: يعني إيه /24؟ تقدر تعدّيه في أول قراية

هتشوف كتير كتابة زي `192.168.1.0/24` . ده اسمه **CIDR**. الرقم بعد الـ `/` معناه "أول كام Bit من الـ 32 ثابتين وبيمثّلوا الشبكة". `/24` = أول 24 Bit (يعني أول 3 أرقام `192.168.1`) ثابتين، والباقي (آخر رقم) متاح للأجهزة، يعني 256 عنوان. و `/16` = أول رقمين ثابتين، يعني 65,536 عنوان. وكل ما الرقم بعد الـ `/` يصغر، الشبكة تكبر. ودي نفس الكتابة اللي ممكن تقابلها في إعدادات Docker أو Cloud.

3.4 الحل الجذري: IPv6

الـ NAT شغال، بس ترقيع. الحل الحقيقي إن العناوين تكبر. فاتعمل **IPv6**، وبدل 32 Bit بقى 128 Bit. الرقم ده كبير لدرجة صعب تتخيلها: يكفي تدي عنوان لكل حبة رمل على الأرض وتفضل معاك عناوين. وشكله مختلف، بيتكتب بالـ Hex (أرقام من 0 لـ 9 وحروف من a لـ f) في 8 مجموعات بينهم `:` :

تشريح العنوان

```
1    2  3    4
2606:4700:10::6814:179a
```

1	`2606:4700`	كل مجموعة 4 خانات Hex = 16 Bit. أول مجموعات غالباً بتحدد الشبكة (هنا شبكة Cloudflare).
2	`:`	الفاصل بين المجموعات بدل النقطة في IPv4.
3	`::`	اختصار معناه "هنا شوية مجموعات كلها أصفار". ومينفعش يتكتب غير مرة واحدة في العنوان، عشان الجهاز يعرف يحسب الأصفار كام.
4	`6814:179a`	آخر المجموعات. والأصفار اللي على الشمال في أي مجموعة ممكن تتشال: `0db8` = `db8` .

وده العنوان الحقيقي (واحد منهم) لـ `example.com` على IPv6. والـ localhost في IPv6 بيتكتب `::1` ، يعني "كله أصفار وآخره 1". أغلب الأجهزة النهارده بتشتغل بالاتنين مع بعض (اسمها Dual stack)، والمتصفح بيجرب الاتنين ويستخدم الأسرع.

⚠ خلي بالك: localhost ممكن يبقى ::1 مش 127.0.0.1

من Node 17 وطالع، كلمة `localhost` ممكن تتحول لـ `::1` (IPv6) الأول. فلو سيرفر شغال على `127.0.0.1` بس، وأداة تانية بتحاول توصله على `localhost` ، ممكن تاخد `ECONNREFUSED ::1:3000` وتستغرب. الحل: استخدم نفس الكلمة في الحتتين، أو اكتب `127.0.0.1` صريحة. هتقابل المشكلة دي غالباً لما تبدأ Node.

3.5 اعرف عناوينك على الـ Mac

خلينا نشوف الكلام ده على جهازك. أول حاجة: العنوان الخاص بتاعك جوه الشبكة.

تشريح الأمر

```
ipconfig getifaddr en0
```
(1) (2) (3)

1. `ipconfig` أداة في macOS لإعدادات الشبكة. (خلي بالك: ده مش نفس `ipconfig` بتاع Windows، الاسم بس متشابه.)
2. `getifaddr` اختصار get interface address: "هاتلي عنوان الـ IPv4 لكارت شبكة معين".
3. `en0` اسم كارت الشبكة. على أغلب أجهزة الـ MacBook، `en0` هو الواي فاي. لو مطلعش حاجة جرّب `en1`.

Terminal

```
user@MacBook ~ % ipconfig getifaddr en0
192.168.1.15
user@MacBook ~ % route -n get default | grep gateway
    gateway: 192.168.1.1
user@MacBook ~ % curl -4 ifconfig.me
41.33.12.7
```

التلات أوامر

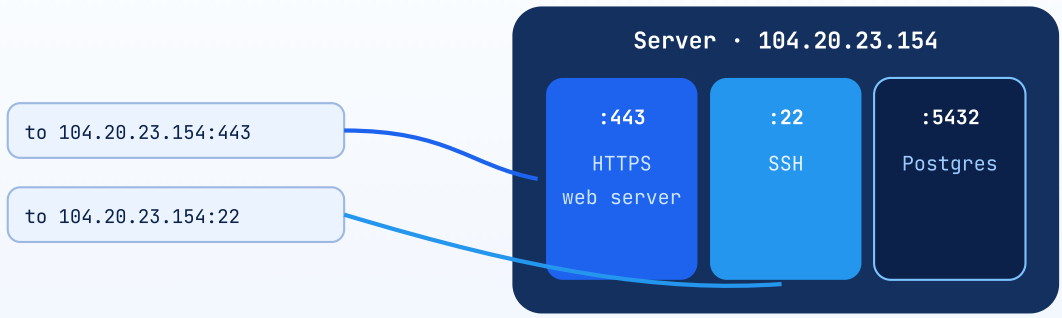
`ipconfig getifaddr en0`	(1) عنوانك الخاص (Private) جوه شبكة البيت.
`route -n get default`	(2) `route get default` = "الـ Packets اللي رايحة لأي حتة برّه بتروح على مين؟". و `-n` = Numeric: اطبع أرقام من غير ما تحاول تجيب أسامي. و `| grep gateway` بياخد سطر الـ `gateway` بس: ده عنوان الراوتر.
`curl -4 ifconfig.me`	(3) بنكلم موقع بيرد بالعنوان اللي شايفك بيه، يعني الـ Public IP بتاع الراوتر. و `-4` = استخدم IPv4 بس.

حاجة مشتركة: الـ Pipe |

الـ `|` في `route -n get default | grep gateway` هو نفس الـ Pipe اللي اتعلمناه في كتاب الـ Terminal: بياخد ناتج الأمر اللي على شماله ويدخّله للأمر اللي على يمينه. وهنستخدمه كتير في الكتاب ده مع `curl` و `dig` و `lsof` عشان نفلتر ناتج طويل.

3.6 الـ Port: رقم الشقة

تمام، الـ Packet وصلت للجهاز الصح. بس الجهاز ده شغال عليه برامج كتير: سيرفر ويب، وسيرفر Database، وبرنامج إيميل. الـ Packet دي لمين فيهم؟ نفس الجواب اللي وصل للعمارة الصح: لازم يبقى مكتوب عليه رقم الشقة. فاتطور حل: كل برنامج بيستنى اتصالات ياخد رقم اسمه **Port**، من `0` لـ `65535`. وكل Packet مكتوب عليها الـ Port اللي رايحة له. والـ IP مع الـ Port بيتكتبوا كده: `104.20.23.154:443`، يعني "العمارة دي، الشقة دي".



شكل 3.2 نفس الـ IP، بس كل Port بيوصّلك لبرنامج مختلف.

وفيه أرقام اتفق عليها العالم كله لخدمات معينة، عشان لما تكتب `https://example.com` المتصفح يعرف لوحده إنه يروح على 443 من غير ما تكتبها. وفيه أرقام اتعودت عليها أدوات البرمجة لسيرفرات التطوير. الجدول ده هترجعله كتير:

الـ Port	مين	ملاحظة
`80`	HTTP	الويب من غير تشفير. المتصفح بيفترضه لو كتبت `http://`
`443`	HTTPS	الويب المتأمّن. الافتراضي لـ `https://`
`53`	DNS	السؤال عن عناوين الأسماء (الفصل 5)
`22`	SSH	اللي بتستخدمه مع `git@github.com` في كتاب Git
`3000`	React / Next.js / Express / Node (CRA)	أشهر Port هتشوفه وانت بتتعلم
`5173`	Vite	لما تعمل مشروع React بـ Vite
`8000` / `8080`	سيرفرات تطوير تانية	`python3 -m http.server` بيفتح 8000
`5432` / `27017` / `6379`	Postgres / MongoDB / Redis	قواعد بيانات هتقابلها مع Node

جدول 3.2 أشهر الـ Ports

يعني إيه Database أصلاً؟

الـ **Database** (قاعدة البيانات) برنامج متخصص في إنه يحفظ البيانات ويرجّعها بسرعة: المستخدمين، والطلبات، والمقالات. بدل ما الكود يحفظ في ملفات بإيده، بيكلّم الـ Database على Port بتاعها ويقولها "احفظ" أو "هات". **Postgres** و MySQL بيحفظوا في جداول (صفوف وأعمدة زي Excel)، و **MongoDB** بيحفظ Documents شبه JSON، و **Redis** بيحفظ في الذاكرة فبيبقى سريع جداً (غالباً للكاش). هتقابل أساميهم كتير في Docker و Kubernetes، ومش محتاج تعرف أكتر من كده دلوقتي.

الـ Ports تحت 1024

الأرقام من 0 لـ 1023 اسمها **Well-known ports**، ومحجوزة للخدمات الرسمية، وأغلب الأنظمة كانت بتطلب صلاحيات Admin عشان برنامج يفتحها. عشان كده أدوات التطوير بتختار أرقام فوق 1024 زي 3000 و 5173. وعلى الناحية التانية، لما المتصفح بتاعك يكلم سيرفر، هو كمان بيفتح Port عشوائي عالي عنده (زي `54821`) عشان الرد يرجعله. الأرقام دي اسمها **Ephemeral ports** يعني "مؤقتة".

3.7 localhost: الجهاز بيكلم نفسه

أول ما تشغّل مشروع وانت بتتعلم، هتلاقي الـ Terminal بيقولك حاجة زي `Local: http://localhost:5173/`. والسؤال: localhost ده فين؟ الإجابة: هو **جهازك نفسه**. فيه عنوان خاص اسمه **Loopback**، أي Packet رايحة له مبتطلعش من الجهاز أصلاً، بتلف وترجع جوه نفس الجهاز. والعنوان ده `127.0.0.1` في IPv4 و `::1` في IPv6، والاسم بتاعه `localhost`. ليه ده مفيد؟ لأنه بيخليك تجرب سيرفر وClient على نفس الجهاز، من غير إنترنت خالص، ومن غير ما حد برّه يشوف شغلك.

وفيه عنوان تالت هتقابله وبيلخبط: `0.0.0.0`. ده مش عنوان تروح له، ده معناه عند السيرفر "اسمع على **كل** العناوين اللي عندي". والفرق ده عملي جداً:

السيرفر سامع على 127.0.0.1

محدش يقدر يوصله غير من نفس الجهاز. موبايلك على نفس الواي فاي مش هيقدر يفتح الموقع. ده الافتراضي في أغلب أدوات التطوير، وده أأمن.

السيرفر سامع على 0.0.0.0

أي جهاز على نفس الشبكة يقدر يوصله بعنوانك الخاص، زي `http://192.168.1.15:5173`. مفيد لما تجرب الموقع على الموبايل. في Vite بتعمله بـ `npm run dev -- --host`.

3.8 معمل: شغّل سيرفر واعرف مين ماسك الـ Port

يلا نعمل سيرفر حقيقي على جهازك في ثانية. الـ Mac فيه Python 3 لو كنت سطبت أدوات المطورين (لو سطبت Homebrew فأنت عندك). اعمل فولدر فيه ملف HTML صغير وشغّل:

تشريح الأمر

```
python3 -m http.server 8000 --bind 127.0.0.1
```

1. `python3` شغّل Python.
2. `-m http.server` = Module: "شغّل الـ Module ده كأنه برنامج". و `http.server` سيرفر ويب بسيط جاي مع Python بيعرض ملفات الفولدر الحالي.
3. `8000` الـ Port اللي يسمع عليه.
4. `--bind 127.0.0.1` اسمع على localhost بس. من غيرها هيسمع على كل العناوين (زي `0.0.0.0`).

افتح `http://localhost:8000` في المتصفح، هتلاقي ملفاتك. وبص على الـ Terminal: هتلاقي سطر لكل طلب، زي `"GET / HTTP/1.1" 200 -` . ده السيرفر بيكتب كل طلب جاله. ودلوقتي افتح Terminal تاني وخلينا نسأل: مين ماسك Port 8000؟

تشريح الأمر

```
lsof -nP -iTCP:8000 -sTCP:LISTEN
```

1. `lsof` — List Open Files. في Unix كل حاجة "ملف"، حتى الاتصالات، فـ lsof بيعرض الاتصالات المفتوحة كمان.
2. `-nP` — حرفين مع بعض: `-n` = متحوّلش الـ IP لأسامي، و`-P` = متحوّلش رقم الـ Port لاسم (يعني اكتب `8000` مش `irdmi`). الاتنين بيخلوا الأمر أسرع وأوضح.
3. `-iTCP:8000` — `-i` = Internet: اعرض اتصالات الشبكة بس، و`TCP:8000` = اللي على TCP وعلى Port 8000.
4. `-sTCP:LISTEN` — `-s` = State: اعرض اللي في حالة `LISTEN` بس، يعني البرنامج اللي "فاتح الباب ومستني"، مش الاتصالات العادية.

Terminal

```
user@MacBook ~ % lsof -nP -iTCP:8000 -sTCP:LISTEN
COMMAND   PID USER   FD   TYPE  DEVICE SIZE/OFF NODE NAME
Python  48213 user    4u  IPv4  0x9a...      0t0  TCP 127.0.0.1:8000 (LISTEN)
```

قراءة ناتج lsof

1 اسم البرنامج اللي ماسك الـ Port.	`COMMAND  Python`
2 رقم العملية. ده اللي هتستخدمه مع `kill` لو عاوز توقفه.	`PID  48213`
3 سامع على IPv4. لو `IPv6` يبقى على `::1` أو `*` .	`TYPE  IPv4`
4 سامع على localhost بس، Port 8000. لو مكتوب `*:8000` يبقى سامع على كل العناوين.	`127.0.0.1:8000 (LISTEN)`

مشكلة 1 **Error: EADDRINUSE :listen :::3000 address already in use**

شغّلت `npm run dev` أو `node server.js` والـ Terminal رمى الخطأ ده. معناه إن فيه برنامج تاني ماسك نفس الـ Port (غالباً نسخة قديمة من نفس المشروع نسيتها مفتوحة في تابة تانية). و`:::3000` = "كل عناوين IPv6، Port 3000".

الحل: اعرف مين ماسكه: `lsof -nP -iTCP:3000 -sTCP:LISTEN` . خد الـ PID ووقّفه: `kill 48213` (ولو معاندش: `kill -9 48213`). أو ببساطة شغّل مشروعك على Port تاني، زي `PORT=3001 npm start` .

اختصار يستاهل تحفظه

بدل ما تحفظ الأمر الطويل، حط Function صغيرة في `~/.zshrc` زي ما اتعلمنا في كتاب الـ Terminal: `whoport() { lsof -nP -iTCP:"$1" -sTCP:LISTEN; }` . وبعد `source ~/.zshrc` تكتب `whoport 3000` وخلاص. و `$1` = أول Argument بتكتبه بعد اسم الـ Function، والـ Quotes حواليه عشان لو القيمة فاضية أو فيها مسافة متبوظش الأمر.

ملخص الفصل

- الـ IP = عنوان الجهاز. IPv4 أربع أرقام من 0 لـ 255 (32 Bit)، وعناوينه خلصت.
- الحل الأول: Private IPs جوه البيت (`192.168.x.x`) وعنوان عام واحد للراوتر، والـ NAT بيترجم بينهم.
- الحل الجذري: IPv6 (128 Bit، Hex، و `::` اختصار للأصفار).
- الـ Port = رقم البرنامج جوه الجهاز. 80 لـ HTTP، 443 لـ HTTPS، 3000 و 5173 للتطوير.
- `localhost` = `127.0.0.1` أو `::1` = جهازك. و `0.0.0.0` عند السيرفر = اسمع على كل العناوين.
- `lsof -nP -iTCP:PORT -sTCP:LISTEN` يعرّفك مين ماسك أي Port.

صح ولا غلط؟

1. `192.168.1.300` عنوان IPv4 صحيح. صح / غلط
2. المواقع اللي بتفتحها بتشوف الـ Private IP بتاعك. صح / غلط
3. سيرفر سامع على `127.0.0.1` تقدر تفتحه من موبايلك على نفس الواي فاي. صح / غلط

أسئلة مراجعة

1. ليه ممكن جهازك وجهاز صاحبك يبقى ليهم نفس الـ IP من غير مشكلة؟
2. إيه الفرق بين `localhost:3000` و `localhost:5173` لو الاتنين على نفس الجهاز؟
3. اكتب العنوان `2001:0db8:0000:0000:0000:0000:0000:0001` بأقصر شكل.
4. اشرح كل حرف في `lsof -nP -iTCP:3000 -sTCP:LISTEN` .

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعرف الـ Private IP والـ Gateway والـ Public IP بتوعك بالتلات أوامر اللي في الفصل.
2. متوسط شغّل `python3 -m http.server 8000` (من غير `--bind`) وافتحه من موبايلك على `http://IP-بتاعك:8000` . وبعدين شغّله بـ `--bind 127.0.0.1` وجرب تاني. إيه اللي حصل وليه؟
3. صعب شغّل سيرفرين على نفس الـ Port في تابتين. اقرا رسالة الخطأ، وبعدين استخدم `lsof` و `kill` عشان تحرر الـ Port.

04

المرحلة 2 · العناوين والتوصيل

الفصل الرابع

الطبقات و TCP و UDP: إزاي الكلام يوصل سليم

الـ IP بيوصّل الـ Packet للعنوان الصح، بس من غير أي ضمان. ممكن تضيع، أو توصل مرتين، أو توصل بالعكس. طب إزاي صفحة فيها آلاف الـ Packets بتوصلك كاملة ومترتبة؟ هنا ييجي دور TCP. وهنفهم كمان ليه فيه حاجة تانية اسمها UDP بتضحي بالضمان ده عن قصد، وإزاي كل الـ Protocols دي راكبة فوق بعض في طبقات.

بعد الفصل ده هتكون عارف

- فكرة الطبقات (Layers) وإزاي كل طبقة بتلف اللي فوقها زي الظرف
- ليه IP لوحده مش كفاية، و TCP بيضيف إيه
- الـ TCP Handshake خطوة خطوة (SYN و SYN-ACK و ACK) ويعني إيه RTT
- UDP إمتى أحسن من TCP، و QUIC
- تستخدم ping و traceroute و nc على الـ Mac وتقرا ناتجهم
- تكلم سيرفر ويب بإيدك بـ nc وتشوف إن HTTP مجرد نص

4.1 المشكلة: الـ IP مبيضمنش حاجة

الـ IP اللي اتكلمنا عنه في الفصل اللي فات شغلته واحدة: يوصّل الـ Packet للعنوان. بس هو بيعملها بطريقة اسمها **Best effort**، يعني "هحاول وماعرفش". كل Router في السكة ممكن يكون زحمة فيرمي Packets. وممكن Packet تمشي في طريق أطول فتوصل بعد اللي بعدها. وممكن Packet تتكرر. تخيل بقى إنك بتحمّل ملف JavaScript مقطّع 50 حتة، وحتة رقم 23 ضاعت، والحتة 40 وصلت قبل 39. الملف هيبوظ، وصفحتك مش هتشتغل.

فاتطور حل: نحط فوق الـ IP طبقة تانية مسؤولة عن **الضمان**: ترقّم الحتت، وتتأكد إن كل حتة وصلت، وتعيد بعت اللي ضاع، وترتّب اللي وصل بالعكس. الطبقة دي اسمها **TCP** (Transmission Control Protocol). وعشان كده الاسم بيتقال دايماً مع بعض: **TCP/IP**. الـ IP بيوصّل، والـ TCP بيضمن.

4.2 الطبقات: ظرف جوه ظرف

الفكرة دي، إن كل طبقة بتحل مشكلة واحدة وتسيب الباقي للي تحتها، هي أهم فكرة في تصميم الشبكات. وأسهل طريقة تتخيلها بيها: **ظرف جوه ظرف**. المتصفح بيكتب رسالة HTTP (الجواب نفسه)، وبيديها لـ TCP اللي بيحطها في ظرف مكتوب عليه رقم الـ Port ورقم الحتة، وبعدين IP بيحط الظرف ده في ظرف أكبر مكتوب عليه عناوين الـ IP، وبعدين كارت الواي فاي بيحط الكل في ظرف أخير يناسب الموجات اللاسلكية. وعند السيرفر بيحصل العكس: كل طبقة تفتح الظرف بتاعها وتدي اللي جواه للي فوقها.



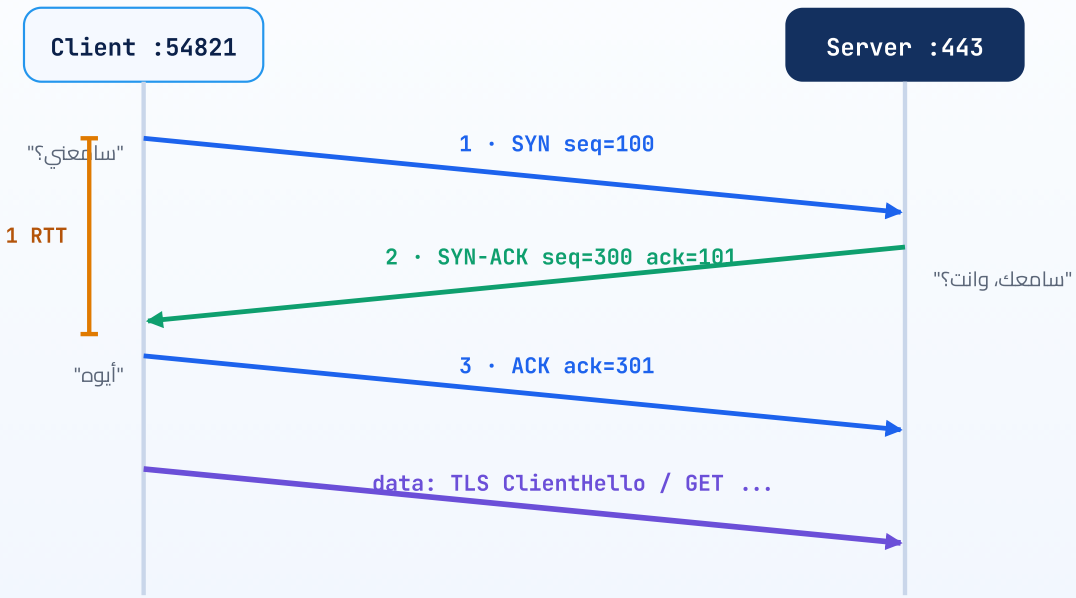
شكل 4.1 الطبقات الأربعة في نموذج TCP/IP، وإزاي الرسالة بتتلف ظرف جوه ظرف (Encapsulation).

الجميل في الفكرة إن كل طبقة **مش مهتمة** باللي فوقها ولا اللي تحتها. الـ HTTP مش فارق معاه انت على واي فاي ولا كابل ولا G5. والـ Router في النص مش بيفتح غير ظرف الـ IP، ومش فارق معاه جواه HTTP ولا لعبة ولا إيميل. وده اللي خلّى الإنترنت يكبر بالشكل ده: تقدر تخترع تطبيق جديد فوق، أو نوع كابل جديد تحت، من غير ما تغيّر حاجة في النص.

> **هتسمع عن OSI و 7 طبقات**
>
> في الكتب والمقابلات هتسمع عن نموذج اسمه **OSI** فيه 7 طبقات، وكلام زي "Layer 7" و"Layer 4". ده نموذج نظري أقدم وأكتر تفصيلاً، والنموذج اللي فوق (4 طبقات) هو اللي الإنترنت ماشي بيه فعلاً. اللي محتاج تفتكره: **Layer 7** = التطبيق (HTTP)، و**Layer 4** = النقل (TCP/UDP)، و**Layer 3** = الشبكة (IP). فلو حد قال "Load balancer بيشتغل على Layer 7" يبقى قصده إنه بيفهم HTTP ويقدر يقرا الـ URL والـ Headers.

4.3 الـ TCP Handshake: "سامعني؟"

قبل ما TCP يبعت أي بيانات، لازم الطرفين يتفقوا إن الاتصال موجود، ويتفقوا على أرقام يبدأوا يعدّوا منها الحتت. ده بيحصل في تلات رسايل، والعملية اسمها **Three-way handshake**، زي مكالمة تليفون في شبكة ضعيفة: "ألو، سامعني؟" ... "أيوه سامعك، وانت سامعني؟" ... "أيوه، يلا نتكلم".



شكل 4.2 التلات رسايل قبل أي بيانات. أول بيانات حقيقية بتتبعت بعد رحلة رايح جاي كاملة (RTT).

تشريح الـ Handshake

```
1   2          3                4          5
SYN seq=100 → SYN-ACK seq=300 ack=101 → ACK ack=301
```

1. `SYN` — Synchronize: "عاوز أفتح اتصال، وأنا هبدأ أعدّ من 100". الرقم ده عشوائي عشان محدش يخمّنه.
2. `seq=100` — Sequence number: رقم أول Byte هيبعته الـ Client. كل Byte بعد كده ليه رقم، وكده السيرفر يعرف يرتّب ويعرف لو حاجة ناقصة.
3. `SYN-ACK` — السيرفر بيقول حاجتين في رسالة واحدة: "أنا كمان عاوز أفتح (SYN) وهبدأ من 300"، و"استلمت رسالتك (ACK)".
4. `ack=101` — Acknowledgment: "استلمت لحد 100، ومستني 101". الـ ACK دايماً رقم **الجاي** المتوقع.
5. `ACK ack=301` — الـ Client: "استلمت بداية عدّك، ومستني 301". وكده الطرفين متفقين، والاتصال مفتوح.

وبعد كده، طول الكلام، كل ما طرف يستلم حتت يرد بـ ACK. ولو الطرف التاني ماستلمش ACK لحتة معينة في وقت معين، يفترض إنها ضاعت **ويبعتها تاني** (اسمها Retransmission). وكمان TCP بيراقب الزحمة: لو حس إن حتت كتير بتضيع، يبطّأ البعت عشان ميخنقش الشبكة (Congestion control). وفي الآخر، لما الكلام يخلص، كل طرف بيبعت `FIN` ("خلصت") والتاني يرد بـ ACK، وده قفل الاتصال بأدب.

الـ RTT: أهم رقم في سرعة الويب

RTT = Round-Trip Time، الوقت اللي رسالة بتاخده رايح جاي. ولاحظ في الرسمة إن أول بيانات مبتتبعتش غير بعد RTT كامل للـ Handshake. ولو الموقع HTTPS، الـ TLS محتاج RTT تاني (أو أكتر في النسخ القديمة). يعني لو السيرفر في أمريكا والـ RTT من مصر 150 ملي ثانية، انت دفعت 300 ملي ثانية قبل ما تبعت حرف واحد من الطلب. عشان كده المتصفحات بتحاول **تعيد استخدام الاتصال المفتوح** بدل ما تفتح جديد كل مرة، وعشان كده اتعمل HTTP/3 (الفصل 11).

4.4 UDP: ابعت وخلاص

طب كل الضمان ده حلو، بس له تمن: وقت. تخيل مكالمة فيديو. لو حتة صغيرة من الصوت ضاعت، هل عاوز البرنامج يوقف ويستنى يعيد بعتها؟ لأ طبعاً، لما توصل هتبقى قديمة ومالهاش لازمة، وانت عاوز الكلام اللي بيتقال **دلوقتي**. نفس الكلام في الألعاب الأونلاين: مكان اللاعب من نص ثانية مش مهم، المهم مكانه دلوقتي.

عشان كده فيه بروتوكول نقل تاني اسمه **UDP** (User Datagram Protocol)، فكرته عكس TCP بالظبط: مفيش Handshake، ومفيش ترقيم، ومفيش إعادة، ومفيش ضمان. بتحط الرسالة في ظرف عليه الـ Port وترميها. سريع وخفيف، وأي ضمان محتاجه التطبيق يعمله بنفسه. والـ DNS نفسه بيستخدم UDP في أغلب الأحوال، لأن السؤال والإجابة صغيرين، ولو السؤال ضاع ابعته تاني وخلاص، أرخص من Handshake كامل.

	TCP	UDP
اتصال قبل البيانات	أيوه (Handshake)	لأ
الحتت بتوصل كلها ومترتبة	مضمون	مش مضمون
السرعة والتأخير	أبطأ شوية	أسرع وأخف
بيستخدمه	HTTP/1.1 و HTTP/2، والإيميل، و SSH، و Git	DNS، ومكالمات الفيديو، والألعاب، و**QUIC** (اللي تحت HTTP/3)

جدول 4.1 TCP و UDP جنب بعض

لو حابب تتعمق: QUIC، أحسن ما في الاتنين

تقدر تعدّيه في أول قراية

المهندسين في Google سألوا: ليه منعملش بروتوكول فيه ضمان TCP بس من غير بطئه؟ فعملوا **QUIC**، وبقى Standard رسمي (`RFC 9000`). QUIC شغال **فوق UDP**، وبيعمل الترقيم والإعادة بنفسه، وبيدمج الـ Handshake بتاع الاتصال والتشفير في رحلة واحدة. وده الأساس اللي HTTP/3 راكب عليه. هنرجعله في الفصل 11. وأحلى حاجة إنك هتشوف في DevTools عمود Protocol فيه `h3` لمواقع زي Google و YouTube.

4.5 ping: "انت عايش؟"

كفاية نظري، يلا نلعب بالشبكة. أبسط أداة هي `ping`، وفكرتها زي السونار في الغواصات: بتبعت رسالة صغيرة لجهاز وتستنى ترجع، وتقيس الوقت.

تشريح الأمر

```
ping -c 4 example.com
```

1. `ping` — ابعت رسايل ICMP Echo Request واستنى الرد (Echo Reply). ICMP بروتوكول صغير جنب IP للرسايل الإدارية والأخطاء.
2. `-c 4` — Count = ابعت 4 بس واقف. من غيرها `ping` على الـ Mac هيفضل شغال لحد ما تدوس `Ctrl+C`.
3. `example.com` — الهدف. ممكن تكتب IP على طول زي `1.1.1.1`.

Terminal

```
user@MacBook ~ % ping -c 4 example.com
PING example.com (104.20.23.154): 56 data bytes
64 bytes from 104.20.23.154: icmp_seq=0 ttl=57 time=21.4 ms
64 bytes from 104.20.23.154: icmp_seq=1 ttl=57 time=19.8 ms
64 bytes from 104.20.23.154: icmp_seq=2 ttl=57 time=20.3 ms
64 bytes from 104.20.23.154: icmp_seq=3 ttl=57 time=22.0 ms

--- example.com ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 19.8/20.9/22.0/0.8 ms
```

قراية ناتج ping

1 ping عمل DNS الأول وطلّع الـ IP، وكده تعرف إن الـ DNS شغال.	(104.20.23.154)
2 رقم الرسالة. لو لقيت رقم ناقص (0 و 1 و 3) يبقى 2 ضاعت.	icmp_seq=0
3 عدّاد بيقل واحد مع كل Router في السكة (هنشرحه مع traceroute تحت).	ttl=57
4 ده الـ RTT اللي اتكلمنا عنه، رايح جاي.	time=21.4 ms
5 نسبة الرسايل اللي ضاعت. أي رقم فوق 1-2% على اتصال ثابت يبقى فيه مشكلة.	0.0% packet loss
6 أقل وأوسط وأعلى RTT، والـ stddev بيقيس قد إيه الأرقام متذبذبة (الـ Jitter).	min/avg/max/stddev

> ⚠ **خلي بالك: ping فاشل مش معناه إن الموقع واقع**
>
> سيرفرات وشركات كتير بتقفل ICMP عن قصد لأسباب أمنية. فممكن `ping` يطلعلك `Request timeout` والموقع نفسه شغال عادي في المتصفح. ping مفيد يقولك "الشبكة عندي شغالة؟" (جرّب `ping -c 3 1.1.1.1`)، بس مش حكم نهائي على موقع معين. الحكم النهائي على موقع ويب هو `curl`.

4.6 traceroute: السكة فيها كام محطة؟

عمرك سألت نفسك الـ Packet بتاعتك بتعدّي على كام Router قبل ما توصل لـ Google؟ `traceroute` بيوريك السكة كلها، وفكرته ذكية جداً. كل Packet فيها عدّاد اسمه **TTL** (Time To Live)، وكل Router بيعدّيها بينقّص منه واحد، ولما يوصل صفر الـ Router يرميها ويبعت للمرسل رسالة "الـ Packet دي ماتت عندي". العداد ده اتعمل في الأصل عشان لو حصل لف في الشبكة، الـ Packet متفضلش تلف للأبد. traceroute بيستغله: يبعت Packet بـ TTL = 1، فأول Router يرد، وبعدين TTL = 2 فتاني Router يرد، وهكذا لحد ما يوصل.

تشريح الأمر

```
traceroute -n -m 15 example.com
```

1. `traceroute` اعرض كل Router في السكة لحد الهدف.
2. `-n` زي `lsof` بالظبط: اطبع الأرقام ومتدورش على أسامي. أسرع بكتير.
3. `-m 15` = Max hops: متعدّيش 15 محطة (الافتراضي 64).
4. `example.com` الهدف.

Terminal

```
user@MacBook ~ % traceroute -n -m 15 example.com
traceroute to example.com (104.20.23.154), 15 hops max, 40 byte packets
 1  192.168.1.1  3.1 ms  2.4 ms  2.2 ms
 2  10.37.0.1  8.7 ms  9.1 ms  8.3 ms
 3  * * *
 4  196.204.12.33  12.9 ms  13.4 ms  12.6 ms
 5  104.20.23.154  20.8 ms  21.1 ms  20.6 ms
```

قراية traceroute

1 أول محطة دايماً الراوتر بتاعك (الـ Gateway اللي عرفناه في الفصل 3).	`1  192.168.1.1`
2 شبكة شركة الإنترنت. لاحظ إنه Private IP، عادي جوه شبكات الـ ISP.	`2  10.37.0.1`
3 traceroute بيجرب كل محطة 3 مرات، وده الـ RTT لكل مرة.	`3.1 ms  2.4 ms  2.2 ms`
4 Router مارضيش يرد (بيتجاهل الرسايل دي). عادي جداً ومش معناه إن فيه مشكلة طالما اللي بعده كمّل.	`3  * * *`

الأرقام والمحطات توضيحية، والسكة عندك هتبقى مختلفة حسب شركة الإنترنت.

4.7 nc: سكينة الشبكة السويسري

`nc` (اختصار netcat) أداة بسيطة جداً وقوية جداً: بتفتح اتصال TCP (أو UDP) لأي IP وأي Port، وتبعت فيه اللي تكتبه، وتطبع اللي يرجع. ومن غير أي فهم لـ HTTP أو أي Protocol. وعشان كده هي أحسن أداة تثبتلك إن كل الكلام اللي في الكتاب ده مجرد **نص ماشي على TCP**. أول استخدام: هل الـ Port ده مفتوح؟

تشريح الأمر

```
nc -zv example.com 443
```

1. `nc` netcat. موجود على الـ Mac جاهز.

2	`-z`	Zero I/O: "افتح الاتصال بس (اعمل الـ Handshake) ومتبعتش بيانات". يعني اختبار سريع للـ Port.
3	`-v`	Verbose: قولي النتيجة بالكلام. من غيرها مش هيطبع حاجة.
4	`example.com 443`	الجهاز والـ Port. لاحظ إنهم منفصلين بمسافة هنا، مش `:`.

Terminal

```
user@MacBook ~ % nc -zv example.com 443
Connection to example.com port 443 [tcp/https] succeeded!
user@MacBook ~ % nc -zv localhost 3000
nc: connectx to localhost port 3000 (tcp) failed: Connection refused
```

`succeeded` = الـ Handshake نجح، فيه برنامج سامع. و`Connection refused` = الجهاز موجود بس مفيش حد سامع على الـ Port ده (يعني سيرفرك مش شغال). ولو فضل مستني ومردش خالص، يبقى غالباً فيه Firewall بيرمي الطلب في صمت. الفرق ده بين **refused** و**timeout** هيفرق معاك جداً في الـ Debugging بعدين.

كلّم سيرفر ويب بإيدك

ودلوقتي الحتة الممتعة. هنكتب طلب HTTP بإيدنا حرف حرف، ونبعته بـ `nc` على Port 80 (HTTP من غير تشفير، عشان nc ميعرفش TLS):

تشريح الأمر

```
printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n' | nc example.com 80
```

1	`printf`	اطبع النص ده بالظبط. بنستخدمه بدل `echo` عشان بيفهم `\r\n` على كل الأنظمة.
2	`'...'`	الـ Single quotes بتخلّي الـ zsh ميلمسش أي حاجة جوه النص.
3	`GET / HTTP/1.1`	سطر الطلب: هات الصفحة الرئيسية `/` بـ HTTP/1.1 (الفصل 7).
4	`\r\n`	نهاية السطر في HTTP: `\r` = Carriage Return و`\n` = New Line. HTTP/1.1 اتفق على الاتنين مع بعض (زي الآلة الكاتبة: رجّع للأول وانزل سطر).
5	`Host: example.com`	Header إجباري في HTTP/1.1: انت عاوز أنهي موقع (لأن سيرفر واحد ممكن يشيل مواقع كتير).
6	`Connection: close`	"اقفل الاتصال بعد ما ترد"، عشان nc يخلص ويرجعلك الـ Prompt.
7	`\r\n\r\n`	سطر فاضي = "خلصت الـ Headers". من غيره السيرفر هيفضل مستني.
8	`| nc example.com 80`	ادّي النص ده لـ nc يبعته على Port 80، ويطبع اللي يرجع.

```
Terminal
user@MacBook ~ % printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n' |
nc example.com 80
HTTP/1.1 200 OK
Date: Thu, 22 Oct 2026 20:24:10 GMT
Content-Type: text/html; charset=utf-8
Connection: close
Server: cloudflare

<!doctype html><html lang="en"><head><title>Example Domain</title>...
```

لو رجعلك `301 Moved Permanently` بدل `200 OK`، يبقى السيرفر بيقولك "روح على HTTPS"، وده نفسه درس هنشرحه في الفصل 8.

شفت؟ مفيش سحر. المتصفح، و`curl`، و`fetch` في JavaScript، كلهم في الآخر بيبعتوا نص زي ده بالظبط على اتصال TCP، وبيستلموا نص زي ده. كل الفرق إنهم بيعملوا ده بشياكة، وبيضيفوا TLS للتشفير. ولما تفهم ده، كل حاجة في الفصول الجاية هتبقى منطقية.

مثال عملي: شات بين تابتين

افتح تابتين Terminal. في الأولى: `nc -l 1234` (`-l` = Listen: "اقعد استنى على Port 1234"، يعني انت بقيت Server). وفي التانية: `nc localhost 1234` (انت الـ Client). اكتب أي حاجة في أي تابة ودوس Enter، هتظهر في التانية. انت كده عملت Client و Server على TCP بإيدك. اقفل بـ `Ctrl+C`.

ملخص الفصل

- الشبكة طبقات: Link ثم IP ثم TCP/UDP ثم التطبيق (HTTP)، وكل طبقة بتلف اللي فوقها.
- IP بيوصّل من غير ضمان، و TCP بيضيف الترقيم والتأكيد والإعادة والترتيب.
- الـ Handshake: SYN ثم SYN-ACK ثم ACK، وبيكلّف RTT كامل قبل أول بيانات.
- UDP بيبعت من غير ضمان، وده أحسن للصوت والألعاب والـ DNS، و QUIC (تحت HTTP/3) راكب عليه.
- `ping` للـ RTT والـ Loss، و`traceroute` للسكة (بيستغل الـ TTL)، و`nc` لفتح اتصال TCP بإيدك.
- HTTP مجرد نص ماشي على TCP، وده اللي شفناه بـ `nc`.

صح ولا غلط؟

1. الـ Router في النص بيقرا رسالة الـ HTTP عشان يعرف يوديها فين. صح / غلط
2. UDP بيضمن إن الرسايل توصل بالترتيب. صح / غلط
3. لو `ping` فشل يبقى الموقع أكيد واقع. صح / غلط

أسئلة مراجعة

1. إيه التلات مشاكل اللي ممكن تحصل للـ Packets، و TCP بيحل كل واحدة إزاي؟
2. في الـ Handshake، ليه السيرفر بيرد بـ `ack=101` لما الـ Client بعت `seq=100`؟
3. ليه مكالمات الفيديو بتفضّل UDP؟
4. إيه الفرق بين `Connection refused` وإن `nc` يفضل مستني من غير رد؟
5. اشرح `\r\n\r\n` في آخر طلب الـ HTTP اللي بعتناه بـ nc.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل `ping -c 5` لـ `1.1.1.1` و`google.com` وموقع بعيد زي `abc.net.au`. قارن الـ avg، وفكّر ليه مختلفين.
2. متوسط اعمل `traceroute -n` لـ `github.com`. عدّ المحطات، وشوف أنهي محطة الوقت قفز فيها جامد (غالباً لما الطلب عدّى البحر).
3. صعب شغّل `python3 -m http.server 8000`، وبعدين كلّمه بـ nc بإيدك: `printf 'GET / HTTP/1.1\r\nHost: localhost\r\n\r\n' | nc localhost 8000`. بص على الرد وعلى سطر اللوج في تابة السيرفر.

05

المرحلة 2 · العناوين والتوصيل

الفصل الخامس

DNS: دليل التليفونات بتاع الإنترنت

عرفنا إن الأجهزة بتكلم بعض بأرقام، والبشر بيحفظوا أسامي. طب مين بيترجم example.com لـ 104.20.23.154؟ وإزاي نظام واحد بيجاوب مليارات الأسئلة في اليوم من غير ما يقع؟ الفصل ده عن DNS: حكايته، والرحلة اللي السؤال بيعملها، وأنواع السجلات، وإزاي تسأله بنفسك من الـ Terminal.

بعد الفصل ده هتكون عارف

- ليه اتعمل DNS والمشكلة اللي قبله (ملف HOSTS.TXT)
- تشريح أي دومين: الـ Root والـ TLD والـ Subdomain
- رحلة السؤال خطوة خطوة من كاش المتصفح لحد الـ Authoritative server
- أنواع السجلات A و AAAA و CNAME و MX و TXT و NS، والـ TTL
- تستخدم dig و nslookup و dscacheutil وتقرا ناتجهم
- تعدّل /etc/hosts وتمسح كاش الـ DNS على الـ Mac، وتفهم أخطاء DNS في Chrome

5.1 قبل DNS: ملف واحد للدنيا كلها

في أيام ARPANET، كانت الأجهزة كام مية بس. والحل اللي الناس عملته عشان متحفظش أرقام كان بسيط جداً: ملف نصي واحد اسمه `HOSTS.TXT`، فيه كل اسم وجنبه رقمه، ومحفوظ على جهاز واحد في معهد اسمه SRI في كاليفورنيا. أي حد عاوز يعرف الأسامي، ينزّل الملف ده على جهازه كل فترة. ولو عاوز تضيف جهاز جديد، تبعت إيميل للناس اللي في SRI يضيفوه بإيدهم.

وهنا كانت المشكلة. الشبكة كبرت لآلاف الأجهزة، والملف كبر، والكل بينزّله كل شوية فالجهاز اللي عليه بقى مخنوق. ومفيش حاجة تمنع اتنين يختاروا نفس الاسم. وأي تعديل بياخد أيام لحد ما يوصل للكل. نظام مركزي واحد ببساطة مكانش هيستحمل. فاتطور حل: سنة 1983 مهندس اسمه **Paul Mockapetris** صمم **DNS** (Domain Name System)، وفكرته إن مفيش ملف واحد خالص. الأسامي بقت **شجرة**، وكل فرع في الشجرة مسؤول عنه جهة مختلفة، وكل جهة بتدير الأسامي اللي تحتها بس. وبالمناسبة، الملف القديم ده لسه عايش على جهازك لحد النهارده باسم `/etc/hosts`، وهنلعب بيه في آخر الفصل.

5.2 تشريح الدومين: بيتقرا من اليمين

الدومين بيتقرا من اليمين للشمال، من العام للخاص، زي العنوان البريدي بالعكس: البلد، وبعدين المدينة، وبعدين الشارع.

تشريح الدومين

```
api.shop.example.com.
5 1 4   3     2
```

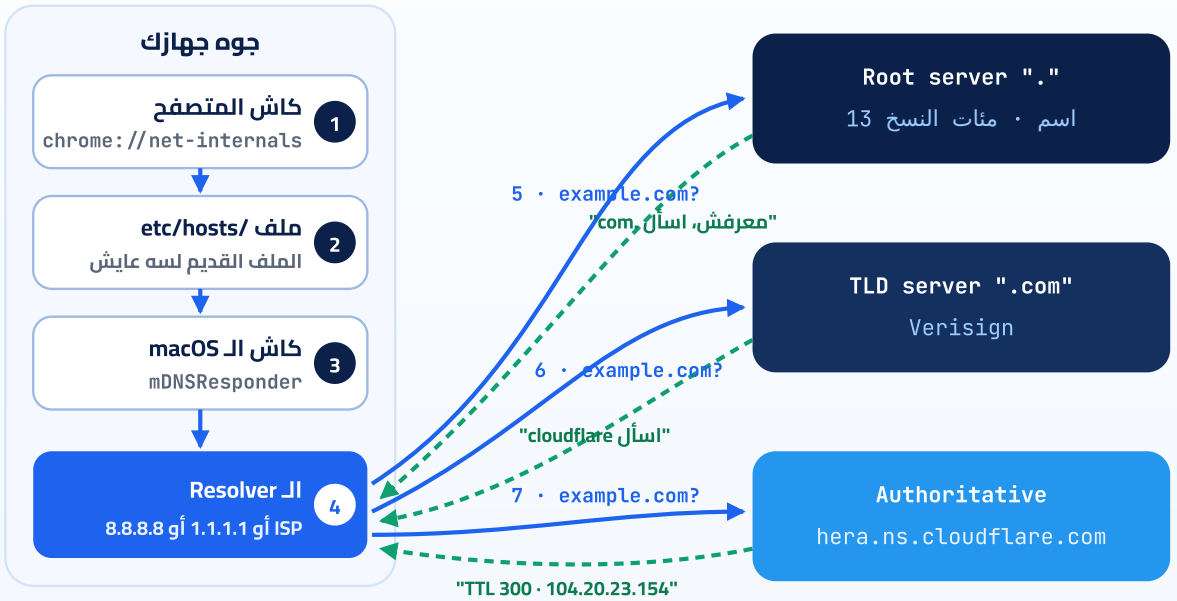
1. `.` النقطة الأخيرة اللي محدش بيكتبها: دي الـ **Root**، جذر الشجرة. كل دومين في الدنيا بيخلص بيها في الحقيقة، والأدوات بتضيفها لوحدها.
2. `com` الـ **TLD** (Top-Level Domain). فيه عامة زي `com` و `org` و `dev`، وفيه بلاد زي `eg` لمصر و `uk` .

3. `example` الاسم اللي انت بتشتريه (اسمه Second-level domain). `example.com` كله اسمه الـ **Apex** أو الـ Root domain بتاعك.
4. `shop` Subdomain: انت كصاحب `example.com` تعمل اللي انت عاوزه تحته ببلاش، زي `shop` و `blog` و `www`.
5. `api` Subdomain جوه Subdomain. وكلمة `www` نفسها مجرد Subdomain عادي، مش حاجة سحرية.

> **مين بيبيعلك الدومين؟**
>
> لما تشتري دومين من شركة زي Namecheap أو Cloudflare أو GoDaddy (اسمها **Registrar**)، انت في الحقيقة بتأجر الاسم لمدة (سنة غالباً)، والشركة بتسجّله عند الجهة اللي ماسكة الـ TLD (اسمها **Registry**، زي Verisign لـ `.com`). وبتقولهم "الأسامي اللي تحت دوميني، اسألوا عليها السيرفرات دي"، ودي اسمها **Nameservers**.

5.3 رحلة السؤال: example.com فين؟

خلينا نمشي مع المتصفح وهو بيدوّر على `example.com` لأول مرة خالص. أهم حاجة تلاحظها إن كل خطوة فيها **كاش**: قبل ما حد يسأل اللي بعده، بيبص الأول "هل أنا سألت السؤال ده قريب؟". ولو أيوه، الرحلة بتقف عنده. ومن غير الكاش ده، الـ Root servers كانت هتنهار من أول يوم.



8 · الرجوع: الـ Resolver بيرجّع الإجابة لجهازك، وكل خطوة في السكة بتحفظها في الكاش لمدة الـ TTL

المرة الجاية (لحد ما الـ TTL يخلص) الرحلة كلها بتقف عند خطوة 1 أو 3 أو 4، ومبتوصلش للـ Root خالص.

الخطوات 5 و 6 و 7 الـ Resolver هو اللي بيعملها نيابة عنك (اسمه Recursive resolver).

شكل 5.1 رحلة السؤال عن example.com: من الكاش اللي جوه جهازك لحد السيرفر اللي عنده الإجابة الأكيدة.

نحكيها بالكلام. أول حاجة المتصفح بيبص في **الكاش بتاعه** (خطوة 1): Chrome بيحفظ إجابات DNS لوقت قصير. مفيش؟ بيسأل نظام التشغيل، والـ macOS بيبص الأول في **ملف** `/etc/hosts` (خطوة 2)، وبعدين في **الكاش بتاعه** (خطوة 3) اللي ماسكه برنامج

في الخلفية اسمه `mDNSResponder` . لسه مفيش؟ هنا جهازك بيبعت السؤال لبرّه، لسيرفر اسمه **Recursive Resolver** (خطوة 4). ده غالباً بتاع شركة الإنترنت، أو واحد عام انت اخترته زي `1.1.1.1` بتاع Cloudflare أو `8.8.8.8` بتاع Google. والـ Resolver ده "الموظف" اللي هيلف المصالح بدالك.

لو الـ Resolver نفسه معندوش الإجابة في الكاش، بيبدأ من فوق خالص. يسأل **Root server** (خطوة 5): "فين `example.com` ؟". الـ Root معندوش فكرة عن `example.com` ، بس عارف مين مسؤول عن كل TLD، فيرد: "معرفش، بس دول سيرفرات `.com` ، اسألهم". يروح يسأل **سيرفر الـ** `.com` (خطوة 6)، وده برضه معندوش الرقم، بس عارف مين مسؤول عن `example.com` تحديداً: "اسأل `hera.ns.cloudflare.com` ". وأخيراً يسأل الـ **Authoritative server** (خطوة 7)، وده اللي صاحب الدومين حاطط عنده السجلات، فيرد بالإجابة الأكيدة: " `104.20.23.154` ، واحفظها 300 ثانية". والـ Resolver يرجّعها لجهازك (خطوة 8)، وكل واحد في السكة يحفظها.

لو حابب تتعمق: 13 Root server بس؟ تقدر تعدّيه في أول قراية

هتقرا إن فيه "13 Root server" في الدنيا. ده نص الحقيقة: فيه 13 **اسم** (من `a.root-servers.net` لـ `m.root-servers.net`)، بس كل اسم وراه مئات الأجهزة في بلاد كتير بتشارك نفس الـ IP، بتقنية اسمها **Anycast**: الـ Packet بتاعتك بتروح لأقرب نسخة. نفس التقنية اللي `1.1.1.1` والـ CDNs بيستخدموها (الفصل 15). ولو عاوز تشوف الرحلة بنفسك من الـ Root لحد الآخر: `dig +trace example.com` .

5.4 أنواع السجلات: مش أرقام بس

الـ DNS مش بيخزن IP بس. كل دومين ليه **سجلات** (Records) من أنواع مختلفة، كل نوع بيجاوب سؤال مختلف. لو انت عاوز تعرف "أبعت الإيميل لـ `sara@example.com` فين؟"، ده سؤال تاني غير "الموقع على أنهي IP؟". وأهم الأنواع اللي هتقابلها:

النوع	بيجاوب على	مثال
`A`	عنوان IPv4 للاسم ده	`example.com → 104.20.23.154`
`AAAA`	عنوان IPv6 (أربع A لأنه 4 أضعاف الحجم)	`→ 2606:4700:10::6814:179a`
`CNAME`	"الاسم ده اسم تاني لـ..." (Alias)	`www.site.com → site.netlify.app`
`MX`	سيرفر الإيميل للدومين	`10 mail.example.com`
`TXT`	أي نص، غالباً للتحقق من الملكية وإعدادات الإيميل	`"v=spf1 include:_spf.google.com ~all"`
`NS`	مين الـ Authoritative servers للدومين	`hera.ns.cloudflare.com`

جدول 5.1 أشهر أنواع سجلات DNS

مثال عملي: هتستخدم ده إمتى؟

لما تنشر مشروع React على Netlify أو Vercel وتشتري دومين، الموقع هيقولك "اعمل `CNAME` لـ `www` يشاور على `your-site.netlify.app` "، أو "اعمل `A` record للـ Apex يشاور على IP كذا". ولما تربط الدومين بـ Google Workspace عشان الإيميل، هيطلب منك `MX` و `TXT` . يعني الجدول ده هيبقى شغلك فعلاً.

الـ TTL وخرافة "الـ DNS لسه بيتنشر"

كل سجل معاه رقم اسمه **TTL** بالثواني: "احفظ الإجابة دي في الكاش الوقت ده بس". TTL = 300 يعني 5 دقايق. ولو غيّرت IP موقعك، الناس اللي عندهم الإجابة القديمة في الكاش هيفضلوا يروحوا للقديم لحد ما الـ TTL بتاعهم يخلص. وده اللي الناس بتسميه "DNS propagation" وتقول "استنى 48 ساعة". في الحقيقة مفيش حاجة "بتتنشر"، فيه كاشات بتخلص في أوقات مختلفة. وعشان كده الشركات بتقلل الـ TTL قبل أي نقل بيوم، عشان الكاشات تخلص بسرعة.

> **حاجة مشتركة: TTL ليها معنيين**
>
> خلي بالك إن كلمة **TTL** قابلتنا مرتين بمعنيين مختلفين: في الفصل 4 كانت عدّاد المحطات جوه الـ Packet (اللي traceroute بيستغله)، وهنا مدة الكاش بالثواني لسجل DNS. الاسم واحد (Time To Live: "يعيش قد إيه") والفكرة قريبة، بس السياق مختلف. وهتقابل نفس الفكرة تالت مرة في كاش HTTP بس باسم `max-age` (الفصل 15).

5.5 dig: اسأل الـ DNS بنفسك

الأداة الأشهر لسؤال DNS هي `dig` (Domain Information Groper)، وجاية جاهزة على الـ Mac. أبسط استخدام:

Terminal

```
user@MacBook ~ % dig example.com

; <<>> DiG 9.10.6 <<>> example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41267
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            300     IN      A       104.20.23.154
example.com.            300     IN      A       172.66.147.243

;; Query time: 18 msec
;; SERVER: 192.168.1.1#53(192.168.1.1)
;; WHEN: Thu Oct 22 23:30:12 EEST 2026
;; MSG SIZE  rcvd: 72
```

قراية ناتج dig

1 السؤال اتجاوب. لو `NXDOMAIN` = الاسم ده مش موجود أصلاً. لو `SERVFAIL` = الـ Resolver فشل يجيب إجابة.	`status: NOERROR`
2 `qr` = ده رد، و `rd` = طلبنا بحث كامل (Recursion desired)، و `ra` = الـ Resolver بيعمل ده (available).	`flags: qr rd ra`

3 السؤال: سجل `A` لـ `example.com.` (لاحظ نقطة الـ Root في الآخر). و `IN` = Internet.	`;example.com.  IN  A`
4 الإجابة: الاسم، والـ TTL المتبقي (300 ثانية)، والنوع، والقيمة. وفيه عنوانين، والمتصفح ممكن يستخدم أي واحد.	`example.com. 300 IN A 104.20.23.154`
5 الوقت اللي الإجابة خدته. جرّب تاني على طول، هتلاقيه قرّب من 0 لأنه بقى في الكاش.	`Query time: 18 msec`
6 مين جاوب: هنا الراوتر بتاعك على Port `53` (الـ `#` بتفصل الـ IP عن الـ Port في ناتج dig). والراوتر بيوصّل السؤال للـ Resolver بتاع شركة الإنترنت.	`SERVER: 192.168.1.1#53`

الـ TTL والأرقام عندك هتختلف. والسطور اللي بتبدأ بـ `;` في ناتج dig مجرد تعليقات وعناوين للأقسام.

الناتج الكامل ده مفيد لما تحقق في مشكلة، بس أغلب الوقت انت عاوز الإجابة وبس. ودي أهم أشكال dig:

تشريح الأمر

```
1   2      3     4           5
dig +short AAAA example.com @1.1.1.1
```

1. `dig` اسأل الـ DNS.
2. `+short` في dig، الـ Options بتبدأ بـ `+` مش `-` (استثناء غريب). `+short` = اطبع الإجابة بس من غير كل التفاصيل.
3. `AAAA` نوع السجل. من غيره dig بيسأل عن `A`. جرّب `MX` و `TXT` و `NS` و `CNAME`.
4. `example.com` الاسم.
5. `@1.1.1.1` الـ `@` = "اسأل السيرفر ده بالذات" بدل الـ Resolver الافتراضي. مفيد جداً تقارن: هل الإجابة عند Cloudflare زي اللي عند شركة الإنترنت؟

Terminal

```
user@MacBook ~ % dig +short example.com
104.20.23.154
172.66.147.243
user@MacBook ~ % dig +short NS example.com
hera.ns.cloudflare.com.
elliott.ns.cloudflare.com.
user@MacBook ~ % dig +short www.github.com
github.com.
140.82.121.4
```

لاحظ آخر مثال: `www.github.com` رجّع سطرين. الأول `github.com.` ده CNAME ("www اسم تاني لـ github.com")، والتاني الـ IP بتاع `github.com`. يعني dig مشي الـ CNAME لوحده لحد ما وصل لرقم.

nslookup: الأداة الأقدم

هتلاقي في شروحات كتير `nslookup` بدل dig، خصوصاً اللي جاية من Windows لأنها موجودة هناك. بتعمل نفس الشغلانة بناتج أبسط:

```
Terminal

user@MacBook ~ % nslookup example.com
Server:         192.168.1.1
Address:        192.168.1.1#53

Non-authoritative answer:
Name:   example.com
Address: 104.20.23.154
Name:   example.com
Address: 172.66.147.243
```

`Non-authoritative answer` معناها "الإجابة دي جاية من كاش الـ Resolver، مش من الـ Authoritative server نفسه". عادي جداً وده اللي بيحصل أغلب الوقت. وتقدر تحدد السيرفر برضه: `nslookup example.com 1.1.1.1` (من غير `@` هنا).

> **خلي بالك: dig و nslookup مش بيبصوا في /etc/hosts**
>
> دي نقطة بتضيّع وقت ناس كتير. `dig` و`nslookup` بيكلموا سيرفر DNS على طول، ومش بيعدّوا على `/etc/hosts` ولا على كاش الـ macOS. أما المتصفح و`curl` و`ping` فبيستخدموا نظام التشغيل اللي بيبص في الكل. فلو ضفت سطر في `/etc/hosts` وdig مش شايفه، ده طبيعي. عشان تسأل زي ما المتصفح بيسأل بالظبط على الـ Mac استخدم: `dscacheutil -q host -a name example.com` (`-q host` = اسأل عن جهاز، و`-a name` = بالاسم ده).

5.6 /etc/hosts: اكذب على جهازك

فاكر ملف `HOSTS.TXT` القديم؟ أهو لسه عايش على كل جهاز Mac و Linux باسم `/etc/hosts`، وجهازك بيبص فيه **قبل** أي DNS. يعني أي سطر تكتبه فيه بيكسب. وده مفيد جداً في التطوير: عاوز تفتح مشروعك على اسم شيك زي `myshop.test` بدل `localhost`؟ أو عاوز تجرب سيرفر جديد قبل ما تغيّر الـ DNS الحقيقي؟

تشريح الأمر

```
sudo  nano  /etc/hosts
 1     2       3
```

1. `sudo` — الملف ملك النظام، فمحتاج صلاحيات Admin تعدّله. هيطلب باسورد الـ Mac (ومش هتشوف النجوم وانت بتكتب، عادي).
2. `nano` — Editor بسيط في الـ Terminal. للحفظ `Ctrl+O` ثم Enter، وللخروج `Ctrl+X`.
3. `/etc/hosts` — المسار الكامل للملف.

```
/etc/hosts

##
# Host Database
##
127.0.0.1       localhost
255.255.255.255 broadcasthost
::1             localhost

# مشاريعي
127.0.0.1       myshop.test api.myshop.test
```

كل سطر: IP وبعده اسم أو أكتر بينهم مسافات. والـ `#` تعليق. بعد السطر الأخير، لو كتبت `http://myshop.test:5173` في المتصفح هيفتح مشروعك اللي على localhost. واستخدم دايماً `.test` للأسامي اللي بتخترعها، لأنه TLD محجوز رسمياً للتجارب ومستحيل يبقى موقع حقيقي (`.local` متستخدمهوش على الـ Mac لأنه محجوز لحاجة تانية اسمها Bonjour).

امسح كاش الـ DNS على الـ Mac

عدّلت `/etc/hosts` أو غيّرت DNS موقعك والجهاز لسه بيروح للقديم؟ امسح الكاش:

تشريح الأمر

```
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```

1. `sudo dscacheutil -flushcache` — أداة كاش خدمات الدليل في macOS: "امسح الكاش".
2. `;` — نفّذ الأمر اللي بعدي بعد ما ده يخلص، سواء نجح أو لأ (غير `&&` اللي بتنفّذ بس لو نجح).
3. `sudo killall -HUP mDNSResponder` — `killall` = ابعت إشارة لكل البرامج اللي بالاسم ده. و`-HUP` إشارة اسمها Hang Up معناها هنا "اعمل Reload وانسى الكاش"، مش "موت". و`mDNSResponder` البرنامج اللي ماسك DNS في macOS.

وChrome كمان ليه كاش لوحده. افتح `chrome://net-internals/#dns` ودوس **Clear host cache**.

5.7 لما الـ DNS يبوظ: إزاي تعرف؟

مشكلة 1 — **net::ERR_NAME_NOT_RESOLVED أو DNS_PROBE_FINISHED_NXDOMAIN**

Chrome بيقولك "This site can't be reached" و"server IP address could not be found". يعني الـ DNS ملقاش الاسم ده خالص. في الـ Console لو الطلب جاي من `fetch` هتلاقي `net::ERR_NAME_NOT_RESOLVED`.

الحل: (1) اتأكد من الإملاء: `gogle.com` ؟ (2) جرّب `dig +short الاسم` : لو فاضي أو `NXDOMAIN` يبقى الاسم مش موجود فعلاً. (3) جرّب `dig +short 1.1.1.1@ الاسم` : لو اشتغل هنا ومش شغال من غيرها، يبقى المشكلة في الـ Resolver بتاع شركة الإنترنت، فغيّر الـ DNS من **System Settings › Network › Wi-Fi › Details › DNS**. (4) لو دومين لسه شاريه، الـ Nameservers ممكن تكون لسه متظبطتش.

مشكلة 2 الموقع شغال عند ناس ومش شغال عندي بعد ما غيّرت الـ DNS

نقلت موقعك لسيرفر جديد، وصاحبك شايف الجديد وانت لسه شايف القديم.

الحل: ده الكاش. قارن `dig +short الاسم` بـ `dig +short 1.1.1.1@ الاسم` . لو مختلفين استنى الـ TTL يخلص، أو امسح كاش الـ Mac و Chrome زي ما شرحنا. واتأكد إن مفيش سطر قديم في `/etc/hosts` .

لو حابب تتعمق: DNS over HTTPS

تقدر تعدّيه في أول قراية

الـ DNS العادي بيمشي على UDP Port 53 من غير تشفير، يعني شركة الإنترنت (أو أي حد على نفس الواي فاي) يقدر يشوف كل الأسامي اللي بتسأل عنها، حتى لو المواقع نفسها HTTPS. فاتطور حل اسمه **DoH** (DNS over HTTPS): السؤال بيتبعت كطلب HTTPS عادي لـ Resolver زي `1.1.1.1` ، فمحدش يقدر يقراه. Chrome بيدعمه من **Use › Security › Privacy and security › Settings** **DNS secure**. وتقدر تجربه بـ curl:

```
'accept: application/dns-json' 'https://cloudflare-dns.com/dns-query?name=example.com&type=A'
```

، وهيرجعلك الإجابة JSON.

ملخص الفصل

- DNS اتعمل بدل ملف `HOSTS.TXT` المركزي، وفكرته شجرة موزعة كل فرع مسؤول عنه جهة.
- الدومين بيتقرا من اليمين: Root ثم TLD ثم الاسم ثم الـ Subdomains.
- الرحلة: كاش المتصفح، `/etc/hosts` ، كاش الـ OS، الـ Resolver، وبعدين Root ثم TLD ثم Authoritative. وكل خطوة بتكاش لمدة الـ TTL.
- السجلات: `A` و `AAAA` للعناوين، و `CNAME` للـ Alias، و `MX` للإيميل، و `TXT` للتحقق، و `NS` للمسؤول.
- `dig +short` و `@server` وأنواع السجلات، ومعاهم `dscacheutil -q host` اللي بيسأل زي المتصفح.
- `/etc/hosts` بيكسب على DNS، واستخدم `.test` لأساميك، وامسح الكاش بعد أي تعديل.

صح ولا غلط؟

1. الـ Root server عارف الـ IP بتاع كل موقع في الدنيا. صح / غلط
2. `dig` بيقرا ملف `/etc/hosts` قبل ما يسأل. صح / غلط
3. `www` مجرد Subdomain عادي. صح / غلط

أسئلة مراجعة

1. إيه المشاكل اللي كانت في `HOSTS.TXT` ، و DNS حلها إزاي؟
2. اشرح ليه الكاش مهم جداً لـ DNS، وإيه علاقته بالـ TTL؟
3. إمتى تستخدم `CNAME` وإمتى `A` ؟
4. عدّلت `/etc/hosts` و `dig` مش شايف التعديل. ليه؟ وتتأكد إزاي؟
5. اشرح كل حتة في `sudo killall -HUP mDNSResponder` .

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعرف سجلات `A` و `AAAA` و `MX` و `NS` لـ `gmail.com` بـ `dig +short` .
2. متوسط ضيف `127.0.0.1 hello.test` لـ `/etc/hosts` ، وشغّل `python3 -m http.server 8000` ، وافتح `http://hello.test:8000` . وبعدين قارن `dig +short hello.test` بـ `dscacheutil -q host -a name hello.test` . (متنساش تشيل السطر بعدها.)
3. صعب جرّب `dig +trace github.com` ، وحدد في الناتج: مين الـ Root اللي رد، ومين سيرفرات `.com` ، ومين الـ Authoritative.

06

المرحلة 3 · لغة الويب: HTTP

الفصل السادس

الـ URL متشرّح: كل حرف في العنوان ليه شغلانة

الـ URL هو أول حاجة بتكتبها وأول حاجة المتصفح بيفصّصها. وكل حتة فيه بتستخدم في مرحلة مختلفة: حتة للـ DNS، وحتة للـ Port، وحتة للسيرفر، وحتة مبتطلعش من المتصفح أصلاً. والفصل ده هيوصلك لمفهوم الـ Origin، اللي هو مفتاح فهم الكوكيز والـ CORS بعدين.

بعد الفصل ده هتكون عارف

- كل جزء في الـ URL: Scheme و Host و Port و Path و Query و Fragment
- ليه الـ Fragment (#) عمره ما بيوصل للسيرفر
- الـ Query string وإزاي تبعت بيانات فيه
- الـ Percent-encoding: ليه المسافة بتبقى %20، والعربي في الـ URL
- العناوين النسبية والمطلقة (/ و ./ و ../)
- يعني إيه Origin، وإزاي تقارن اتنين
- ليه لازم تحط الـ URL بين Quotes في zsh

6.1 المشكلة: عنوان واحد يقول كل حاجة

قبل الويب، عشان تجيب مستند من جهاز تاني كان لازم تعرف تلات حاجات منفصلة وتعمل بيهم خطوات منفصلة: أنهي جهاز، وأنهي برنامج تكلمه عليه (FTP؟ Gopher؟)، وفين الملف جوه الجهاز. والمعلومات دي كانت بتتقال بالكلام أو بتتكتب في إيميل: "ادخل على الجهاز الفلاني بـ FTP، وروح للفولدر الفلاني، وهات الملف ده". فـ Tim Berners-Lee عمل حاجة بسيطة وعبقرية: **جمع التلاتة في سطر واحد** بقواعد ثابتة، أي برنامج يقدر يقراه ويعرف يعمل إيه. وده الـ **URL** (Uniform Resource Locator)، يعني "عنوان موحّد لأي مورد".

6.2 التشريح الكامل

خلينا ناخد URL فيه كل الحتت مرة واحدة. في الحياة العادية مش هتلاقي كلهم مع بعض، بس كل حتة منهم هتقابلها:



شكل 6.1 كل لون حتة ليها شغلانة مختلفة. والتلات حتت الأولى مع بعض اسمهم الـ Origin.

تشريح الـ URL

```
1  2      3           4    5          6       7
https://shop.example.com:8443/products/shoes?color=red&size=42#reviews
```

1. `https` — الـ **Scheme**: بأنهي Protocol نتكلم. `https` و `http` للويب، وفيه غيرهم زي `mailto:` و `file:` و `ws:` (WebSocket).
2. `://` — فاصل ثابت بعد الـ Scheme في عناوين الويب. Tim Berners-Lee نفسه قال بعدين إن الـ `//` مكانتش ليها لازمة، بس فضلت.
3. `shop.example.com` — الـ **Host**: اسم الجهاز، وده اللي بيروح لـ DNS (الفصل 5). ممكن يبقى IP على طول زي `127.0.0.1`.
4. `:8443` — الـ **Port** (الفصل 3). لو مكتوبش، المتصفح بيستخدم الافتراضي: 443 لـ https و 80 لـ http. عشان كده `localhost:3000` لازم تكتب فيه الرقم.
5. `/products/shoes` — الـ **Path**: "عاوز أنهي حاجة على السيرفر ده". زمان كان دايماً مسار ملف حقيقي، النهارده السيرفر بيفهمه زي ما هو عاوز (Route).
6. `?color=red&size=42` — الـ **Query string**: بعد علامة `?`، أزواج `key=value` بينهم `&`. بيستخدم للبحث والفلترة والصفحات.
7. `#reviews` — الـ **Fragment**: بعد `#`. ده للمتصفح بس: "انزل لحد العنصر اللي `id` بتاعه `reviews`". **مبيتبعتش للسيرفر أبداً**.

ليه الـ Fragment مبيروحش للسيرفر؟

لأنه اتعمل أصلاً عشان يشاور على مكان **جوه** المستند اللي عندك، والسيرفر مش محتاج يعرف انت عاوز تقرا أنهي حتة. وعشان كده لو فتحت DevTools وبصيت على الطلب هتلاقي الـ URL من غير الـ `#`. والمعلومة دي استخدمتها تطبيقات قديمة (زي React Router في وضع `HashRouter`) عشان تغيّر "الصفحة" من غير ما تكلم السيرفر خالص: `site.com/#/about`.

6.3 الـ Query string: بيانات في العنوان

الـ Query string هو أبسط طريقة تبعت بيها بيانات للسيرفر، وهتستخدمه كتير في كورس JavaScript مع APIs زي `?q=egypt`. القواعد بسيطة: يبدأ بـ `?`، وكل معلومة `key=value`، وبينهم `&`، والترتيب مش مهم. وممكن نفس الـ Key يتكرر: `?tag=js&tag=css`، والسيرفر هو اللي يقرر يفهمها إزاي.

وفي JavaScript مش محتاج تقطّع الـ URL بإيدك، فيه أدوات جاهزة. افتح الـ Console في Chrome وجرّب:

```
Console · Chrome DevTools
> const u = new URL('https://shop.example.com:8443/products?color=red&size=42#reviews')
> u.hostname
'shop.example.com'
> u.port
'8443'
> u.searchParams.get('color')
'red'
> u.origin
'https://shop.example.com:8443'
> u.hash
'#reviews'
```

`new URL()` بياخد الـ URL ويفصّصه زي ما المتصفح بيعمل بالظبط، و`searchParams` بيديك الـ Query جاهز. ولو عاوز تبني Query بأمان: `new URLSearchParams({ q: 'آيفون 15', page: 2 }).toString()` هيرجعلك `q=%D8%A2%D9%8A%D9%81%D9%88%D9%86+15&page=2`. والشكل الغريب ده هو موضوعنا الجاي.

6.4 الـ Percent-encoding: ليه المسافة بقت %20؟

المشكلة: الـ URL اتصمم من الأول إنه يتكتب بحروف إنجليزي وأرقام وكام رمز بس. وفيه رموز ليها معنى خاص جواه: `?` بتبدأ الـ Query، و`&` بتفصل، و`#` بتبدأ الـ Fragment، و`/` بتفصل المسار. طب لو عاوز تبحث عن "`rock & roll`"؟ الـ `&` هتتفهم إنها فاصل، والمسافة مش مسموحة أصلاً. ولو عاوز تبحث بالعربي؟

فاتطور حل اسمه **Percent-encoding**: أي حرف مش مسموح أو ليه معنى خاص، بيتحول لـ `%` وبعدها رقمه بالـ Hex. المسافة رقمها 32 يعني `20` بالـ Hex، فبقت `%20`. والحروف العربي بتتحول لـ Bytes بنظام UTF-8 (كل حرف عربي = 2 Bytes)، وكل Byte يتكتب `%XX`. عشان كده كلمة "مصر" بقت `%D9%85%D8%B5%D8%B1`.

الحرف	شكله في الـ URL	ملاحظة
مسافة	`%20` أو `+`	`+` في الـ Query بس (من أيام الفورمز)، و`%20` في أي حتة
`&`	`%26`	عشان متتفهمش فاصل
`=`	`%3D`	
`?` / `#` / `/`	`%3F` / `%23` / `%2F`	لما يبقوا جزء من القيمة نفسها
`%` نفسه	`%25`	عشان يتفهم إنه حرف مش بداية كود

جدول 6.1 أشهر الحروف المتشفّرة

في JavaScript: encodeURIComponent

لما تبني URL بإيدك من قيمة جاية من المستخدم، متلزقهاش كده: `'?q=' + search`. استخدم `'?q=' + encodeURIComponent(search)`، أو الأحسن `URLSearchParams` اللي بيعملها لوحده. ولو نسيت، أول مستخدم يكتب `&` أو `#` هيكسر الطلب بتاعك بطريقة غريبة جداً ومش هتفهمها.

لو حابب تتعمق: الدومين بالعربي تقدر تعدّيه في أول قراية

الـ Path والـ Query بيتشفّروا بالـ Percent-encoding، بس الدومين نفسه ليه نظام تاني. فيه دومينات بالعربي فعلاً (زي `. مصر` كـ TLD رسمي لمصر)، والـ DNS نفسه مبيفهمش غير إنجليزي، فالاسم بيتحول لشكل اسمه **Punycode** يبدأ بـ `xn--` . يعني `مصر` بتبقى `xn--wgbh1c` . المتصفح بيعرضلك العربي، وبيبعت الـ Punycode من ورا.

6.5 العناوين النسبية: من غير ما تكتب كل حاجة

جوه ملف HTML، نادراً ما بتكتب الـ URL كامل. بتكتب `<img src="logo.png">` وخلاص، والمتصفح بيكمّل الباقي من عنوان الصفحة اللي انت فيها. ودي قواعد التكميل، لو انت في صفحة `https://site.com/blog/post.html` :

اللي كتبته	بيبقى	القاعدة
`logo.png`	`https://site.com/blog/logo.png`	نفس فولدر الصفحة
`./logo.png`	`https://site.com/blog/logo.png`	`./` = الفولدر الحالي (زي الـ Terminal)
`../logo.png`	`https://site.com/logo.png`	`../` = فولدر لفوق
`/logo.png`	`https://site.com/logo.png`	يبدأ بـ `/` = من جذر الموقع
`//cdn.com/x.js`	`https://cdn.com/x.js`	يبدأ بـ `//` = نفس الـ Scheme بس جهاز تاني
`?page=2`	`https://site.com/blog/post.html?page=2`	نفس الصفحة بـ Query جديد

جدول 6.2 إزاي المتصفح بيكمّل العنوان النسبي

حاجة مشتركة: . و .. و / زي الـ Terminal

الـ `./` والـ `../` والـ `/` في العناوين النسبية نفس معناهم بالظبط في مسارات الـ Terminal اللي اتعلمناها: الحالي، واللي فوقه، والجذر. والفرق الوحيد إن "الجذر" هنا جذر الموقع مش جذر الجهاز. وأشهر مشكلة هتقابلك: الصور شغالة على `localhost` ومش شغالة بعد النشر على GitHub Pages، لأن الموقع بقى في `user.github.io/project/` فالـ `/logo.png` بقى بيروح لجذر `user.github.io` مش لمشروعك. الحل غالباً: `./logo.png` .

6.6 الـ Origin: أهم مفهوم في أمان الويب

دلوقتي جه وقت أهم فكرة في الفصل. فاكر الخط اللي فوق الرسمة الأولى؟ الـ **Scheme** والـ **Host** والـ **Port** مع بعض اسمهم **Origin**. والـ Origin هو "الهوية" بتاعة أي صفحة في المتصفح. المتصفح بيعامل كل Origin كأنه بيت لوحده، ليه كوكيزه وتخزينه، وصفحة من Origin مش مسموحلها تقرا حاجات Origin تاني إلا لو التاني سمح. ودي قاعدة اسمها **Same-Origin Policy**، وهي أساس فصل الكوكيز والـ CORS كله.

القاعدة: اتنين URL ليهم نفس الـ Origin **بس لو** التلات حتت متطابقين بالظبط. والـ Path والـ Query مش داخلين في الحسبة. خلينا نقارن بـ `https://shop.example.com` :

الـ URL	نفس الـ Origin؟	ليه
`https://shop.example.com/cart?id=5`	أيوه	الـ Path والـ Query مش داخلين
`http://shop.example.com`	لأ	الـ Scheme اختلف (http)
`https://api.example.com`	لأ	الـ Host اختلف، حتى لو نفس الدومين الكبير
`https://shop.example.com:8443`	لأ	الـ Port اختلف (الأصلي 443)
`http://localhost:5173` **مقابل** `http://localhost:3000`	لأ	Port مختلف. ودي اللي هتقابلك أول ما تكلم React Node

جدول 6.3 نفس الـ Origin ولا لأ؟

> ⚠ **خلي بالك: Origin مش Site**
>
> هتقابل كلمة تانية قريبة اسمها **Site**. الـ Site أوسع: `shop.example.com` و`api.example.com` نفس الـ **Site** (`example.com`) بس مش نفس الـ **Origin**. الكوكيز (الفصل 13) بتفكر بالـ Site في حاجات كتير زي `SameSite`، والـ CORS (الفصل 14) بيفكر بالـ Origin. افتكر الفرق ده عشان هيرجعلك.

6.7 URLs في الـ Terminal: الـ Quotes إجبارية

آخر حاجة، ودي هتوفرلك وجع دماغ كبير. جرّب تكتب في zsh على الـ Mac:

```
Terminal

user@MacBook ~ % curl https://httpbin.org/get?color=red&size=42
zsh: no matches found: https://httpbin.org/get?color=red
```

إيه اللي حصل؟ حاجتين في نفس السطر. الـ `?` في zsh رمز **Globbing** معناه "أي حرف واحد"، فـ zsh حاول يدوّر على ملف اسمه كده، وملقاش، فوقف بالخطأ ده. ولو كانت عدّت، كانت الـ `&` هتعمل مشكلة تانية: في الـ Shell معناها "شغّل اللي قبلي في الخلفية"، فالأمر كان هيتقطع عندها والـ `size=42` كانت هتتفهم أمر لوحده. الحل بسيط: **حط أي URL بين Quotes** دايماً:

تشريح الأمر

```
curl 'https://httpbin.org/get?color=red&size=42'
```

1. `curl` اطلب الـ URL ده.
2. `'` الـ Single quote بتقول للـ zsh "متلمسش أي حرف جوايا"، فالـ `?` والـ `&` بيوصلوا لـ curl زي ما هما. (الـ Double quotes `"` كمان تنفع هنا، بس الـ Single أأمن لأن zsh مبيفسرش `$` جواها.)
3. `?color=red&size=42` الـ Query. و`httpbin.org/get` موقع للتجارب بيرجعلك الطلب اللي بعته كـ JSON، فتشوف الـ Query وصل إزاي.

حاجة مشتركة: نفس القاعدة من كتاب الـ Terminal

دي نفس قاعدة الـ Quoting اللي اتعلمناها في كتاب الـ Terminal: أي حاجة فيها رموز ليها معنى في الـ Shell (`? * & # $ ! ( )` والمسافات) حطها بين Single quotes. والـ `#` كمان ممكن تتفهم تعليق لو جت بعد مسافة. فخليها عادة: أي URL في الـ Terminal بين `'...'`.

ملخص الفصل

- الـ URL = Scheme + `://` + Port + Host + (اختياري) Path + Query (`?`) + Fragment (`#`).
- الـ Port الافتراضي 443 لـ https و 80 لـ http، والـ Fragment مبيوصلش للسيرفر.
- الـ Query أزواج `key=value` بينهم `&`، وفي JS استخدم `URL` و`URLSearchParams`.
- Percent-encoding: أي حرف خاص أو عربي بيتحول لـ `%XX`، والمسافة `%20` أو `+`.
- الـ Origin = Scheme + Host + Port، ولازم التلاتة يتطابقوا. ده أساس الكوكيز والـ CORS.
- في zsh حط أي URL بين `'...'` عشان `?` و`&`.

صح ولا غلط؟

1. `https://site.com` و`https://site.com:443` نفس الـ Origin. صح / غلط
2. السيرفر يقدر يعرف انت فاتح `#section2` في الصفحة من الطلب. صح / غلط
3. `http://localhost:3000` و`http://localhost:5173` نفس الـ Origin لأنهم نفس الجهاز. صح / غلط

أسئلة مراجعة

1. فصّص `http://127.0.0.1:8000/api/users?page=2&limit=10#top` لكل حتته.
2. ليه كلمة "مصر" في الـ URL بقت 6 أكواد `%XX`؟
3. انت في `https://site.com/docs/a/page.html` . `../img/x.png` هيبقى إيه؟
4. ليه `curl https://x.com/?a=1&b=2` من غير Quotes بيبوظ في zsh؟ اذكر السببين.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل في الـ Console: `new URL(location.href)` على أي موقع، واستكشف الخصائص (`protocol` و`host` و`pathname` و`search`).
2. متوسط جرّب `curl 'https://httpbin.org/get?q=rock%20%26%20roll&lang=ar'` وبص على `args` في الرد: الـ `%26` اتفهمت إزاي؟ وبعدين جرّبها بـ `&` عادية بدل `%26` وقارن.
3. صعب اكتب في الـ Console Function بتاخد Object وترجع URL كامل بالـ Query متشفّر صح باستخدام `URL` و `searchParams.set`. جربها بقيمة فيها عربي و `&`.

07

المرحلة 3 · لغة الويب: HTTP

الفصل السابع

HTTP: الطلب والرد سطر سطر

ده قلب الكتاب. HTTP هي اللغة اللي المتصفح و fetch و curl والسيرفر بيتكلموا بيها. هنعرف الحكاية: بدأت بسطر واحد وكبرت ليه. وبعدين نفصّص الطلب والرد سطر سطر، ونفهم الـ Methods (GET و POST وأخواتهم) ويعني إيه Safe و Idempotent، وإزاي البيانات بتتبعت في الـ Body.

بعد الفصل ده هتكون عارف

- إزاي HTTP اتطور من سطر واحد سنة 1991 لشكله النهارده
- شكل رسالة الطلب: Request line و Headers وسطر فاضي و Body
- شكل رسالة الرد: Status line و Headers و Body
- الـ Methods: GET و POST و PUT و PATCH و DELETE و HEAD و OPTIONS
- يعني إيه Safe ويعني إيه Idempotent، وليه زرار الدفع مبيتضغطش مرتين
- أنواع الـ Body: JSON و Form و Multipart، والـ Content-Type
- نفس الطلب بـ fetch في JavaScript
- يعني إيه HTTP Stateless

7.1 الحكاية: من سطر واحد لـ لغة كاملة

أول نسخة من HTTP سنة 1991 (اسمها بعدين **HTTP/0.9**) كانت بسيطة لدرجة مضحكة. الطلب كله سطر واحد: `GET /page.html`. والرد: الـ HTML على طول، وبعدين السيرفر يقفل الاتصال. مفيش أي حاجة تانية. وده كان كفاية لفكرة Tim الأولى: مستندات نصية فيها روابط.

وهنا كانت المشكلة. الناس بدأت تحط صور، وملفات، وصفحات بتتعمل من برامج. والمتصفح بيستلم Bytes ومش عارف دي HTML ولا صورة ولا PDF. ولو الصفحة مش موجودة، السيرفر يرجّع إيه؟ صفحة HTML مكتوب فيها "مش موجود"؟ طب المتصفح هيعرف منين إنها خطأ مش صفحة عادية؟ ومفيش طريقة تبعت بيها بيانات للسيرفر غير في الـ URL. فاتطور حل: **HTTP/1.0** سنة 1996 ضاف تلات حاجات غيرت كل حاجة: **رقم النسخة** في الطلب، و**Status code** في أول سطر من الرد (200 تمام، 404 مش موجود)، و**Headers**: سطور بتوصف الرسالة نفسها، زي `Content-Type: image/png`.

وبعدها على طول ظهرت مشكلة تانية. السيرفرات بقت بتشيل مواقع كتير على نفس الـ IP (عشان IPv4 غالي)، والطلب مفيهوش غير المسار، فالسيرفر مش عارف انت عاوز أنهي موقع. وكل ملف بيفتح اتصال TCP جديد، والـ Handshake كل مرة بطيء. فجه **HTTP/1.1** (1997 ثم اتظبط 1999): خلّى Header اسمه `Host` إجباري، وخلّى الاتصال **يفضل مفتوح** لطلبات كتير (Keep-alive). والنسخة دي لسه شغالة لحد النهارده، ونسخ 2 و 3 (الفصل 11) غيروا طريقة النقل بس، لكن **المعاني** نفسها (Methods و Status و Headers) زي ما هي، ومكتوبة النهارده في `RFC 9110`.

7.2 رسالة الطلب: أربع حتت

أي طلب HTTP، من المتصفح أو `fetch` أو `curl`، شكله كده. خلينا ناخد طلب حقيقي: انت بتعمل طلب أوردر جديد في متجر:



شكل 7.1 تشريح رسالة طلب HTTP/1.1. كل سطر بيخلص بـ \r\n، والسطر الفاضي هو الحد بين الـ Headers والـ Body.

تشريح سطر الطلب

```
1    2          3
POST /api/orders HTTP/1.1
```

1. `POST` — الـ **Method** (أو الفعل): "عاوز أعمل إيه". هنا: "ابعتلك بيانات تعمل بيها حاجة جديدة". وبيتكتب كابيتال دايماً.
2. `/api/orders` — الـ **Target**: الـ Path والـ Query من الـ URL. لاحظ إن الـ Host مش هنا، بيتكتب في Header لوحده. والـ `#fragment` مش هنا أكيد.
3. `HTTP/1.1` — نسخة الـ Protocol اللي الـ Client بيتكلمها، عشان السيرفر يرد بنفس اللغة.

وبعد سطر الطلب تيجي الـ **Headers**، كل واحد في سطر: اسم، وبعدين `:`، وبعدين القيمة. وأسامي الـ Headers **مش حساسة لحالة الحروف**: `Content-Type` و `content-type` نفس الحاجة (وفي HTTP/2 بقت كلها small). هنفصّص أهم الـ Headers في الفصل 9، بس لاحظ دلوقتي إن الـ Headers دي هي "الظرف" اللي بيوصف الجواب: الجواب رايح لمين (`Host`)، ومين باعته (`User-Agent`)، وعاوز الرد بأنهي شكل (`Accept`)، والبيانات اللي جوه نوعها إيه وحجمها قد إيه (`Content-Type` و `Content-Length`).

7.3 رسالة الرد: نفس الشكل تقريباً

والسيرفر بيرد برسالة شكلها نفس الفكرة، بس أول سطر مختلف:



شكل 7.2 تشريح رسالة الرد. أهم حاجة فيها الرقم اللي في أول سطر.

تشريح سطر الحالة

```
HTTP/1.1 201 Created
①        ②   ③
```

1	`HTTP/1.1`	النسخة اللي السيرفر بيرد بيها.
2	`201`	الـ **Status code**: 3 أرقام، وأول رقم بيقول الفئة (2 = نجح). البرامج بتقرا الرقم ده بس. الفصل 8 كله عنه.
3	`Created`	الـ Reason phrase: جملة للبني آدمين بس، والبرامج بتتجاهلها. وفي HTTP/2 اتشالت خالص، فهتشوف `HTTP/2 201` من غيرها.

HTTP مالوش ذاكرة (Stateless)

معلومة هتفسرلك حاجات كتير: كل طلب HTTP **مستقل تماماً** عن اللي قبله. السيرفر مبيفتكرش إنك لسه عامل Login من ثانيتين. كل طلب لازم يحمل معاه كل اللي السيرفر محتاجه يعرفه عنك. طب إزاي المواقع بتفتكرك؟ ده موضوع الفصل 13 (الكوكيز)، وشايف في الرسمة الأولى Header اسمه `Cookie` ؟ هو ده السر.

7.4 الـ Methods: انت عاوز تعمل إيه؟

في الأول كان فيه `GET` بس، لأن الويب كان للقراية بس. ولما الناس احتاجت تبعت فورمز (تسجيل، بحث، تعليق) اتضاف `POST`. ومع الوقت، والـ APIs بقت بتتعامل مع بيانات زي قاعدة بيانات (تقرا وتضيف وتعدل وتمسح)، بقى فيه Method لكل فعل. وفكرة إن الـ URL يمثل "حاجة" (`/api/products/17`) والـ Method يمثل "الفعل" ده أساس طريقة تصميم اسمها **REST** هتقابلها في Node:

الـ Method	معناه	مثال	Body؟
`GET`	هاتلي (قراية بس)	`GET /api/products/17`	لأ
`POST`	خد البيانات دي واعمل بيها حاجة (غالباً حاجة جديدة)	`POST /api/products`	أيوه
`PUT`	حط الحاجة دي كاملة مكان القديمة (أو اعملها)	`PUT /api/products/17`	أيوه، كاملة
`PATCH`	عدّل جزء بس	`PATCH /api/products/17` بـ `{"price":90}`	أيوه، الجزء بس
`DELETE`	امسح	`DELETE /api/products/17`	غالباً لأ
`HEAD`	زي GET بالظبط بس من غير Body: الـ Headers بس	`curl -I`	لأ
`OPTIONS`	"إيه المسموح هنا؟" المتصفح بيستخدمه في الـ CORS Preflight (الفصل 14)	`OPTIONS /api/orders`	لأ

جدول 7.1 الـ Methods الأساسية

Safe و Idempotent: ليه زرار الدفع مبيتضغطش مرتين

تخيل إنك دوست "ادفع" والنت قطع ثانية، والصفحة مردتش. تدوس تاني؟ لو الطلب الأول كان وصل فعلاً، هتدفع مرتين! المشكلة دي حقيقية، وعشان كده HTTP بيقسم الـ Methods بصفتين مهمين جداً، والمتصفحات والسيرفرات والكاش بيعتمدوا عليهم:

Safe: مبيغيرش حاجة

`GET` و `HEAD` و `OPTIONS` . المفروض إنهم قراية بس، فالمتصفح يقدر يعيدهم براحته، ويعملهم مسبقاً (Prefetch)، والكاش يحفظهم. عشان كده عمرك ما تعمل `GET /delete-account` : أي Bot بيمشي على الروابط هيمسح الحسابات!

Idempotent: تكراره زي مرة واحدة

الـ Safe كلهم، ومعاهم `PUT` و `DELETE` . لو بعت `DELETE /items/5` عشر مرات، النتيجة نفس مرة واحدة: العنصر ممسوح. أما `POST` مش Idempotent: كل مرة ممكن تعمل أوردر جديد. عشان كده المتصفح بيسألك "Confirm Form Resubmission" لو عملت Refresh بعد POST.

في الشغل الحقيقي

لما تيجي تعمل API في Node: القراية دايماً `GET` ، والإنشاء `POST` ، والتعديل `PATCH` (أو `PUT` لو بتستبدل كله)، والمسح `DELETE` . ومتحطش أفعال في الـ URL زي `/getProducts` أو `/deleteUser?id=5` ، الـ Method هو الفعل والـ URL هو الحاجة. وفي الواجهة، امنع الضغط المتكرر على أزرار الـ POST المهمة (اعمل الزرار `disabled` لحد ما الرد ييجي).

7.5 الـ Body والـ Content-Type: البيانات شكلها إيه؟

لما تبعت بيانات في الـ Body، السيرفر بيستلم Bytes. ومحتاج يعرف يقراها إزاي. هنا شغل Header اسمه `Content-Type` ، وهو بيقول "اللي في الـ Body ده نوعه كذا". وفيه تلات أشكال هتقابلهم في كل حتة:

1 · JSON: الأشهر في الـ APIs

```
POST /api/orders HTTP/1.1
Content-Type: application/json

{"productId":17,"quantity":2,"note":"باب العمارة"}
```

2 · Form عادي: اللي الفورم بيبعته افتراضياً

```
POST /login HTTP/1.1
Content-Type: application/x-www-form-urlencoded

email=sara%40example.com&password=s3cret
```

3 · Multipart: لما فيه ملفات

```
POST /upload HTTP/1.1
Content-Type: multipart/form-data; boundary=----X9f2

------X9f2
Content-Disposition: form-data; name="avatar"; filename="me.png"
Content-Type: image/png

(Bytes الصورة)
------X9f2--
```

الفرق بين التلاتة

1 الـ Body نص JSON. في JS بتعمله بـ `JSON.stringify(obj)`. وده اللي هتستخدمه مع أغلب الـ APIs.	`application/json`
2 نفس شكل الـ Query string بالظبط (`key=value&...`) بس في الـ Body، ومتشفّر Percent-encoding (لاحظ `@` بقت `%40`). ده اللي بيبعته `<form method="post">`.	`application/x-www-form-urlencoded`
3 الـ Body متقسم أجزاء، وكل جزء ليه Headers لوحده. والـ `boundary` = النص اللي بيفصل الأجزاء. ده الوحيد اللي ينفع للملفات. في JS بتعمله بـ `FormData`.	`multipart/form-data; boundary=…`

خلي بالك: أشهر غلطة مع fetch

لو بعت JSON من غير ما تقول إنه JSON، السيرفر ممكن ميعرفش يقراه، ويرجعلك `400` أو يلاقي الـ Body فاضي. يعني لازم الاتنين مع بعض: `body: JSON.stringify(data)` و `headers: { 'Content-Type': 'application/json' }`. والعكس كمان: لو بعت `FormData` متحطش `Content-Type` بإيدك، سيب المتصفح يحطه عشان يكتب الـ `boundary` صح.

7.6 نفس الكلام في JavaScript

Promise و async/await في دقيقة (لو لسه موصلتلهمش في الكورس)

الطلب على الشبكة بياخد وقت، و JavaScript مش هتقف تستنى وتعلّق الصفحة. فـ `fetch` مبيرجعش الرد على طول، بيرجع **Promise**: "وعد" إن الرد هييجي بعدين، زي البيجر اللي بيدّوهولك في المطعم وبرن لما الأكل يجهز. الـ Promise ليها حالتين في الآخر: يا **اتحققت** (Fulfilled) بالنتيجة، يا **اترفضت** (Rejected) بـ Error.

- `await` قبل الـ Promise = "استنى هنا لحد ما البيجر يرن، وهات النتيجة". ومينفعش تستخدمها غير جوه دالة مكتوب قبلها `async` (أو في الـ Console مباشرة).
- `.then(fn)` = نفس الفكرة بالطريقة القديمة: "لما النتيجة تيجي، شغّل fn".
- `try / catch` حوالين الـ `await` بتمسك الـ Rejection.

وكمان **JSON**: نص شكله زي الـ Object في JavaScript (`{"id": 1, "name": "Sara"}`)، والـ Keys لازم بين `"..."`. `JSON.stringify(obj)` بتحوّل الـ Object لنص علشان يتبعت، و `res.json()` بتعمل العكس. الشرح الكامل لـ JSON في كتاب APIs و JSON (فصل 4 و 5)، والـ fetch بالتفصيل هناك في فصل 10.

لما توصل لجزء `fetch` في كورس Jonas، افتكر إن كل سطر في `fetch` بيتحول لحتة من الرسالة اللي فوق. شوف الطلب اللي رسمناه في أول الفصل لو اتكتب بـ JS:

JavaScript

```
const res = await fetch('https://shop.example.com/api/orders', {
  method: 'POST',                                  // سطر الطلب: الـ Method
  headers: { 'Content-Type': 'application/json',
             'Accept': 'application/json' },       // الـ Headers
  body: JSON.stringify({ productId: 17, quantity: 2 }), // الـ Body
});
console.log(res.status);        // 201 ← من سطر الحالة
console.log(res.ok);            // true ← أي كود من 200 لـ 299
console.log(res.headers.get('Location')); // '/api/orders/981'
const data = await res.json(); // اقرا الـ Body كـ JSON
```

كل حتة في fetch رايحة فين

1 أول كلمة في سطر الطلب. لو مكتبتهاش، الافتراضي `GET`.	`method: 'POST'`
2 الـ Headers. المتصفح بيضيف لوحده حاجات زي `Host` و `User-Agent` و `Cookie`، ومش هيسيبك تغيرها.	`headers: {...}`
3 الـ Body كنص. ومينفعش تبعت Body مع `GET` في fetch، هيرمي Error.	`body: JSON.stringify(...)`

`res.status / res.ok`	4 رقم الحالة، و `ok` = هل الرقم بين 200 و 299.
`await res.json()`	5 اقرا الـ Body وحوّله Object. ليه `await` تاني؟ لأن الـ Headers بتوصل الأول، والـ Body ممكن لسه بيتحمل.

خلي بالك: fetch مبيعملش Error على 404 و 500

دي هتقابلك في الكورس أكيد. `fetch` بيعتبر أي رد من السيرفر "نجاح"، حتى لو 404 أو 500، لأن الطلب راح ورجع فعلاً. الـ Promise مبترفضش (Reject) غير لو الطلب **فشل يوصل** أصلاً: نت قاطع، DNS فشل، CORS منع. عشان كده لازم دايماً تشيك بنفسك: `if (!res.ok) throw new Error(`HTTP ${res.status}`)` .

حاجة مشتركة: نفس الرسالة في 4 أدوات

الرسالة اللي فصّصناها هي هي في كل حتة، والفرق في طريقة الكتابة بس. في `curl` : `-X POST` للـ Method، و `-H` للـ Header، و `-d` للـ Body (الفصل 10). وفي `fetch` : `method` و `headers` و `body` . وفي DevTools: تابة **Headers** وتابة **Payload** (الفصل 19). وفي Node بعدين: `req.method` و `req.headers` و `req.body` . اتعلمها مرة، تقراها في الأربعة.

ملخص الفصل

- HTTP بدأ سطر واحد (0.9)، و 1.0 ضاف النسخة والـ Status والـ Headers، و 1.1 ضاف `Host` الإجباري والاتصال المفتوح.
- الطلب: Request line (Version + Target + Method)، Headers، سطر فاضي، Body.
- الرد: Status line (Reason + Code + Version)، Headers، سطر فاضي، Body.
- الـ Methods: GET قراية، POST إنشاء، PUT استبدال، PATCH تعديل جزئي، DELETE مسح، HEAD الـ Headers بس، OPTIONS سؤال.
- Safe = مبيغيرش (GET و HEAD و OPTIONS)، و Idempotent = تكراره زي مرة (ومعاهم PUT و DELETE). POST لا ده ولا ده.
- `Content-Type` بيقول شكل الـ Body: JSON أو Form أو Multipart.
- HTTP Stateless، و `fetch` مبيرفضش على 404/500، شيك على `res.ok` .

صح ولا غلط؟

1. أسامي الـ Headers حساسة لحالة الحروف. صح / غلط
2. `DELETE` Idempotent. صح / غلط
3. `fetch` بيرمي Error لو السيرفر رجّع 404. صح / غلط

؟ أسئلة مراجعة

1. إيه المشكلة اللي الـ Header الـ `Host` حلها في HTTP/1.1؟
2. السيرفر بيعرف إن الـ Headers خلصت والـ Body بدأ إزاي؟
3. إيه الفرق بين `PUT` و `PATCH`؟ اديني مثال لكل واحد على `/api/users/3`.
4. ليه غلط تعمل Route زي `GET /logout` أو `GET /delete?id=5`؟
5. عاوز ترفع صورة بروفايل من فورم. أنهي `Content-Type`؟ وليه مش JSON؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل في الـ Console على أي صفحة:
   `const r = await fetch('https://httpbin.org/get'); r.status; await r.json()`. بص على الـ `headers` اللي httpbin رجعهالك: دي الـ Headers اللي المتصفح بعتها.
2. متوسط ابعت POST بـ JSON لـ `https://httpbin.org/post` من الـ Console، وشوف في الرد قيمة `json` وقيمة `headers['Content-Type']`. وبعدين ابعته من غير الـ Header الـ Content-Type وقارن.
3. صعب اكتب Function اسمها `getJSON(url)` بترمي Error فيه رقم الـ Status لو `res.ok` بـ false، وجربها على `https://httpbin.org/status/404`.

08

المرحلة 3 · لغة الويب: HTTP

الفصل التامن

الـ Status Codes: السيرفر بيقولك إيه بالأرقام

أول حاجة هتبص عليها في أي طلب في DevTools هي الرقم اللي في عمود Status. الرقم ده هو أسرع تشخيص في الدنيا: في ثانية بيقولك المشكلة عندك ولا عند السيرفر، ولا مفيش مشكلة أصلاً. الفصل ده هيخليك تقرا الأرقام دي زي ما الدكتور بيقرا التحاليل.

بعد الفصل ده هتكون عارف

- ليه اتعملت الأرقام، ومنطق الفئات الخمسة xx1 لـ xx5
- أكواد النجاح 200 و 201 و 204 و 206
- التحويلات 301 و 302 و 307 و 308، و 304 اللي مش تحويل
- أخطاء الـ Client: 400 و 401 و 403 و 404 و 405 و 409 و 422 و 429
- أخطاء السيرفر: 500 و 502 و 503 و 504، وإزاي تعرف مين اللي رد
- تجرب أي كود بـ curl و httpbin وتطلّع الكود بس بـ -w

8.1 المشكلة: البرنامج مش بيقرا كلام

في HTTP/0.9، لو طلبت صفحة مش موجودة، السيرفر كان بيرجعلك صفحة HTML مكتوب فيها "Not found" وخلاص. الإنسان يفهمها، بس البرنامج؟ المتصفح مش هيعرف إن دي صفحة خطأ، والكاش هيحفظها كأنها الصفحة الحقيقية، ومحرك البحث هيضيفها لنتائجه. ولو عندك برنامج بيجيب بيانات من سيرفر، هيعرف منين إن العملية فشلت؟ يقرا النص ويدوّر على كلمة "error"؟ وبالإنجليزي ولا بالعربي؟

فاتطور حل بسيط جداً في HTTP/1.0: أول سطر في أي رد فيه **رقم من 3 خانات**، ثابت ومتفق عليه، البرنامج يقراه من غير ما يفهم أي لغة. وأحلى حاجة في التصميم إن **أول رقم لوحده بيقولك الفئة**. يعني حتى لو قابلت كود عمرك ما شفته، زي `425`، هتعرف على طول إنه "غلطة من ناحية الـ Client" لأنه بيبدأ بـ 4.



شكل 8.1 أول رقم بيقولك الحكاية كلها: 2 تمام، 3 روح حتة تانية، 4 المشكلة في طلبك، 5 المشكلة عند السيرفر.

أول سؤال في أي Debugging

لما حاجة متشتغلش، بص على الرقم قبل أي حاجة. **xx4** يبقى ابدأ دوّر عندك: الـ URL، والـ Headers، والـ Body، والـ Login. **xx5** يبقى غالباً الكود بتاعك سليم والمشكلة في السيرفر، فبص على لوجات السيرفر أو كلّم اللي ماسك الـ API. والتقسيمة البسيطة دي لوحدها هتوفرلك ساعات. هنبني عليها خطة كاملة في الفصل 22.

8.2 xx2: نجح، بس بأنهي طريقة؟

ممكن تقول: نجح يعني نجح، ليه أكتر من كود؟ لأن فيه فرق بين "أهو اللي طلبته" و"عملت الحاجة الجديدة" و"عملت اللي قلت عليه ومعنديش حاجة أرجعها". والفرق ده بيفرق مع البرنامج اللي بيقرا الرد:

`200 OK`	الافتراضي: الطلب نجح، والـ Body فيه النتيجة. أي `GET` ناجح غالباً 200.
`201 Created`	اتعملت حاجة جديدة (بعد `POST` غالباً). وعادة معاه Header اسمه `Location` فيه عنوان الحاجة الجديدة، زي اللي شفناه في الفصل 7.
`204 No Content`	نجح، و**مفيش Body خالص**. مشهور بعد `DELETE`. خلي بالك: لو عملت `await res.json()` على 204 هيرمي Error لأن مفيش JSON أصلاً.
`206 Partial Content`	رجعتلك حتة من الملف بس، لأنك طلبت كده بـ Header اسمه `Range`. ده اللي بيخلي الفيديو يبدأ من النص لما تقدّم، والتحميلات تكمّل بعد ما النت يقطع.

8.3 xx3: "مش هنا، روح هناك"

المواقع بتتنقل. الصفحة اللي كانت `/blog/old-post` بقت `/articles/new-post`. الموقع انتقل من `http` لـ `https`، أو من `site.com` لـ `www.site.com`. والمشكلة: الناس عندها الروابط القديمة، ومحركات البحث كمان. فاتطور حل اسمه **Redirect**: السيرفر يبرد بكود من فئة 3، ومعاه Header اسمه `Location` فيه العنوان الجديد، والمتصفح لوحده يبروح هناك من غير ما تحس.

Terminal

```
user@MacBook ~ % curl -I http://github.com
HTTP/1.1 301 Moved Permanently
Content-Length: 0
Location: https://github.com/
```

شفت؟ طلبت `http`، و GitHub قالك "اتنقلت لـ `https` نهائياً". بس فيه أكتر من نوع Redirect، والفرق بينهم حاجتين: **نهائي ولا مؤقت؟** و**المتصفح يحافظ على الـ Method ولا يحولها GET؟**

الكود	نهائي؟	الـ Method	هتستخدمه إمتى
`301 Moved Permanently`	أيوه	ممكن يتحول GET	الصفحة اتنقلت للأبد. محركات البحث بتنقل ترتيبها للجديدة، والمتصفح بيكاشه
`302 Found`	لأ	ممكن يتحول GET	تحويل مؤقت، زي بعد الـ Login روح للـ Dashboard
`303 See Other`	لأ	دايماً GET	بعد POST فورم: "تمام، روح شوف النتيجة هنا" (عشان Refresh ميعيدش الـ POST)
`307 Temporary Redirect`	لأ	زي ما هو	زي 302 بس بيحافظ على POST وعلى الـ Body
`308 Permanent Redirect`	أيوه	زي ما هو	زي 301 بس بيحافظ على الـ Method. مهم للـ APIs

جدول 8.1 أنواع الـ Redirect

> ⚠ **خلي بالك: الـ 301 بيتكاش جامد**
>
> المتصفح بيحفظ الـ `301` في الكاش، وممكن ميسألش السيرفر تاني خالص لفترة طويلة. يعني لو عملت 301 بالغلط وصلّحته، الناس اللي زاروا الموقع هيفضلوا يتحولوا للعنوان الغلط. وانت كمان! عشان كده وانت بتجرب، استخدم `302` أو `307` لحد ما تتأكد، وفي DevTools فعّل **Disable cache** (الفصل 18).

304 Not Modified: مش تحويل خالص

كود غريب في الفئة دي: `304`. هو مش "روح هناك"، هو "النسخة اللي عندك لسه زي ما هي، استخدمها". المتصفح بيسأل السيرفر "معايا نسخة من الملف ده من يوم كذا، اتغير؟"، والسيرفر يرد بـ 304 من غير Body، فيوفر تحميل الملف كله تاني. هتشوفه كتير جداً في تابة Network، وده **شيء كويس** مش خطأ. تفاصيله في فصل الكاش (الفصل 15).

8.4 xx4: المشكلة في طلبك

الفئة دي هي اللي هتقابلها أكتر حاجة وانت بتتعلم، لأن أغلبها معناه "انت بعت حاجة غلط"، وده بيحصل كتير لما تبدأ تكلم APIs. وكل كود هنا بيقولك **نوع** الغلطة بالظبط، فافهمهم كويس:

`400 Bad Request`	"مش فاهم طلبك". الـ JSON بايظ، أو حقل مطلوب ناقص، أو قيمة شكلها غلط. اقرا الـ Body بتاع الرد، غالباً السيرفر كاتب فيه السبب.
`401 Unauthorized`	"**مين انت؟**". مش عامل Login، أو الـ Token غلط أو انتهى. الاسم مضلل: المقصود Unauthenticated. وغالباً معاه Header اسمه `WWW-Authenticate`.
`403 Forbidden`	"**عارف انت مين، ومش مسموحلك**". عامل Login بس صلاحياتك متسمحش. Login تاني مش هيحل حاجة.
`404 Not Found`	"مفيش حاجة على العنوان ده". غلطة في الـ URL أو الـ Path، أو الحاجة اتمسحت. وأحياناً السيرفر بيرد 404 بدل 403 عشان ميعرفكش إن الحاجة موجودة أصلاً.

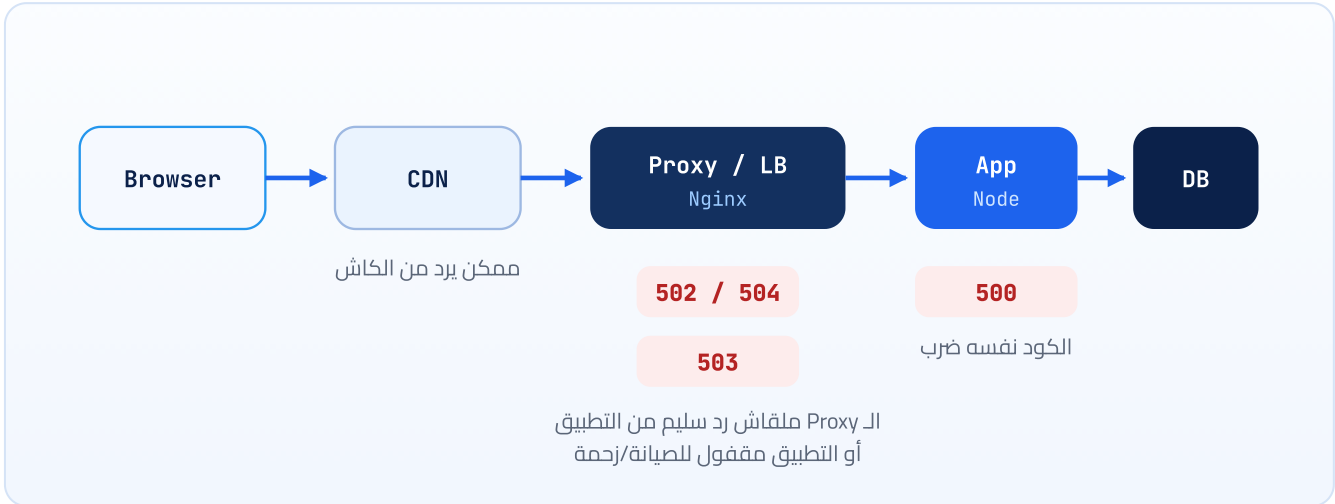
`405 Method Not Allowed`	العنوان موجود بس مش بيقبل الـ Method ده. بعت `POST` لحاجة بتقبل `GET` بس. الرد فيه Header اسمه `Allow` بالمسموح.
`409 Conflict`	الطلب بيتعارض مع الحالة الحالية: الإيميل ده مسجل قبل كده، أو حد عدّل نفس الحاجة قبلك.
`413 / 415`	`413 Content Too Large` : الـ Body (غالباً ملف) أكبر من المسموح. `415 Unsupported Media Type` : الـ `Content-Type` مش اللي السيرفر مستنيه (بعت Form والسيرفر عاوز JSON).
`422 Unprocessable Content`	"فهمت طلبك، بس البيانات نفسها مش منطقية": الإيميل شكله غلط، أو التاريخ في الماضي. APIs كتير بتستخدمه لـ Validation بدل 400.
`429 Too Many Requests`	"اهدى شوية". بعت طلبات كتير في وقت قليل (Rate limit). بص على Header اسمه `Retry-After` : هيقولك تستنى قد إيه. هيقابلك مع APIs المجانية.

401 و 403: مثال الفندق

تخيل فندق. لو جيت تدخل الأسانسير من غير كارت خالص، الأمن هيقولك "انت مين؟ وريني الكارت" = **401**. ولو معاك كارت سليم بس بتحاول تدخل جناح الإدارة، هيقولك "عارف انت مين، بس الكارت ده مش بيفتح هنا" = **403**. فلو جالك 401: اعمل Login أو جدد الـ Token. لو جالك 403: محتاج صلاحية، مش Login.

8.5 xx5: السيرفر هو اللي وقع

هنا المشكلة مش منك. طلبك ممكن يكون سليم 100%، والسيرفر هو اللي معرفش يتعامل. بس عشان تفهم الأكواد دي كويس، لازم تعرف حاجة عن شكل السيرفرات الحقيقية: نادراً ما بيبقى فيه سيرفر واحد. غالباً طلبك بيعدّي على طبقات: CDN، وبعدين **Reverse proxy** أو **Load balancer** (زي Nginx)، وبعدين التطبيق نفسه (Node مثلاً)، اللي ممكن يكلم Database. وكل طبقة ممكن هي اللي ترد بالخطأ:



شكل 8.2 مين يرد بأنهي xx5: الـ 500 من التطبيق نفسه غالباً، والـ 502 و 504 من اللي واقف قدامه.

`500 Internal Server Error`	"حصل Error في الكود ومعرفتش أعمل إيه". في Node ده غالباً Exception متمسكتش. الرسالة المفيدة في لوج السيرفر، مش في المتصفح.
`502 Bad Gateway`	الـ Proxy كلّم التطبيق اللي وراه، ورجعله رد بايظ أو مردش خالص (التطبيق واقع أو بيعيد التشغيل). مشهور جداً وقت الـ Deploy.
`503 Service Unavailable`	"مش قادر دلوقتي": صيانة أو ضغط زيادة. ممكن يبقى معاه `Retry-After` . مؤقت بطبيعته.
`504 Gateway Timeout`	الـ Proxy استنى التطبيق وزهق. التطبيق بطيء جداً، غالباً Query تقيلة على الـ Database.

مثال عملي: مين اللي رد؟

لو صفحة الخطأ شكلها بسيط ومكتوب فيها `nginx` أو `cloudflare` ، يبقى الطبقة اللي قدام هي اللي ردت، والتطبيق نفسه غالباً واقع أو بطيء. بص على الـ Header `Server` في الرد (الفصل 9)، وعلى Headers زي `cf-ray` اللي بتقولك إن Cloudflare في السكة. المعلومة دي بتفرق لما تبلّغ حد بالمشكلة: "الـ API بيرجّع 502 من Nginx" أوضح بكتير من "الموقع مش شغال".

8.6 xx1: الأكواد اللي مش هتشوفها كتير

الفئة دي رسايل مؤقتة قبل الرد الحقيقي، ونادراً ما هتتعامل معاها بنفسك. أشهرهم `101 Switching Protocols` : "تمام، هنقلب من HTTP لـ WebSocket" (الشات والإشعارات اللحظية). و `103 Early Hints` : السيرفر لسه بيجهّز الصفحة، فيقول للمتصفح بدري "ابدأ نزّل ملف الـ CSS ده على ما أخلص"، وهتلاقيه في DevTools في عمود Initiator باسم Early Hints.

لو حابب تتعمق: I'm a teapot 418

تقدر تعدّيه في أول قراية

فيه كود اسمه `418 I'm a teapot` ("أنا براد شاي")، جه من RFC اتكتب كنكتة يوم 1 أبريل 1998 عن Protocol للتحكم في أباريق القهوة. والكود فضل عايش لحد النهارده، وفيه مواقع بترجعه كهزار، و `https://httpbin.org/status/418` هيرجعهولك برسمة براد. مش هتستخدمه، بس دلوقتي لما تشوفه هتضحك بدل ما تتخض.

8.7 معمل: اطلب أي كود بإيدك

موقع `httpbin.org` اتعمل للتجارب، وعنده عناوين بترجع أي كود تطلبه. وفيه طريقتين مهمين تشوف بيهم الكود من curl. الأولى `-I` اللي شفناها: بتعمل طلب `HEAD` وتطبع الـ Headers بس. والتانية بتطبع الرقم لوحده، ودي اللي هتستخدمها في الـ Scripts:

تشريح الأمر

```
curl -s -o /dev/null -w '%{http_code}\n' https://httpbin.org/status/404
```

1. `-s` Silent: متطبعش شريط التقدم والرسايل. curl بيطبع شريط تقدم لما الناتج رايح لملف، و `-s` بتخفيه.
2. `-o /dev/null` ارمي الـ Body (زي ما شفنا في الفصل 2).

3 `-w` Write-out: "بعد ما تخلص، اطبع النص ده". وجواه متغيرات بين `%{...}` curl بيملاها.

4 `'%{http_code}\n'` متغير الـ Status code، وبعده `\n` سطر جديد عشان الـ Prompt ميلزقش في الرقم. والـ Single quotes عشان zsh ميلمسش الـ `%` والـ `{}`.

5 `https://httpbin.org/status/404` عنوان بيرجّع الكود اللي في آخره.

```
Terminal
user@MacBook ~ % curl -s -o /dev/null -w '%{http_code}\n' https://httpbin.org/status/404
404
user@MacBook ~ % curl -s -o /dev/null -w '%{http_code}\n' https://example.com
200
user@MacBook ~ % curl -sI https://httpbin.org/redirect/2
HTTP/2 302
date: Thu, 22 Oct 2026 20:41:03 GMT
content-type: text/html; charset=utf-8
content-length: 247
location: /relative-redirect/1
```

وآخر مثال بيوريك إن curl **مبيتبعش الـ Redirect لوحده** زي المتصفح، بيوقف عند أول رد ويوريهولك. ولو عاوزه يكمّل زي المتصفح، ضيف `-L`:

تشريح الأمر

```
curl -sIL https://httpbin.org/redirect/2
     123  4
```

1 `-s` Silent.

2 `I` الـ `-I` = اطبع الـ Headers بس (طلب HEAD). وتقدر تلزق الحروف القصيرة ورا بعض: `-sIL` = `-s -I -L`.

3 `L` الـ `-L` = Location: "لو الرد Redirect، روح على الـ `Location` وكمّل". هتشوف كل رد في السلسلة ورا بعض لحد الـ 200.

4 `https://httpbin.org/redirect/2` عنوان بيحوّلك مرتين قبل ما يوصّلك.

حاجة مشتركة: لزق الـ Flags القصيرة

زي ما شفنا في كتاب الـ Terminal مع `ls -la` و `tar -xzf`، الـ Flags القصيرة (حرف واحد بشرطة واحدة) بتتلزق ورا بعض: `-sIL`. بس خلي بالك: لو فيه Flag محتاج قيمة (زي `-o file` أو `-w '...'`) خليه آخر واحد في المجموعة أو لوحده، عشان اللي بعده هيتفهم إنه قيمته. والطويلة (`--silent`) مبتتلزقش أبداً.

الكود	في كلمة	أول حاجة تعملها
`200` / `201` / `204`	تمام / اتعمل / تمام من غير Body	ولا حاجة، أو اقرا الـ Body
`301` / `308`	اتنقل نهائي	حدّث الرابط عندك

الكود	في كلمة	أول حاجة تعملها
`302` / `307`	اتنقل مؤقتاً	عادي، المتصفح هيتبعه
`304`	نسختك لسه سليمة	ولا حاجة، ده كاش شغال
`400` / `422`	البيانات غلط	اقرا Body الرد وراجع اللي بعته (Payload)
`401`	مين انت؟	Login أو Token في `Authorization`
`403`	مش مسموحلك	صلاحيات، أو CSRF، أو الـ API Key مش ليها الحق ده
`404`	مش موجود	راجع الـ URL حرف حرف، والـ Method
`429`	كتير قوي	استنى `Retry-After`، وقلل الطلبات
`500`	الكود ضرب	لوج السيرفر
`502` / `503` / `504`	اللي قدام التطبيق مش لاقيه / مشغول / زهق	استنى وجرب تاني، أو بلّغ صاحب السيرفر

جدول 8.2 ورقة المراجعة: أهم الأكواد في سطر

ملخص الفصل

- الـ Status code 3 أرقام، والأول هو الفئة: 1 معلومة، 2 نجاح، 3 تحويل، 4 غلطة الـ Client، 5 غلطة السيرفر.
- 201 بعد إنشاء ومعاه `Location`، و 204 من غير Body (متعملش `.json()`).
- 301/308 نهائي و 302/307 مؤقت، و 307/308 بيحافظوا على الـ Method. و 304 = استخدم الكاش.
- 401 = مين انت (Login)، و 403 = مش مسموحلك (صلاحية)، و 404 = مش موجود، و 429 = بطّأ.
- 500 من التطبيق، و 502/504 من الـ Proxy اللي قدامه، و 503 مؤقت.
- `curl -s -o /dev/null -w '%{http_code}\n'` للرقم بس، و `-I` للـ Headers، و `-L` عشان يتبع التحويلات.

صح ولا غلط؟

1. الـ 304 معناه إن الصفحة اتنقلت لعنوان تاني. صح / غلط
2. لو جالك 403، أول حل إنك تعمل Login تاني. صح / غلط
3. curl بيتبع الـ Redirects لوحده زي المتصفح. صح / غلط

أسئلة مراجعة

1. ليه فكرة "أول رقم = الفئة" ذكية؟ إيه فايدتها لما تقابل كود جديد؟
2. بعد فورم تسجيل بـ POST، تحوّل المستخدم بأنهي كود؟ وليه مش 307؟
3. API رجّع 502 وصاحبك قالك "الكود بتاعك بايظ". ترد تقوله إيه؟
4. اشرح كل حتة في `curl -s -o /dev/null -w '%{http_code}\n' URL`.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اطلع الكود بتاع `https://httpbin.org/status/201` و `/status/429` و `/status/503` بأمر الـ `-w` .
2. متوسط جرّب `curl -sIL http://github.com` و `curl -sIL http://www.google.com` . كام Redirect في كل واحد، ولأنهي عنوان؟
3. صعب اكتب Script صغير في zsh بيلف على 3 مواقع ويطبع جنب كل واحد الكود بتاعه:

```
//httpbin.org/status/500; do echo "$u $(curl -s -o /dev/null -w '%{http_code}' "$u")"; done
```

. واشرح الـ `$()` فيه.

09

المرحلة 3 · لغة الويب: HTTP

الفصل التاسع

الـ Headers: الكلام اللي على الظرف

الـ Headers هي المكان اللي أغلب الحكاية بتحصل فيه: مين انت، وعاوز الرد بأنهي لغة وشكل، والرد متضغوط ولا لأ، ويتحفظ في الكاش قد إيه، وكوكيز، وتصاريح. ولما تفتح أي طلب في DevTools، هتلاقي عشرات الـ Headers. الفصل ده هيخليك تعرف تقرا أهمهم وتعرف كل واحد جه ليه.

بعد الفصل ده هتكون عارف

- فكرة الـ Headers وأنواعهم (طلب، رد، وصف المحتوى)
- Host و User-Agent وحكاية Mozilla/5.0
- الـ Content negotiation: Accept و Accept-Language و Accept-Encoding
- Content-Type والـ MIME types والـ charset (وليه العربي بيطلع حروف غريبة)
- الضغط: gzip و br، وتقيس الفرق بـ curl
- Authorization و Bearer tokens، و Referer و Origin
- Headers الأمان المشهورة، والـ Headers المخصوصة
- تبعت أي Header بـ curl -H وتشوف اللي اتبعت بـ httpbin

9.1 الفكرة: معلومات عن الرسالة، مش الرسالة

فاكر لما قلنا إن HTTP/1.0 ضاف الـ Headers؟ الفكرة كانت حل لمشكلة واضحة: الطرفين محتاجين يقولوا لبعض معلومات **عن** الرسالة، مش جزء منها. زي الجواب في البوسطة: فيه الجواب نفسه جوه، وفيه على الظرف من برّه: لمين، ومن مين، ومستعجل ولا عادي، وقابل للكسر ولا لأ. موظف البوسطة بيتعامل مع الظرف من غير ما يفتح الجواب. والـ Headers هي الكلام اللي على الظرف ده.

وأحلى حاجة في التصميم إنه **مفتوح**: أي حد يقدر يخترع Header جديد، والطرف التاني لو مش فاهمه يتجاهله وخلاص من غير ما حاجة تبوظ. وده اللي خلّى HTTP يتطور 30 سنة من غير ما يتكسر: كل ميزة جديدة (كوكيز، كاش، CORS، أمان) جت كـ Headers جديدة. وعشان كده فيه مئات الـ Headers، بس انت محتاج تعرف حوالي 25 بس، ودول اللي هنمشي عليهم.

شكل أي Header

`Name: value`. الاسم مش حساس لحالة الحروف، وبعده `:` ومسافة اختيارية والقيمة. وفي HTTP/2 و HTTP/3 الأسامي بتتكتب كلها small، وهتلاقي Headers غريبة بتبدأ بـ `:` زي `:method` و `:path` و `:authority`. دول اسمهم **Pseudo-headers**، وهما بالظبط حتت سطر الطلب القديم (الـ Method والـ Path والـ Host) اتحولوا لـ Headers. هتشوفهم في DevTools.

9.2 مين انت؟ Host و User-Agent

`Host` شرحناه: السيرفر الواحد بيشيل مواقع كتير، فلازم تقوله انت عاوز أنهي واحد. ده الـ Header الوحيد الإجباري في HTTP/1.1.

`User-Agent` بقى حكايته طريفة. الفكرة الأصلية: المتصفح يعرّف نفسه، عشان السيرفر يعرف مثلاً يبعت نسخة الموبايل. بس في التسعينيات، متصفح اسمه Netscape (واسمه الكودي Mozilla) كان بيدعم حاجات جديدة، فالمواقع بقت تبص: "لو الـ User-Agent فيه Mozilla، ابعتله الصفحة الحلوة". فلما Internet Explorer جه، عمل نفسه Mozilla عشان ياخد الصفحة الحلوة.

وبعدين Safari عمل نفسه Mozilla و KHTML و Gecko. وبعدين Chrome عمل نفسه كل اللي قبله. والنتيجة الطريفة النهارده: كل متصفح في الدنيا بيقول إنه Mozilla:

تشريح الـ User-Agent

```
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
```

1	`Mozilla/5.0`	"أنا متصفح حديث". كل المتصفحات بتقولها من أيام حرب التسعينيات، ومالهاش معنى حقيقي النهارده.
2	`(Macintosh; Intel Mac OS X 10_15_7)`	نظام التشغيل. ولاحظ: حتى على Mac بـ Apple Silicon و macOS حديث، Chrome بيكتب الرقم ده ثابت عن قصد، عشان يقلل المعلومات اللي بتميزك (Privacy).
3	`AppleWebKit/537.36`	محرك الرسم. Chrome أصله من WebKit (بتاع Safari)، والرقم ده متجمّد من سنين.
4	`(KHTML, like Gecko)`	"أنا زي KHTML وزي Gecko" (محركات تانية)، عشان المواقع القديمة اللي بتدور عليهم.
5	`Chrome/150.0.0.0`	الجزء الحقيقي المفيد: المتصفح ونسخته الرئيسية (الرقم عندك هيختلف). والباقي أصفار عن قصد برضه.
6	`Safari/537.36`	كمان "أنا زي Safari"، لنفس السبب.

متبنيش منطقك على الـ User-Agent

عشان الفوضى دي، الـ User-Agent مش مصدر موثوق، وسهل تغيّره (curl بيبعت `curl/8.7.1`، وتقدر تخليه أي حاجة). لو محتاج تعرف المتصفح يدعم ميزة ولا لأ، اختبر الميزة نفسها في JS: `if ('share' in navigator)`. والمواقع اللي بترفض curl بتبص غالباً على الـ User-Agent، فممكن تجرب `-A 'Mozilla/5.0'` (الفصل 10).

9.3 عاوز الرد شكله إيه؟ الـ Content negotiation

نفس الصفحة ممكن تبقى عند السيرفر بأكتر من شكل: بالعربي وبالإنجليزي، وصورة بصيغة قديمة (JPEG) وجديدة أصغر (AVIF)، ومضغوطة وعادية. طب السيرفر يبعت أنهي واحدة؟ فاتطور حل اسمه **Content negotiation**: الـ Client يقول في الطلب "أنا أفضّل كذا"، والسيرفر يختار، ويقول في الرد اختار إيه.

تشريح Accept

```
Accept-Language: ar-EG,ar;q=0.9,en;q=0.8
```

1	`Accept-Language:`	"اللغات اللي بفهمها، بالترتيب". المتصفح بياخدها من إعدادات اللغة عندك.
2	`ar-EG`	عربي مصري أولاً. الكود: اللغة (`ar`) وبعدها البلد (`EG`).
3	`,`	الفاصلة بتفصل الاختيارات.
4	`ar;q=0.9`	أي عربي، بدرجة تفضيل 0.9. الـ `q` = Quality: رقم من 0 لـ 1، واللي مكتوبش جنبه `q` يبقى 1.

5 `en;q=0.8` وإنجليزي لو مفيش، بدرجة 0.8.

في الطلب	بيسأل عن	والسيرفر بيرد في
`Accept: text/html,application/json;q=0.9,*/*;q=0.8`	نوع المحتوى. و `*/*` = أي حاجة	`Content-Type`
`Accept-Language: ar-EG,ar;q=0.9`	اللغة	`Content-Language`
`Accept-Encoding: gzip, deflate, br, zstd`	الضغط	`Content-Encoding`

جدول 9.1 عائلة الـ Accept

والسيرفر لما بيختار حسب Header معين، المفروض يقول كده في Header اسمه `Vary`، مثلاً `Vary: Accept-Language`. ده مهم للكاش: "متديش نسخة العربي لحد طالب إنجليزي". هنرجعله في الفصل 15.

9.4 Content-Type: إيه اللي جوه؟

ده أهم Header في الرد. المتصفح بيستلم Bytes، ومن غير `Content-Type` مش هيعرف يعرضها صورة ولا يشغلها JavaScript ولا يرسمها HTML. والقيمة اسمها **MIME type** (اتأخذت من الإيميل)، وشكلها `نوع/نوع-فرعي`:

الـ Content-Type	يعني
`text/html; charset=utf-8`	صفحة HTML بترميز UTF-8
`text/javascript` / `text/css`	ملف JS / ملف CSS
`application/json`	بيانات JSON. ده رد أغلب الـ APIs
`image/png` / `image/webp` / `image/avif` / `image/svg+xml`	صور
`application/octet-stream`	"Bytes ومعرفش نوعها": المتصفح هيحمّلها كملف

جدول 9.2 أشهر الـ MIME types

تشريح Content-Type

```
Content-Type: text/html; charset=utf-8
              ①    ②③   ④
```

1. `text` النوع الكبير: نص.
2. `/html` النوع الفرعي: HTML تحديداً.
3. `;` بعدها Parameters إضافية للنوع.
4. `charset=utf-8` الترميز: الـ Bytes دي تتحول لحروف إزاي. UTF-8 هو اللي بيدعم كل لغات الدنيا، والعربي منهم.

> ⚠ **خلي بالك: العربي طالع ØªÙ…Ø§Ù…©**
>
> لو شفت صفحة أو JSON فيه حروف غريبة زي `Ø§Ù„` بدل العربي، المشكلة دايماً تقريباً في الـ `charset`: البيانات مكتوبة UTF-8، والمتصفح فاكرها ترميز قديم (زي Latin-1). الحل: السيرفر يبعت `charset=utf-8` في الـ Content-Type، وفي ملفات HTML حط `<meta charset="utf-8">` أول سطر في `<head>`. والظاهرة دي ليها اسم ياباني ظريف: **Mojibake**.

> ⚠ **خلي بالك: Content-Type غلط = حاجات بتتمنع**
>
> المتصفحات بقت صارمة: لو ملف JS جاي بـ `Content-Type: text/plain` أو `text/html`، Chrome هيرفض يشغله ويكتبلك في الـ Console حاجة زي `Refused to execute script ... because its MIME type ('text/html') is not executable`. وده بيحصل كتير لما المسار غلط فالسيرفر يرجع صفحة HTML 404 بدل الملف. فلما تشوف الرسالة دي، بص الأول على الـ Status بتاع الملف.

9.5 الضغط: نفس المحتوى بعُشر الحجم

الـ HTML والـ CSS والـ JS والـ JSON كلهم نص، والنص فيه تكرار كتير (`<div class=` بتتكرر مية مرة). فالضغط بيوفر جامد. المتصفح بيقول `Accept-Encoding: gzip, deflate, br, zstd`، والسيرفر يختار واحد ويضغط ويقول `Content-Encoding: br`، والمتصفح يفك الضغط لوحده. `gzip` هو الأقدم والمدعوم في كل حتة، و`br` (Brotli، من Google) أحسن للنصوص، و`zstd` الأحدث. وتقدر تشوف الفرق بعينك:

تشريح الأمر

```
curl -s --compressed URL -o /dev/null -w '%{size_download}\n'
 1    2        3    4          5
```

1. `curl -s` اطلب بهدوء.
2. `--compressed` ابعت `Accept-Encoding` بالأنواع اللي curl بيدعمها، وفك الضغط لوحدك. من غيرها curl مبيطلبش ضغط خالص.
3. `URL` أي صفحة كبيرة، والمثال تحت بصفحة من MDN.
4. `-o /dev/null` ارمي المحتوى.
5. `-w '%{size_download}\n'` اطبع عدد الـ Bytes اللي اتنزلت فعلاً على الشبكة.

Terminal

```
user@MacBook ~ % curl -s https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview -o
/dev/null -w '%{size_download}\n'
228532
user@MacBook ~ % curl -s --compressed https://developer.mozilla.org/en-
US/docs/Web/HTTP/Guides/Overview -o /dev/null -w '%{size_download}\n'
26548
```

228 كيلو من غير ضغط، و 26 كيلو بالضغط. يعني تقريباً **تُمن الحجم**. وفي DevTools هتلاقي عمود Size بيوريك الحجم اللي اتنقل على الشبكة (المضغوط)، ولو فعّلت Big request rows هيوريك تحته الحجم الحقيقي بعد فك الضغط (الفصل 18).

9.6 Authorization: الكارت بتاعك

لما API يطلب منك تثبت انت مين (فاكر 401؟)، الطريقة القياسية Header اسمه `Authorization`. وفيه شكلين هتقابلهم:

تشريح Authorization

```
1           2       3
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0MiJ9.dBjf…
```

1. `Authorization:` "دي بياناتي اللي بتثبت أنا مين".
2. `Bearer` الـ **Scheme**: نوع البيانات. Bearer يعني "حامله": أي حد معاه الـ Token ده يعتبر هو صاحبه، زي تذكرة السينما. عشان كده الـ Token ده سر.
3. `eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0MiJ9.dBjf…` الـ Token نفسه. الشكل ده (3 حتت بينهم نقط وبيبدأ بـ `eyJ`) اسمه **JWT**، وهنفهمه في الفصل 13.

والشكل التاني الأقدم `Authorization: Basic c2FyYTpzM2NyZXQ=`، والحتة اللي بعد Basic هي `user:password` متحولين بـ Base64، و Base64 **مش تشفير**، أي حد يفكه في ثانية (جرّب `echo c2FyYTpzM2NyZXQ= | base64 -d`). عشان كده Basic مينفعش غير على HTTPS. وفيه APIs كتير بتستخدم Header مخصوص للـ API Key زي `x-api-key: ...`.

> ⚠ **خلي بالك: الـ Tokens في الـ Screenshots**
>
> لما تاخد Screenshot من DevTools عشان تسأل حد أو تسأل الـ AI، أو تعمل Copy as cURL وتلزقه في Issue على GitHub، خلي بالك إن الـ `Authorization` والـ `Cookie` بيبقوا جواه. امسحهم الأول. نفس الكلام لملفات HAR (الفصل 19).

9.7 جاي منين؟ Referer و Origin

لما تدوس على رابط في صفحة، أو الصفحة تطلب صورة، المتصفح بيقول للسيرفر التاني "أنا جاي من الصفحة دي" في Header اسمه `Referer`. وأيوه الاسم مكتوب غلط (المفروض Referrer بـ r اتنين)، الغلطة دخلت الـ Standard سنة 1996 ومحدش يقدر يصلحها دلوقتي لأن كل السيرفرات بتدور على الكلمة الغلط. وده اللي بيخلي أدوات زي Google Analytics تعرف الزيارات جاية منين. والمتصفحات النهارده بتبعت الـ Origin بس (من غير الـ Path) للمواقع التانية افتراضياً، عشان الخصوصية، وده بيتظبط بـ Header اسمه `Referrer-Policy` (المرة دي مكتوب صح!).

و `Origin` Header شبهه بس أقصر وأهم: فيه الـ Origin بس (Port + Host + Scheme) اللي الطلب جاي منه، والمتصفح بيبعته لوحده مع طلبات `fetch` للـ Origins التانية ومع أي POST. وده بالظبط اللي السيرفر بيبص عليه عشان يقرر في الـ CORS (الفصل 14).

9.8 Headers تانية هتشوفها في كل رد

الـ Header	معناه	تفاصيله
`Content-Length: 1256`	حجم الـ Body بالـ Bytes	عشان المستلم يعرف الرسالة خلصت فين
`Date`	وقت الرد بتوقيت GMT	مهم للكاش
`Server: nginx`	نوع السيرفر	بيساعد تعرف مين رد (الفصل 8)
`Location`	العنوان التاني	مع الـ xx3 والـ 201
`Set-Cookie`	"احفظ الكوكي دي"	الفصل 13
`Age` / `Last-Modified` / `ETag` / `Cache-Control`	قواعد الكاش	الفصل 15
`Access-Control-Allow-Origin`	إذن الـ CORS	الفصل 14
`Retry-After: 30`	جرّب تاني بعد 30 ثانية	مع 429 و 503
`Alt-Svc: h3=":443"`	"أنا بدعم HTTP/3 كمان"	الفصل 11

جدول 9.3 Headers الرد المشهورة

Headers الأمان: السيرفر بيحميك

فيه مجموعة Headers وظيفتها إن السيرفر يطلب من المتصفح يشد الأمان على الصفحة دي. مش هتظبطهم دلوقتي، بس هتقابلهم في أي موقع كبير ولازم تعرف معناهم:

`Strict-Transport-Security: max-age=31536000`	اسمه **HSTS**: "لمدة سنة، متكلمنيش غير بـ HTTPS حتى لو كتبت http". الفصل 12.
`Content-Security-Policy: script-src 'self'`	اسمه **CSP**: "متشغلش JS غير من موقعي". بيحمي من حقن الأكواد (XSS). ولو منع حاجة هتشوف رسالة في الـ Console.
`X-Content-Type-Options: nosniff`	"صدّق الـ Content-Type ومتخمنش النوع من المحتوى".
`X-Frame-Options: SAMEORIGIN`	"محدش يحطني جوه `<iframe>` غير موقعي". بيحمي من خدعة اسمها Clickjacking.

لو حابب تتعمق: حكاية X-

تقدر تعدّيه في أول قراية

هتشوف Headers كتير بتبدأ بـ `X-` زي `X-Powered-By: Express` و `X-Request-Id`. زمان الاتفاق كان إن أي Header مخصوص (مش في الـ Standard) يبدأ بـ `X-`. بس المشكلة إن حاجات كتير بدأت مخصوصة وبعدين بقت Standard، واسمها فضل فيه `X-` للأبد. فسنة 2012 اتفقوا (`RFC 6648`) إن الأحسن متستخدمش `X-` للحاجات الجديدة. ونصيحة لما تعمل Express: اقفل `X-Powered-By` بـ `app.disable('x-powered-by')` عشان متقولش للمهاجمين انت شغال بإيه.

9.9 معمل: ابعت Headers وشوف وصلت إزاي

`httpbin.org` عنده عنوان بيرجعلك كل الـ Headers اللي وصلتله، وده أحسن مراية تشوف فيها طلبك:

تشريح الأمر

```
curl -s https://httpbin.org/headers -H 'Accept-Language: ar-EG' -H 'X-Debug: yes' -A 'MyApp/1.0'
```

1. `curl -s` — اطلب بهدوء.
2. `https://httpbin.org/headers` — عنوان بيرد بـ JSON فيه الـ Headers اللي استلمها.
3. `-H 'Accept-Language: ar-EG'` — `-H` = Header: ضيف Header للطلب. الشكل `'Name: value'` بين Quotes عشان المسافة. وتقدر تكرر `-H` قد ما انت عاوز.
4. `-H 'X-Debug: yes'` — Header مخصوص من اختراعك. السيرفر هيستلمه عادي.
5. `-A 'MyApp/1.0'` — `-A` = User-Agent: اختصار لـ `-H 'User-Agent: MyApp/1.0'`.

Terminal

```
user@MacBook ~ % curl -s https://httpbin.org/headers -H 'Accept-Language: ar-EG' -H 'X-Debug: yes' -A 'MyApp/1.0'
{
  "headers": {
    "Accept": "*/*",
    "Accept-Language": "ar-EG",
    "Host": "httpbin.org",
    "User-Agent": "MyApp/1.0",
    "X-Debug": "yes"
  }
}
```

لاحظ إن curl ضاف `Accept: */*` و`Host` لوحده. ولو عاوز تمسح Header curl بيحطه لوحده، ابعته فاضي بنقطتين: `-H 'Accept:'`. وجرب نفس العنوان من المتصفح: افتح `https://httpbin.org/headers` في Chrome، هتلاقي Headers أكتر بكتير، زي `Sec-Ch-Ua` و`Sec-Fetch-Mode`. دول Headers بيبعتها Chrome لوحده وبتبدأ بـ `Sec-` عشان JavaScript ميقدرش يزوّرها.

حاجة مشتركة: -H في كل حتة

الـ `-H` بنفس الشكل `'Name: value'` هي اللي هتلاقيها في أي Copy as cURL من DevTools، وفي توثيق أي API ("`curl -H 'Authorization: Bearer YOUR_KEY'`"). وفي `fetch` بتبقى `headers: { 'Name': 'value' }`. وفي Postman أو أي أداة APIs تابة اسمها Headers. نفس الفكرة بأشكال مختلفة.

ملخص الفصل

- الـ Headers = كلام على الظرف بيوصف الرسالة، والتصميم المفتوح خلّى HTTP يتطور من غير ما يتكسر.
- `Host` إجباري، و`User-Agent` فوضى تاريخية متعتمدش عليه.
- الـ Content negotiation: `Accept*` في الطلب، و`Content-*` في الرد، و`q` للتفضيل، و`Vary` للكاش.
- `Content-Type` أهم Header في الرد، والـ `charset=utf-8` هو علاج العربي البايظ.
- الضغط (`gzip` و`br` و`zstd`) بيوفر لحد 80-90% من حجم النصوص.
- `Authorization: Bearer ...` للـ Tokens، و Basic مجرد Base64 مش تشفير.
- `curl -H` و`-A` وموقع `httpbin.org/headers` كمراية لطلبك.

صح ولا غلط؟

1. Chrome على Mac حديث بيكتب نسخة macOS الحقيقية في الـ User-Agent. صح / غلط
2. Base64 في `Authorization: Basic` بيشفّر الباسورد. صح / غلط
3. لو السيرفر مفهمش Header معين، الطلب كله بيفشل. صح / غلط

أسئلة مراجعة

1. اشرح `Accept-Language: en-US,en;q=0.9,ar;q=0.7` : المتصفح عاوز إيه بالترتيب؟
2. صفحتك العربي طالعة حروف غريبة. إيه السبب الأشهر، وإيه الحلين؟
3. إيه الفرق بين `Content-Type` و`Content-Encoding` ؟
4. ليه Chrome بيرفض يشغل ملف JS أحياناً ويقول إن الـ MIME type غلط؟ وأول حاجة تشيك عليها؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح `https://httpbin.org/headers` في Chrome وقارن الـ Headers بتاعته باللي curl بيبعتها.
2. متوسط قيس حجم 3 صفحات كبيرة من غير `--compressed` وبيها. احسب نسبة التوفير لكل واحدة.
3. صعب اعمل `curl -sI` لـ `https://github.com` و`https://www.google.com`، ولخّص كل Header أمان لقيته (HSTS و CSP و X-Frame-Options) وقيمته بتقول إيه.

المرحلة 3 · لغة الويب: HTTP

الفصل العاشر

معمل curl: متصفح في سطر واحد

استخدمنا curl في كل فصل لحد دلوقتي بشكل متفرق. الفصل ده بقى نجمع فيه كل حاجة: نعمل كل أنواع الطلبات على API حقيقي للتجارب، ونبعت JSON وفورمز وملفات، ونحفظ كوكيز، ونقيس الوقت بالملي ثانية لكل مرحلة من مراحل الرحلة، ونعمل طلبات بتفشل بصوت عالي في الـ Scripts. كل Flag متشرّح.

بعد الفصل ده هتكون عارف

- ليه curl أداة أساسية جنب DevTools مش بديل ليها
- قراية الرد: i- و I- و v- والفرق بينهم
- CRUD كامل على API: X- و H- و d- و json--
- رفع الملفات بـ F-، والـ Query بـ G- و data-- urlencode
- التحويلات والملفات: L- و o- و O-
- الهدوء والفشل الصريح: s- و S- و f- و fail-with-- body
- قياس الوقت بـ w- (DNS و Connect و TLS و TTFB)
- الكوكيز والـ Login بـ u- و k- والـ Timeouts و resolve--

10.1 ليه curl لما عندي متصفح؟

سؤال منطقي. المتصفح بيعمل طلبات طول اليوم، و DevTools بيوريك كل حاجة. ليه أتعب نفسي بالـ Terminal؟ عشان المتصفح بيعمل حاجات كتير من ورا ضهرك: بيبعت كوكيز لوحده، وبيستخدم الكاش، وبيطبّق CORS، وعنده Extensions ممكن تعدّل الطلبات، وبيتبع الـ Redirects من غير ما يقولك. فلما حاجة تبوظ، صعب تعرف هل المشكلة في السيرفر ولا في حاجة من دول. والحل: **تجرب الطلب "عريان"**، من غير أي حاجة زيادة. لو اشتغل بـ curl ومش شغال في المتصفح، يبقى المشكلة في الحاجات اللي المتصفح بيضيفها (غالباً CORS أو كوكيز أو كاش). ولو مش شغال بـ curl كمان، يبقى المشكلة في السيرفر أو في طلبك نفسه.

وكمان curl بيتكتب في Scripts، وبيتبعت لزميلك في رسالة، وتوثيق أي API في الدنيا بيكتب أمثلته بـ curl. وأحلى حاجة: DevTools يقدر يحوّلك أي طلب لأمر curl بزرار (Copy as cURL، الفصل 19). فاتعلمه كويس، هتستخدمه كل أسبوع.

> **الـ API اللي هنجرب عليه**
>
> هنستخدم `jsonplaceholder.typicode.com` : API مجاني للتجارب فيه `/posts` و `/users` و `/todos` ، وبيقبل كل الـ Methods. بس خلي بالك: هو بيرد كأنه نفّذ (يرجعلك 201 والبيانات)، لكنه **مش بيحفظ** حاجة فعلاً. ومعاه `httpbin.org` اللي بيرجعلك طلبك كمراية.

10.2 تقرا الرد: i- و I- و v-

من غير أي Flags، curl بيطبع الـ Body بس. بس انت غالباً محتاج تشوف الـ Status والـ Headers. وفيه تلات درجات:

```
curl -i URL
```

`-i` = Include: اطبع Headers الرد **وبعدها** الـ Body. الطلب نفسه عادي (GET).

`curl -I URL`	`-I` = Head: ابعت طلب `HEAD`، فالرد Headers بس. أسرع، بس خلي بالك إن سيرفرات قليلة بترد على HEAD بشكل مختلف عن GET.
`curl -v URL`	`-v` = Verbose: كل حاجة: الـ DNS والاتصال والـ TLS، والطلب اللي اتبعت (`>`)، والرد (`<`)، والـ Body. ده اللي بتستخدمه في الـ Debugging.

حاجة مشتركة: i صغيرة و I كبيرة

زي كتير من أدوات Unix، الحرف الصغير والكبير ممكن يبقوا حاجتين مختلفين خالص في نفس الأداة: `-i` (ضمّن الـ Headers) و`-I` (HEAD بس)، و`-o` (احفظ باسم أنا بختاره) و`-O` (احفظ بنفس اسم الملف)، و`-s` (اسكت) و`-S` (بس قولي الأخطاء). فلو أمر طلع ناتج غريب، أول حاجة تشيك عليها حالة الحرف.

10.3 CRUD كامل: اقرا واعمل وعدّل وامسح

اقرا: GET هو الافتراضي، فمش محتاج تكتب Method:

Terminal

```
user@MacBook ~ % curl -s https://jsonplaceholder.typicode.com/posts/1
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur ..."
}
```

اعمل: POST بـ JSON. ودي الصيغة اللي هتكتبها كتير، فخلينا نفصّصها كويس:

تشريح الأمر

```
curl -i -X POST https://jsonplaceholder.typicode.com/posts -H 'Content-Type:
application/json' -d '{"title":"First post","userId":1}'
```

1. `curl -i` — اطلب، واطبع Headers الرد مع الـ Body عشان نشوف الـ Status.
2. `-X POST` — `-X` = Request method: استخدم الـ Method ده. (الحقيقة: لما تستخدم `-d`، curl بيعمل POST لوحده، فـ `-X POST` هنا للوضوح بس.)
3. `https://jsonplaceholder.typicode.com/posts` — الـ Collection: "ضيف بوست جديد للبوستات".
4. `-H 'Content-Type: application/json'` — قول للسيرفر إن الـ Body JSON. فاكر الغلطة المشهورة في الفصل 7؟ هنا نفس الكلام.
5. `-d` — `-d` = Data: الـ Body. من غير الـ Header اللي فوق، curl بيبعته على إنه `application/x-www-form-urlencoded`.

6	`'{"title":"First post","userId":1}'`	الـ JSON نفسه بين Single quotes، عشان الـ Double quotes اللي جوه JSON متتلخبطش مع الـ zsh.

```
Terminal
HTTP/2 201
content-type: application/json; charset=utf-8
location: https://jsonplaceholder.typicode.com/posts/101
etag: W/"3e-Jwo8gJL384l8HW9eGTvv/o+Y2dI"

{
  "title": "First post",
  "userId": 1,
  "id": 101
}
```

شوف الحاجات اللي اتعلمناها كلها في رد واحد: `201` لأن حاجة اتعملت، و`location` فيه عنوان الحاجة الجديدة، و`id: 101` اتعمل من السيرفر. وفيه اختصار أحدث بيعمل الـ Header والـ Body مع بعض:

تشريح الأمر

```
curl --json '{"title":"First post","userId":1}'
https://jsonplaceholder.typicode.com/posts
```

1	`curl`	اطلب.
2	`--json`	اختصار (من curl 7.82) بيعمل 3 حاجات مرة واحدة: `-d` للـ Body، و`Content-Type: application/json`، و`Accept: application/json`. والـ POST تلقائي.
3	`'{"title":"First post","userId":1}'`	الـ JSON.
4	`https://jsonplaceholder.typicode.com/posts`	العنوان. لاحظ إن ترتيب الـ Flags والـ URL في curl مش مهم.

عدّل وامسح: نفس الفكرة بـ Methods تانية:

`curl -X PATCH --json '{"title":"New title"}' https://jsonplaceholder.typicode.com/posts/1`	عدّل جزء. و`-X` هنا لازم، عشان `--json` لوحده بيعمل POST.
`curl -X PUT --json '{"id":1,"title":"x","userId":1}' https://jsonplaceholder.typicode.com/posts/1`	استبدل كله. لاحظ إن PUT بيبعت الـ Object كامل.

امسح. الرد `200` ببوست فاضي `{}` هنا، وفي APIs تانية `204`.	`curl -i -X DELETE https://jsonplaceholder.typicode.com/posts/1`

JSON من ملف بدل ما تكتبه

لو الـ JSON طويل، اكتبه في ملف `post.json` وابعته بـ `@`:

تشريح الأمر

```
curl --json @post.json https://jsonplaceholder.typicode.com/posts
```

1. `curl --json` ابعت JSON.
2. `@post.json` الـ `@` في curl معناها "اقرا القيمة من الملف ده" بدل ما تكون مكتوبة. بتشتغل مع `-d` و`--json` و`-F`. و `@-` = اقرا من الـ stdin (اللي جاي من Pipe).
3. `https://jsonplaceholder.typicode.com/posts` العنوان.

مثال عملي: JSON مقروء بـ jq

ردود الـ APIs بتيجي JSON في سطر واحد طويل أحياناً. أداة `jq` (موجودة في macOS 15 وطالع، ولو مش عندك: `brew install jq`) بتنسقه وتلونه وتطلع منه اللي انت عاوزه: `curl -s https://jsonplaceholder.typicode.com/users | jq '.[0].email'`. الـ `.` = الـ JSON كله، و `[0]` = أول عنصر في الـ Array، و `.email` = خاصية الإيميل. يعني نفس اللي بتكتبه في JS: `data[0].email`.

10.4 فورمز وملفات و Query

مش كل حاجة JSON. لو بتجرب فورم قديم، أو بترفع ملف، أو عاوز تبني Query string بأمان:

تشريح الأمر

```
curl -F 'name=Sara' -F 'avatar=@me.png;type=image/png' https://httpbin.org/post
```

1. `curl` اطلب.
2. `-F 'name=Sara'` `-F` = Form: ابعت `multipart/form-data` (زي فورم فيه ملفات). وكل `-F` حقل.
3. `-F 'avatar=@me.png;type=image/png'` حقل اسمه `avatar` قيمته **محتوى** الملف `me.png` (الـ `@` تاني)، و `;type=` بتحدد الـ Content-Type بتاع الجزء ده. لو مكتبتهاش curl بيخمّن من الامتداد.
4. `https://httpbin.org/post` هيرجعلك الـ `form` والـ `files` اللي استلمها.

تشريح الأمر

```
curl -G https://httpbin.org/get --data-urlencode 'q=rock & roll' --data-urlencode 'lang=ar'
```

1. `curl -G` = `-G`: Get = "خد أي `--data-urlencode` أو `-d` وحطهم في الـ Query string بدل الـ Body، والطلب يبقى "GET".
2. `https://httpbin.org/get` العنوان من غير `?`. curl هيضيفها.
3. `--data-urlencode 'q=rock & roll'` اعمل Percent-encoding للقيمة لوحدك (فاكر الفصل 6؟). الناتج `q=rock%20%26%20roll` (وممكن تشوف `+` مكان المسافة، والاتنين صح في الـ Query string). المهم إن الـ `&` بقت `%26` فمش هتتفهم إنها فاصل بين Parameters. وكده مش محتاج تحسب الأكواد بإيدك.
4. `--data-urlencode 'lang=ar'` Parameter تاني. curl هيحط `&` بينهم.

10.5 التحويلات والتحميل: -L و -o و -O

`curl -L http://github.com`	`-L`: اتبع الـ Redirects لحد الآخر (الفصل 8). من غيرها هتاخد الـ 301 وبس.
`curl -L -o logo.svg https://example.com/a/b/logo.svg`	`-o file`: احفظ الـ Body في ملف بالاسم ده.
`curl -LO https://example.com/files/report.pdf`	`-O`: احفظ بنفس اسم الملف اللي في الـ URL (`report.pdf`) في الفولدر الحالي.
`curl -C - -LO https://example.com/big.zip`	`-C -`: كمّل تحميل اتقطع من مكان ما وقف. ده بيستخدم `Range` و `206` اللي في الفصل 8. والـ `-` بعد `-C` معناها "احسب انت وقفت فين".

10.6 الـ Scripts: اسكت، بس افشل بصوت عالي

لما تحط curl في Script، فيه مشكلتين. الأولى: شريط التقدم بيبوظ الناتج. والتانية أخطر: curl بيعتبر أي رد من السيرفر نجاح (زي `fetch` بالظبط!)، يعني لو السيرفر رد 500، curl هيطبع صفحة الخطأ ويخرج بـ Exit code صفر، والـ Script هيكمّل كأن كل حاجة تمام. والحل في تلات Flags:

تشريح الأمر

```
curl -sSf https://httpbin.org/status/500 -o data.json || echo "فشل"
```

1. `-s` Silent: مفيش شريط تقدم ولا رسايل.
2. `S` الـ `-S` = Show error: "حتى لو ساكت، لو حصل خطأ اطبعه". من غيرها `-s` بتخفي الأخطاء كمان.

3 `f` الـ `-f` = Fail: لو الـ Status 400 أو أكتر، متطبعش الـ Body واخرج بـ Exit code غير صفر (22). ده اللي بيخلي الـ Script يعرف إن فيه مشكلة.

4 `-o data.json` احفظ الناتج لو نجح.

5 `|| echo "فشل"` الـ `||` = "لو اللي قبلي فشل (Exit code مش صفر)، نفّذ اللي بعدي". نفس الـ `||` اللي في كتاب الـ Terminal.

Terminal

```
user@MacBook ~ % curl -sSf https://httpbin.org/status/500 -o data.json || echo "فشل"
curl: (22) The requested URL returned error: 500
فشل
user@MacBook ~ % echo $?
0
```

الـ `echo $?` طبع 0 لأنه بيطبع Exit code آخر أمر، وآخر أمر هنا كان `echo "فشل"` اللي نجح.

`--fail-with-body` :احسن الاتنين

مشكلة `-f` إنها بترمي Body الرد، وغالباً السيرفر كاتب فيه سبب الخطأ. من curl 7.76 فيه `--fail-with-body` : يفشل زي `-f` بالظبط، بس يطبعلك الـ Body. وده اللي تستخدمه وانت بتجرب API ومش فاهم ليه بيرجّع 400.

10.7 قيس الرحلة بالملي ثانية

فاكر رحلة الطلب في الفصل 2؟ curl يقدر يقيسلك كل محطة فيها، ودي نفس الأرقام اللي هتشوفها في تابة Timing في DevTools.

اعمل ملف اسمه `timing.txt` فيه:

timing.txt

```
     dns:  %{time_namelookup}s\n
 connect:  %{time_connect}s\n
     tls:  %{time_appconnect}s\n
    ttfb:  %{time_starttransfer}s\n
   total:  %{time_total}s\n
  status:  %{http_code}  ·  %{size_download} bytes  ·  http/%{http_version}\n
```

تشريح الأمر

```
curl -s -o /dev/null -w @timing.txt https://example.com
     1          2      3              4
```

1 `curl -s -o /dev/null` اسكت، وارمي الـ Body.

2 `-w` اطبع بعد ما تخلص.

3 `@timing.txt` نفس الـ `@` اللي فوق: "الشكل اللي تطبعه موجود في الملف ده". أسهل من كتابة سطر طويل.

4	العنوان اللي هنقيسه. `https://example.com`

Terminal

```
user@MacBook ~ % curl -s -o /dev/null -w @timing.txt https://example.com
      dns:  0.012s
  connect:  0.031s
      tls:  0.074s
     ttfb:  0.118s
    total:  0.119s
   status:  200  ·  577 bytes  ·  http/2
```

الأرقام دي بتقيس إيه بالظبط

`time_namelookup`	1 من البداية لحد ما الـ DNS خلص. لو الرقم ده كبير، المشكلة في الـ DNS.
`time_connect`	2 لحد ما الـ TCP Handshake خلص. الفرق بينه وبين اللي قبله ≈ RTT واحد.
`time_appconnect`	3 لحد ما الـ TLS خلص (للـ HTTPS بس). الفرق عن connect = وقت التشفير.
`time_starttransfer`	4 الـ **TTFB** (Time To First Byte): لحد ما أول Byte من الرد وصل. الفرق عن appconnect = وقت السيرفر وهو بيفكر + RTT. لو الرقم ده الكبير، السيرفر بطيء.
`time_total`	5 كله. الفرق عن starttransfer = وقت تحميل الـ Body.

خلي بالك: الأرقام تراكمية

كل رقم من دول محسوب **من بداية الطلب**، مش مدة المرحلة لوحدها. يعني وقت الـ TLS لوحده = `tls - connect` = 0.043 ثانية في المثال. وفي DevTools، تابة Timing بتوريك كل مرحلة لوحدها، فخلي بالك وانت بتقارن.

10.8 حاجات تانية هتحتاجها

`curl -c jar.txt -b jar.txt URL`	`-c file` = Cookie jar: احفظ أي `Set-Cookie` في الملف ده. و `-b file` = ابعت الكوكيز اللي فيه. الاتنين مع بعض = curl بيفتكر الـ Login زي المتصفح. و `-b 'sid=abc'` تبعت كوكي بإيدك.
`curl -u sara:s3cret URL`	`-u` = User: بيعمل `Authorization: Basic` لوحده. لو كتبت `-u sara` بس، هيسألك عن الباسورد من غير ما يظهر في تاريخ الأوامر.
`curl -H "Authorization: Bearer $TOKEN" URL`	Token من متغير. هنا لازم **Double** quotes عشان zsh يحط قيمة `$TOKEN`. وده أأمن من إنك تلزق الـ Token في الأمر نفسه.

`curl --max-time 10 --connect-timeout 3 URL`	متستناش للأبد: الطلب كله 10 ثواني بالكتير، وفتح الاتصال 3 ثواني. مهمين في الـ Scripts.
`curl --http1.1 URL` / `curl --http2 URL`	أجبره على نسخة معينة من HTTP (الفصل 11). مفيد لما حاجة شغالة في نسخة ومش شغالة في التانية.
`curl --resolve shop.test:443:127.0.0.1 https://shop.test`	"لما تدور على `shop.test` على Port 443، استخدم الـ IP ده". زي `/etc/hosts` بس للأمر ده بس، ومن غير sudo.
`curl -k https://localhost:8443`	`-k` = Insecure: متتحققش من الشهادة. **للتجربة على جهازك بس** (شهادة Self-signed). على أي موقع حقيقي، ده بيلغي الحماية كلها (الفصل 12).

خلي بالك: الأسرار في تاريخ الـ zsh

أي أمر بتكتبه بيتحفظ في `~/.zsh_history`. يعني لو كتبت `curl -H 'Authorization: Bearer sk-live-...'`، الـ Token بقى محفوظ في ملف نصي على جهازك. الأحسن تحطه في متغير بيتقرا من مكان آمن، أو تبدأ الأمر بمسافة (لو مفعّل `setopt HIST_IGNORE_SPACE` في `~/.zshrc`، الأمر اللي بيبدأ بمسافة مبيتحفظش).

ملخص الفصل

- curl = الطلب من غير إضافات المتصفح، فبيفصل مشاكل السيرفر عن مشاكل المتصفح.
- `-i` Body + Headers، و`-I` طلب HEAD، و`-v` كل حاجة.
- `-X` للـ Method، و`-H` للـ Header، و`-d` للـ Body، و`--json` اختصار JSON، و`@file` من ملف.
- `-F` للفورمز والملفات، و`-G --data-urlencode` للـ Query المتشفّر.
- `-L` اتبع التحويلات، و`-o` / `-O` احفظ، و`-sSf` أو `--fail-with-body` في الـ Scripts.
- `-w` بالمتغيرات `time_*` بيقيس كل محطة، والأرقام تراكمية من البداية.
- `-c` / `-b` للكوكيز، و`-u` للـ Basic، و`--resolve` بدل hosts، و`-k` للتجربة المحلية بس.

صح ولا غلط؟

1. من غير `-f`، curl بيرجّع Exit code صفر حتى لو السيرفر رد 500. صح / غلط
2. `-I` و`-i` نفس الحاجة. صح / غلط
3. في `-w`، قيمة `time_appconnect` هي وقت الـ TLS لوحده. صح / غلط

؟ أسئلة مراجعة

1. إمتى تقول "المشكلة في المتصفح مش في السيرفر" بعد ما تجرب بـ curl؟
2. إيه التلات حاجات اللي `--json` بيعملهم؟ وليه محتاج `-X PATCH` معاه في التعديل؟
3. إيه الفرق بين `-d 'q=a b'` مع `-G` و `--data-urlencode 'q=a b'` مع `-G`؟
4. عندك `dns 0.010` و `connect 0.040` و `tls 0.090` و `ttfb 1.900`. المشكلة فين؟
5. ليه في `-H "Authorization: Bearer $TOKEN"` استخدمنا Double quotes مش Single؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل CRUD كامل على `/todos` في jsonplaceholder: GET لـ todo رقم 5، و POST لواحدة جديدة، و PATCH يخليها `completed: true`، و DELETE. اكتب الـ Status بتاع كل واحد.
2. متوسط اعمل ملف `timing.txt` وقيس 3 مواقع: واحد في مصر، و `example.com`، وواحد في آسيا. قارن الـ TTFB والـ TLS.
3. صعب استخدم `-c` و `-b` مع `https://httpbin.org/cookies/set?name=sara` وبعدين `https://httpbin.org/cookies`: اثبت إن curl فاكر الكوكي في الطلب التاني. (تلميح: الأول بيعمل Redirect، فمحتاج `-L`.)

11

المرحلة 3 · لغة الويب: HTTP

الفصل الحادي عشر

HTTP/1.1 و 2 و 3: نفس اللغة، طرق أسرع

الـ Methods والـ Status codes والـ Headers اللي اتعلمناها ماتغيرتش من التسعينات تقريباً. اللي اتغير هو إزاي الكلام ده بيتنقل على السلك. الصفحات بقت 100 ملف بدل ملف واحد، و HTTP/1.1 بقى بيتخنق. فاتطور HTTP/2 وبعده HTTP/3. هنفهم كل مشكلة والحل بتاعها، ونشوف النسخة في DevTools و curl.

بعد الفصل ده هتكون عارف

- ليه HTTP/1.1 اتخنق مع الصفحات الحديثة
- Head-of-line blocking على مستوى HTTP وعلى مستوى TCP
- HTTP/2: اتصال واحد و Streams و Binary frames و HPACK
- HTTP/3: QUIC فوق UDP وإزاي بيحل مشكلة TCP
- Alt-Svc: إزاي المتصفح يعرف إن فيه HTTP/3
- تعرف النسخة من عمود Protocol و curl -w

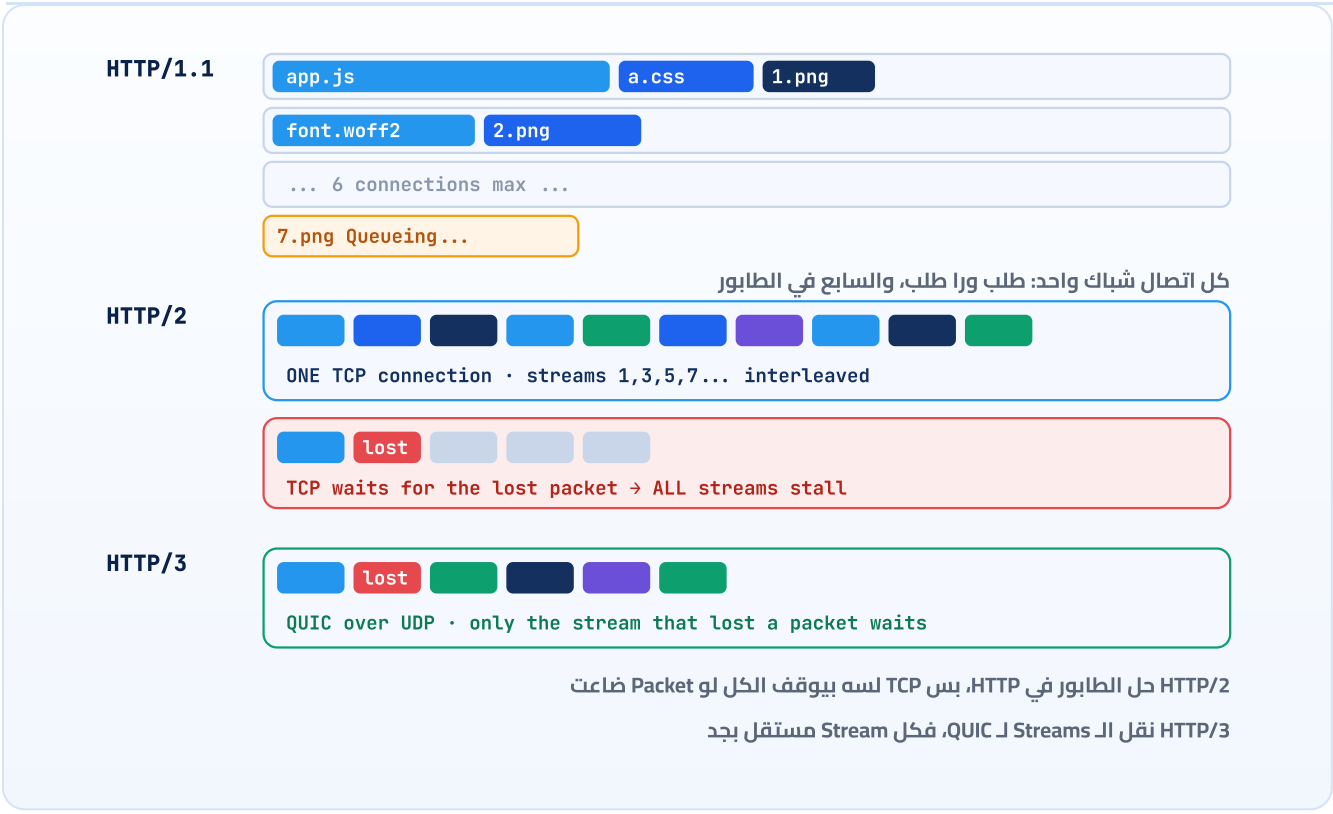
11.1 المشكلة: صفحة واحدة بقت 100 طلب

سنة 1997 لما HTTP/1.1 اتعمل، صفحة الويب العادية كانت ملف HTML وكام صورة. النهارده، افتح أي موقع أخبار وبص على Network: 100 أو 200 طلب. ملفات JS كتير، و CSS، وخطوط، وصور، وإعلانات. وهنا ظهرت مشكلة HTTP/1.1 الكبيرة: **على الاتصال الواحد، طلب واحد في المرة**. تبعت طلب، تستنى الرد كله، وبعدين تبعت اللي بعده. زي شباك واحد في البنك والطابور طويل.

فالمتصفحات عملت حل مؤقت: **افتح 6 اتصالات** لنفس السيرفر في نفس الوقت. 6 شبابيك بدل شباك. وده هو السبب اللي هتشوف بيه في DevTools حاجة اسمها **Queueing** (الفصل 18): الطلب السابع واقف مستني شباك يفضى. والمطورين عملوا حيل أكتر: يجمعوا كل ملفات JS في ملف واحد كبير (Bundling)، ويحطوا كل الأيقونات في صورة واحدة (Sprites)، ويوزعوا الملفات على كذا Domain (`static1.site.com` ، `static2...`) عشان ياخدوا 6 اتصالات من كل واحد. حيل، مش حلول.

Head-of-line blocking: العربية المعطلة في أول الطابور

المشكلة ليها اسم: **Head-of-line blocking** (الحجز من أول الطابور). لو أول طلب في الطابور ملف كبير أو السيرفر بطيء فيه، كل اللي وراه واقفين، حتى لو صغيرين وجاهزين. زي ما عربية واحدة معطلة في حارة واحدة بتوقف الحارة كلها.



شكل 11.1 تلات طرق لنقل نفس الطلبات: 6 طوابير، ثم اتصال واحد متداخل، ثم Streams مستقلة فعلاً

11.2 HTTP/2: اتصال واحد، طلبات كتير مع بعض

جوجل جربت بروتوكول اسمه **SPDY** سنة 2009، ونجح، فبقى أساس **HTTP/2** (سنة 2015، دلوقتي RFC 9113). الفكرة الأساسية: بدل ما الطلب ينزل على الاتصال كتلة واحدة، **قطّعه حتت صغيرة** اسمها **Frames**، وكل Frame عليه رقم الطلب بتاعه (اسمه **Stream ID**). وكده تقدر تبعت حتة من طلب 1، وحتة من طلب 3، وحتة من 5، على نفس الاتصال في نفس الوقت، والناحية التانية تجمّع كل طلب من الحتت بتاعته. ده اسمه **Multiplexing**. اتصال واحد بيعمل شغل 6 وأكتر، والـ Queueing تقريباً اختفى.

Binary بدل نص

HTTP/1.1 كان نص تقدر تكتبه بإيدك في nc (عملناها في الفصل 4). HTTP/2 Binary: أسرع في القراية للأجهزة، بس انت مش هتقدر تكتبه بإيدك. عشان كده curl و DevTools بيترجموه لشكل نصي ليك.

HPACK: ضغط الـ Headers

كل طلب بيبعت نفس الـ User-Agent والكوكيز تاني وتاني، يعني مئات البايتات متكررة. HPACK بيعمل جدول بالـ Headers اللي اتبعتت، والمرة الجاية يبعت "رقم 62" بدل السطر كله.

Pseudo-headers

السطر الأول اتقسم لـ `:method` و `:path` و `:scheme` و `:authority` (الفصل 9)، وأسماء الـ Headers بقت **small letters** إجباري. عشان كده بتشوف `content-type` مش `Content-Type` في ردود HTTP/2.

من غير Reason phrase

سطر الحالة بقى `HTTP/2 200` من غير "OK". الكلمة كانت زينة أصلاً، والبرامج بتقرا الرقم بس.

الحيل القديمة بقت مضرة أحياناً

مع HTTP/2، تقسيم الملفات على كذا Domain بقى بيضر، لأن كل Domain = اتصال جديد = DNS و TCP و TLS من الأول، والمتصفح كان ممكن يستخدم اتصال واحد للكل. والـ Bundling لسه مفيد بس بشكل أخف. فلو قريت نصيحة أداء قديمة من 2012، اسأل نفسك: هي لسه صح مع HTTP/2؟

11.3 HTTP/3: المشكلة كانت في TCP نفسه

HTTP/2 حل الطابور على مستوى HTTP. بس فاكر في الفصل 4 إن TCP بيضمن إن البيانات توصل **بالترتيب**؟ هنا بقت مشكلة. TCP ميعرفش حاجة عن Streams. هو شايف شريط بايتات واحد. فلو Packet واحدة ضاعت (وده بيحصل كتير على Wi-Fi و G4)، TCP بيوقف **كل حاجة بعدها** لحد ما تتبعت تاني، حتى لو اللي بعدها تبع Streams تانية خالص وكاملة. يعني Head-of-line blocking رجع، بس من تحت.

والحل كان جريء: **سيب TCP خالص**. جوجل عملت **QUIC** فوق **UDP** (اللي مبيضمنش حاجة)، وبنت جواه كل اللي محتاجينه: الترتيب والتأكد من الوصول، بس **لكل Stream لوحده**. فلو Packet ضاعت من Stream الصورة، الـ JS والـ CSS بيكملوا عادي. وده بقى **HTTP/3** (سنة 2022، RFC 9114).

Streams مستقلة بجد

Packet ضاعت = Stream واحد بيستنى، مش الكل.

Handshake أسرع

QUIC بيدمج الاتصال والـ TLS 1.3 في خطوة واحدة: RTT واحد بدل اتنين قبل أول طلب (الفصل 12).

الاتصال بيعيش لما الشبكة تتغير

نزلت من الـ Wi-Fi للـ G4؟ مع TCP الاتصال بيموت لأن الـ IP اتغير. QUIC عنده Connection ID بيفضل زي ما هو.

المتصفح بيعرف منين إن فيه HTTP/3؟

المتصفح مبيقدرش يجرب HTTP/3 على طول، لأنه UDP على Port 443، وممكن يكون مقفول في شبكة الشركة. فأول مرة بيروح بـ HTTP/2 العادي، والسيرفر بيقوله في الرد: "على فكرة، أنا بعرف HTTP/3":

تشريح الـ Header

```
alt-svc: h3=":443"; ma=86400
```

1. `alt-svc` — Alternative Service: "فيه طريقة تانية توصلي بيها".
2. `h3` — البروتوكول: HTTP/3.
3. `=":443"` — على نفس الـ Host، Port 443 (بس UDP المرة دي).
4. `ma=86400` — Max age: افتكر المعلومة دي 86400 ثانية (يوم). فالطلبات الجاية بتروح HTTP/3 على طول.

وعشان كده أول تحميل لموقع ممكن تلاقيه h2 في DevTools، وبعد ما تعمل Reload تلاقيه h3. ده مش عطل، ده التصميم. وفيه طريقة أحدث كمان: السيرفر يحط المعلومة دي في الـ DNS نفسه (Record اسمه `HTTPS`)، فالمتصفح يعرف من الأول.

11.4 تشوف النسخة بعينك

في DevTools: في Network، اعمل Right-click على عناوين الأعمدة وفعّل **Protocol**. هتلاقي قيم زي `h2` و `h3` و `http/1.1`، وكمان ممكن `data` للحاجات المدمجة في الصفحة (الفصل 18).

في curl:

`curl -sI https://example.com | head -1`	أول سطر في الرد بيقولك النسخة: `HTTP/2 200`.
`curl -s -o /dev/null -w '%{http_version}\n' https://example.com`	النسخة بس: `2`.
`curl --http1.1 -sI https://example.com | head -1`	أجبره على 1.1. هتلاقي `HTTP/1.1 200 OK`، والـ Reason phrase رجع.
`curl -sI https://example.com | grep -i alt-svc`	هل السيرفر بيعلن عن HTTP/3؟
`curl -V | grep -o HTTP3`	هل الـ curl بتاعك بيعرف HTTP/3 أصلاً؟ لو ماطلعش حاجة، يبقى `--http3` مش هيشتغل معاك.

على الـ Mac: curl بتاع Apple

الـ curl اللي جاي مع macOS بتاع Apple نفسها، وغالباً مش متبني بـ HTTP/3. لو عاوز تجرب `--http3`، نزّل نسخة Homebrew: `brew install curl`. وخلي بالك إن Homebrew مش بيحطها مكان بتاعة النظام (عشان متكسرش حاجة)، فهتلاقي المسار بتاعها في رسالة التثبيت، زي `/opt/homebrew/opt/curl/bin/curl`. وفيه طريقة أسهل: افتح الموقع في Chrome وبص على عمود Protocol.

11.5 النهارده: مين بيستخدم إيه؟

حسب تقرير Cloudflare لسنة 2025، توزيع الطلبات اللي عدت عليهم كان تقريباً: **HTTP/2 حوالي 50%**، و**HTTP/1.x حوالي 29%** (أغلبه Bots و APIs وبرامج قديمة)، و**HTTP/3 حوالي 21%**. يعني الـ 3 نسخ عايشين مع بعض، والمتصفح والسيرفر بيتفقوا على أحسن نسخة الاتنين بيعرفوها. وده سبب إن التلاتة لازم تكون عارفهم.

الحاجة	HTTP/1.1	HTTP/2	HTTP/3
السنة	1997	2015	2022
فوق إيه	TCP	TCP	QUIC فوق UDP
الشكل	نص	Binary frames	Binary frames
طلبات متوازية	طلب لكل اتصال، فـ 6 اتصالات	Multiplexing على اتصال واحد	Multiplexing، و Streams مستقلة
HOL blocking	على مستوى HTTP	على مستوى TCP بس	محلول
ضغط الـ Headers	لأ	HPACK	QPACK

الحاجة	HTTP/1.1	HTTP/2	HTTP/3
التشفير	اختياري	عملياً إجباري في المتصفحات	إجباري (TLS 1.3 جواه)
في DevTools	`http/1.1`	`h2`	`h3`

جدول 11.1 ملخص: 3 نسخ لنفس اللغة

لو حابب تتعمق: ليه مش عدلوا TCP وخلاص؟

تقدر تعدّيه في أول قراية

سؤال منطقي. المشكلة إن TCP متبني جوه أنظمة التشغيل نفسها (Kernel)، وجوه ملايين الراوترات والـ Firewalls حوالين العالم اللي "فاهمة" شكل TCP وممكن ترمي أي حاجة شكلها غريب. فأي تعديل في TCP محتاج سنين لحد ما يوصل لكل الأجهزة. ده اسمه **Ossification** (التكلّس). لكن UDP بسيط، وكل الأجهزة بتعديه. فـ QUIC اتبنى فوق UDP، ومتشفّر بالكامل حتى في الـ Headers بتاعته، عشان الأجهزة اللي في النص متقدرش تقراه ولا "تتعود" على شكله، فيقدر يتطور بسرعة.

ملخص الفصل

- اللغة (Methods و Status و Headers) واحدة في الـ 3 نسخ. اللي بيتغير طريقة النقل.
- HTTP/1.1: طلب واحد لكل اتصال، فالمتصفح بيفتح 6 اتصالات، والزيادة بتقف في Queueing.
- HTTP/2: Frames و Streams على اتصال واحد (Multiplexing)، و HPACK، وأسماء Headers صغيرة.
- TCP لسه بيوقف كل الـ Streams لو Packet ضاعت. HTTP/3 حل ده بـ QUIC فوق UDP.
- المتصفح بيعرف بـ HTTP/3 من `alt-svc` أو من DNS، فأول زيارة ممكن تبقى h2.
- تشوف النسخة من عمود Protocol في DevTools أو `curl -w '%{http_version}'`.

صح ولا غلط؟

1. HTTP/2 غيّر أسماء الـ Status codes. صح / غلط
2. HTTP/3 شغال فوق UDP مش TCP. صح / غلط
3. لو أول زيارة طلعت h2 وبعد Reload طلعت h3، يبقى فيه عطل. صح / غلط

أسئلة مراجعة

1. اشرح Head-of-line blocking بمثال من حياتك، وإزاي HTTP/2 خففه.
2. ليه HTTP/2 لوحده مكفاش على شبكات الموبايل؟
3. إيه معنى `alt-svc: h3=":443"; ma=86400`؟
4. ليه بتشوف أسماء الـ Headers small letters في ردود كتير؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح 3 مواقع كبيرة في Chrome، وفعّل عمود Protocol. مين بيستخدم h3؟ ومين لسه http/1.1؟
2. متوسط قارن `curl --http1.1 -sI` بـ `curl -sI` على نفس الموقع. إيه الفروق اللي شايفها في الشكل؟
3. صعب افتح موقع كبير في Chrome بـ Disable cache، وبص على عمود Waterfall. هل شايف Queueing؟ لو الموقع h2، ليه ممكن تشوف Queueing برضه؟ (تلميح: Domains مختلفة، والأولوية Priority.)

12

المرحلة 4 · الأمان والحالة

الفصل التاني عشر

HTTPS و TLS: القفل اللي جنب العنوان

كل اللي بعتناه لحد دلوقتي بيعدي على راوترات ناس منعرفهمش: الكافيه، ومزوّد الإنترنت، ودول كاملة. HTTP العادي نص مكشوف لأي حد في النص. الفصل ده عن الحل: TLS. هنفهم المشكلتين اللي بيحلهم (حد يقرا، وحد ينتحل)، والـ Handshake خطوة بخطوة، والشهادات وسلسلة الثقة، وإزاي تقرا كل ده في curl و openssl و Chrome، وأشهر أخطاء الشهادات.

بعد الفصل ده هتكون عارف

- المشكلتين: التصنت والانتحال، وليه التشفير لوحده مش كفاية
- فكرة المفتاحين (Public/Private) والمفتاح المشترك
- TLS 1.3 Handshake خطوة بخطوة، و RTT-0
- الشهادة وسلسلة الثقة والـ CAs و Let's Encrypt
- عمر الشهادات اللي بيقصر: 200 يوم ثم 100 ثم 47
- HSTS والـ Mixed content
- تقرا TLS في curl -v و openssl s_client و Chrome
- أخطاء NET::ERR_CERT_* ومعناها

12.1 المشكلة: كلامك ماشي في الشارع مكشوف

افتكر رحلة الطلب: الـ Packets بتعدي على راوتر الكافيه، وبعدين مزوّد الإنترنت، وبعدين 10 أو 15 راوتر تانيين بتوع شركات مختلفة. مع HTTP العادي، كل واحد من دول يقدر يقرا كل حاجة: الباسورد اللي كتبته، والكوكي بتاع الـ Session، والصفحة اللي بتقراها. ومش يقرا وبس، يقدر **يعدّل**: يحقن إعلانات في الصفحة (ومزوّدين إنترنت عملوا كده فعلاً)، أو يغيّر رابط التحميل لبرنامج فيه فيروس.

طب الحل سهل: شفّر الكلام. بس هنا فيه مشكلة أخبث: **انت بتشفّر لمين؟** لو الراوتر اللي في الكافيه قال للمتصفح "أنا `bank.com`، ابعتلي مشفر بالمفتاح بتاعي"، المتصفح هيشفّر كويس جداً... للحرامي. ده اسمه **Man-in-the-middle** (حد في النص). فالحل محتاج حاجتين مش حاجة:

1. التشفير (Confidentiality)

محدش في النص يقدر يقرا.

2. السلامة (Integrity)

محدش يقدر يعدّل حرف من غير ما يتكشف.

3. الهوية (Authentication)

انت متأكد إنك بتكلم `bank.com` الحقيقي، مش حد بيقلده.

البروتوكول اللي بيعمل التلاتة اسمه **TLS** (Transport Layer Security)، واسمه القديم **SSL** (لسه الناس بتقول "شهادة SSL" بالعادة). و**HTTPS** = HTTP عادي بالظبط، بس ماشي جوه نفق TLS. نفس الـ Methods والـ Headers اللي اتعلمناها، بس محدش يشوفهم غير الطرفين.

12.2 فكرة المفتاحين: قفل أي حد يقفله، ومفتاح مع واحد بس

مشكلة التشفير القديمة: لو أنا وانت عاوزين نتكلم مشفر، لازم نتفق على مفتاح سري الأول. بس إزاي نتفق عليه والخط نفسه متراقب؟ لو بعتّه، اتكشف. وهنا جه اختراع من السبعينات غيّر كل حاجة: **التشفير بمفتاحين** (Public-key cryptography).

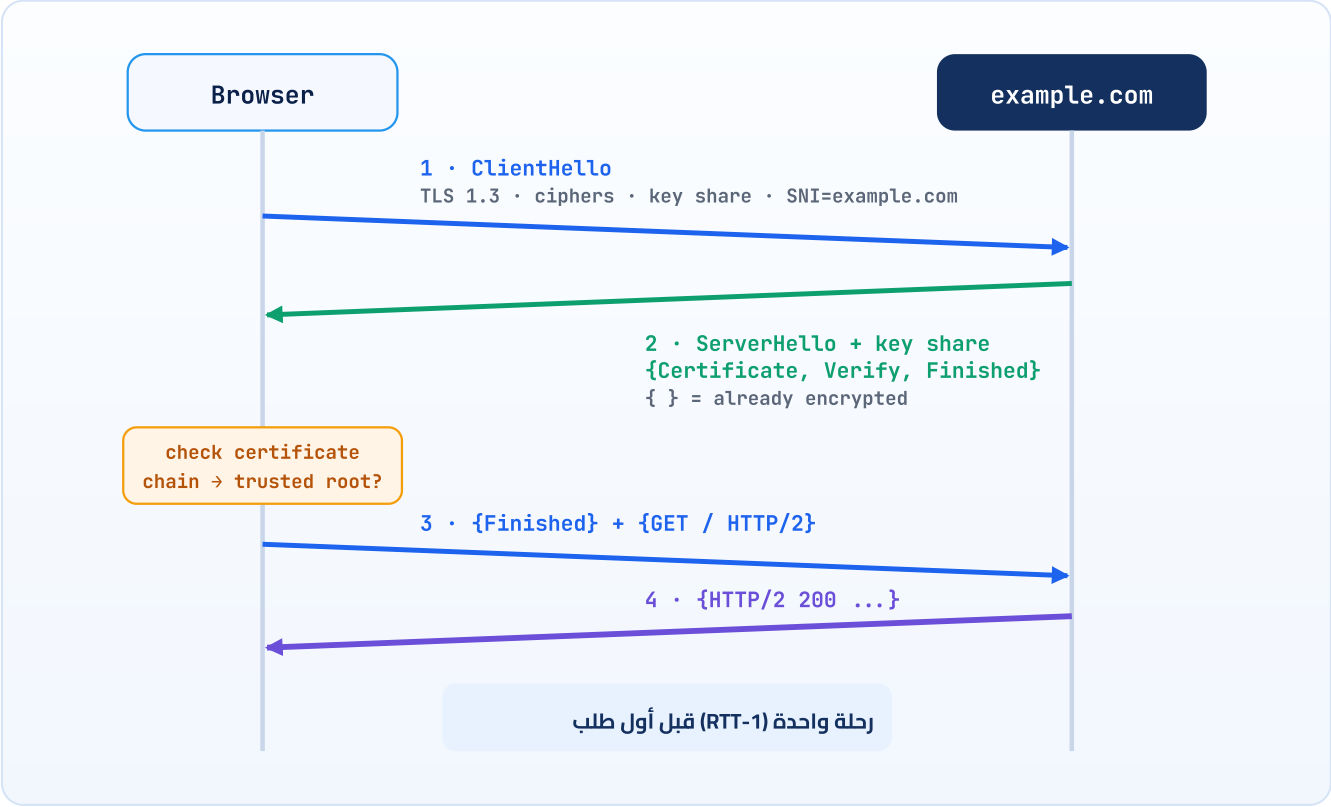
تخيل إن السيرفر معاه **قفل** بيوزّع منه نسخ على أي حد (ده الـ **Public key**)، ومعاه **المفتاح** الوحيد اللي بيفتحه، ومخبيه عنده (ده الـ **Private key**). أي حد يقدر ياخد نسخة من القفل ويقفل بيها صندوق، بس محدش يفتحه غير السيرفر. والعكس كمان مفيد: السيرفر يقدر "يمضي" على حاجة بالـ Private key، وأي حد يتأكد بالـ Public key إن المضا ده بتاعه هو. وده اللي بيحل مشكلة الهوية.

بس التشفير بالمفتاحين بطيء. فـ TLS بيستخدمه في البداية بس، عشان الطرفين يتفقوا بأمان على **مفتاح مشترك** سريع (Session key)، وبعدين كل الكلام بيتشفر بيه. والطريقة اللي بيتفقوا بيها اسمها **Key exchange**، وأشهرها حالياً **X25519**.

> **طريقة الاتفاق اللي عاملة زي السحر**
>
> فيه طريقة اسمها Diffie-Hellman بتخلي اتنين يطلعوا بنفس الرقم السري، وهما بيتكلموا قدام الناس كلها، ومحدش من الناس يقدر يحسبه. التشبيه المشهور: كل واحد عنده لون سري، وبيخلطه مع لون عام ويبعت الخلطة. كل واحد يضيف لونه السري على خلطة التاني، فالاتنين بيطلعوا بنفس اللون النهائي. واللي سامع الكلام شايف الخلطات بس، ومش عارف يفصل الألوان.

12.3 TLS 1.3 Handshake خطوة بخطوة

بعد ما الـ TCP Handshake يخلص (الفصل 4)، المتصفح والسيرفر بيعملوا Handshake تاني بتاع TLS. ونسخة **TLS 1.3** (سنة 2018، RFC 8446) خلته **رحلة واحدة رايح جاي (1-RTT)** بدل اتنين في 1.2:



شكل 12.1 TLS 1.3: المتصفح بيبعت مفتاحه من أول رسالة، فالسيرفر يرد بالشهادة متشفرة، والطلب بيمشي في الرحلة التانية

1. **ClientHello**: المتصفح بيقول "أنا بعرف TLS 1.3، وبعرف طرق التشفير دي، ودي حتة المفتاح بتاعتي (Key share) عشان نتفق". وبيقول كمان اسم الموقع اللي عاوزه في حاجة اسمها **SNI**، لأن السيرفر الواحد ممكن يشيل آلاف المواقع على نفس الـ IP (زي Cloudflare)، ولازم يعرف يطلع الشهادة الصح.
2. **ServerHello**: السيرفر بيختار طريقة، ويبعت حتة المفتاح بتاعته. هنا الاتنين بقى معاهم المفتاح المشترك، فكل اللي بعد كده **متشفّر**: الشهادة، وإمضا بيثبت إن معاه الـ Private key بتاعها (CertificateVerify)، و Finished.
3. **المتصفح بيتحقق** من الشهادة (الجزء الجاي)، ولو تمام يبعت Finished، ومعاه على طول **أول طلب HTTP**.
4. السيرفر يرد. ومن هنا كله HTTP عادي جوه النفق.

> **لو حابب تتعمق: RTT-0 والتشفير الجاي بعد الكمبيوتر الكمّي** — تقدر تعدّيه في أول قراية
>
> **RTT-0**: لو المتصفح كلم السيرفر قبل كده، ممكن يبعت الطلب مع الـ ClientHello نفسه، من غير ما يستنى أي رحلة. أسرع حاجة ممكنة، بس فيها خطر: حد يسجل الرسالة دي ويعيد بعتها (Replay)، فبتستخدم للطلبات الآمنة زي GET بس. **Post-quantum**: لو كمبيوتر كمّي قوي اتعمل في المستقبل، ممكن يكسر X25519، وحد ممكن يسجل كلامك النهارده ويفكه بعد 10 سنين. فالمتصفحات بقت بتستخدم طريقة مختلطة اسمها `X25519MLKEM768`، ودي اللي ظهرت في curl بتاعنا مع example.com. وحسب Cloudflare، أكتر من نص الاتصالات اللي عدت عليهم آخر 2025 كانت بتستخدم النوع ده.

12.4 الشهادة: مين يضمن إن ده bank.com بجد؟

رجعنا للمشكلة: السيرفر بعتلي Public key وقالي "ده بتاع example.com". أصدقه إزاي؟ الحرامي برضه يقدر يعمل مفتاح ويقول نفس الكلام. والحل زي البطاقة الشخصية: انت مش بتصدق إن ده "أحمد" عشان هو قال، بتصدق عشان **الحكومة** (طرف انت واثق فيه) مضت على البطاقة. على الويب، البطاقة اسمها **Certificate**، والحكومة اسمها **CA** (Certificate Authority).

الشهادة ملف فيه: اسم الموقع (أو المواقع)، والـ Public key بتاعه، وتاريخ البداية والنهاية، واسم الـ CA اللي أصدرها، وإمضا الـ CA. ونظام التشغيل والمتصفح جايين معاهم قايمة بحوالي 150 CA موثوقين اسمها **Root store**. على الـ Mac تقدر تشوفها في برنامج **Keychain Access** تحت System Roots.

سلسلة الثقة (Chain of trust)

الـ Root CAs مفاتيحهم غالية جداً ومحفوظة في خزن، مبيستخدموهاش كل يوم. فبيمضوا على CA وسيط (Intermediate)، والوسيط هو اللي بيمضي على شهادات المواقع. فالمتصفح بيمشي في سلسلة: شهادة الموقع ممضية من الوسيط، والوسيط ممضي من الـ Root، والـ Root موجود في القايمة. لو أي حلقة باظت، الاتصال بيقف. ودي السلسلة الحقيقية بتاعة example.com:

Terminal

```
user@MacBook ~ % echo | openssl s_client -connect example.com:443 -servername example.com
2>/dev/null | head -12
CONNECTED(00000003)
---
Certificate chain
 0 s:CN=example.com
   i:C=US, O=SSL Corporation, CN=Cloudflare TLS Issuing ECC CA 3
   v:NotBefore: Sep 26 22:49:11 2026 GMT; NotAfter: Dec 25 22:56:35 2026 GMT
 1 s:C=US, O=SSL Corporation, CN=Cloudflare TLS Issuing ECC CA 3
   i:C=US, O=SSL Corporation, CN=SSL.com TLS Transit ECC CA R2
 2 s:C=US, O=SSL Corporation, CN=SSL.com TLS Transit ECC CA R2
   i:C=US, O=SSL Corporation, CN=SSL.com TLS ECC Root CA 2022
```

تشريح الأمر

```
1    2          3                    4                    5      6
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null |
head -12
```

1. `echo |` ابعت سطر فاضي للأمر اللي بعده. ليه؟ `s_client` بيفتح اتصال ويفضل مستنيك تكتب. السطر الفاضي بيخليه يقفل ويخرج على طول.
2. `openssl s_client` عميل TLS خام: بيعمل Handshake ويطبع كل التفاصيل. زي `nc` بس بالتشفير.
3. `-connect example.com:443` الـ Host والـ Port.
4. `-servername example.com` ابعت SNI بالاسم ده. من غيره، سيرفر زي Cloudflare ممكن ميعرفش يطلعلك أنهي شهادة.
5. `2>/dev/null` ارمي الـ stderr (رسايل التحقق الكتير).
6. `| head -12` أول 12 سطر بس.

تقرا السلسلة إزاي

1 الحلقة 0 = شهادة الموقع نفسه. `s:` = Subject (الشهادة دي بتاعة مين). `CN` = Common Name.	`0 s:CN=example.com`
2 `i:` = Issuer (مين اللي مضى عليها). ده الوسيط. لاحظ إن الـ i هنا هو نفسه الـ s في الحلقة اللي بعدها: ده معنى "سلسلة".	`i:...Cloudflare TLS Issuing ECC CA 3`
3 فترة الصلاحية: من 26 سبتمبر لـ 25 ديسمبر 2026. يعني الشهادة دي عمرها **90 يوم** بس.	`v:NotBefore ... NotAfter`
4 آخر Issuer = الـ Root اللي المفروض يكون في الـ Root store عندك.	`2 ... i:SSL.com TLS ECC Root CA 2022`

على الـ Mac: openssl بتاع Apple

الـ `openssl` اللي جاي مع macOS هو في الحقيقة **LibreSSL** (نسخة متفرعة منه)، والأمر ده شغال عليه عادي. بس لو قريت أوامر openssl أحدث على النت وفيها Flags مش شغالة، نزّل النسخة الأصلية بـ `brew install openssl`. وعشان تشوف الأسماء اللي الشهادة بتغطيها: `... | openssl x509 -noout -text | grep -A1 'Subject Alternative'`، وهتلاقي `DNS:example.com, DNS:*.example.com`.

الشهادة بتغطي أنهي أسماء؟

زمان كان الاسم في `CN`. دلوقتي المتصفحات بتبص على حقل اسمه **SAN** (Subject Alternative Name) بس، وفيه قايمة أسماء. و`*.example.com` = **Wildcard**: أي Subdomain مستوى واحد، زي `www.example.com` و`api.example.com`، بس **مش** `a.b.example.com`. ولو فتحت موقع باسم مش في القايمة، هتاخد `NET::ERR_CERT_COMMON_NAME_INVALID`.

12.5 Let's Encrypt والشهادات اللي عمرها بيقصر

قبل 2015، الشهادة كانت بتتشترى بفلوس (أحياناً مئات الدولارات في السنة)، وتطلبها بإيميلات وأوراق، وتركبها بإيدك. فمواقع كتير كانت HTTP وخلاص. وبعدين جت **Let's Encrypt**: CA مجاني وأوتوماتيكي بالكامل. برنامج على السيرفر بيثبت إنك متحكم في الدومين (مثلاً بيحط ملف معين على الموقع، والـ CA يدخل يشوفه)، وياخد شهادة، ويجددها لوحده. النتيجة: HTTPS بقى الطبيعي، وأغلب الصفحات اللي Chrome بيفتحها دلوقتي HTTPS.

والصناعة ماشية في اتجاه **شهادات عمرها أقصر**، لأن الشهادة لو اتسرقت، كل ما عمرها أقصر كل ما الضرر أقل. منظمة اسمها CA/Browser Forum (الـ CAs والمتصفحات مع بعض) قررت إن أقصى عمر للشهادة:

من تاريخ	أقصى عمر
قبل 15 مارس 2026	398 يوم
15 مارس 2026	200 يوم
15 مارس 2027	100 يوم
15 مارس 2029	47 يوم

جدول 12.1 أقصى عمر لشهادة TLS (قرار CA/B Forum)

الرسالة واضحة: محدش هيجدد شهادة بإيده كل 47 يوم. التجديد لازم يبقى أوتوماتيك. ولو انت مستخدم Vercel أو Netlify أو Cloudflare، ده بيحصل من غير ما تحس.

12.6 HSTS: متجيش هنا من غير قفل تاني

فيه ثغرة صغيرة: لما تكتب `bank.com` في العنوان من غير `https://`، المتصفح ممكن يروح HTTP الأول، والسيرفر يرد بـ 301 لـ HTTPS. بس الطلب الأول ده كان مكشوف، وحد في النص يقدر يخطفه ويفضل يكلمك HTTP. فالسيرفر يبعت Header بيقول: "من هنا ورايح، متكلمنيش غير HTTPS":

تشريح الـ Header

```
1                2                3          4
strict-transport-security: max-age=31536000; includeSubDomains; preload
```

1. `strict-transport-security` اسم الـ Header، واختصاره **HSTS**.
2. `max-age=31536000` افتكر القاعدة دي سنة (بالثواني). طول المدة دي المتصفح هيحوّل أي `http://` لـ `https://` بنفسه، من غير ما يبعت أي حاجة.
3. `includeSubDomains` وكل الـ Subdomains كمان.
4. `preload` "حطوني في القايمة المدمجة". فيه قايمة جوه Chrome نفسه بالمواقع دي، فحتى أول زيارة بتبقى HTTPS.

وخلي بالك من أثر جانبي: الموقع اللي عنده HSTS، لو شهادته فيها مشكلة، Chrome **مش هيسيبك تعدي** (مفيش زرار "Proceed anyway"). وده مقصود. وعشان كده دومينات زي `.dev` و `.app` كلها HSTS إجباري، ولازم HTTPS.

Mixed content

صفحة HTTPS بتحمّل سكريبت أو صورة من `http://`؟ ده اسمه **Mixed content**، ومعناه إن جزء من الصفحة ماشي مكشوف، فحد في النص يقدر يبدّل السكريبت بسكريبت بتاعه ويتحكم في الصفحة كلها. فالمتصفح بيمنع السكريبتات والـ `fetch` دي خالص، والصور بيحاول يحولها لـ HTTPS. وهتشوف ده في الـ Console (الفصل 20)، بس الفكرة: **كل حاجة في صفحة HTTPS لازم تبقى HTTPS**.

12.7 تقرا TLS بعينك

في curl -v، السطور اللي بتبدأ بـ `*` بعد الاتصال:

Terminal

```
user@MacBook ~ % curl -v -o /dev/null https://example.com
* Connected to example.com (104.20.23.154) port 443
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519MLKEM768 / id-ecPublicKey
* ALPN: server accepted h2
* Server certificate:
*  subject: CN=example.com
*  expire date: Dec 25 22:56:35 2026 GMT
*  subjectAltName: host "example.com" matched cert's "example.com"
*  issuer: C=US; O=SSL Corporation; CN=Cloudflare TLS Issuing ECC CA 3
*  SSL certificate verify ok.
```

السطور دي بتقول إيه	
1 **ALPN** = الاتفاق على نسخة HTTP جوه الـ TLS Handshake نفسه. curl بيقول "بعرف h2 و 1.1".	`ALPN: curl offers h2,http/1.1`
2 الخطوة 1 في الرسمة. OUT = طالع مني، IN = جايلي.	`(OUT) Client hello`
3 الخطوة 2: كله جه في رحلة واحدة.	`(IN) Server hello / Certificate / CERT verify / Finished`
4 النسخة، وطريقة تشفير الكلام (AES-256)، وطريقة الاتفاق على المفتاح (المختلطة Post-quantum).	`TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519MLKEM768`
5 اتفقنا على HTTP/2.	`ALPN: server accepted h2`
6 الاسم اللي طلبته موجود في الشهادة.	`subjectAltName: host ... matched`
7 السلسلة كلها سليمة لحد Root موثوق. ده السطر اللي بتدور عليه.	`SSL certificate verify ok.`

السطور دي مختصرة (شلنا سطور زي Change cipher spec و CAfile). والـ curl بتاع Apple على الـ Mac بيطبع السطور دي بشكل مختلف شوية، وطريقة الاتفاق عندك ممكن تطلع X25519 بس. الفكرة واحدة.

في Chrome: اضغط على الأيقونة اللي جنب العنوان، وبعدين "Connection is secure"، وبعدين "Certificate is valid"، هتشوف الشهادة والسلسلة. وفي DevTools فيه تابة اسمها **Security** (ممكن تلاقيها في قايمة `»`) بتوريك النسخة وطريقة التشفير والشهادة لكل Origin في الصفحة. وفي تابة Timing لأي طلب، مرحلة **SSL** هي وقت الـ Handshake ده.

12.8 أخطاء الشهادات: الصفحة الحمرا

لما التحقق يفشل، Chrome بيوقفك بصفحة "Your connection is not private"، وتحتها كود صغير هو أهم حاجة في الصفحة:

الكود	المعنى	السبب الشائع
`NET::ERR_CERT_DATE_INVALID`	الشهادة منتهية أو لسه مبدأتش	التجديد الأوتوماتيك وقف، أو ساعة جهازك غلط
`NET::ERR_CERT_COMMON_NAME_INVALID`	الاسم مش في الشهادة	فتحت `www` والشهادة على الاسم من غيرها، أو العكس
`NET::ERR_CERT_AUTHORITY_INVALID`	مش واصل لـ Root موثوق	شهادة Self-signed، أو السيرفر مش باعت الوسيط
`NET::ERR_CERT_REVOKED`	الشهادة اتلغت	الـ Private key اتسرق أو اتلغت لسبب تاني
`ERR_SSL_PROTOCOL_ERROR`	الـ Handshake نفسه فشل	سيرفر متظبط غلط، أو بتكلم HTTPS على Port بيتكلم HTTP

جدول 12.2 أشهر أخطاء الشهادات في Chrome

مشكلة 1 **الموقع شغال في Chrome، بس curl بيقول certificate problem**

```
curl: (60) SSL certificate problem: unable to get local issuer certificate
```

غالباً السيرفر مش باعت الشهادة الوسيطة. Chrome ساعات بيكمّل السلسلة لوحده (عنده كاش أو بينزلها)، لكن curl والـ Node والبرامج التانية لأ. اتأكد بـ `openssl s_client -showcerts` : لو مفيش غير حلقة 0، يبقى الوسيط ناقص، والحل على السيرفر. **متستخدمش** `-k` كحل: كده انت بتلغي التحقق خالص.

خلي بالك: الكتابة thisisunsafe وأخواتها

هتلاقي نصايح على النت لتجاوز صفحة الخطأ في Chrome. على موقع بتطوره على جهازك بشهادة Self-signed، ماشي. لكن لو ظهرت الصفحة دي على موقع حقيقي، خصوصاً في شبكة عامة، **متعديش**. دي بالظبط اللحظة اللي حد في النص بيحاول فيها، والصفحة دي معمولة عشان توقفه.

HTTPS على جهازك وانت بتطور

أغلب الوقت `http://localhost` كفاية، لأن المتصفحات بتعتبر localhost "Secure context" حتى من غير HTTPS. لو محتاج HTTPS محلي بجد (عشان كوكي `Secure` مثلاً على دومين زي `shop.test`)، فيه أداة اسمها `mkcert` (`brew install mkcert`) بتعمل CA صغير على جهازك ويثق فيه الـ Mac، وتطلع منه شهادات من غير أي تحذيرات.

ملخص الفصل

- HTTP مكشوف لأي حد في النص. TLS بيدي تشفير وسلامة و هوية ، و HTTPS = HTTP جوه TLS.
- المفتاحين (Public/Private) بيستخدموا في البداية بس، عشان يتفقوا على مفتاح مشترك سريع.
- TLS 1.3: رحلة واحدة. ClientHello بالمفتاح و SNI، والسيرفر يرد بالشهادة متشفرة، وبعدين الطلب.
- الشهادة = اسم + Public key + مدة + إمضا CA. والمتصفح بيمشي في السلسلة لحد Root موثوق.
- الأسماء في SAN، و `*.` بتغطي مستوى واحد. والعمر بيقصر: 200 يوم من مارس 2026 ولحد 47 يوم في 2029.
- HSTS بيمنع أي HTTP بعد كده، و Mixed content بيتمنع في صفحات HTTPS.
- كود الخطأ (`NET::ERR_CERT_*`) هو اللي بيقولك المشكلة فين.

صح ولا غلط؟

1. التشفير لوحده من غير التحقق من الهوية بيحميك من Man-in-the-middle. صح / غلط
2. شهادة `*.example.com` بتغطي `api.example.com` . صح / غلط
3. `curl -k` حل مناسب لخطأ الشهادة على موقع حقيقي. صح / غلط

؟ أسئلة مراجعة

1. ليه HTTPS محتاج "هوية" مش تشفير بس؟ احكي سيناريو الكافيه.
2. إيه فايدة الـ SNI؟ وليه Cloudflare بالذات محتاجاه؟
3. في سلسلة example.com، مين مضى على مين؟ ومين المفروض يكون في الـ Root store؟
4. إيه اللي بيحصل لو موقع عنده HSTS وشهادته خلصت؟
5. ليه الصناعة بتقصّر عمر الشهادات؟ وإيه الأثر على أصحاب المواقع؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح شهادة 3 مواقع في Chrome (من الأيقونة جنب العنوان). مين الـ CA؟ وعمر كل شهادة كام يوم؟
2. متوسط استخدم `openssl s_client` على `github.com` و `google.com`، واكتب السلسلة كاملة لكل واحد.
3. صعب جرب المواقع اللي على `badssl.com` (زي `expired.badssl.com` و `wrong.host.badssl.com` و `self-signed.badssl.com`) في Chrome و curl، واكتب كود الخطأ في كل واحد.

13

المرحلة 4 · الأمان والحالة

الفصل التالت عشر

الكوكيز والـ Sessions: إزاي السيرفر بيفتكرك

في الفصل 7 قلنا إن HTTP بينسى: كل طلب لوحده، والسيرفر ميعرفش إنك نفس الشخص اللي عمل Login من ثانية. طب إزاي أي موقع بيفتكرك؟ الحل كان تذكرة صغيرة اسمها Cookie. هنحكي القصة، ونفصّص Set-Cookie وكل Attribute فيه، ونفهم الـ Sessions والـ Tokens، والـ Third-party cookies ووضعها النهارده، وأماكن التخزين التانية في المتصفح.

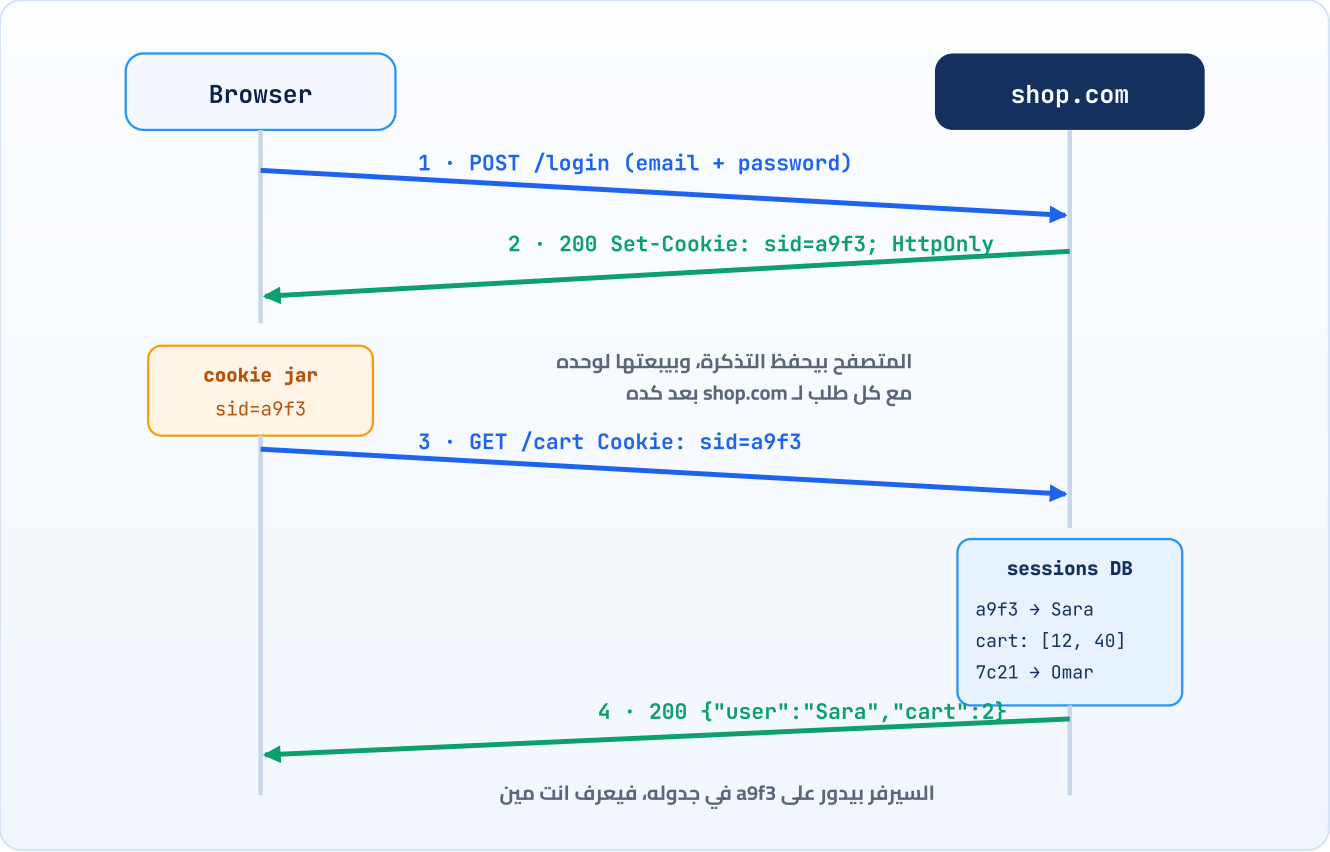
بعد الفصل ده هتكون عارف

- المشكلة: HTTP بينسى، وإزاي الكوكي حل ده
- رحلة الكوكي: Set-Cookie ثم Cookie في كل طلب
- كل Attribute: Domain و Path و Expires و Max-Age و Secure و HttpOnly و SameSite و Partitioned
- Session ID على السيرفر مقابل Token زي JWT
- First-party و Third-party ووضعهم في 2026
- localStorage و sessionStorage و IndexedDB والفرق عن الكوكيز
- تتابع الكوكيز في curl و DevTools

13.1 قبل الكوكيز: عربية تسوق بتفضى كل صفحة

سنة 1994، شركة Netscape كانت بتعمل موقع تسوق لعميل، ووقعت في مشكلة: الزبون يضيف منتج للعربية، يروح للصفحة اللي بعدها، والسيرفر **ميعرفوش**. ده طلب جديد من غير أي ذاكرة، فالعربية فاضية. كان فيه حلول وحشة: تحط رقم الزبون في كل لينك في الـ URL (`?user=4821`)، وده بيتسرب لو بعت اللينك لحد، وبيضيع لو فتحت الموقع من جديد.

فمبرمج اسمه Lou Montulli اقترح فكرة بسيطة جداً: السيرفر يدّي المتصفح **تذكرة صغيرة** في الرد، والمتصفح **يحفظها ويرجّعها لوحده مع كل طلب** لنفس الموقع. زي التذكرة اللي بتاخدها في الجراج: انت مش بتفتكر عربيتك فين، الجراج بيفتكر، وانت معاك الرقم بس. الاسم جه من "Magic cookie"، مصطلح قديم عند المبرمجين لحتة بيانات بتتبعت رايح جاي من غير ما تتفهم.



شكل 13.1 رحلة الـ Session: السيرفر بيدّي تذكرة (Set-Cookie)، والمتصفح بيرجعها (Cookie)، والسيرفر بيدور عليها في جدوله

لاحظ إن الكوكي نفسها مفيهاش "Sara" ولا العربية. فيها رقم عشوائي بس (**Session ID**). المعلومات الحقيقية على السيرفر، في جدول. ده اسمه **Session**: حالة متخزنة على السيرفر، والكوكي هي المفتاح بتاعها.

13.2 Set-Cookie: كل كلمة ليها معنى

الرد بيحط الكوكي بـ Header اسمه `Set-Cookie` (واحد لكل كوكي). وده شكل كوكي Session متظبطة صح:

تشريح الـ Header

```
1          2              3              4          5          6       7
Set-Cookie: sid=a9f3c1; Domain=shop.com; Path=/; Max-Age=86400; Secure; HttpOnly;
8
SameSite=Lax
```

1. `Set-Cookie:` — "احفظ الكوكي دي". وكل كوكي ليها سطر `Set-Cookie` لوحدها.
2. `sid=a9f3c1` — الاسم والقيمة. ده الجزء الوحيد اللي هيرجع في الطلبات. كل اللي بعده تعليمات للمتصفح بس.
3. `Domain=shop.com` — ابعتها لـ `shop.com` وكل الـ **Subdomains** (`api.shop.com`). لو مكتبتهاش، بتتبعت للـ Host اللي حطها بالظبط بس، ودي أضيق وأأمن.
4. `Path=/` — ابعتها مع أي مسار. لو `Path=/admin`، تتبعت مع `/admin/...` بس.
5. `Max-Age=86400` — عيشها يوم (بالثواني). وفيه `Expires=` بتاريخ، ولو الاتنين موجودين، `Max-Age` بيكسب. ولو مفيش ولا واحد = **Session cookie**، بتتمسح لما تقفل المتصفح (أو ممكن تعيش لو المتصفح بيرجّع الـ Tabs القديمة).

6 `Secure` ابعتها على HTTPS بس. على `localhost` مسموح استثناء.

7 `HttpOnly` **الـ JS ميشوفهاش.** `document.cookie` مش هيرجعها. لو حد قدر يحقن سكريبت في الصفحة (XSS)، مش هيقدر يسرق الـ Session.

8 `SameSite=Lax` إمتى تتبعت لو الطلب جاي من موقع تاني (الجزء الجاي).

وفي الطلبات اللي بعد كده، المتصفح بيبعت الأسماء والقيم بس، كلهم في Header واحد مفصولين بـ `;` :

```http
GET /cart HTTP/2
Host: shop.com
Cookie: sid=a9f3c1; theme=dark; lang=ar
```

حاجة مشتركة: الـ ; في الاتنين بس بمعنيين

في `Set-Cookie` ، الـ `;` بتفصل بين الكوكي والتعليمات بتاعتها (`sid=a9f3; Secure; HttpOnly`)، فده كوكي **واحدة**. لكن في `Cookie` ، الـ `;` بتفصل بين كوكيز **مختلفة** (`sid=a9f3; theme=dark`). نفس الرمز، وظيفتين. فلو شفت `Secure` في Header اسمه `Cookie` ، يبقى فيه حاجة غلط.

13.3 SameSite: الحماية من الطلبات اللي جاية من بره

هنا فيه مشكلة تاريخية خطيرة. المتصفح بيبعت الكوكي "لوحده مع كل طلب لنفس الموقع"، حتى لو الطلب ده **موقع تاني هو اللي عمله**. يعني لو انت عامل Login في البنك، ودخلت موقع شرير فيه فورم مخفي بيعمل `POST bank.com/transfer` ، المتصفح هيبعت كوكي البنك معاه، والبنك هيفتكر إنك انت اللي بتحوّل. الهجوم ده اسمه **CSRF** (Cross-Site Request Forgery).

والحل: الكوكي نفسها تقول "متتبعتش مع طلبات جاية من مواقع تانية". وفاكر الفرق بين Origin و Site في الفصل 6؟ هنا بيفرق. SameSite بيبص على الـ **Site**:

القيمة	طلب من نفس الموقع	ضغطت لينك من موقع تاني (Navigation)	fetch أو فورم POST أو صورة من موقع تاني
`Strict`	تتبعت	لأ	لأ
`Lax`	تتبعت	تتبعت (GET بس)	لأ
`None`	تتبعت	تتبعت	تتبعت، **ولازم** معاها `Secure`

جدول 13.1 قيم SameSite

`Strict` أأمن، بس مزعجة: لو حد بعتلك لينك لـ GitHub ودخلت منه، هتلاقي نفسك مش عامل Login لحد ما تعمل Reload. عشان كده `Lax` هي التوازن المعقول. ومن Chrome 2020، بيعامل أي كوكي **من غير SameSite** على إنها `Lax` . وده كسر مواقع كتير وقتها، وبيفسر رسايل هتشوفها في DevTools عن كوكيز "اتمنعت".

Prefixes: أسماء ليها قواعد

لو اسم الكوكي بدأ بـ `__Secure-`، المتصفح مش هيقبلها غير لو معاها `Secure`. ولو بدأ بـ `__Host-`، لازم `Secure` و `Path=/` ومن غير `Domain`، يعني مربوطة بالـ Host ده بس. دي حماية من إن Subdomain تاني يكتب كوكي بنفس الاسم ويخدع السيرفر.

13.4 First-party و Third-party: وإعلان الشوز اللي بيمشي وراك

انت بتقرا مقالة في `news.com`، والصفحة فيها إعلان جاي من `ads.net`. الإعلان ده طلب لـ `ads.net`، فيقدر يحط كوكي لـ `ads.net`. وبكرة لما تفتح `recipes.com` وفيه نفس شركة الإعلانات، المتصفح هيبعت نفس الكوكي. والنتيجة: `ads.net` عارف إنك نفس الشخص اللي كان في الموقعين. كده الإعلان بيمشي وراك. الكوكي اللي بتاعة الموقع اللي انت فيه اسمها **First-party**، واللي بتاعة موقع تاني جوه الصفحة اسمها **Third-party**.

الوضع النهارده (2026) مختلف بين المتصفحات:

Safari

بيمنع الـ Third-party cookies من 2020 (ITP). يعني على الـ Mac لو في Safari، حاجات زي "Login بحساب في iframe" ممكن تتصرف بشكل مختلف.

Firefox

بيعزل كل موقع في "برطمان" لوحده (Total Cookie Protection). `ads.net` جوه `news.com` غير `ads.net` جوه `recipes.com`.

Chrome

كان ناوي يمنعها، وبعد سنين من التأجيل أعلن في 2025 إنه **هيسيبها**، وقفل أغلب مشروع Privacy Sandbox. بس في Incognito بيمنعها.

والحل اللي اتفقوا عليه للحالات المشروعة (زي Chat widget جوه مواقع كتير) اسمه **CHIPS**: الكوكي تتعلم بـ `Partitioned`، فتتحفظ منفصلة لكل موقع أب. كده الـ Widget يفتكرك جوه `news.com`، بس ميقدرش يربطك بـ `recipes.com`. وده العمود اللي اسمه **Partition Key** في DevTools (الفصل 21).

13.5 Session على السيرفر ولا Token معاك؟

الطريقة اللي في الرسمة (رقم عشوائي، والبيانات في جدول على السيرفر) اسمها **Server-side session**. فيه طريقة تانية انتشرت مع الـ APIs و الـ SPAs (زي تطبيقات React): السيرفر يدّيك **Token** فيه البيانات نفسها، وممضي، فالسيرفر يتأكد من الإمضا من غير ما يدور في جدول. أشهر شكل ليه **JWT** (JSON Web Token):

تشريح الـ Token

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0MiIsIm5hbWUiOiJTYXJhIiwiZXhwIjoxNzk1MDAwMDAwfQ.k3J8x.
..
```

1. `eyJhbGciOiJIUzI1NiJ9` الجزء الأول (Header): Base64 لـ `{"alg":"HS256"}`، يعني طريقة الإمضا.
2. `.` النقط بتفصل التلات أجزاء.

3 `eyJzdWIiOiI0MiIsIm5hbWUiOiJTYXJhIiwiZXhwIjoxNzk1MDAwMDAwfQ` الجزء التاني (Payload): الـ Base64 لـ `ame":"Sara","exp":1795000000}`. `sub` = المستخدم، و `exp` = إمتى ينتهي. فمتحطش فيه أسرار.

4 `k3J8x...` الجزء التالت (Signature): الإمضا. لو حد غيّر حرف في الـ Payload، الإمضا مش هيطابق، والسيرفر هيرفضه.

والـ Token ده بيتبعت إزاي؟ يا إما في كوكي (زي الـ Session ID بالظبط)، يا إما الـ JS يحفظه ويبعته في `Authorization: Bearer ...` (الفصل 9). والنقاش المشهور: لو حفظته في `localStorage`، أي سكريبت شرير في الصفحة يقدر يقراه. ولو في كوكي `HttpOnly`، الـ JS مش شايفه، بس لازم تهتم بـ SameSite و CSRF. مفيش إجابة واحدة، بس الأهم إنك لما تفتح DevTools تعرف تدوّر في المكانين.

الحاجة	Server-side session	Token (زي JWT)
البيانات فين	على السيرفر، والكوكي فيها رقم بس	جوه الـ Token نفسه
السيرفر بيتحقق إزاي	بيدور في جدول	بيتأكد من الإمضا
تسجيل خروج فوري	سهل: امسح السطر	صعب: الـ Token صالح لحد `exp`
بيتبعت في	`Cookie`	`Cookie` أو `Authorization`

جدول 13.2 ملخص: Session مقابل Token

13.6 أماكن تخزين تانية في المتصفح

الكوكيز صغيرة (حوالي 4 KB للكوكي)، وبتتبعت مع **كل** طلب، فلو حطيت فيها حاجات كبيرة كل طلب هيتقل. عشان كده المتصفح فيه أماكن تخزين **مبتتبعتش** للسيرفر خالص، الـ JS بس هو اللي بيستخدمها:

المكان	بيتبعت للسيرفر؟	بيعيش قد إيه	الحجم تقريباً	تستخدمه في
Cookies	أيوه، مع كل طلب	حسب Max-Age (Chrome أقصاه 400 يوم)	~4 KB للكوكي	الـ Session والـ Login
`localStorage`	لأ	لحد ما يتمسح	~5 MB لكل Origin	الإعدادات: الثيم واللغة
`sessionStorage`	لأ	لحد ما التابة تتقفل	~5 MB	حاجة مؤقتة في تابة واحدة: فورم بتكتبه
IndexedDB	لأ	لحد ما يتمسح	كبير (جزء من مساحة الديسك)	بيانات كتير، شغل Offline

جدول 13.3 أماكن التخزين في المتصفح

Console · Chrome DevTools

```
> localStorage.setItem('theme', 'dark')
> localStorage.getItem('theme')
'dark'
> localStorage.setItem('user', {name: 'Sara'})
> localStorage.getItem('user')
'[object Object]'
# بيخزن نصوص بس! لازم JSON.stringify الأول
> localStorage.setItem('user', JSON.stringify({name: 'Sara'}))
> JSON.parse(localStorage.getItem('user')).name
'Sara'
> document.cookie
'theme=dark; lang=ar'
# الـ sid مش ظاهرة، لأنها HttpOnly
```

> ⚠ **خلي بالك: التخزين مربوط بالـ Origin**
>
> `localStorage` بتاع `http://localhost:3000` غير بتاع `http://localhost:5173`، لأن الـ Port مختلف يعني Origin مختلف (الفصل 6). فلو غيّرت Port السيرفر وانت بتطور، هتلاقي "البيانات اختفت". هي ما اختفتش، انت في Origin تاني.

13.7 تتابع الكوكيز في curl

curl مبيحفظش كوكيز من نفسه. ودي فرصة تشوف الآلية من غير سحر:

Terminal

```
user@MacBook ~ % curl -si 'https://httpbin.org/cookies/set?name=sara' | grep -i -e '^HTTP' -e
set-cookie -e location
HTTP/2 302
location: /cookies
set-cookie: name=sara; Path=/
user@MacBook ~ % curl -s -c jar.txt -b jar.txt -L 'https://httpbin.org/cookies/set?name=sara'
{
  "cookies": {
    "name": "sara"
  }
}
```

تشريح الأمر

```
curl -s -c jar.txt -b jar.txt -L 'https://httpbin.org/cookies/set?name=sara'
     1  2          3  4
```

1. `-s` اسكت.
2. `-c jar.txt` أي `Set-Cookie` تجيلك، احفظها في `jar.txt` (الملف بيتكتب في الآخر).

3	`-b jar.txt`	وابعت الكوكيز اللي في الملف ده مع الطلبات. وكمان، أول ما تدّي curl `-b` ، بيشغّل "محرك الكوكيز" بتاعه، فيفتكر الكوكي حتى بين الـ Redirects في نفس الأمر.
4	`-L`	اتبع الـ 302 لـ `/cookies` ، اللي بيرجعلك الكوكيز اللي وصلت للسيرفر.
5	`'...cookies/set?name=sara'`	بين Quotes عشان الـ `?` (zsh، فاكر؟).

ولو فتحت `jar.txt` هتلاقي سطر فيه الدومين والمسار و Secure ووقت الانتهاء والاسم والقيمة، مفصولين بـ Tabs. ده "برطمان" الكوكيز بتاع curl، نفس فكرة اللي جوه المتصفح بالظبط.

ملخص الفصل

- HTTP بينسى، والكوكي تذكرة: السيرفر يبعتها بـ `Set-Cookie` ، والمتصفح يرجعها لوحده في `Cookie` .
- Session = البيانات على السيرفر، والكوكي فيها رقم عشوائي بس.
- `Domain` و`Path` = تتبعت لمين، و`Expires` / `Max-Age` = تعيش قد إيه.
- `Secure` = HTTPS بس، و`HttpOnly` = الـ JS ميشوفهاش، و`SameSite` = ضد CSRF، والافتراضي في Chrome `Lax` .
- Third-party cookies: Safari بيمنع، و Firefox بيعزل، و Chrome لسه بيسمح. و`Partitioned` (CHIPS) حل وسط.
- JWT = بيانات ممضية، وبتقري بسهولة، فمتحطش فيه أسرار.
- `localStorage` و`sessionStorage` و IndexedDB مبيتبعتوش للسيرفر، ومربوطين بالـ Origin.

صح ولا غلط؟

1. كوكي عليها `HttpOnly` مش هتظهر في `document.cookie` . صح / غلط
2. `localStorage` بيتبعت للسيرفر مع كل طلب زي الكوكيز. صح / غلط
3. محتوى الـ JWT متشفّر ومحدش يقدر يقراه غير السيرفر. صح / غلط

أسئلة مراجعة

1. ليه الكوكي بتاعة الـ Session فيها رقم عشوائي، مش اسم المستخدم؟
2. اشرح هجوم CSRF، وإزاي `SameSite=Lax` بيمنعه.
3. إيه الفرق بين الـ `;` في `Set-Cookie` والـ `;` في `Cookie` ؟
4. ليه ممكن تلاقي بياناتك في `localStorage` "اختفت" لما غيّرت Port السيرفر؟
5. Session على السيرفر مقابل JWT: إيه أسهل في تسجيل الخروج الفوري؟ وليه؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل الأمر بتاع httpbin بـ `-c` و `-b`، وافتح `jar.txt` واشرح كل عمود فيه.
2. متوسط خد JWT من أي موقع تعليمي (أو اعمل واحد على `jwt.io`)، وفك الجزء التاني بـ `atob()` في الـ Console. إيه البيانات اللي جواه؟
3. صعب افتح أي موقع كبير، ومن Network شوف ردود فيها `Set-Cookie` من دومينات تانية غير الموقع نفسه. دي Third-party؟ وإيه الـ SameSite بتاعها؟

14

المرحلة 4 · الأمان والحالة

الفصل الرابع عشر

CORS: أشهر خطأ أحمر في الـ Console

هتكتب fetch بسيط لـ API، وتلاقي في الـ Console سطر أحمر طويل فيه كلمة CORS، والطلب نفسه شغال كويس في curl. ده أكتر خطأ بيلخبط المبتدئين، لأنه بيبان كأنه مشكلة في كودك، وهو في الحقيقة قرار أمان من المتصفح ومكان حله غالباً على السيرفر. هنحكي القصة من الأول: ليه المتصفح بيمنع، والاستثناء اللي اسمه CORS، والـ Preflight، وتقرا الخطأ صح، وتحله صح.

بعد الفصل ده هتكون عارف

- Same-origin policy: ليه المتصفح بيمنع قراية ردود مواقع تانية
- CORS: السيرفر هو اللي بيدّي الإذن، مش انت
- Simple requests مقابل Preflight بـ OPTIONS
- كل Header: Allow-Origin و Methods و Headers و Credentials و Max-Age
- تقرا رسالة الخطأ في الـ Console وتلاقي الطلب في Network
- الحلول الصح (والغلط): السيرفر، و Proxy في Vite، و no-cors
- ليه curl مبيتأثرش بـ CORS خالص

14.1 ليه المتصفح بيمنع أصلاً؟

انت عامل Login في بنكك في تابة. وفي تابة تانية فتحت موقع شرير. الموقع ده فيه سطر JS بسيط: `fetch('https://bank.com/api/balance')`. المتصفح هيبعت الطلب، ومعاه كوكي البنك بتاعتك (هو بيعمل كده لوحده، الفصل 13)، والبنك هيرد برصيدك. لو الـ JS الشرير قدر **يقرا** الرد ده، يبقى يقدر يقرا إيميلاتك وكل حاجة انت عامل فيها Login. كارثة.

فالمتصفحات من التسعينات عندها قاعدة اسمها **Same-origin policy** (سياسة نفس الأصل): **الـ JS اللي جاي من Origin معين ميقدرش يقرا ردود Origin تاني**. وفاكر الـ Origin من الفصل 6؟ `scheme + host + port`. يعني `http://localhost:5173` و`http://localhost:3000` Origins مختلفين، حتى لو الاتنين على جهازك.

بس هنا ظهرت مشكلة تانية: الويب اتغير. بقى عندك Frontend على `app.com` ومحتاج يكلم API على `api.app.com`، أو API عام زي الطقس عاوز أي موقع يستخدمه. والقاعدة بتمنع ده كله. فكان لازم طريقة **آمنة** لعمل استثناء. والمهم: مين اللي يقرر الاستثناء؟ مش الـ JS (ده ممكن يكون الشرير)، لكن **السيرفر صاحب البيانات**. الطريقة دي اسمها **CORS** (Cross-Origin Resource Sharing).

> **الفكرة في جملة**
>
> CORS مش حماية. CORS **استثناء** من حماية. المتصفح بيمنع، والسيرفر يقدر يقول "أنا موافق إن الـ Origin ده يقرا ردودي" بـ Headers. لو السيرفر مقالش، المتصفح بيفضل مانع. فالخطأ معناه: "السيرفر مدّاش الإذن".

14.2 أبسط حالة: Access-Control-Allow-Origin

الـ Frontend بتاعك على `http://localhost:5173` عمل `fetch('https://api.weather.dev/today')`. المتصفح بيبعت الطلب عادي، ومعاه Header بيقول هو جاي منين:

```http
GET /today HTTP/2
Host: api.weather.dev
Origin: http://localhost:5173
```

ولما الرد يرجع، المتصفح بيدور فيه على Header واحد:

تشريح الـ Header

```
access-control-allow-origin: http://localhost:5173
```

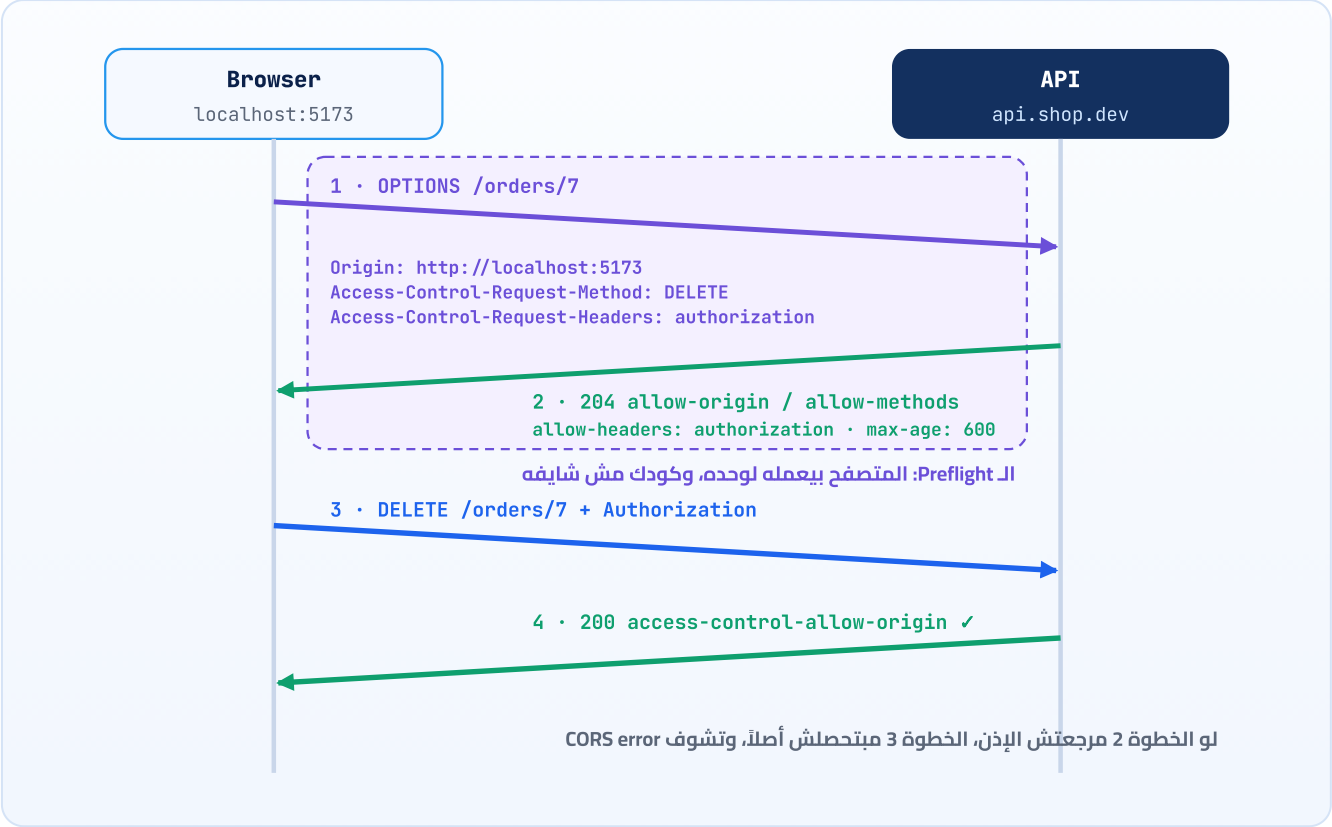
1. `access-control-allow-origin` — "مسموح للـ Origin ده يقرا الرد". اختصاره **ACAO**.
2. `http://localhost:5173` — لازم يطابق الـ `Origin` اللي اتبعت **بالظبط** (الـ Scheme والـ Port كمان). أو `*` = "أي حد".

لو الـ Header موجود ومطابق، المتصفح يسلّم الرد للـ JS. لو مش موجود، **الطلب اتبعت والرد وصل فعلاً**، بس المتصفح بيحجزه وبيرمي خطأ. ودي نقطة مهمة جداً: CORS مش بيمنع الطلب من إنه يوصل للسيرفر في الحالة دي، بيمنع الـ JS من إنه يقرا الرد.

14.3 الـ Preflight: "ممكن أبعت؟" قبل ما تبعت

طب لو الطلب مش مجرد قراية؟ لو هو `DELETE` أو `PUT`، أو `POST` بـ JSON، أو فيه `Authorization`؟ هنا الخطر أكبر: لو المتصفح بعت الطلب الأول وبعدين سأل، يبقى الحذف حصل فعلاً حتى لو الرد اتحجز. وكمان فيه سيرفرات قديمة كانت متصممة على إن "مفيش موقع تاني يقدر يبعت DELETE"، لأن الفورمز العادية متقدرش تبعته.

فالمتصفح في الحالات دي **بيسأل الأول** بطلب صغير اسمه **Preflight**، بالـ Method `OPTIONS`، ومفيهوش Body ولا كوكيز. ولو السيرفر وافق، يبعت الطلب الحقيقي:



شكل 14.1 الـ Preflight: سؤال بـ OPTIONS قبل الطلب الحقيقي، والطلب الحقيقي مبيتبعتش غير لو السيرفر وافق

إمتى يحصل Preflight؟

المتصفح بيعدي من غير Preflight لو الطلب "بسيط"، يعني حاجة كانت الفورمز العادية تقدر تعملها من زمان. أي حاجة غير كده بتاخد Preflight:

الحاجة	من غير Preflight (Simple)	بيعمل Preflight
الـ Method	`GET` و `HEAD` و `POST`	`PUT` و `PATCH` و `DELETE`
الـ Content-Type	`text/plain` و `multipart/form-data` و `application/x-www-form-urlencoded`	`application/json` وأي حاجة تانية
Headers إضافية	مفيش غير الأساسية (زي `Accept` و `Accept-Language`)	`Authorization` أو أي `X-...` من عندك

جدول 14.1 Simple request ولا Preflight؟

يعني أبسط `fetch` بـ JSON في الدنيا: `fetch(url, {method:'POST', headers:{'Content-Type':'application/json'}, body})` بيعمل Preflight. وده بيفاجئ ناس كتير: "أنا بعت POST، ليه لقيت OPTIONS في Network؟".

كل Header في الرد

`access-control-allow-origin`	مين مسموحله يقرا. Origin واحد بالظبط أو `*`. مينفعش قايمة: لو السيرفر عاوز يسمح لكذا Origin، لازم يقرا الـ `Origin` اللي جاي ويرجعه لو هو في القايمة، ويحط `Vary: Origin` (الفصل 9).
`access-control-allow-methods`	في رد الـ Preflight: الـ Methods المسموحة.
`access-control-allow-headers`	في رد الـ Preflight: الـ Headers اللي مسموح تبعتها (زي `authorization` و`content-type`).
`access-control-allow-credentials: true`	مسموح تبعت كوكيز وتقرا الرد. ولازم الـ JS يطلبها: `fetch(url, {credentials: 'include'})`. ومعاها **مينفعش** `allow-origin: *`.
`access-control-max-age: 600`	افتكر رد الـ Preflight 600 ثانية، فمتسألش تاني كل مرة.
`access-control-expose-headers`	الـ JS بيشوف Headers قليلة في الرد بشكل افتراضي. لو عاوز يقرا `Location` مثلاً، السيرفر يحطه هنا. (شفناه في رد jsonplaceholder في الفصل 10.)

14.4 تقرا الخطأ صح

ده شكل الخطأ في الـ Console. طويل ومخيف، بس كل حتة فيه بتقول حاجة:

Console · Chrome DevTools

```
Access to fetch at 'https://api.shop.dev/orders/7' from origin 'http://localhost:5173' has
been blocked by CORS policy: Response to preflight request doesn't pass access control check:
No 'Access-Control-Allow-Origin' header is present on the requested resource.
DELETE https://api.shop.dev/orders/7 net::ERR_FAILED
Uncaught (in promise) TypeError: Failed to fetch
```

فصّص الرسالة

1 الطلب اللي اتمنع، وكان بـ `fetch` (ممكن تلاقي XMLHttpRequest لو مكتبة زي axios).	`Access to fetch at '...orders/7'`
2 الـ Origin بتاعك. قارنه بالقيمة اللي السيرفر مسموح بيها.	`from origin 'http://localhost:5173'`
3 المشكلة في الـ **OPTIONS**، مش في الطلب الحقيقي. ولو الجملة دي مش موجودة، المشكلة في الرد العادي.	`Response to preflight request ...`

4 السبب بالظبط. وفيه أسباب تانية هتلاقيها مكانها، زي: `value '...' that is not equal to the supplied origin`، أو إن الـ Header فيه `*` وانت باعت `credentials`، أو `Request header field authorization is not allowed`.	`No 'Access-Control-Allow-Origin' header`
5 ده اللي الكود بتاعك بيشوفه في الـ `catch`. لاحظ إنه مفيهوش أي تفاصيل، عن قصد: عشان الـ JS الشرير ميعرفش أي حاجة عن الموقع التاني. التفاصيل للـ Console بس، يعني ليك انت كمطور.	`TypeError: Failed to fetch`

وفي **Network**، الطلب هيظهر وعليه في عمود Status: `CORS error`. ولو فيه Preflight، هتلاقي سطر تاني لوحده نوعه (Type) `preflight`. اضغط عليه وشوف الـ Response Headers بتاعته: هل فيها `access-control-allow-origin`؟ هل الـ Status 2xx؟ ده أول مكان تبص فيه.

> ⚠ **خلي بالك: الـ CORS error ساعات بيخبي خطأ تاني**
>
> لو السيرفر وقع ورد بـ `500` أو `404`، صفحة الخطأ دي غالباً مفيهاش الـ CORS Headers (الكود اللي بيحطهم مشتغلش). فالمتصفح يقولك "CORS error"، والمشكلة الحقيقية خطأ في السيرفر. عشان كده بص على الـ Status الحقيقي في Network، أو جرب نفس الطلب في curl.

14.5 ليه curl مبيتأثرش؟

CORS قاعدة **جوه المتصفح**، معمولة عشان تحمي اليوزر من صفحات شريرة شغالة في نفس المتصفح بكوكيزه. curl مش متصفح: مفيش صفحة، ومفيش كوكيز مستخبية، ومفيش Origin. انت بتطلب بنفسك. فمحدش بيمنع. وده بالظبط اللي يخلي curl أداة ممتازة لتشخيص CORS: تقدر تعمل الـ Preflight بإيدك وتشوف السيرفر بيرد بإيه:

تشريح الأمر

```
curl -si -X OPTIONS https://httpbin.org/post -H 'Origin: http://localhost:5173' -H 'Access-Control-Request-Method: POST' -H 'Access-Control-Request-Headers: content-type'
```

1. `curl -si` اسكت، واطبع الـ Headers.
2. `-X OPTIONS` نفس الـ Method بتاع الـ Preflight.
3. `https://httpbin.org/post` العنوان اللي الـ Frontend عاوز يكلمه.
4. `-H 'Origin: http://localhost:5173'` نتظاهر إننا المتصفح وجايين من الـ Origin ده.
5. `-H 'Access-Control-Request-Method: POST'` "عاوز أبعت POST، ينفع؟"
6. `-H 'Access-Control-Request-Headers: content-type'` "وعاوز أبعت Header اسمه content-type."

Terminal

```
HTTP/2 200
allow: POST, OPTIONS
access-control-allow-origin: http://localhost:5173
access-control-allow-credentials: true
access-control-allow-methods: GET, POST, PUT, DELETE, PATCH, OPTIONS
access-control-max-age: 3600
access-control-allow-headers: content-type
```

هنا السيرفر وافق على كل حاجة: الـ Origin والـ Method والـ Header. لو جربت نفس الأمر على الـ API بتاعك ومالقيتش السطور دي، يبقى عرفت المشكلة فين قبل ما تكتب سطر JS.

14.6 تحلها إزاي؟

1. على السيرفر (الحل الحقيقي)

لو الـ API بتاعك أو بتاع فريقك: السيرفر يرجّع الـ Headers الصح، ويرد على `OPTIONS` . في Express مثلاً فيه Package اسمها `cors` بتعمل ده في سطر. وحط الـ Origins اللي مسموح بيها بالاسم، مش `*` ، لو فيه كوكيز أو بيانات خاصة.

2. Proxy وانت بتطور

خلي الطلب يروح لنفس الـ Origin بتاعك (`/api/...`)، وسيرفر التطوير يوصله للـ API من وراك. كده مفيش Cross-origin أصلاً من وجهة نظر المتصفح. ده موجود في Vite و Next.js وغيرهم.

vite.config.js

```
export default {
  server: {
    proxy: {
      // أي طلب يبدأ بـ /api يروح للـ API الحقيقي
      '/api': {
        target: 'https://api.shop.dev',
        changeOrigin: true,              // غيّر الـ Host للـ API
        rewrite: (p) => p.replace(/^\/api/, ''),
      },
    },
  },
}
```

وفي الكود بتاعك تكتب `fetch('/api/orders/7')` بدل العنوان الكامل. المتصفح شايفه طلب لـ `localhost:5173` نفسه، وسيرفر Vite (اللي شغال على جهازك، ومش متصفح) يبعته للـ API. وخلي بالك إن ده للتطوير بس؛ في الـ Production محتاج حل حقيقي (السيرفر نفسه، أو Proxy في الاستضافة).

حلول غلط هتلاقيها على النت

`mode: 'no-cors'` : الخطأ هيختفي، بس الرد هيبقى **Opaque**: Status صفر، ومفيش Body تقدر تقراه. يعني مش حل، انت بس سكّتّ الرسالة. **Extensions بتلغي CORS**: هتشتغل عندك بس، ومش عند اليوزرز، وبتفتح ثغرة في متصفحك لكل المواقع. **تشغيل Chrome بـ** `--disable-web-security` : نفس الكلام وأخطر. **"Proxies" عامة على النت**: كل بياناتك بتعدي على سيرفر حد معرفوش.

لو حابب تتعمق: Opaque responses وليه الصور شغالة عادي

تقدر تعدّيه في أول قراية

ممكن تسأل: طب إزاي `<img src="https://other.com/a.png">` شغالة من غير CORS؟ لأن الـ JS مبيقراش الصورة، المتصفح بيعرضها بس. وكذلك `<script src>` و `<link rel=stylesheet>` . القاعدة عن **قراية** البيانات بالـ JS. لو حاولت ترسم الصورة دي على `<canvas>` وتقرا البيكسلات، هنا CORS هيدخل، لأنك بتحاول تقرا. وده نفس منطق الـ Opaque response في `no-cors` : "اعرضه أو خزنه، بس متقراهوش".

ملخص الفصل

- Same-origin policy: الـ JS ميقدرش يقرا ردود Origin تاني. CORS استثناء السيرفر هو اللي بيدّيه.
- الطلب البسيط بيتبعت، والمتصفح بيحجز الرد لو مفيش `access-control-allow-origin` مطابق.
- الطلبات غير البسيطة (PUT/DELETE، JSON، Authorization) بتاخد Preflight بـ `OPTIONS` الأول.
- `allow-credentials: true` مينفعش معاها `*` ، والـ JS لازم يطلب `credentials: 'include'` .
- اقرا رسالة الـ Console كلها: هل المشكلة في الـ Preflight؟ إيه السبب بالظبط؟ وبص على Status الحقيقي في Network.
- curl مبيتأثرش بـ CORS، فاستخدمه تعمل الـ Preflight بإيدك.
- الحل: على السيرفر، أو Proxy وانت بتطور. `no-cors` مش حل.

صح ولا غلط؟

1. CORS error معناه إن الكود بتاعك فيه Syntax error. صح / غلط
2. `fetch` بـ POST و `Content-Type: application/json` بيعمل Preflight. صح / غلط
3. `mode: 'no-cors'` بيخليك تقرا الرد عادي. صح / غلط

أسئلة مراجعة

1. احكي سيناريو البنك والتابة الشريرة، وإزاي Same-origin policy بتحمي منه.
2. ليه الإذن في CORS لازم يكون من السيرفر مش من الـ JS؟
3. إيه اللي بيخلي طلب "مش بسيط"؟ اذكر 3 أسباب.
4. ليه ممكن تشوف CORS error والمشكلة الحقيقية 500؟
5. إزاي Proxy التطوير في Vite بيخلي المشكلة تختفي؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل من Console أي موقع (زي `example.com`)، اعمل `fetch('https://jsonplaceholder.typicode.com/posts/1').then(r=>r.json()).then(console.log)` . اشتغل؟ ليه؟ (بص على الـ Response headers في Network.)
2. متوسط من Console صفحة `example.com` نفسها، اعمل `fetch('https://www.google.com/')` . اقرا الخطأ كامل وفصّصه: هو في Preflight ولا لأ؟ وليه؟ (لو جربت من موقع تاني ولقيت خطأ فيه `Content Security Policy` بدل CORS، ده نوع منع تاني من الموقع نفسه، هنشوفه في الفصل 20.)
3. صعب اعمل Preflight بـ curl على `httpbin.org/delete` بـ Method `DELETE` و Header `authorization`، وقارن الرد بالرد اللي في الفصل. وبعدين جرب نفس الطلب بـ `fetch` من Console أي موقع، ولاقي سطر الـ `preflight` في Network.

15

المرحلة 5 · السرعة والعرض

الفصل الخامس عشر

الكاش والـ CDN: أسرع طلب هو اللي مبيتبعتش

كل طلب فيه DNS و TCP و TLS ورحلة للسيرفر ورجوع. طب لو الملف ماتغيرش من امبارح، ليه نجيبه تاني؟ الكاش هو إن حد في السكة يحتفظ بنسخة. هنفهم مين بيكاش (المتصفح والـ CDN)، وإزاي السيرفر بيقول 'خلي ده عندك قد إيه'، وإزاي المتصفح يسأل 'لسه زي ما هو؟' ويرد عليه 304، وليه أحياناً الكاش هو اللي مبوظ عليك التعديلات.

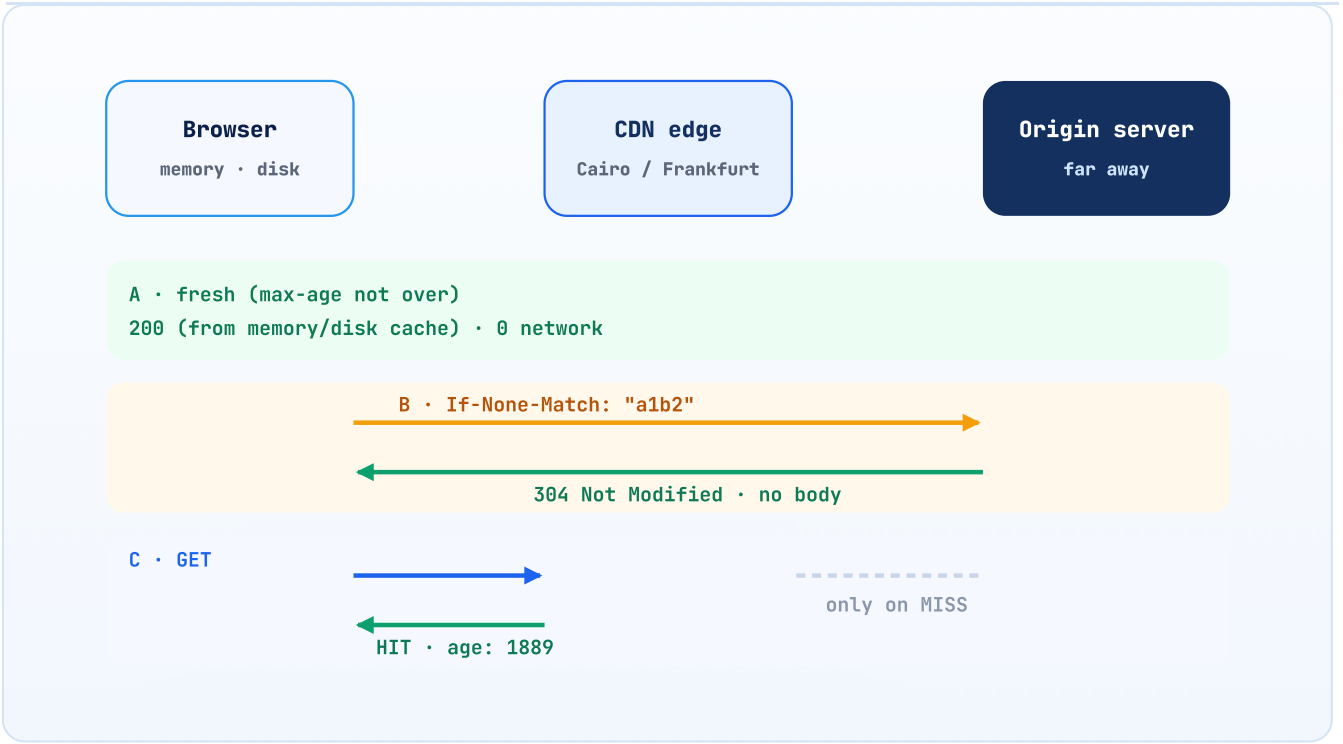
بعد الفصل ده هتكون عارف

- المشكلة: نفس الملف بيتجاب مليون مرة
- Cache-Control وكل Directive فيه: max-age و no-cache و no-store و private و public و immutable
- الـ Validation: ETag و If-None-Match و Last-Modified، و 304
- Fingerprinting: app.3f9a1c.js وليه بيحل كل حاجة
- memory cache و disk cache في DevTools
- Hard reload و Disable cache و Empty cache
- الـ CDN: Edge و Age و cf-cache-status و x-cache

15.1 ليه نجيب نفس الحاجة كل مرة؟

افتح Facebook. اللوجو، وملفات CSS، والخطوط، وملفات JS اللي حجمها ميجات. كل ده ماتغيرش من ساعة ما فتحته امبارح. لو المتصفح جابه تاني كل مرة، هتستنى، وهتصرف باقة، والسيرفر هيتعب من مليار طلب على نفس الملف. والحل القديم جداً في علوم الكمبيوتر: **احتفظ بنسخة قريبة**. ده هو الـ **Cache**.

بس الكاش جواه مشكلة صعبة جداً: **إمتى النسخة اللي عندي تبقى قديمة؟** لو السيرفر غيّر الـ CSS وانت لسه شايف القديم، الصفحة هتبوظ. وفيه نكتة مشهورة بين المبرمجين: "فيه حاجتين صعبين بس في علوم الكمبيوتر: إبطال الكاش (Cache invalidation)، وتسمية الحاجات". فـ HTTP بيدّي السيرفر أدوات يقول بيها للكاش إمتى يثق في النسخة وإمتى يسأل.



شكل 15.13 سيناريوهات: النسخة طازة في المتصفح فمفيش شبكة أصلاً (A)، أو المتصفح يسأل "اتغيرت؟" ويرجعله 304 (B)، أو الـ CDN القريب يرد بدل السيرفر البعيد (C)

15.2 Cache-Control: السيرفر بيقول للكاش يعمل إيه

الـ Header الأساسي اسمه `Cache-Control`، والسيرفر بيحطه في الرد. جواه تعليمات (Directives) مفصولة بـ `,`. ده رد حقيقي من MDN:

```
Terminal
user@MacBook ~ % curl -sI https://developer.mozilla.org/en-US/ | grep -i -e cache -e '^age' -e
etag -e last-mod
cache-control: public, max-age=3600
etag: "adbb7a40fd40be73d624e80eab1af10f"
last-modified: Thu, 22 Oct 2026 01:11:30 GMT
age: 1889
x-cache: MISS, HIT, HIT
```

تشريح الـ Header

```
cache-control: public, max-age=3600
```

1. `cache-control:` تعليمات الكاش.
2. `public` أي كاش في السكة (المتصفح أو CDN أو Proxy) يقدر يحتفظ بنسخة.
3. `,` فاصل بين التعليمات.

4 `max-age=3600` النسخة "طازة" (Fresh) ساعة من وقت ما اتعملت. طول الساعة دي، الكاش يستخدمها من غير ما يسأل حد.

وده كل اللي هتشوفه تقريباً:

التعليمة	معناها	تستخدمها في
`max-age=N`	طازة N ثانية	أي حاجة ثابتة شوية
`s-maxage=N`	زي اللي فوق بس للكاش المشترك (CDN) بس	عاوز الـ CDN يحتفظ أكتر من المتصفح
`public`	أي كاش مسموحله	ملفات عامة
`private`	متصفح اليوزر بس، مش CDN	صفحة فيها بيانات اليوزر
`no-cache`	احتفظ بيها، بس **اسأل كل مرة** قبل ما تستخدمها	HTML لازم يبقى أحدث حاجة
`no-store`	**متحتفظش بأي نسخة** خالص	بيانات حساسة: كشف حساب البنك
`immutable`	مش هيتغير أبداً، متسألش حتى في Reload	ملفات باسم فيه Hash (الجزء الجاي)
`must-revalidate`	لما تخلص، لازم تسأل، متستخدمش القديمة أبداً	لما الدقة أهم من السرعة
`stale-while-revalidate=N`	بعد ما تخلص، استخدم القديمة N ثانية وانت بتجيب الجديدة من ورا	سرعة مع تحديث معقول

جدول 15.1 أشهر تعليمات Cache-Control

> ⚠ **خلي بالك: no-cache مش معناها 'متكاشش'**
>
> دي أشهر لخبطة في الكاش كله. `no-cache` معناها "خزّن، بس متستخدمش من غير ما تسأل السيرفر". واللي معناها "متخزنش خالص" هي `no-store`. الأسماء وحشة، بس هي كده من التسعينات.

15.3 "لسه زي ما هي؟" : ETag و 304

الـ `max-age` خلص، أو السيرفر قال `no-cache`. المتصفح معاه نسخة، بس مش متأكد هي لسه صح. هل يجيب الملف كله تاني؟ لأ، فيه طريقة أذكى: **يسأل**. ودي اسمها **Revalidation**.

السيرفر لما بعت الملف أول مرة حط عليه "بصمة" اسمها **ETag**، زي `"a1b2c3"`: كود بيتغير لو المحتوى اتغير. المتصفح بيحفظها، ولما يحب يسأل، يبعتها:

Terminal

```
user@MacBook ~ % curl -sI https://jsonplaceholder.typicode.com/posts/1 | grep -i etag
etag: W/"124-yiKdLzqO5gfBrJFrcdJ8Yq0LGnU"
user@MacBook ~ % curl -sI https://jsonplaceholder.typicode.com/posts/1 -H 'If-None-Match:
W/"124-yiKdLzqO5gfBrJFrcdJ8Yq0LGnU"' | head -1
HTTP/2 304
```

تشريح الأمر

```
curl -sI https://jsonplaceholder.typicode.com/posts/1 -H 'If-None-Match: W/"124-
yiKdLzqO5gfBrJFrcdJ8Yq0LGnU"'
```

1. `curl -sI` — HEAD، والـ Headers بس.
2. `https://.../posts/1` — نفس الملف.
3. `-H 'If-None-Match: ...'` — "ابعتلي الملف **لو** بصمته مش زي دي". يعني "لو اتغير".
4. `W/` — في قيمة الـ ETag: Weak، يعني "المحتوى نفس المعنى" حتى لو اتضغط بشكل مختلف. من غيرها Strong: كل Byte زي التاني.
5. `"124-yiKd..."` — البصمة نفسها. لازم بالـ Quotes، وعشان كده الـ Header كله بين Single quotes.

السيرفر رد **304 Not Modified** من غير Body. "اللي عندك صح، استخدمه". ووفرنا تحميل الملف كله. وفيه طريقة أقدم بالتاريخ بدل البصمة: السيرفر يبعت `Last-Modified`، والمتصفح يسأل بـ `If-Modified-Since`. نفس الفكرة، بس أقل دقة (بالثانية).

السيرفر بعت	المتصفح بيسأل بـ	الإجابة لو ماتغيرش
`ETag: "a1b2"`	`If-None-Match: "a1b2"`	`304` من غير Body
`Last-Modified: تاريخ`	`If-Modified-Since: تاريخ`	`304` من غير Body

جدول 15.2 ملخص: الأسئلة والإجابات

15.4 الحيلة اللي بتحل كل حاجة: اسم الملف فيه بصمته

هنا التعارض: عاوز الكاش طويل (عشان السرعة)، وعاوز التعديلات توصل فوراً (عشان الصح). الاتنين ضد بعض... إلا لو **غيّرت اسم الملف** كل ما يتغير. وده اللي أدوات زي Vite و Webpack بتعمله لوحدها لما تعمل Build:

dist/index.html

```
<link rel="stylesheet" href="/assets/index-3f9a1c.css">
<script type="module" src="/assets/index-b72e0d.js"></script>
```

الـ `3f9a1c` ده Hash من محتوى الملف. لو غيّرت سطر في الـ CSS، الاسم بيبقى `index-8c41e2.css`، ودي بالنسبة للمتصفح حاجة جديدة تماماً، مالهاش نسخة في الكاش. فالاستراتيجية الشائعة:

الملفات اللي باسمها Hash

`Cache-Control: public, max-age=31536000, immutable`
. سنة، ومتسألش. لأن لو اتغيرت، اسمها هيتغير.

الـ HTML نفسه

`Cache-Control: no-cache` . اسأل كل مرة.
لأنه هو اللي فيه أسماء الملفات الجديدة، ولو اتكاش، اليوزر هيفضل بيطلب الملفات القديمة.

15.5 الكاش في DevTools

في Network، عمود **Size** هيقولك الحاجة جت منين:

`(memory cache)`	من الـ RAM. أسرع حاجة، وغالباً حاجات اتحملت في نفس الصفحة أو التابة. Time تقريباً 0 ms.
`(disk cache)`	من الديسك. من زيارة قبل كده. برضه من غير شبكة خالص.
`304` + حجم صغير	سأل السيرفر، والسيرفر قال "زي ما هي". اتبعت طلب، بس من غير Body.
`(ServiceWorker)`	جه من Service Worker (كود JS بيعمل كاش بنفسه، الفصل 21).
حجم عادي، زي `14.2` kB	اتجاب من الشبكة فعلاً. وفي الحالة دي فيه رقمين لو فعّلت Big request rows: المنقول (مضغوط) والحقيقي.

وفيه 3 طرق تعدّي بيها الكاش، ومهم تعرف الفرق:

الطريقة	بتعمل إيه
Reload عادي ⌘R	الصفحة نفسها بتتسأل (Revalidate)، والملفات اللي لسه طازة ممكن تيجي من الكاش
Hard reload ⇧⌘R	تجاهل الكاش للصفحة دي وملفاتها في الـ Load ده
Empty cache and hard reload	Right-click على زرار Reload **و DevTools مفتوح**: يمسح الكاش ويحمّل. أقوى حاجة
Disable cache في Network	طول ما DevTools **مفتوح**، ولا طلب يستخدم الكاش (الفصل 18). أحسن حاجة وانت بتطور

جدول 15.3 طرق تخطي الكاش في Chrome على الـ Mac

مشكلة 1 **عدّلت الـ CSS والتعديل مش ظاهر**

غيّرت لون الزرار، ورفعت الموقع، وعملت Reload، ولسه اللون القديم.

افتح Network وبص على ملف الـ CSS: لو مكتوب `(disk cache)` أو `(memory cache)` ، الكاش هو السبب. جرب ⇧⌘R أو Disable cache. ولو اتحل، يبقى الحل الدائم: أسماء ملفات فيها Hash، أو `max-age` أقصر. ولو لسه مش ظاهر حتى من غير كاش المتصفح، بص على `age` و `cf-cache-status` : ممكن الـ CDN هو اللي لسه ماسك النسخة القديمة (الجزء الجاي).

15.6 الـ CDN: نسخة في كل مدينة

السيرفر بتاعك في فرجينيا في أمريكا. اليوزر في القاهرة. كل طلب بيعدي المحيط رايح جاي، يعني 150 أو 200 ms RTT على الأقل، ودي فيزيا مش حاجة تتحل بسيرفر أسرع. والحل: **CDN** (Content Delivery Network). شركات زي Cloudflare و Akamai و Fastly عندها سيرفرات في مئات المدن اسمها **Edge**. الـ DNS (أو Anycast، فاكر الفصل 5؟) بيوديك لأقرب واحد، وهو يرد من الكاش بتاعه. لو معندوش نسخة، يجيبها مرة من السيرفر الأصلي (**Origin**) ويحتفظ بيها للي بعدك.

وعشان كده الـ CDN بيستخدم نفس `Cache-Control` بتاعنا (خصوصاً `public` و`s-maxage`)، وبيضيف Headers تقولك حصل إيه:

MDN الـ CDN في رد example.com و Headers

1 Cloudflare: النسخة جت من الكاش بتاع الـ Edge. وفيه `MISS` (راح للـ Origin)، و`EXPIRED`، و`DYNAMIC` (مش بيتكاش أصلاً)، و`BYPASS`.	`cf-cache-status: HIT`
2 النسخة دي بقالها 1889 ثانية (نص ساعة تقريباً) في الكاش. مع `max-age=3600`، فاضلها 1711 ثانية وتبقى قديمة.	`age: 1889`
3 في CDNs زي Fastly: فيه أكتر من طبقة كاش، وكل واحدة بتقول حصلها إيه. هنا الطبقة الأولى ماكانش عندها، والتانية كان عندها.	`x-cache: MISS, HIT, HIT`
4 أسماء الـ Edge servers. الحروف غالباً كود المطار في المدينة: `IAD` = واشنطن.	`x-served-by: cache-bfi-..., cache-iad-...`

حاجة مشتركة: نفس الفكرة في 4 أماكن

شفنا الكاش في الـ DNS (الـ TTL في الفصل 5)، وفي المتصفح، وفي الـ CDN. وكلهم نفس الفكرة بالظبط: نسخة قريبة، ورقم بيقول تعيش قد إيه، ومشكلة "إمتى أعرف إنها قديمة". لو فهمت واحد، فهمت الكل. وكمان Docker بيعمل كاش لطبقات الـ Image بنفس المنطق: لو الطبقة ماتغيرتش، استخدم القديمة.

لو حابب تتعمق: Vary ومفتاح الكاش

تقدر تعدّيه في أول قراية

الكاش بيحفظ النسخة تحت "مفتاح"، وأساسه الـ URL. بس لو نفس الـ URL بيرجع عربي لواحد وإنجليزي لواحد (حسب `Accept-Language`)، أو مضغوط Brotli لواحد و Gzip لواحد، المفتاح لازم يشمل الـ Header ده، وإلا اليوزر العربي ياخد الإنجليزي. وده بالظبط شغل `Vary: Accept-Language, Accept-Encoding` اللي اتكلمنا عنه في الفصل 9. وكمان `Vary: Origin` مع CORS، عشان الكاش ميرجعش رد عليه إذن لـ Origin لحد تاني.

ملخص الفصل

- الكاش = نسخة قريبة. والمشكلة الصعبة: إمتى تبقى قديمة.
- `Cache-Control: max-age` = طازة قد إيه، و`public` / `private` = مين يكاش.
- `no-cache` = اسأل كل مرة، و`no-store` = متخزنش. مش نفس الحاجة.
- الـ Revalidation: `ETag` + `If-None-Match`، أو `Last-Modified` + `If-Modified-Since`، والرد `304` من غير Body.
- اسم الملف فيه Hash + `immutable` سنة للملفات، و`no-cache` للـ HTML.
- في DevTools: `(memory cache)` و`(disk cache)` و 304، و Disable cache وانت بتطور.
- الـ CDN = كاش في Edge قريب. و`age` و`cf-cache-status` و`x-cache` بيقولوا حصل إيه.

صح ولا غلط؟

1. `Cache-Control: no-cache` معناها إن المتصفح ميخزنش أي نسخة. صح / غلط
2. رد الـ `304` فيه الملف كامل. صح / غلط
3. ملف اسمه `app-3f9a1c.js` آمن نديله كاش سنة. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين Fresh و Stale؟ وإيه اللي بيحصل لما النسخة تبقى Stale؟
2. اشرح رحلة `ETag` من أول تحميل لحد الـ `304`.
3. ليه الـ HTML بالذات ميتكاشش طويل، والـ JS يتكاش سنة؟
4. في رد فيه `max-age=3600` و`age: 3500`، النسخة هتفضل طازة قد إيه؟
5. إيه الفرق بين `(disk cache)` و`304` في عمود Size؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح موقع كبير مرتين، والمرة التانية بص على عمود Size: كام ملف جه من `memory cache` وكام من `disk cache`؟
2. متوسط اعمل تجربة الـ ETag بـ curl على 3 ملفات مختلفة (صورة و JS و API)، ولاقي مين بيرجع 304.
3. صعب اعمل `curl -sI` على `example.com` 3 مرات ورا بعض، وراقب `age` و`cf-cache-status`. وبعدين ضيف `?v=123` في الآخر. إيه اللي اتغير؟ وليه؟

16

المرحلة 5 · السرعة والعرض

الفصل السادس عشر

من HTML لبيكسلات: المتصفح بيرسم إزاي

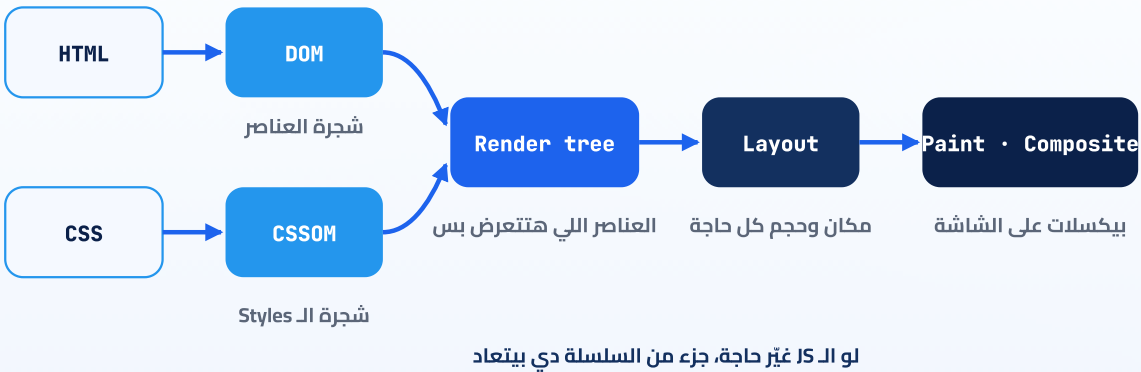
وصل الرد، وجواه HTML. لسه الصفحة مش ظاهرة. المتصفح محتاج يقرا النص ده، ويكتشف منه طلبات تانية (CSS و JS وصور)، ويبني منه شجرة، ويحسب مكان كل حاجة، ويرسمها. الفصل ده قصير وعملي: هنفهم الخطوات عشان تفهم الخطين الأزرق والأحمر في Network، وليه سكريبت واحد ممكن يوقف الصفحة كلها، وإيه فايدة defer و async.

بعد الفصل ده هتكون عارف

- الخطوات: Parse و DOM و CSSOM و Render tree و Layout و Paint
- ليه المتصفح بيبعت طلبات جديدة وهو بيقرا الـ HTML
- Render-blocking: الـ CSS والـ script العادي بيوقفوا إيه
- defer و async و type=module والفرق بينهم
- DOMContentLoaded و load: الخطين في Waterfall
- يعني إيه Initiator: Parser و Script

16.1 الرحلة من النص للشاشة

الـ HTML اللي جالك ده نص، زي اللي شفناه في curl. المتصفح محتاج يعمل منه صورة. وبيعدي على خطوات ثابتة اسمها **Critical rendering path**:



شكل 16.1 المتصفح بيبني شجرتين (DOM و CSSOM)، يدمجهم، يحسب الأماكن، وبعدين يرسم

1. **Parse الـ HTML → DOM**: المتصفح بيقرا الـ HTML من فوق لتحت، وبيحوّل كل Tag لعنصر في شجرة اسمها **DOM**. ودي نفس الشجرة اللي بتكلمها في JS بـ `document.querySelector`، واللي بتشوفها في تابة Elements.
2. **بيكتشف طلبات**: وهو بيقرا، كل ما يلاقي `<link rel=stylesheet>` أو `<script src>` أو `<img>`، بيبعت طلب جديد. وعشان كده في Network، عمود **Initiator** للحاجات دي بيقول `Parser` : الـ Parser هو اللي لقاها. ولو طلب اتعمل من كود JS (زي `fetch`)، الـ Initiator بيبقى `Script` واسم الملف ورقم السطر.
3. **CSSOM → CSS**: نفس الفكرة للـ CSS: شجرة فيها كل Style ينطبق على إيه.
4. **Render tree**: الشجرتين مع بعض، بس العناصر اللي هتظهر. `display:none` مش هنا، و `<head>` مش هنا.
5. **Layout** (أو Reflow): يحسب كل عنصر فين وحجمه قد إيه، حسب عرض الشاشة.
6. **Paint و Composite**: يرسم الألوان والنصوص والصور في طبقات، ويركبهم على بعض.

16.2 مين بيوقف مين؟

هنا فيه مشكلتين ليهم أثر مباشر على سرعة أي موقع:

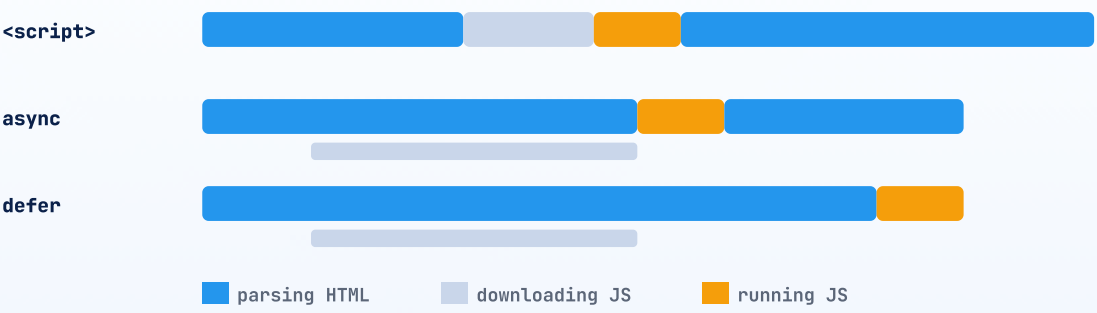
الـ CSS بيوقف الرسم

المتصفح **مش بيرسم** لحد ما الـ CSS اللي في `<head>` يوصل ويتقرا. ليه؟ لو رسم الصفحة من غير Styles وبعدين جه الـ CSS، هتشوف الصفحة "بتنط" (اسمه FOUC). فالـ CSS اسمه **Render-blocking**.

الـ script العادي بيوقف القراية

`<script src="app.js">` من غير أي حاجة: المتصفح **بيقف يقرا الـ HTML**، يجيب الملف، ويشغله، وبعدين يكمل. ليه؟ لأن السكريبت ممكن يعمل `document.write` أو يغيّر الـ DOM، فلازم يتنفذ في مكانه بالظبط.

وعشان كده النصيحة القديمة كانت: "حط الـ scripts في آخر الـ `<body>`". والحل الأحدث: خلي السكريبت يقول للمتصفح إنه مش محتاج يوقف حاجة:



شكل 16.2 العادي بيوقف القراية وهو بيحمّل وبيشغّل. async بيحمّل جنب القراية ويشغّل أول ما يوصل. defer بيحمّل جنب القراية ويشغّل بعد ما الـ HTML يخلص، وبالترتيب

الشكل	بيوقف قراية الـ HTML؟	بيتنفذ إمتى	بالترتيب؟
`<script src>`	أيوه	فوراً في مكانه	أيوه
`<script async src>`	لأ (بس بيوقف وهو بيتنفذ)	أول ما يوصل	لأ، اللي يوصل الأول
`<script defer src>`	لأ	بعد ما الـ HTML يخلص، قبل DOMContentLoaded	أيوه
`<script type="module" src>`	لأ	زي defer بالظبط	أيوه

جدول 16.1 ملخص: طرق تحميل السكريبت

القاعدة العملية

سكريبت التطبيق بتاعك: `defer` أو `type="module"` (وده اللي Vite بيعمله لوحده). سكريبت مستقل مالوش علاقة بحاجة، زي Analytics: `async`. وكورس Jonas هيشرح الفرق ده بالتفصيل في جزء الـ DOM، فلو شفته هناك، افتكر الرسمة دي.

16.3 الخطين الأزرق والأحمر

في Waterfall بتاع Network (الفصل 18)، هتلاقي خطين رأسيين، وتحت في الـ Status bar رقمين بنفس اللون:

`DOMContentLoaded` (أزرق)	الـ HTML اتقرا كله، والـ DOM جاهز، والسكريبتات العادية و defer اتنفذت. الصور ممكن لسه بتتحمل. ده الحدث اللي بتستناه لما تكتب `ocument.addEventListener('DOMContentLoaded', ...)`.
`Load` (أحمر)	كل حاجة اتحملت: الصور والـ CSS والـ iframes. ده `window.onload`.

لو الخط الأزرق متأخر قوي، دور على سكريبت عادي تقيل في الـ `<head>`. ولو الأحمر متأخر، دور على صور تقيلة. بس خلي بالك: المواقع الحديثة بتحمّل حاجات كتير بعد Load (بـ `fetch`)، فالـ Load مش معناه إن الصفحة "خلصت" من وجهة نظر اليوزر. وعشان كده فيه مقاييس أحدث زي **LCP** (إمتى أكبر حاجة ظهرت)، تقدر تشوفها في تابة **Performance** و Lighthouse.

لو حابب تتعمق: الـ Preload scanner و preload

تقدر تعدّيه في أول قراية

المتصفح عنده Parser تاني خفيف بيجري قدام الـ Parser الأساسي وهو واقف مستني سكريبت، ويبعت طلبات للحاجات اللي هيلاقيها بعد كده. عشان كده هتلاقي طلبات كتير بتبدأ مع بعض في الـ Waterfall حتى لو فيه سكريبت عادي. ولو حاجة مهمة جداً مش ظاهرة في الـ HTML (زي خط جوه CSS)، تقدر تقول للمتصفح يجيبها بدري: `<link rel="preload" href="/font.woff2" as="font" crossorigin>`. والسيرفر ممكن يقول ده قبل ما الـ HTML نفسه يجهز بـ Status `103 Early Hints` (الفصل 8)، وده واحد من قيم الـ Initiator.

ملخص الفصل

- HTML → DOM، و CSS → CSSOM، والاتنين → Render tree → Layout → Paint.
- الـ Parser بيكتشف الطلبات وهو بيقرا، فالـ Initiator بتاعها `Parser`. وطلبات `fetch` بتاعها `Script`.
- الـ CSS بيوقف الرسم، والـ script العادي بيوقف قراية الـ HTML.
- `defer` و`type="module"`: بعد الـ HTML وبالترتيب. و`async`: أول ما يوصل.
- الأزرق = DOMContentLoaded، والأحمر = Load.

صح ولا غلط؟

1. السكريبتات اللي عليها `async` بتتنفذ بنفس ترتيبها في الـ HTML دايماً. صح / غلط
2. `<script type="module">` بيتصرف زي `defer`. صح / غلط
3. Load بيحصل قبل DOMContentLoaded. صح / غلط

أسئلة مراجعة

1. رتّب الخطوات من الـ HTML لحد البيكسلات، وقول كل خطوة بتعمل إيه.
2. ليه المتصفح بيستنى الـ CSS قبل ما يرسم؟
3. إيه الفرق بين `Parser` و`Script` في عمود Initiator؟
4. سكريبت Analytics وسكريبت التطبيق: كل واحد تحطه بـ async ولا defer؟ وليه؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح أي موقع، وفي Network بص على الرقمين DOMContentLoaded و Load تحت. وبعدين اعمل نفس الحاجة مع Slow 4G (الفصل 18).
2. متوسط اعمل صفحة فيها `<script src="a.js">` و`<script src="b.js">`، كل واحد بيعمل `console.log`. جرب الترتيب مع العادي و async و defer.
3. صعب في صفحة كبيرة، دوس ⇧ وحط الماوس على طلب JS في Network: هتشوف مين طلبه (أخضر) ومين هو طلبه (أحمر). ارسم السلسلة.

17

المرحلة 6 · DevTools والـ Debugging

الفصل السابع عشر

DevTools: الشبابيك اللي جوه المتصفح

كل اللي فات كان عن اللي بيحصل في السلك. من هنا ورايح هنشوفه بعينينا، من جوه المتصفح نفسه. Chrome DevTools عبارة عن مجموعة أدوات مدمجة في المتصفح بتوريك كل طلب، وكل خطأ، وكل كوكي، وكل عنصر في الصفحة. الفصل ده جولة: إزاي تفتحه، وكل Panel بيعمل إيه، والاختصارات اللي هتوفر عليك وقت كل يوم.

بعد الفصل ده هتكون عارف

- ليه DevTools اتعمل، وإزاي كان الـ Debugging قبله
- تفتحه بـ ⌘⌥I و ⌘⌥J و ⌘⌥C
- خريطة الـ Panels: Elements و Console و Sources و Network و Application وغيرهم
- الـ Drawer بـ Esc، و Command Menu بـ ⌘⇧P
- مكانه في الشاشة، والإعدادات، واللغة
- حاجات تعملها في أول 5 دقايق
- أدوات الـ AI في DevTools وحدودها

17.1 قبل DevTools: alert وربنا يستر

في أول الألفينات، لو الـ JS بتاعك بايظ، المتصفح كان بيعمل... ولا حاجة. أيقونة صغيرة صفرا في الركن تحت في Internet Explorer لو كنت محظوظ. فالمطورين كانوا بيكتبوا `alert('وصلت هنا')` في كل حتة، ويقفلوا 30 نافذة عشان يعرفوا الكود وقف فين. ومفيش أي طريقة تشوف الطلبات اللي الصفحة بتعملها.

سنة 2006 جت Extension لـ Firefox اسمها **Firebug**، وغيرت كل حاجة: Console حقيقي، وتقدر تدوس على أي عنصر وتشوف الـ HTML والـ CSS بتوعه وتعدلهم مباشرة، وقايمة بكل الطلبات. المطورين حبوها لدرجة إن كل المتصفحات بنت نسخ مدمجة منها: Safari عمل Web Inspector، و Chrome من أول نسخة سنة 2008 كان فيه **Developer Tools**. والنهارده DevTools متطور لدرجة إنه بقى أهم أداة عند أي Frontend developer بعد المحرر نفسه.

17.2 تفتحه إزاي

الاختصار	بيفتح فين	تستخدمه لما
⌘⌥I	آخر Panel كنت فاتحه	الطريقة العامة. Inspect = I
⌘⌥J	الـ Console مباشرة	عاوز تشوف الأخطاء أو تجرب سطر JS
⌘⌥C	Elements، ومعاه وضع اختيار العنصر	عاوز تعرف الحتة دي في الصفحة إيه
Right-click ← Inspect	Elements، على العنصر اللي دوست عليه	نفس اللي فوق بالماوس
View ← Developer ← Developer Tools	من القايمة	لو نسيت الاختصار

جدول 17.1 فتح DevTools على الـ Mac

> **حاجة مشتركة: ⌥ هي Option**
>
> على كيبورد الـ Mac، ⌘ = Command، و⌥ = Option (مكتوب عليه alt كمان)، و⇧ = Shift، و^ = Control. ولما تقرا دوكيومنتيشن مكتوبة للـ Windows، بدّل `Ctrl` بـ ⌘، و `Alt` بـ ⌥، غالباً. يعني `Ctrl+Shift+I` = ⌘⌥I (دي حالة خاصة) و `Ctrl+Shift+P` = ⌘⇧P.

17.3 خريطة الـ Panels

فوق في DevTools فيه صف تابات، كل واحدة اسمها **Panel**. مش محتاج تعرفهم كلهم النهارده. دول اللي هتستخدمهم، ومرتبين حسب أهميتهم في الكتاب ده:

Network

كل طلب الصفحة بتعمله: الـ Status والـ Headers والـ Body والوقت. نجم الكتاب ده، وليه فصلين كاملين (18 و 19).

Console

الأخطاء والتحذيرات، والـ `console.log` بتاعك، ومكان تكتب فيه JS على الصفحة نفسها. الفصل 20.

Application

الكوكيز و localStorage و sessionStorage و IndexedDB و Service Workers. تشوفهم وتعدلهم وتمسحهم. الفصل 21.

Elements

الـ DOM الحقيقي (مش الـ HTML اللي جه من السيرفر، لكن بعد ما الـ JS غيّر فيه)، والـ CSS بتاع أي عنصر، وتعدّل وتشوف النتيجة فوراً.

Sources

ملفات الصفحة، والـ Debugger: توقف الكود عند سطر وتمشي سطر سطر. هنلمس منه الأساسيات في الفصل 22.

Performance و Lighthouse

ليه الصفحة بطيئة: تسجيل لكل حاجة المتصفح عملها، وتقرير بالدرجات والنصايح. خارج نطاق الكتاب، بس اعرف إنهم موجودين.

ولو مش لاقي Panel (زي Application أو Security)، هتلاقيه في زرار `»` آخر صف التابات. وتقدر تسحب التابات وترتبها زي ما تحب.

17.4 الـ Drawer والـ Command Menu: اتنين هيغيروا حياتك

الـ Drawer: Console تحت أي Panel

وانت في Network، عاوز تكتب حاجة في الـ Console. هتسيب Network؟ لأ. دوس Esc، هيطلعلك **Drawer** (درج) تحت فيه Console. يعني Network فوق و Console تحت في نفس الوقت. وفي الدرج ده كمان تابات تانية زي **Network conditions** و**Request conditions** و**Search**. وEsc تاني يقفله.

Command Menu: أي حاجة بالكتابة

DevTools فيه مئات الأوامر. مش لازم تفتكر هي فين. دوس ⌘⇧P واكتب اللي عاوزه، زي Spotlight في الـ Mac بالظبط:

⌘⇧P ← screenshot	"Capture full size screenshot": صورة للصفحة كلها من فوق لتحت، حتى الجزء اللي مش ظاهر.
⌘⇧P ← dark	غيّر ثيم DevTools نفسه لـ Dark أو Light.
⌘⇧P ← disable javascript	شغّل الصفحة من غير JS خالص. مفيد تعرف الموقع بيعتمد على JS قد إيه.
⌘⇧P ← show network conditions	افتح تابة تغيير السرعة والـ User-Agent في الـ Drawer.
⌘P (من غير ⇧)	افتح أي ملف من ملفات الصفحة بالاسم (زي VS Code بالظبط).
⌘⌥F	دوّر على كلمة في **كل** ملفات الصفحة مرة واحدة (تابة Search في الـ Drawer).

17.5 ظبّطه مرة واحدة

⋮ ← Dock side	مكان DevTools: تحت، أو يمين، أو شمال، أو في نافذة لوحده. على شاشة Laptop، **يمين** غالباً أحسن عشان الصفحة تفضل عريضة كفاية. و⌘⇧D بيبدّل بين آخر مكانين.
⚙ أو ? أو Fn+F1	الإعدادات (الـ `?` وانت واقف جوه DevTools). فيها Preferences و Shortcuts و Experiments.
Settings ← Language	لغة DevTools. سيبها **English**: رسايل الأخطاء ومحركات البحث والدوكيومنتيشن وكل الكتاب ده بالإنجليزي، فهتلاقي الكلام بنفس الأسماء.
⌘⇧M	Device mode: شوف الصفحة كأنها على موبايل بمقاس معين. (والـ Network throttling اللي هنشوفه بعدين جزء منه.)
⌘⇧+ / ⌘- / ⌘0	كبّر أو صغّر **خط DevTools نفسه** (وانت واقف جواه). مفيد على شاشة صغيرة.

5 دقايق هتوفر عليك ساعات

1) اعمل Dock على اليمين. 2) في Network، فعّل **Disable cache** (هيشتغل بس وDevTools مفتوح، الفصل 15). 3) في Network، فعّل **Preserve log** لو بتطور صفحة فيها Redirect أو Login (الفصل 18). 4) في Console، فعّل **Preserve log** برضه. 5) اعمل Profile منفصل في Chrome للتطوير من غير Extensions، لأن الـ Extensions بتحقن كود وبتعمل طلبات وبتطلع أخطاء مش بتاعتك.

> **خلي بالك: الـ Extensions بتلخبط**
>
> هتلاقي في الـ Console أخطاء زي `chrome-extension://abc.../content.js` أو في Network طلبات مش فاكر إنك عملتها. دي غالباً من Extensions زي Ad blockers أو Password managers أو Grammarly. قبل ما تقلق، جرب الصفحة في **Incognito** (⇧⌘N)، لأن الـ Extensions مقفولة فيه بشكل افتراضي. ولو الخطأ اختفى، مش بتاعك.

17.6 أدوات الـ AI جوه DevTools

من 2024، Chrome بقى فيه حاجات بتستخدم Gemini جوه DevTools: زرار **Understand this error** جنب الأخطاء في الـ Console (اسمها Console insights)، و Panel اسمه **AI assistance** تسأله عن طلب في Network أو عنصر في Elements. ممكن تكون مفيدة جداً وانت بتتعلم، بس اعرف حدودها:

- محتاجة تكون عامل Sign in في Chrome، وسنك 18+، وفي دولة متاحة فيها الخدمة، ولغة DevTools **إنجليزي**، وتفعّلها من Settings ← AI Innovations. فممكن متلاقيهاش خالص عندك.
- لما تستخدمها، بيانات من الصفحة (الخطأ، والكود اللي حواليه، والطلب) **بتتبعت لجوجل**. متستخدمهاش على حاجة فيها بيانات حساسة أو كود شغل سري.
- زي أي AI: ممكن تغلط بثقة. اعتبرها زميل بيقترح، مش حكم نهائي. والكتاب ده هدفه إنك تفهم الرسالة بنفسك الأول.

ملخص الفصل

- DevTools اتولد من Firebug، والنهارده مدمج في كل متصفح.
- ⌥⌘I للفتح، و⌘J للـ Console، و⌘C لاختيار عنصر.
- أهم Panels في الكتاب: Network و Console و Application، ومعاهم Elements و Sources.
- Esc = Drawer فيه Console تحت أي Panel، و⇧⌘P = أي أمر بالكتابة.
- سيب DevTools بالإنجليزي، وفعّل Disable cache و Preserve log، وجرّب في Incognito لو شاكك في Extension.
- أدوات الـ AI ليها شروط، وبتبعت بيانات لجوجل، وممكن تغلط.

صح ولا غلط؟

1. ⌥⌘J بيفتح الـ Console مباشرة. صح / غلط
2. تابة Elements بتوريك الـ HTML زي ما جه من السيرفر بالظبط، من غير أي تعديلات من الـ JS. صح / غلط
3. Disable cache بيفضل شغال حتى بعد ما تقفل DevTools. صح / غلط

؟ أسئلة مراجعة

1. إيه المشكلة اللي Firebug حلها للمطورين؟
2. عاوز تشوف Network و Console في نفس الوقت. تعمل إيه؟
3. ليه يفضّل تسيب DevTools بالإنجليزي؟
4. لقيت خطأ غريب في الـ Console من ملف `chrome-extension://`. تعمل إيه؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح DevTools بالـ 3 اختصارات، واعمل Dock على اليمين، وفعّل Disable cache.
2. متوسط استخدم Command Menu تعمل Screenshot كامل لصفحة طويلة، وبعدين شغّل الصفحة من غير JS. إيه اللي اختفى؟
3. صعب افتح موقع في Profile عادي وفي Incognito، وقارن عدد الطلبات في Network وعدد الرسايل في Console. الفرق جاي منين؟

18

المرحلة 6 · DevTools والـ Debugging

الفصل التامن عشر

Network Panel (1): تقرا قايمة الطلبات

ده الفصل اللي كل اللي فات كان بيمهدله. Network Panel بيوريك كل طلب الصفحة عملته، بنفس الكلام اللي اتعلمناه: Methods و Status codes و Headers وكاش و CORS وتوقيت. هنفصّص الشاشة حتة حتة: شريط الأدوات، والفلاتر، وكل عمود، والـ Waterfall، وتابة Timing مرحلة مرحلة، وإزاي تقلّد نت بطيء على جهازك.

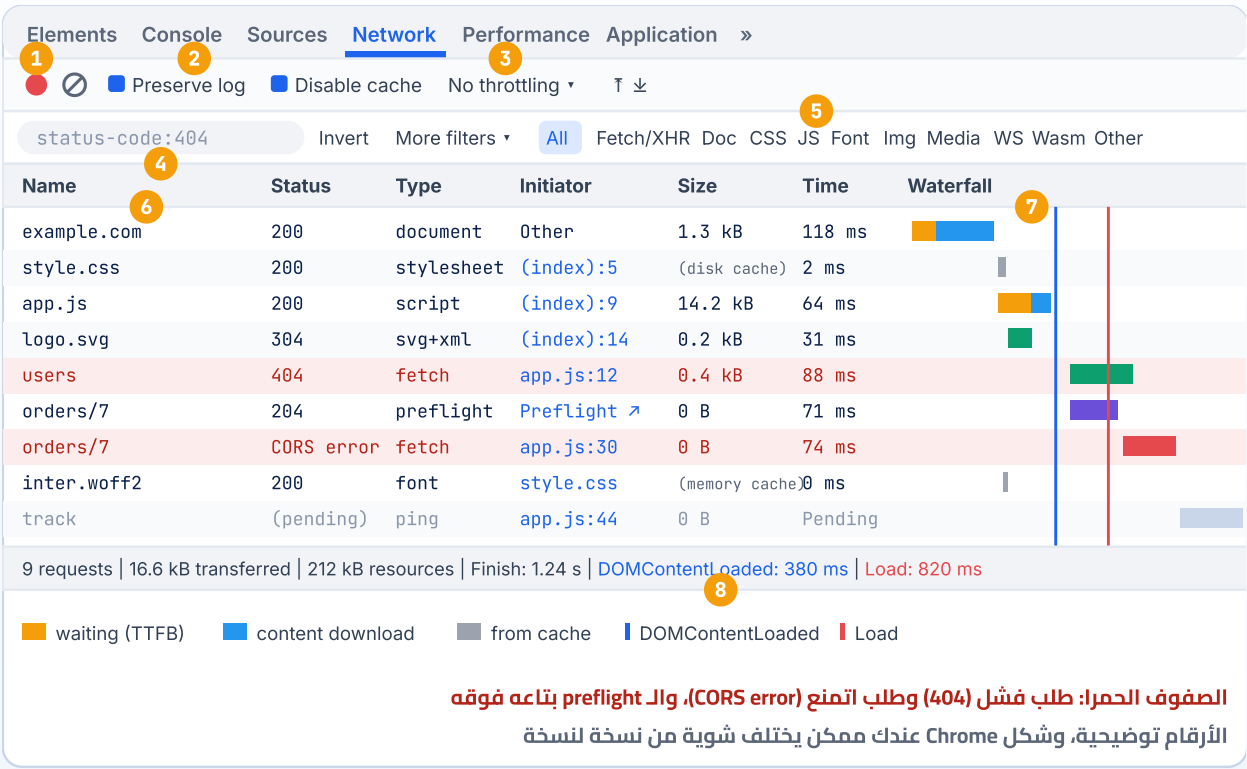
بعد الفصل ده هتكون عارف

- ليه لازم تفتح Network قبل ما الصفحة تحمّل
- شريط الأدوات: Record و Clear و Preserve log و Disable cache و Throttling
- الفلاتر: الأنواع و Property filters زي status-code: و method: و domain:
- كل عمود: Name و Status و Type و Initiator و Size و Time، وتضيف Protocol وغيره
- قيم Status الخاصة: (pending) و (canceled) و (failed) و CORS error و (blocked)
- الـ Waterfall: Queueing و Stalled و DNS و SSL و Waiting (TTFB) و Download
- Throttling: تقلّد Slow 4G و Offline
- Status bar: requests و transferred و resources و Finish

18.1 أول قاعدة: افتحه الأول، وبعدين حمّل

Network Panel بيسجل الطلبات **من ساعة ما يتفتح**. لو فتحته بعد ما الصفحة خلصت تحميل، هتلاقيه فاضي أو فيه طلبات قليلة، وتقول "فين الطلبات؟". فالطريقة دايماً: افتح DevTools، روح Network، وبعدين اعمل Reload (⌘R). وهو حتى بيقولك كده لما يبقى فاضي: "Recording network activity... Perform a request or hit ⌘ R to record the reload".

18.2 الشاشة حتة حتة



شكل 18.1 نسخة مرسومة من Network Panel، فيها أغلب الحالات اللي هتقابلها: Document وكاش و 304 و 404 و Preflight و CORS error وطلب لسه شغال

الأرقام 1 و 2 و 3: شريط الأدوات

1 زرار التسجيل (أحمر = شغال) وجنبه المسح. **2** Preserve log و Disable cache. **3** قايمة Throttling، وبعدها Import/Export.

الرقمين 4 و 5: الفلاتر

4 خانة الفلتر بالكتابة (هنا `status-code:404`) ومعاها Invert و More filters. **5** أزرار الأنواع: All و Fetch/XHR و JS وغيرهم.

رقم 6: الأعمدة

Name و Status و Type و Initiator و Size و Time. اعمل Right-click على العناوين تضيف أعمدة تانية.

الرقمين 7 و 8: الوقت والملخص

7 الـ Waterfall والخطين الأزرق (DOMContentLoaded) والأحمر (Load). **8** شريط الملخص تحت: العدد والحجم والوقت.

18.3 شريط الأدوات

● (أحمر)	**Record**: شغال أو واقف. لو رمادي، مش بيسجل حاجة. ⌘E يبدّل.
⊘	**Clear**: امسح القايمة. مفيد قبل ما تدوس زرار في الصفحة عشان تشوف طلباته هو بس.

`Preserve log`	متمسحش القايمة لما الصفحة تتغير. من غيره، أي Navigation أو Redirect بيمسح كل حاجة، فلو فورم Login بيعمل POST وبعدين Redirect، هتشوف الصفحة الجديدة بس وتضيع الـ POST اللي انت عاوزه.
`Disable cache`	متستخدمش الكاش طول ما DevTools مفتوح (الفصل 15). كده بتشوف اللي هيحصل لزائر جديد.
`No throttling ▾`	قلّد شبكة أبطأ (آخر الفصل).
`⤒ ⤓`	Import و Export لملف HAR (الفصل 19).
`⚙` (يمين الشريط)	إعدادات الـ Panel: **Big request rows** (كل صف سطرين، فيه الرقمين بتوع الحجم والوقت)، و**Show overview** (شريط الوقت فوق)، و Group by frame.

18.4 الفلاتر: تلاقي الإبرة في 200 طلب

فلاتر الأنواع

الأزرار `All` و`Fetch/XHR` و`Doc` و`CSS` و`JS` و`Font` و`Img` و`Media` و`Manifest` و`WS` (WebSocket) و`Wasm` و`Other`. وأهمهم ليك: **Fetch/XHR**، لأنه بيوريك طلبات الـ API اللي كودك بيعملها بس، من غير الصور والخطوط. وتقدر تختار أكتر من نوع بـ ⌘+Click.

الفلتر بالكتابة

اكتب أي كلمة وهيوريك الطلبات اللي الـ URL بتاعها فيه الكلمة دي. بس القوة الحقيقية في الـ **Property filters**: `خاصية:قيمة`:

الفلتر	بيوريك
`status-code:404`	الطلبات اللي رجعت 404 بالظبط
`method:POST`	طلبات POST بس
`domain:api.shop.dev`	طلبات الدومين ده. وبيقبل `*`: `domain:*.google.com`
`-domain:localhost`	الـ `-` في الأول = **عكس** الفلتر: كل حاجة ما عدا localhost
`larger-than:100k`	الملفات اللي أكبر من 100 KB. لقيت ليه الصفحة تقيلة
`mime-type:application/json`	الردود اللي نوعها JSON
`has-response-header:set-cookie`	الردود اللي بتحط كوكيز
`scheme:http`	اللي ماشي من غير HTTPS (شاكك في Mixed content؟)
`is:running`	الطلبات اللي لسه شغالة
`/api\/v\d/`	بين `/ /` = Regex

جدول 18.1 أشهر Property filters

وتقدر تكتب أكتر من فلتر بمسافة بينهم، ومعناهم "و": `method:POST status-code:500` = طلبات POST اللي رجعت 500. وجنب خانة الفلتر فيه **Invert** (اعكس النتيجة كلها)، و**More filters** وفيها: Hide data URLs، و Hide extension URLs (اخفي طلبات الـ Extensions، مفيد جداً)، و Blocked response cookies، و Blocked requests، و 3rd-party requests.

مثال عملي: فين الـ API اللي بيفشل؟

صفحة فيها 150 طلب، وفيه حاجة مش ظاهرة. دوس **Fetch/XHR**، واكتب `-status-code:200`. فاضل قدامك الطلبات اللي كودك عملها ورجعت أي حاجة غير 200. كده في 3 ثواني.

18.5 الأعمدة: كل واحد بيقولك إيه

Name	الأعمدة الافتراضية
Name	آخر جزء من الـ URL. حط الماوس عليه تشوف الـ URL كامل. ودوس عليه تفتح التفاصيل (الفصل 19).
Status	الـ Status code من الفصل 8، أو حالة خاصة (الجزء الجاي). وفي Big request rows بيظهر تحته الـ Reason phrase زي `Not Found`.
Type	نوع الحاجة: `document` (الصفحة) و `stylesheet` و `script` و `xhr` و `fetch` و `font` و `png` و `preflight` و `ping` و `websocket`.
Initiator	مين اللي طلب الحاجة دي (الفصل 16): `Parser` (الـ HTML)، أو `Script` بسطر معين (`app.js:12`، دوس عليه يوديك للسطر في Sources)، أو `Redirect`، أو `Preflight`، أو `Other` (زي إنك كتبت العنوان بنفسك).
Size	الحجم **المنقول على الشبكة** (مضغوط ومعاه الـ Headers). ولو جه من الكاش هتلاقي `(disk cache)` أو `(memory cache)`. وفي Big request rows الرقم التاني = الحجم الحقيقي بعد فك الضغط.
Time	من بداية الطلب لآخر Byte. والرقم التاني في Big request rows = الـ Latency (لحد أول Byte).
Waterfall	كل طلب كخط على محور الوقت: بدأ إمتى وخلص إمتى. حط الماوس على أي خط يطلعلك تفصيل المراحل.

وفيه أعمدة مستخبية مفيدة جداً. اعمل **Right-click على عناوين الأعمدة** وفعّل:

`Method`	GET ولا POST ولا OPTIONS. أول عمود أنصحك تفعّله.
`Domain`	الطلب ده رايح لمين. مفيد تشوف كام Third-party في الصفحة.

`Protocol`	`h2` و `h3` و `http/1.1` (الفصل 11).
`Remote address`	الـ IP والـ Port اللي اتكلمنا معاه فعلاً، زي `104.20.23.154:443`. يأكدلك إن الـ DNS ودّاك المكان الصح، أو إن `/etc/hosts` اشتغل.
`Priority`	أولوية الطلب عند المتصفح (Highest لحد Lowest).
`Cookies / Set cookies`	عدد الكوكيز اللي اتبعتت، واللي السيرفر حطها.
`Response Headers ←`	تضيف أي Header كعمود، زي `cache-control` أو `cf-cache-status`. وتقدر تعمل **Manage Header Columns** وتضيف أي اسم.

وأي عمود تدوس على عنوانه يرتّب القايمة بيه: رتّب بـ Size تلاقي أتقل ملف، وبـ Time تلاقي أبطأ طلب.

18.6 قيم Status الخاصة

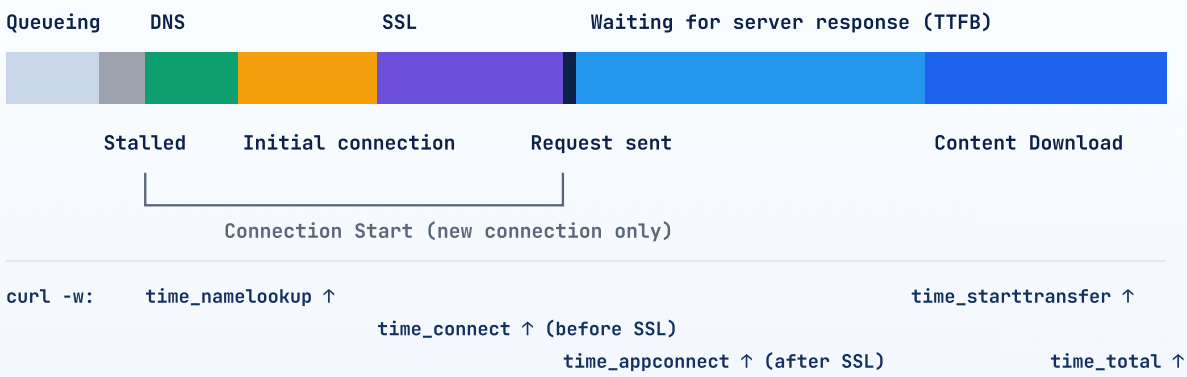
مش كل الطلبات بترجع رقم. لو الطلب ماوصلش لرد HTTP أصلاً، هتلاقي حاجة بين أقواس، ودي مهمة جداً في الـ Debugging لأنها بتقولك إن المشكلة **قبل** السيرفر:

القيمة	معناها	تعمل إيه
`(pending)`	الطلب لسه شغال، مستني رد	لو فضل كده كتير: السيرفر مش بيرد، أو Long polling
`(canceled)`	المتصفح لغاه	غالباً طبيعي: الصفحة اتغيرت، أو الكود عمل `abort()`، أو نفس الصورة اتطلبت تاني
`(failed)`	مفيش رد خالص	دوس عليه وشوف `net::ERR_...`: الـ DNS، أو السيرفر مقفول، أو الشبكة (الفصل 20)
`CORS error`	الرد اتحجز بسبب CORS	شوف سطر الـ `preflight` والـ Console (الفصل 14)
`(blocked:mixed-content)`	طلب `http://` من صفحة HTTPS	حوّله HTTPS (الفصل 12)
`(blocked:devtools)`	انت اللي منعته من DevTools	شيل الـ Block من Request conditions (الفصل 19)
`(blocked:other)`	اتمنع من حاجة في المتصفح	غالباً Ad blocker أو Extension، وفي الـ Console هتلاقي `net::ERR_BLOCKED_BY_CLIENT`. جرب Incognito
`(blocked:csp)`	الـ Content Security Policy بتاعة الصفحة منعته	الموقع نفسه مش سامح بالدومين ده (الفصل 20)

جدول 18.2 قيم في عمود Status مش أرقام

18.7 الـ Waterfall و Timing: الرحلة بالملي ثانية

فاكر رحلة الطلب في الفصل 2، والأرقام اللي طلّعناها من curl بـ `-w` في الفصل 10؟ دي نفسها. دوس على أي طلب ← تابة **Timing**، أو حط الماوس على خطه في الـ Waterfall:



شكل 18.2 مراحل الطلب في تابة Timing بالترتيب، وتحتها أرقام curl اللي بتقابلها (curl بيقيس تراكمي من البداية)

المرحلة	اللي بيحصل	لو طويلة يبقى
Queueing	الطلب واقف قبل ما يبدأ	فيه طلبات أولويتها أعلى، أو 6 اتصالات مشغولين على HTTP/1.1 (الفصل 11)
Stalled	اتأخر لأي سبب تاني قبل ما يتبعت	نفس أسباب Queueing، أو مستني اتصال يفضى
DNS Lookup	ترجمة الاسم لـ IP (الفصل 5)	DNS بطيء، أو Third-party دومينات كتير
Initial connection	TCP Handshake (الفصل 4)	السيرفر بعيد (RTT عالي)
SSL	TLS Handshake (الفصل 12)	بعيد، أو سيرفر متظبط وحش
Request sent	بعت الطلب نفسه	نادراً طويلة، إلا لو بترفع ملف كبير
Waiting for server response	**TTFB**: السيرفر بيفكر + رحلة	**السيرفر بطيء**: Database، أو كود تقيل. ده أشهر سبب
Content Download	تحميل الـ Body	الملف كبير أو النت بطيء

جدول 18.3 كل مرحلة في Timing وإمتى تقلق منها

حاجة مشتركة: نفس الرحلة في 3 أماكن

رسمة الفصل 2، وأرقام `curl -w`، وتابة Timing: نفس المحطات بالظبط. الفرق إن curl بيقيس **تراكمي** (كل رقم من البداية)، و DevTools بيوريك **كل مرحلة لوحدها**. فلو قالك DevTools إن Waiting = 1.8 s، يبقى المشكلة في السيرفر، ولو جربت بـ curl هتلاقي `time_starttransfer - time_appconnect` قريب من نفس الرقم. ولو الاتنين متفقين، اتأكدت.

18.8 Throttling: جرّب نت وحش من غير ما تنزل الشارع

الموقع شغال طيارة على الـ Wi-Fi بتاعك. بس اليوزر في المترو على G4 ضعيف. عشان تشوف اللي هو شايفه، من قايمة **No throttling** اختار:

`Fast 4G`	نت موبايل كويس.
`Slow 4G`	نت موبايل متوسط. هنا هتبدأ تشوف المشاكل.
`3G`	بطيء. كل حاجة تقيلة هتبان.
`Offline`	مفيش نت خالص. جرب الكود بتاعك بيعمل إيه لو `fetch` فشل: بيوري رسالة كويسة، ولا بيقع؟
`Add… / Custom`	اعمل Profile بسرعة وتأخير من عندك.

ولما تشغّل الـ Throttling، هتلاقي علامة تحذير صفرا جنب اسم Network، عشان متنساش إنه شغال وتقعد تقول "ليه النت بطيء النهارده؟".

خلي بالك: الـ Throttling تقليد، مش حقيقة

هو بيأخر الطلبات ويحدد السرعة داخل المتصفح، بس مش بيقلّد كل حاجة في شبكة موبايل حقيقية (زي Packets بتضيع). كفاية جداً عشان تحس بالمشكلة وتشوف الـ Loading states بتاعتك، بس القياس الدقيق يبقى على أجهزة حقيقية.

18.9 شريط الملخص تحت

9 requests | 16.6 kB transferred | 212 kB resources | Finish: 1.24 s | DOMContentLoaded: 380 ms | Load: 820 ms

`9 requests`	عدد الطلبات (لو فيه فلتر: `3 / 9 requests`).
`16.6 kB transferred`	اللي اتنقل فعلاً على الشبكة. صغير لأن فيه كاش وضغط.
`212 kB resources`	الحجم الحقيقي لكل الحاجات بعد فك الضغط. الفرق بينه وبين اللي فوق = توفير الكاش والضغط.
`Finish: 1.24 s`	آخر طلب خلص إمتى. ممكن يفضل يزيد لو الصفحة بتعمل طلبات طول الوقت (Analytics مثلاً).
`DOMContentLoaded / Load`	الخطين الأزرق والأحمر (الفصل 16).

ملخص الفصل

- افتح Network قبل التحميل، أو اعمل ⌘R وهو مفتوح.
- Preserve log عشان الـ Redirects والـ Login، و Disable cache عشان تشوف زي الزائر الجديد.
- Fetch/XHR يوريك طلبات كودك. والـ Property filters (`status-code:` و`method:` و`domain:` و`-` للعكس) بتلاقي أي حاجة.
- فعّل أعمدة Method و Protocol و Remote address، ورتّب بـ Size و Time.
- Status بين أقواس = مفيش رد HTTP: `(failed)` و`(canceled)` و`(blocked:...)`، و`CORS error`.
- Timing: Queueing و DNS و Initial connection و SSL و Waiting (TTFB) و Download. والـ Waiting الطويل = السيرفر.
- Throttling يقلّد Slow 4G و G3 و Offline، والملخص تحت بيقولك العدد والحجم والوقت.

صح ولا غلط؟

1. الفلتر `-domain:google.com` بيوريك طلبات google.com بس. صح / غلط
2. Status `(failed)` معناه إن السيرفر رد بـ 500. صح / غلط
3. لو مرحلة Waiting for server response هي الأطول، يبقى المشكلة غالباً في السيرفر. صح / غلط

أسئلة مراجعة

1. ليه فورم Login ممكن "يختفي" الطلب بتاعه من Network؟ وإيه الحل؟
2. اكتب فلتر يوريك طلبات POST اللي رجعت أي حاجة غير 201.
3. إيه الفرق بين `transferred` و`resources` في الشريط تحت؟
4. طلب الـ DNS Lookup و Initial connection بتوعه صفر. ليه؟
5. إيه فايدة عمود Remote address لو عدّلت `/etc/hosts`؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح موقع أخبار، وفعّل أعمدة Method و Domain و Protocol، ورتّب بـ Size. إيه أتقل 3 ملفات؟ وكام Domain مختلف؟
2. متوسط نفس الموقع بـ Disable cache، مرة No throttling ومرة Slow 4G. قارن Finish و Load.
3. صعب اختار طلب واحد (الـ document)، واكتب كل مراحل Timing بتاعته، وبعدين قيسه بـ `curl -w @timing.txt`، وقارن كل مرحلة بالتانية.

19

المرحلة 6 · DevTools والـ Debugging

الفصل التاسع عشر

Network Panel (2): جوه الطلب الواحد

القايمة بتقولك فيه مشكلة. التفاصيل بتقولك المشكلة إيه. لما تدوس على أي طلب، بيفتح جنبه لوح فيه كل حاجة: الـ Headers اللي رايحة وجاية، والـ Body اللي اتبعت، والرد خام ومنسق، ومين طلبه، والكوكيز. وبعدين هنشوف الأدوات اللي بتخليك تتحكم: تنسخ الطلب لـ curl أو fetch، وتعدله وتبعته تاني، وتمنع طلبات، وتغير الرد نفسه من غير ما تلمس السيرفر، وتصدّر كل حاجة في ملف HAR.

بعد الفصل ده هتكون عارف

- تابة Headers: General و Request و Response و Raw و Provisional headers
- تابة Payload: Query string و Form data و JSON
- Preview و Response: منسق مقابل خام
- Initiator: مين طلب الطلب ده، وسلسلته
- تابة Cookies: اللي اتبعت، واللي اتحط، واللي اتمنع وليه
- Copy as cURL و Copy as fetch، والتعديل وإعادة الإرسال
- Request conditions: تمنع طلب أو Domain
- Overrides: تغيّر الرد أو الـ Headers من جهازك
- HAR: تصدّر وتستورد، و sanitized مقابل sensitive

19.1 تابة Headers: الحوار كله

دوس على اسم أي طلب. هيفتح لوح فيه تابات، وأولهم **Headers**، ومقسوم 3 أجزاء:

Headers · users (طلب الـ 404 من رسمة الفصل 18)

```
▼ General
Request URL:
https://api.shop.dev/users
Request Method:  GET
Status Code:     ● 404 Not Found
Remote Address:  104.21.48.3:443
Referrer Policy: strict-origin-when-
cross-origin
```

1 **General**: الملخص. الـ URL الكامل (اتأكد إنه اللي انت فاكره، خصوصاً الـ Query)، والـ Method، والـ Status بلون، والـ IP اللي اتكلمنا معاه، وسياسة الـ Referer (الفصل 9).

2 **Response Headers**: اللي السيرفر رد بيه. هنا تدور على `content-type` و`set-cookie` و`cache-control` و`access-control-*`.	▼ Response Headers content-type: application/json; charset=utf-8 cache-control: no-store access-control-allow-origin: http://localhost:5173
3 **Request Headers**: اللي المتصفح بعته فعلاً. اتأكد هنا إن الـ `authorization` اتبعت، وإن الـ `content-type` صح. ولاحظ الـ Pseudo-headers بتوع HTTP/2 (الفصل 9).	▼ Request Headers :authority: api.shop.dev :method: GET :path: /users accept: */* authorization: Bearer eyJhbGciOi... origin: http://localhost:5173

وجنب عنوان كل جزء فيه مفتاح **Raw**: يوريك الـ Headers بالشكل الخام زي ما هي في الـ HTTP (أو أقرب حاجة ليها في HTTP/2)، بترتيبها الحقيقي. مفيد لما تحب تنسخ أو تقارن مع curl.

> ⚠ **Provisional headers are shown**
>
> ساعات هتلاقي تحذير أصفر فوق Request Headers بالجملة دي. معناها: "اللي قدامك مش الـ Headers الحقيقية اللي اتبعتت، دي اللي المتصفح كان ناوي يبعتها". بيحصل لما الطلب جه من الكاش (فمفيش طلب حقيقي اتبعت)، أو اتمنع (CORS أو Extension)، أو لسه مكملش. يعني لو شايف التحذير ده، اسأل نفسك الأول: الطلب ده وصل للسيرفر أصلاً؟

19.2 تابة Payload: اللي انت بعته

بتظهر بس لو الطلب فيه Query string أو Body. وفيها اللي كودك بعته بالظبط، وده أهم مكان تتأكد فيه إن البيانات وصلت بالشكل اللي السيرفر مستنيه:

`Query String Parameters`	الـ `?key=value` متقسمة. وفيه زرار **view URL-encoded** يوريك الشكل المتشفّر (الفصل 6)، و**view decoded** العادي.
`Form Data`	فورم عادي (`urlencoded` أو `multipart`). كل حقل لوحده.
`Request Payload`	Body بـ JSON أو نص. بيتعرض كشجرة تقدر تفتحها، و**view source** يوريك النص زي ما اتبعت حرف حرف.

مشكلة 1 **السيرفر بيقول 400: 'email' missing field، وأنا باعته!**

الـ Console مفيهاش حاجة، والكود شكله سليم: `body: {email, password}`.

افتح Payload و view source: هتلاقي `[object Object]`. نسيت `JSON.stringify`. أو هتلاقي الـ JSON صح، بس في الـ Headers الـ `content-type` هو `text/plain` لأنك نسيت الـ Header، فالسيرفر مش بيقرا الـ Body كـ JSON (الفصل 7). الـ Payload والـ Request Headers مع بعض بيكشفوا المشكلتين في ثانية.

19.3 Preview و Response

Preview

الرد **منسق**: JSON كشجرة تفتحها، وصورة كصورة، و HTML معروض. للقراية السريعة.

Response

الرد **خام** زي ما جه. لو الـ JSON بايظ (فاصلة زيادة، أو جالك صفحة HTML بدل JSON)، Preview مش هيعرضه، و Response هيوريك الحقيقة.

مثال عملي: Unexpected token '<'

الكود `await res.json()` عمل وطلعلك في الـ Console: `SyntaxError: Unexpected token '<', "<!DOCTYPE "... is not valid JSON`. افتح الطلب ← Response: هتلاقي صفحة HTML. يعني الـ URL غلط والسيرفر رجّعلك صفحة 404 بتاعة الموقع، أو الـ Dev server رجّعلك `index.html`. الحرف `<` هو أول حرف في `<!DOCTYPE`. وده من أشهر الأخطاء في كورسات JS كلها.

19.4 تابة Initiator: مين طلب ده؟

لقيت طلب غريب ومش عارف جاي منين؟ تابة Initiator فيها جزئين:

- **Request call stack**: لو الطلب جه من JS، هتلاقي سلسلة الدوال اللي وصلت لحد الـ `fetch`، كل واحدة بملفها ورقم سطرها، ودوس عليها توديك للسطر. يعني "`loadUsers` في `app.js:12`، اتنادت من `init` في `app.js:50`".
- **Initiator chain**: السلسلة على مستوى الطلبات: الصفحة طلبت `app.js`، و`app.js` طلب `users`.

ونفس المعلومة بسرعة في القايمة: دوس ⇧ وحط الماوس على أي طلب، اللي طلبوه يتلونوا **أخضر**، واللي هو طلبهم **أحمر**.

19.5 تابة Cookies: اتبعتت ولا اتمنعت؟

بتظهر لو الطلب فيه كوكيز رايحة أو جاية. فيها جدولين: **Request Cookies** (اللي المتصفح بعتها مع الطلب ده) و**Response Cookies** (اللي السيرفر حطها بـ `Set-Cookie`). وأهم حاجة فيها: لو كوكي **اتمنعت**، هتلاقيها مظللة بلون أصفر وجنبها علامة، وحط الماوس عليها يقولك السبب بالظبط. أمثلة على الأسباب:

`This Set-Cookie didn't specify a "SameSite" attribute…`	اتعاملت `Lax`، فمش هتتبعت مع طلب Cross-site (الفصل 13).
`…had the "SameSite=None" attribute but did not have the "Secure" attribute`	`None` لازم معاها `Secure`. فالمتصفح رفض يحفظها خالص.
`…the "Secure" attribute but was not received over a secure connection`	سيرفر HTTP عادي بيحاول يحط كوكي `Secure`.
`…Domain attribute was invalid with regards to the current host url`	`api.shop.dev` بيحاول يحط كوكي لـ `other.com`. ممنوع.

وفيه Checkbox اسمه **show filtered out request cookies**: يوريك الكوكيز اللي **كانت** موجودة بس المتصفح قرر ميبعتهاش مع الطلب ده وليه. ده بالظبط اللي محتاجه لما تقول "أنا عامل Login، ليه السيرفر بيقولي 401؟".

19.6 انسخ الطلب: cURL و fetch

ودي من أقوى الحاجات في DevTools كله. Right-click على أي طلب ← **Copy**، وهتلاقي:

`Copy URL`	الـ URL بس.
`Copy as cURL`	أمر curl فيه نفس الـ Method والـ Headers والكوكيز والـ Body بالظبط. على الـ Mac بيتنسخ بصيغة bash/zsh. وعلى Windows فيه `Copy as cURL (cmd)` و `(bash)`.
`Copy as fetch`	كود `fetch(...)` جاهز تلزقه في الـ Console أو في كودك.
`Copy as fetch (Node.js)`	نفس اللي فوق، بس بيشيل حاجات خاصة بالمتصفح، ومعاه الكوكيز كـ Header.
`Copy as PowerShell`	لزمايلك على Windows.
`Copy response`	الـ Body بتاع الرد.
`Copy all as cURL / Copy all as HAR (sanitized)`	كل الطلبات اللي في القايمة (بعد الفلتر) مرة واحدة.

وده شكل الناتج، ولاحظ إنك تعرف تقرا كل سطر فيه دلوقتي:

تشريح Copy as cURL

```
curl 'https://api.shop.dev/orders' -H 'accept: application/json' -H 'authorization:
Bearer eyJhbGc...' -H 'content-type: application/json' -H 'origin:
http://localhost:5173' -b 'theme=dark' --data-raw '{"item":12,"qty":2}'
```

1. `curl 'https://api.shop.dev/orders'` — الـ URL بين Single quotes عشان zsh.
2. `-H 'accept: ...'` — كل Request header ليه `-H` لوحده، زي ما في تابة Headers بالظبط.
3. `-H 'authorization: Bearer ...'` — الـ Token اتنسخ **كمان**. خلي بالك وانت بتبعت الأمر ده لحد.
4. `-H 'origin: ...'` — curl مش هيطبّق CORS، بس السيرفر هيشوف نفس الـ Origin.
5. `-b 'theme=dark'` — الكوكيز (لو فيه).
6. `--data-raw '...'` — الـ Body. زي `-d` بس مبيعاملش `@` في الأول كاسم ملف. ووجود Body بيخلي curl يعمل POST لوحده، فمش محتاج `-X POST`.

واستخدامه الأشهر هو الخطوة الأهم في أي Debugging: **"هل المشكلة في المتصفح ولا في السيرفر؟"**. انسخ الطلب اللي فاشل كـ cURL، ولزقه في الـ Terminal، وزوّد `-i`:

- لو فشل بنفس الشكل: المشكلة في **الطلب نفسه أو السيرفر**. عدّل في الأمر (شيل Header، غيّر الـ Body) لحد ما تلاقي السبب.
- لو نجح في curl بس فاشل في المتصفح: المشكلة في حاجة المتصفح بيضيفها: CORS، أو كوكي اتمنعت، أو كاش، أو Extension.

عدّل وابعت تاني

عاوز تجرب نفس الطلب بقيمة مختلفة من غير ما تكتب كود؟ أسهل طريقة: **Copy as fetch**، وروح الـ Console، والزقه، وغيّر اللي عاوزه، وﻫ. الطلب الجديد هيظهر في Network زي أي طلب. وفي نسخ Chrome الحديثة فيه اختصار لده في نفس القايمة (Edit and resend أو حاجة قريبة)، وكمان **Replay XHR** بيعيد طلبات XHR زي ما هي. وفي Safari فيه Edit and resend من زمان.

> **خلي بالك: الطلب المنسوخ فيه أسرارك**
>
> Copy as cURL و fetch بينسخوا الكوكيز والـ `authorization` زي ما هي. يعني اللي يستلم الأمر ده يقدر يعمل اللي انت بتعمله بحسابك لحد ما الـ Session يخلص. قبل ما تبعته في Slack أو تحطه في Issue على GitHub، امسح الـ Tokens والكوكيز أو بدّلهم بـ `XXX`.

19.7 Request conditions: امنع طلب وشوف إيه اللي هيحصل

سؤال مهم: لو ملف Analytics ده وقع، الصفحة هتبوظ؟ لو الـ API ده مرجعش، الكود بتاعي هيتصرف إزاي؟ Right-click على الطلب ← **Block request URL** أو **Block request domain**. هيفتح في الـ Drawer تابة **Request conditions** وفيها القاعدة، وتقدر تكتب قواعد بنفسك بـ `*`، زي `*.analytics.com/*`. واعمل Reload: الطلبات دي هتظهر في Network بـ Status `(blocked:devtools)`.

وفي Chrome الحديث، نفس التابة دي بتسمحلك كمان تعمل **Throttling لطلبات معينة** بس، مش للصفحة كلها. يعني تخلي الـ API بتاعك بس بطيء وتشوف الـ Loading spinner بتاعك.

19.8 Overrides: غيّر الرد من غير ما تلمس السيرفر

تخيل إن الـ Backend لسه مخلصش الـ API، أو بيرجع داتا غلط، وانت عاوز تشوف الـ Frontend هيعمل إيه لو الرد كان شكل تاني. أو عاوز تجرب تشيل Header معين. **Overrides** بيخلي DevTools يرد على المتصفح بنسخة على جهازك بدل الرد الحقيقي:

1. Right-click على الطلب ← **Override content** (للـ Body) أو **Override headers** (للـ Headers).
2. أول مرة هيطلب منك تختار **فولدر** على الـ Mac يحفظ فيه الملفات، ويطلب إذن. اعمل فولدر مخصوص زي `~/devtools-overrides`، ودوس Allow.
3. هيفتح الرد في Sources، عدّله زي ما تحب (مثلاً غيّر `"role":"user"` لـ `"role":"admin"`)، و⌘S.
4. اعمل Reload. الطلب هيظهر في Network وعليه **علامة بنفسجي** تقولك إن ده رد معدل، مش الحقيقي.

> **ده بيتغير عندك انت بس**
>
> Overrides جوه المتصفح بتاعك بس، وطول ما DevTools مفتوح والخاصية شغالة. السيرفر ماعرفش حاجة، واليوزرز مش شايفين حاجة. يعني مش ثغرة: لو غيّرت `role` لـ `admin` وظهرلك زرار Admin، السيرفر المفروض برضه يرفض أي طلب Admin منك، لأن الحماية الحقيقية لازم تبقى على السيرفر. ولو مرفضش، يبقى لقيت Bug أمني حقيقي في الـ Backend. ومتنساش تقفلها من Sources ← Overrides لما تخلص، عشان متقعدش تتجنن ليه التعديلات بتاعتك مش ظاهرة.

19.9 HAR: الجلسة كلها في ملف

زميلك بيقول "الصفحة بايظة عندي" وانت مش عارف تكرر المشكلة. أو فريق الدعم الفني لشركة بتطلب منك تبعتلهم اللي حصل. الحل: **HAR** (HTTP Archive)، ملف JSON فيه كل الطلبات اللي في Network: الـ URLs والـ Headers والـ Bodies والتوقيت.

`Export HAR (sanitized)` ← `⤓`	الافتراضي: بيصدّر من **غير** `Cookie` و `Set-Cookie` و `Authorization`. ده اللي تبعته لحد.
`Export HAR (with sensitive data)`	بالأسرار كلها. مش ظاهر غير لو فعّلت "Allow to generate HAR with sensitive data" من Settings. استخدمه بحذر شديد.
`Network` أو اسحب الملف على `⤒`	Import: تفتح HAR حد بعتهولك، وتشوف كل الطلبات كأنها حصلت عندك.

> **خلي بالك: حتى الـ sanitized ممكن يبقى فيه حاجات**
>
> الـ Sanitized بيشيل الـ Headers دي بس. لكن الـ **Bodies** لسه فيها: لو فورم Login فيه الباسورد في الـ Payload، هيبقى في الملف. ولو فيه Token في الـ URL نفسه (`?token=...`)، هيبقى في الملف. افتح الملف في محرر وادوّر قبل ما تبعته.

> **لو حابب تتعمق: تتحكم في الـ HAR بـ jq** — تقدر تعدّيه في أول قراية
>
> ملف HAR ده JSON عادي، فتقدر تحلله بـ `jq` (الفصل 10). مثلاً تطلع كل الطلبات اللي فشلت مع الـ Status بتاعها:
>
> ```
> jq '.log.entries[] | select(.response.status >= 400) | "\(.response.status) \(.request.url)"' page.har
> ```
>
> الـ `.log.entries[]` = كل الطلبات، و`select` = فلتر زي `filter` في JS، والسطر الأخير بيطبع الرقم والعنوان.

ملخص الفصل

- Headers: General (الملخص)، و Response (رد السيرفر)، و Request (اللي اتبعت فعلاً)، و Raw. و Provisional headers = الطلب غالباً موصلش.
- Payload: اللي كودك بعته. أول مكان تشوف فيه `[object Object]` أو JSON غلط.
- Preview منسق، و Response خام. و `Unexpected token '<'` = جالك HTML بدل JSON.
- Initiator: سلسلة الكود اللي عمل الطلب، و ⇧+Hover في القايمة.
- Cookies: اللي اتبعت واللي اتحط، واللي اتمنع وسببه.
- Copy as cURL يفصل المتصفح عن السيرفر، و Copy as fetch للتعديل وإعادة الإرسال. وفيهم أسرارك.
- Block في Request conditions، و Overrides تغيّر الرد عندك بس، و HAR sanitized عشان تبعت الجلسة.

صح ولا غلط؟

1. Export HAR (sanitized) بيشيل الباسوردات اللي في الـ Body كمان. صح / غلط
2. Overrides بيغيّر الرد للمستخدمين كلهم. صح / غلط
3. لو الطلب المنسوخ كـ cURL نجح في الـ Terminal وفشل في المتصفح، فالمشكلة غالباً حاجة خاصة بالمتصفح زي CORS. صح / غلط

أسئلة مراجعة

1. إيه معنى "Provisional headers are shown"؟ واذكر سببين ليها.
2. إزاي تلاقي `[object Object]` في طلب، وإيه سببه؟
3. إيه الفرق بين Preview و Response؟ وإمتى Preview ميكفيش؟
4. إيه اللي هيحصل لو بعت Copy as cURL في جروب عام؟
5. إزاي تتأكد إن الـ Frontend بتاعك بيتعامل صح لو ملف Analytics وقع؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح `https://jsonplaceholder.typicode.com/` واعمل من الـ Console
`d:'POST', headers:{'Content-Type':'application/json'}, body: JSON.stringify({title:'hi'})})`
. شوف الطلب في Headers و Payload و Preview و Response.
2. متوسط انسخ نفس الطلب كـ cURL، والزقه في الـ Terminal بـ `-i`، وقارن الرد. وبعدين انسخه كـ fetch وغيّر الـ title وابعته تاني.
3. صعب افتح `https://jsonplaceholder.typicode.com/posts/1` كصفحة، واعمل Override content للرد بتاعها، وخلي العنوان كلمة من عندك، واعمل Reload وشوف العلامة البنفسجي. وبعدين صدّر HAR sanitized، وافتحه بـ `jq` وطلّع منه كل الـ URLs.

20

المرحلة 6 · DevTools والـ Debugging

الفصل العشرين

الـ Console: تقرا الأخطاء وتكلم الصفحة

الـ Console ليه وشين. وش بيكلمك: كل خطأ وتحذير ورسالة من كودك أو من المتصفح. ووش انت بتكلمه: تكتب JS يتنفذ على الصفحة نفسها حالاً. هنتعلم نقرا الاتنين: المستويات والفلاتر، وتفصيص الـ Stack trace، وكتالوج بأشهر رسايل الشبكة (CORS و 404 و 401 و 500 و Mixed content و net::ERR_*) ومعنى كل واحدة، وإزاي تعمل fetch صح من الـ Console، وأدوات صغيرة هتستخدمها كل يوم.

بعد الفصل ده هتكون عارف

- مستويات الرسايل: Verbose و Info و Warnings و Errors، والفلاتر والإعدادات
- أكتر من console.log: table و group و time و count و assert و trace
- تقرا الـ Stack trace وتوصل للسطر
- كتالوج رسايل الشبكة: CORS و 404 و 401/403 و 500 و Mixed content و CSP
- عيلة net::ERR_* ومعنى كل واحد
- fetch من الـ Console بـ await، و res.ok و res.status
- أدوات: $0 و $_ و $$ و copy()
- الـ Issues panel و Understand this error

20.1 وشين في مكان واحد

في كورس Jonas هتكتب `console.log` مئات المرات، وده الاستخدام الأول: كودك بيكلمك. بس الـ Console فيه كمان رسايل **المتصفح نفسه**: كل طلب فشل، وكل CORS، وكل تحذير أمان. ولو حد في فريقك قالك "بص في الـ Console"، غالباً يقصد دي، مش الـ logs بتاعتك. والوش التاني: السطر اللي تحت فيه `>`، تكتب فيه أي JS ويتنفذ **جوه الصفحة اللي انت فيها**، بنفس المتغيرات والـ DOM والكوكيز. يعني مش ملعب منفصل، انت جوه الموقع.

20.2 المستويات والفلاتر

كل رسالة ليها مستوى، وكل مستوى ليه شكل:

```
Console · Chrome DevTools
> console.log('عادي'); console.info('معلومة')
عادي
معلومة
> console.warn('تحذير')
تحذير
> console.error('خطأ')
خطأ
> console.debug('تفاصيل')
# مش هيظهر غير لو فعّلت Verbose
```

`Default levels ▾`	اختار تشوف إيه: **Verbose** (`console.debug`) ورسايل تفصيلية من المتصفح، مقفول افتراضياً)، و**Info**، و**Warnings** (أصفر)، و**Errors** (أحمر). لو مش لاقي رسالة متأكد إنها موجودة، بص هنا الأول.
`Filter`	زي Network: كلمة، أو `كلمة-` للعكس، أو `/regex/`، أو `url:app.js` رسايل ملف معين.
`☰ Console sidebar`	قايمة على الجنب بتقسم الرسايل: User messages (بتاعتك) و Errors و Warnings، ومين الملف اللي طلّعها. مفيد جداً لما الصفحة مليانة رسايل.
`top ▾`	الـ **Context**: الكود بتتنفذ فين. `top` = الصفحة الأساسية. لو الصفحة فيها iframe أو Extension، هتلاقيهم هنا. لو كتبت متغير وقالك مش موجود، اتأكد إنك في الـ Context الصح.
`⚙ Console settings`	**Preserve log** (متمسحش مع الـ Reload)، و**Hide network** (اخفي رسايل الشبكة)، و**Log XMLHttpRequests** (اطبع كل XHR/fetch)، و**Show CORS errors in console**، و**Group similar messages**، و**Eager evaluation** (يوريك النتيجة وانت بتكتب).
`⌘K`	امسح الـ Console. أو اكتب `clear()`.

أكتر من console.log

الـ `console` فيه أدوات كتير بتوفر عليك وقت، وكلها شغالة في كودك وفي الـ Console نفسه:

```
جرّبهم في الـ Console
const users = [{id: 1, name: 'Sara', role: 'admin'}, {id: 2, name: 'Omar', role: 'user'}];

console.table(users);                  // جدول بأعمدة id و name و role
console.log({ users });                // اللوج بالاسم: { users: [...] } بدل [...] وخلاص

console.group('Login flow');           // كل اللي بعده متداخل تحت عنوان
console.log('step 1'); console.log('step 2');
console.groupEnd();
```

```
console.time('fetch users');              // ابدأ عداد
await fetch('https://jsonplaceholder.typicode.com/users');
console.timeEnd('fetch users');           // fetch users: 312.4 ms

console.count('render');                  // render: 1، وبعدين render: 2 ...
console.assert(users.length > 5, 'users قليلين!');   // يطبع خطأ بس لو الشرط غلط
console.trace('مين نده عليا؟');           // يطبع سلسلة الدوال اللي وصلت هنا
```

> ⚠ **خلي بالك: الـ Object في اللوج بيتغير**
>
> لو عملت `console.log(user)` وبعدها الكود غيّر `user.name`، وبعدين فتحت السهم جنب الـ Object في الـ Console، ممكن تشوف القيمة **الجديدة** مش اللي كانت وقت اللوج. ليه؟ لأن الـ Console بيحتفظ بـ **Reference** للـ Object، وبيقرا القيم لما تفتحه انت بالظبط: دي اللحظة في القيمة تشوف عشان (ده. بتقولك جنبه صغيرة `i` أيقونة (وفيه `console.log(structuredClone(user))` أو `console.log(JSON.stringify(user))`.

20.3 تقرا الخطأ والـ Stack trace

الخطأ الأحمر مش مجرد "فيه مشكلة". هو بيقولك **إيه** المشكلة، و**فين** بالظبط، و**إزاي** الكود وصل هناك:

Console · Chrome DevTools

```
Uncaught TypeError: Cannot read properties of undefined (reading 'name')
    at renderUser (app.js:22:31)
    at app.js:40:3
    at Array.forEach (<anonymous>)
    at loadUsers (app.js:39:9)
```

فصّص الخطأ

1 محدش عمله `catch`. لو لقيت `Uncaught (in promise)`، يبقى Promise اترفض (زي `fetch` أو `async` function) ومفيش `.catch` ولا `try`.	`Uncaught`
2 نوع الخطأ. `TypeError` = استخدمت قيمة بشكل مش مناسب لنوعها. وفيه `ReferenceError` (متغير مش موجود) و `SyntaxError` (الكود أو الـ JSON مكتوب غلط).	`TypeError`
3 الرسالة: حاولت تقرا `.name` من حاجة قيمتها `undefined`. يعني في `x.name`، الـ `x` هو اللي undefined، مش `name`.	`Cannot read properties of undefined (reading 'name')`
4 أول سطر في الـ Stack = **مكان الخطأ**: دالة `renderUser`، ملف `app.js`، سطر 22، عمود 31. دوس عليه يفتحلك السطر في Sources.	`at renderUser (app.js:22:31)`
5 باقي السلسلة لتحت = **مين نده على مين**: `loadUsers` عملت `forEach`، وجواها Arrow function في سطر 40، وهي اللي نادت `renderUser`. اقرا من فوق لتحت تعرف إزاي الكود وصل للخطأ.	`at app.js:40:3 ... at loadUsers (app.js:39:9)`

الخطأ الأول هو المهم

لو فيه 15 خطأ أحمر، متبدأش من آخرهم. غالباً الأول هو السبب، والباقي نتايج ليه (الـ `users` مجاش، فكل حاجة بتعتمد عليه وقعت). وفي حالة الخطأ اللي فوق، السؤال الحقيقي مش "ليه `renderUser` وقعت" لكن "ليه الـ user كان undefined؟"، والإجابة غالباً في Network: الـ API رجّع شكل غير اللي متوقعه.

20.4 كتالوج رسايل الشبكة

دي الرسايل اللي المتصفح بيكتبها لوحده لما حاجة في الشبكة تبوظ. كل واحدة هتقابلها، فاحفظ شكلها ومعناها:

Console · Chrome DevTools

```
GET https://api.shop.dev/users 404 (Not Found)
Failed to load resource: the server responded with a status of 401 ()
POST https://api.shop.dev/orders 500 (Internal Server Error)
Access to fetch at 'https://api.shop.dev/orders' from origin 'http://localhost:5173' has been
blocked by CORS policy: ...
Mixed Content: The page at 'https://shop.dev/' was loaded over HTTPS, but requested an
insecure resource 'http://cdn.old.com/x.js'. This request has been blocked; the content must
be served over HTTPS.
Refused to connect to 'https://evil.com/' because it violates the following Content Security
Policy directive: "connect-src 'self'".
GET https://apii.shop.dev/users net::ERR_NAME_NOT_RESOLVED
GET http://localhost:3000/api net::ERR_CONNECTION_REFUSED
```

الرسالة	معناها	تبص فين
`404 (Not Found)`	السيرفر رد، بس العنوان ده مش موجود	الـ URL في Headers ← General: غلطة إملائية؟ `/api` ناقصة؟ `/` زيادة؟
`401`	مين انت؟ مفيش Login أو الـ Token غلط أو خلص	Request Headers: الـ `authorization` اتبعت؟ تابة Cookies: الكوكي اتبعتت ولا اتمنعت؟
`403`	عارفك، بس مش مسموحلك	صلاحيات الحساب على السيرفر، مش الـ Frontend
`500`	السيرفر نفسه وقع	Response: ساعات فيها رسالة الخطأ. والحل في Logs السيرفر
`blocked by CORS policy`	السيرفر مدّاش إذن لـ Origin بتاعك	سطر الـ preflight في Network، والفصل 14
`Mixed Content`	صفحة HTTPS بتطلب حاجة `http://`	حوّل الرابط لـ `https://` (الفصل 12)

الرسالة	معناها	تبص فين
`Content Security Policy`	الموقع نفسه حاطط قايمة بالأماكن المسموح يكلمها (بـ Header اسمه `content-security-policy`)، والطلب ده مش فيها	Response Headers بتاعة الصفحة. لو الموقع بتاعك: ضيف الدومين للقايمة

جدول 20.1 معنى كل رسالة وتبص فين

عيلة net::ERR_*: لما مفيش رد أصلاً

دي أهم رسايل لأنها بتقولك إن المشكلة **قبل** HTTP. السيرفر ماردش بأي Status، لأن الطلب ماوصلش أو الاتصال مااكتملش. والاسم نفسه بيقولك المحطة اللي وقع فيها من رحلة الفصل 2:

الخطأ	وقع فين	السبب غالباً
`ERR_NAME_NOT_RESOLVED`	DNS	الاسم غلط (`apii`)، أو الدومين مش موجود. جرب `dig`
`ERR_CONNECTION_REFUSED`	TCP	مفيش حد شغال على الـ Port ده. السيرفر بتاعك مش شغال؟ Port غلط؟ جرب `lsof -i :3000`
`ERR_CONNECTION_TIMED_OUT`	TCP	محدش رد خالص: Firewall، أو IP غلط، أو السيرفر واقع
`ERR_CONNECTION_RESET`	الاتصال	الطرف التاني قفل فجأة. ساعات Proxy أو VPN
`ERR_INTERNET_DISCONNECTED`	جهازك	مفيش نت (أو Offline في Throttling!)
`ERR_SSL_PROTOCOL_ERROR` / `ERR_CERT_*`	TLS	مشكلة شهادة (الفصل 12)
`ERR_TOO_MANY_REDIRECTS`	HTTP	Redirect loop: أ بيحوّل لـ ب وب بيحوّل لـ أ. جرب `curl -sIL`
`ERR_BLOCKED_BY_CLIENT`	المتصفح	Ad blocker أو Extension منع الطلب. جرب Incognito
`ERR_ABORTED`	المتصفح	الطلب اتلغى (الصفحة اتغيرت، أو `AbortController`). غالباً طبيعي
`ERR_EMPTY_RESPONSE`	HTTP	السيرفر قفل من غير ما يبعت حاجة. غالباً كراش في السيرفر

جدول 20.2 أشهر أخطاء net::ERR_*

حاجة مشتركة: نفس المحطات، نفس الأدوات

كل خطأ هنا بيقابله أداة في الـ Terminal من الكتاب ده: DNS ← `dig`، و TCP ← `nc -zv` و `lsof`، و TLS ← `openssl s_client` و `curl -v`، و Redirects ← `curl -sIL`، و HTTP ← `curl -i`. يعني لما الـ Console يقولك "وقعت هنا"، تروح للـ Terminal وتختبر المحطة دي لوحدها.

20.5 fetch من الـ Console

الـ Console بيقبل `await` مباشرة (Top-level await)، فتقدر تجرب أي API في سطر:

Console · Chrome DevTools

```
> const res = await fetch('https://jsonplaceholder.typicode.com/users/99')
> res.status
404
> res.ok
false
> res.headers.get('content-type')
'application/json; charset=utf-8'
> await res.json()
{}
# مفيش Error! fetch مبيرفضش على 404. لازم تشيك على res.ok بنفسك
```

وده الشكل اللي تكتبه في كودك عشان تتعامل مع النوعين: الشبكة وقعت، أو السيرفر رد بخطأ:

getUser.js

```
async function getUser(id) {
  try {
    const res = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);    // 404 و 500 هنا
    return await res.json();
  } catch (err) {
    console.error('getUser failed:', err.message);          // الشبكة أو اللي فوق
    return null;
  }
}
```

> **الـ Origin بتاع الـ Console هو الصفحة**
>
> الـ `fetch` اللي بتكتبه في الـ Console بيتعامل كأنه من الصفحة اللي انت فيها: نفس الـ Origin ونفس الكوكيز ونفس قواعد CORS و CSP. عشان كده لو فتحت الـ Console على `jsonplaceholder.typicode.com` نفسه وعملت `fetch('/users')`، هتشتغل بعنوان نسبي ومفيش CORS خالص. ولو جربت من صفحة تانية، ممكن تتمنع حسب الموقع.

20.6 أدوات الـ Console الصغيرة

`$0`	العنصر اللي انت مختاره في Elements دلوقتي. اختار زرار، وروح الـ Console، واكتب `$0.textContent` أو `$0.click()`.
`$_`	نتيجة آخر سطر اتنفذ. عملت `await res.json()` ونسيت تحفظه؟ `const data = $_`.

`$('css') / $$('css')`	اختصار لـ `querySelector` و`querySelectorAll`، بس `$$` بيرجع Array حقيقية. مثلاً: `$$('a').length`. (لو الصفحة فيها jQuery، `$` هيبقى بتاعها.)
`copy(x)`	انسخ أي قيمة للـ Clipboard. `copy(data)` بيحوّل الـ Object لـ JSON منسق وينسخه. تلزقه في محرر على طول.
↑ / ↓	الأوامر اللي كتبتها قبل كده.
⇧↩	سطر جديد من غير تنفيذ، لو بتكتب كود أطول.

> **خلي بالك: Self-XSS**
>
> لو حد قالك "الزق الكود ده في الـ Console وهتاخد هدية"، **متعملش**. الكود ده هيتنفذ جوه حسابك بكوكيزك، ويقدر يعمل أي حاجة انت تقدر تعملها. عشان كده لو فتحت Console بتاع Facebook مثلاً هتلاقي تحذير كبير. وChrome بيطلب منك تكتب "allow pasting" قبل ما يسيبك تلزق أول مرة، وده لنفس السبب.

20.7 الـ Issues panel و Understand this error

فوق في الـ Console ممكن تلاقي عداد صغير بأيقونة. ده بيفتح Panel اسمه **Issues**: المتصفح بيجمع فيه المشاكل اللي لقاها وشرحها بالتفصيل، بدل ما يملا بيها الـ Console. زي كوكيز اتمنعت بسبب SameSite، و Mixed content، ومشاكل CORS، ومشاكل Accessibility. وكل Issue فيه شرح ولينك للطلب المتأثر في Network. ارجعله لما يكون الـ Console هادي والحاجة مش شغالة.

وجنب الأخطاء هتلاقي أيقونة **Understand this error** لو الخدمة متاحة عندك (الشروط في الفصل 17). بتشرحلك الخطأ بالـ AI بناءً على الكود. جربها **بعد** ما تقرا الخطأ بنفسك، عشان تقارن فهمك بشرحها، مش بداله.

ملخص الفصل

- الـ Console فيه رسايل كودك ورسايل المتصفح، ومكان تشغّل فيه JS جوه الصفحة.
- المستويات: Verbose و Info و Warnings و Errors. والـ Context (`top`) بيحدد الكود بيتنفذ فين.
- `console.table` و`group` و`time` و`count` و`assert` و`trace`. والـ Object في اللوج بيتقري لما تفتحه.
- الـ Stack trace: أول سطر = مكان الخطأ، والباقي = مين نده على مين. والخطأ الأول هو المهم.
- 404 و 401 و 403 و 500 = السيرفر رد. CORS و Mixed content و CSP = المتصفح منع.
- `net::ERR_*` = مفيش رد HTTP، والاسم بيقولك المحطة: DNS أو TCP أو TLS.
- `fetch` مبيرفضش على 404، فشيك على `res.ok`. و`$0` و`$_` و`$$` و`copy()` أدوات يومية.

صح ولا غلط؟

1. `net::ERR_CONNECTION_REFUSED` معناه إن السيرفر رد بـ 403. صح / غلط
2. في الـ Stack trace، أول سطر تحت الرسالة هو المكان اللي الخطأ حصل فيه. صح / غلط
3. `fetch` بيعمل Reject لو السيرفر رد بـ 404. صح / غلط

أسئلة مراجعة

1. في `Cannot read properties of undefined (reading 'name')`، إيه اللي undefined بالظبط؟
2. إيه الفرق بين `ERR_NAME_NOT_RESOLVED` و`ERR_CONNECTION_REFUSED`؟ وتختبر كل واحد بأنهي أمر؟
3. إيه الفرق بين خطأ CORS وخطأ CSP؟ ومين اللي "منع" في كل واحد؟
4. ليه ممكن تشوف قيمة في Object في الـ Console غير اللي كانت وقت `console.log`؟
5. اكتب سطرين في الـ Console يجيبوا `/users/1` ويطبعوا الإيميل بس.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل جرب كل دوال `console` اللي في الفصل على بيانات `/users` من jsonplaceholder: `table` و`time` و`group`.
2. متوسط ولّد بنفسك 3 أخطاء شبكة من الـ Console: `fetch('https://doesnotexist.invalid')` و`fetch('http://localhost:3999')` و`fetch('https://jsonplaceholder.typicode.com/nope')`. اكتب الرسالة ونوعها والمحطة اللي وقعت فيها.
3. صعب اعمل خطأ متعمد في صفحة صغيرة: Array فيها Object ناقص خاصية، و`forEach` بتقرا الخاصية دي. اقرا الـ Stack trace، ودوس عليه لحد السطر، وصلّح.

21

المرحلة 6 · DevTools والـ Debugging

الفصل الحادي والعشرين

Application Panel و Safari: الكوكيز والتخزين بإيدك

في الفصل 13 فهمنا الكوكيز و localStorage نظرياً. Application Panel هو المكان اللي تشوفهم فيه بعينك وتعدلهم وتمسحهم. هنمشي على جدول الكوكيز عمود عمود، ونعدّل التخزين، ونمسح بيانات الموقع كلها لما حاجة تعلق، ونلمس الـ Service Workers. وفي الآخر، بما إنك على Mac: إزاي تفتح Web Inspector في Safari ونفس الأفكار فيه.

بعد الفصل ده هتكون عارف

- خريطة Application Panel
- جدول الكوكيز: كل عمود، والتعديل والمسح، والكوكيز اللي فيها مشكلة
- localStorage و sessionStorage: تشوف وتعدل وتضيف
- Storage و Clear site data: البداية من الصفر
- Service Workers والـ Cache storage باختصار
- Safari Web Inspector: تفعّله وتستخدمه، والفروق المهمة

21.1 فين كل حاجة

افتح DevTools ← **Application** (لو مش ظاهر، في `»`). على الشمال فيه شجرة بأقسام:

`Application`	**Manifest** (لو الموقع PWA)، و**Service workers**، و**Storage** (ملخص المساحة وزرار المسح).
`Storage`	**Local storage** و**Session storage** و**IndexedDB** و**Cookies** و**Cache storage**، وحاجات تانية أقل استخدام. كل واحد تحته الـ Origins اللي ليها بيانات.
`Background services`	تسجيل لحاجات بتحصل في الخلفية زي الـ Push notifications. مش هنحتاجه.
`Frames`	الصفحة وأي iframes جواها، ومعلومات أمان عن كل واحد.

21.2 جدول الكوكيز

Storage ← Cookies ← اختار الـ Origin. هتلاقي جدول، وكل عمود فيه هو Attribute من اللي فصّصناهم في الفصل 13:

العمود	بيقولك
Value / Name	الاسم والقيمة. ودوس على أي صف تشوف القيمة كاملة تحت، ومعاها Checkbox **Show URL-decoded** لو القيمة متشفرة (`%7B%22...`)
Domain	بتتبعت لمين. لو بتبدأ بـ `.` (زي `.shop.com`)، يبقى للـ Subdomains كمان
Path	المسار
Max-Age / Expires	هتخلص إمتى. لو مكتوب `Session`، يبقى Session cookie
Size	الاسم + القيمة بالـ Bytes. لو قربت من 4096، فيه مشكلة
✓ HttpOnly	الـ JS ميشوفهاش. لو فيه علامة، مش هتلاقيها في `document.cookie`
✓ Secure	HTTPS بس
SameSite	`Strict` أو `Lax` أو `None`، أو فاضي (يعني اتعاملت Lax في Chrome)
Partition Key	لو `Partitioned` (CHIPS): مربوطة بأنهي موقع أب
Priority	Low و Medium و High: لو المتصفح اضطر يمسح كوكيز عشان المساحة، يبدأ بأنهي

جدول 21.1 أعمدة جدول الكوكيز

`Double-click` على خانة	عدّلها. غيّر القيمة، أو الـ Expires، أو حتى شيل الـ HttpOnly. مفيد تجرب "لو الكوكي دي اتغيرت، السيرفر هيعمل إيه؟".
`Double-click` على صف فاضي	ضيف كوكي جديدة بإيدك.
اختار صف + `Delete` (⌫)	امسح الكوكي دي. أسرع طريقة تعمل "Logout" وتجرب الصفحة كأنك زائر.
🚫 `Clear all cookies`	امسح كوكيز الـ Origin ده كلها.
`Only show cookies with an issue`	فلتر للكوكيز اللي فيها مشكلة (اتمنعت أو هتتمنع). هتلاقيها متلونة، وحط الماوس على العلامة يقولك السبب.

حاجة مشتركة: نفس الكوكي، 3 أماكن تشوفها

تابة **Cookies** جوه طلب في Network بتقولك الكوكي دي **اتبعتت مع الطلب ده** ولا لأ وليه (الفصل 19). جدول **Application** بيقولك الكوكيز **المحفوظة** عند المتصفح دلوقتي. و `document.cookie` في الـ Console بيقولك اللي **الـ JS شايفه** (من غير HttpOnly). لو الكوكي موجودة في Application ومش في الطلب، يبقى المتصفح قرر ميبعتهاش (Domain أو Path أو SameSite أو Secure). ولو مش موجودة في Application خالص، يبقى السيرفر محطهاش أو المتصفح رفضها.

21.3 localStorage و sessionStorage

Storage ← Local storage ← الـ Origin. جدول بسيط: Key و Value. والـ Value لو JSON، دوس عليه يتعرض كشجرة تحت.

`Double-click`	عدّل أي Key أو Value.
صف فاضي في الآخر	ضيف Key جديد.
⌫ / 🚫	امسح واحد، أو امسح الكل.
⟳	حدّث الجدول. لو الكود غيّر حاجة ومش ظاهرة، دوس عليه.

مثال عملي: الثيم بيرجع Light كل مرة

عملت زرار Dark mode بيعمل `localStorage.setItem('theme','dark')`. ولما تعمل Reload بيرجع Light. افتح Local storage: لو `theme` موجودة بقيمة `dark`، يبقى المشكلة في الكود اللي **بيقرا** القيمة وقت التحميل. ولو مش موجودة، يبقى المشكلة في اللي **بيكتب**. ولو موجودة تحت Origin تاني (Port تاني)، فاكر الفصل 13؟ كده نصيت المشكلة في ثانية بدل ما تقرا الكود كله.

21.4 Clear site data: ابدأ من الصفر

ساعات الموقع "يعلق" في حالة غريبة: Login قديم، أو كاش قديم، أو بيانات اتخزنت غلط من نسخة قديمة من كودك. بدل ما تمسح كل حاجة لوحدها: Application ← **Storage**. هتلاقي رسمة بالمساحة المستخدمة، وتحتها Checkboxes (Cookies و Local and session storage و IndexedDB و Cache storage و Unregister service workers)، وزرار **Clear site data**. ده بيرجّع الموقع ده عندك كأنك أول مرة تفتحه، من غير ما يلمس باقي المواقع.

وفيه طريقة أسرع: Right-click على زرار Reload في Chrome و DevTools مفتوح ← **Empty cache and hard reload** للكاش بس. وفيه طريقة من السيرفر نفسه: Header اسمه `Clear-Site-Data: "cache", "cookies", "storage"` ممكن يتبعت في رد الـ Logout عشان يمسح كل حاجة عند اليوزر.

21.5 Service Workers باختصار

Service Worker = ملف JS الموقع بيسجله في المتصفح، وبيقف **بين الصفحة والشبكة**. أي طلب الصفحة بتعمله بيعدي عليه الأول، ويقدر يرد من كاش بتاعه (اسمه Cache storage) من غير ما يروح للنت. ده اللي بيخلي مواقع تشتغل Offline. بس ليه أثر جانبي مزعج وانت بتطور: ممكن يفضل يديك نسخة قديمة من الموقع حتى بعد ما تغيّر الكود.

`Update on reload`	مع كل Reload، حمّل نسخة الـ Service Worker الجديدة. فعّله وانت بتطور.
`Bypass for network`	خلي الطلبات تروح للشبكة على طول، كأن مفيش Service Worker.
`Unregister`	امسحه خالص.
`Offline`	جرب الموقع كأن مفيش نت.

وفي Network، الطلبات اللي الـ Service Worker رد عليها هتلاقي في Size بتاعها `(ServiceWorker)`، وعليها ترس صغير ⚙ جنب الاسم لو هو اللي عملها.

21.6 Safari Web Inspector

انت على Mac، ويوزرز كتير هيفتحوا موقعك من Safari أو من iPhone (وكل المتصفحات على الـ iPhone جواها محرك Safari). ومحرك Safari (WebKit) مختلف عن Chrome في حاجات، خصوصاً الكوكيز والخصوصية. فلازم تعرف تفتح أدواته:

1. Safari ← **Settings** (⌘,) ← **Advanced** ← فعّل **Show features for web developers**. هيظهرلك قايمة جديدة فوق اسمها **Develop**.
2. افتح Web Inspector: Develop ← Show Web Inspector، أو ⌥⌘I (نفس اختصار Chrome تقريباً)، أو Right-click ← Inspect Element.

نفس الأفكار اللي اتعلمناها موجودة، بأسماء قريبة:

Chrome	Safari	ملاحظة
Network	Network	نفس الفكرة: قايمة وتفاصيل وفلاتر و Export HAR، وفيه Preserve log و Disable caches في الشريط
Console	Console	نفس الرسايل تقريباً، بس النصوص مختلفة شوية
Application	Storage	Cookies و Local/Session storage و IndexedDB
Sources	Sources	الـ Breakpoints والـ Debugger
Performance	Timelines	تسجيل لكل اللي حصل
Device mode	Responsive Design Mode	Enter Responsive Design Mode ← Develop

جدول 21.2 Chrome DevTools مقابل Safari Web Inspector

`Develop ← Empty Caches`	امسح كاش Safari كله. (⌥⌘E)
`Develop ← [اسم الـ iPhone]`	وصّل الـ iPhone بالكابل، وفعّل Web Inspector على الموبايل (Settings ← Apps ← Safari ← Advanced)، وهتقدر تفتح Web Inspector على الـ Mac لصفحة مفتوحة على الموبايل. أقوى طريقة تعمل Debug لموقع على موبايل حقيقي.
`Network ← Edit and resend`	Right-click على طلب: عدّله وابعته تاني من جوه Web Inspector.

ليه موقعك ممكن يشتغل في Chrome ويبوظ في Safari؟

أشهر سبب في الشبكة: Safari بيمنع الـ Third-party cookies من سنين (الفصل 13)، وبيحدد عمر الكوكيز اللي الـ JS بيكتبها في حالات معينة. فلو الـ Login بتاعك معتمد على كوكي من دومين تاني، أو جوه iframe، هيشتغل في Chrome ويفشل في Safari. افتح Storage ← Cookies في Safari وقارن بـ Chrome. والسبب التاني: الـ CSS والـ JS بيختلفوا شوية بين المحركات، وده بتشوفه في Console و Elements.

ملخص الفصل

- Application: Storage فيها Cookies و Local/Session storage و IndexedDB و Cache storage.
- جدول الكوكيز = الـ Attributes كلها: Domain و Path و Expires و HttpOnly و Secure و SameSite و Partition Key.
- Double-click للتعديل، و ⌫ للمسح، و Only show cookies with an issue للمشاكل.
- نفس الكوكي في 3 أماكن: المحفوظة (Application)، واللي اتبعتت (Network ← Cookies)، واللي الـ JS شايفها (`document.cookie`).
- Storage ← Clear site data يرجّع الموقع كأنها أول زيارة.
- Service Worker بيقف بين الصفحة والشبكة، و Update on reload و Bypass for network وانت بتطور.
- Safari: فعّل Show features for web developers، وبعدين ⌥⌘I. و Application = Storage.

صح ولا غلط؟

1. الكوكي اللي عليها HttpOnly مش هتظهر في جدول Application. صح / غلط
2. Clear site data بيمسح بيانات كل المواقع في المتصفح. صح / غلط
3. قايمة Develop في Safari بتظهر لوحدها من غير ما تفعّلها. صح / غلط

أسئلة مراجعة

1. الكوكي موجودة في Application بس مش في Request Cookies لطلب معين. إيه الأسباب المحتملة؟
2. إمتى تستخدم Clear site data بدل إنك تمسح كوكي واحدة؟
3. إيه المشكلة اللي Service Worker ممكن يعملها وانت بتطور؟ وتحلها إزاي؟
4. ليه مهم تجرب موقعك في Safari وانت على Mac؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح أي موقع عامل فيه Login، ولاقي كوكي الـ Session في Application. هي HttpOnly؟ و Secure؟ و SameSite إيه؟
2. متوسط امسح كوكي الـ Session بس، واعمل Reload. حصل إيه؟ وبعدين جرب Clear site data على موقع تاني.
3. صعب فعّل Develop في Safari، وافتح نفس الموقع في Chrome و Safari، وقارن الكوكيز في الاتنين. لو عندك iPhone، جرب Develop ← [الـ iPhone].

22

المرحلة 6 · DevTools والـ Debugging

الفصل التاني والعشرين

الـ Debugging بطريقة: كودي ولا السيرفر؟

عندك دلوقتي كل الأدوات: curl و dig و nc، و Network و Console و Application. بس الأدوات من غير طريقة بتخليك تفتح كل حاجة وتدوّر عشوائي. الفصل ده هو الطريقة: خريطة أسئلة بالترتيب توصّلك للمشكلة، وحالات مشهورة بالحل بتاعها، وفي الآخر أساسيات الـ Breakpoints عشان لما المشكلة تطلع في كودك فعلاً.

بعد الفصل ده هتكون عارف

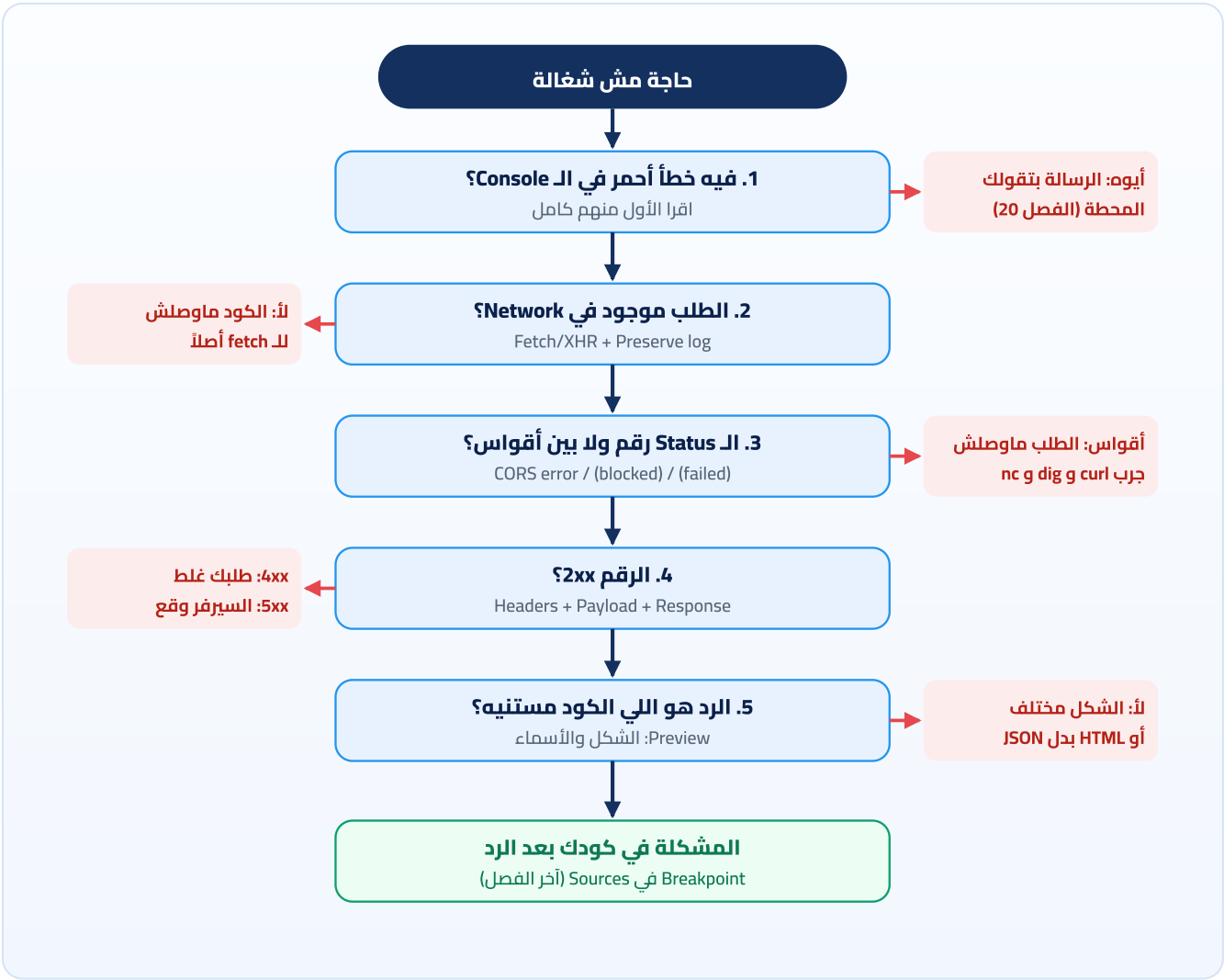
- الطريقة العلمية في الـ Debugging: لاحظ، وخمّن، وجرب، واستبعد
- خريطة: الطلب اتبعت؟ وصل؟ رد بإيه؟ الكود فهم الرد؟
- 7 حالات مشهورة بالأعراض والحل
- Breakpoints: سطر، و debugger، و Conditional، و Logpoints
- Step over و Step into و Scope و Watch
- fetch/XHR breakpoints و Pause on exceptions
- إزاي تكتب سؤال أو Bug report كويس

22.1 الطريقة قبل الأدوات

الـ Debugging مش موهبة. هو طريقة، وشبه شغل المحقق: عندك **عَرَض** ("الصفحة فاضية")، وانت عاوز توصل للـ**سبب**. والغلطة اللي كلنا بنعملها في الأول: نخمّن السبب على طول ونعدّل الكود، ولما ميتحلش نعدّل حاجة تانية، ونفضل كده ساعة. الطريقة الصح:

1. **لاحظ**: إيه اللي حصل بالظبط، وإيه اللي كنت متوقعه؟ اقرا الرسالة كلها، مش أول كلمتين.
2. **نصّ المشكلة**: الرحلة ليها محطات (الفصل 2). المشكلة في أنهي محطة؟ كل سؤال في الخريطة اللي جاية بيشيل نص الاحتمالات.
3. **جرب حاجة واحدة** في المرة. لو غيّرت 3 حاجات ونجحت، مش هتعرف مين اللي حلها.
4. **اتأكد**: المشكلة اتحلت فعلاً ولا بقت مستخبية؟ (Disable cache، Hard reload، Incognito.)

22.2 الخريطة: 5 أسئلة بالترتيب



شكل 22.1 كل سؤال بيشيل نص الاحتمالات. ابدأ من فوق، ومتنطش لكودك غير لما الأربع أسئلة الأولانيين يعدّوا

والسؤال الكبير اللي في عنوان الفصل، "كودي ولا السيرفر؟"، ليه اختبار واحد حاسم: **Copy as cURL** (الفصل 19). لو الطلب نفسه فاشل في curl، المشكلة في الطلب أو السيرفر. لو شغال في curl وفاشل في المتصفح، المشكلة في حاجة المتصفح بيضيفها (CORS أو كوكيز أو كاش أو Extension). ولو شغال في الاتنين والصفحة لسه بايظة، المشكلة في كودك بعد ما الرد وصل.

22.3 7 حالات هتقابلها

مشكلة 1 1. الصفحة فاضية، ومفيش ولا خطأ

مفيش حاجة حمرا في الـ Console، و Network فيه طلبات 200.

اتأكد إن مستوى **Errors** مش مقفول في Default levels، وإن الفلتر فاضي. ولو كله تمام، شوف الـ Response بتاع الـ API: ممكن يكون `[]` فاضي، والكود بيرسم لستة فاضية صح جداً. المشكلة هنا في البيانات مش في الكود.

مشكلة 2 — 2. Failed to fetch :TypeError

الـ `catch` مسك خطأ من غير أي تفاصيل.

دي رسالة `fetch` لأي فشل شبكة، وهي متعمدة من غير تفاصيل. التفاصيل في سطر تاني في الـ Console (CORS أو `net::ERR_*`) وفي عمود Status في Network. ابدأ من هناك، مش من الـ catch.

مشكلة 3 — 3. شغال على جهازي، مش شغال بعد الرفع

على `localhost` تمام. على الموقع الحقيقي، الطلبات فاشلة.

بص على الـ **Request URL** في Headers: لسه بيكلم `http://localhost:3000` ؟ (اليوزر مفيش عنده سيرفر على جهازه!) أو بيكلم `http://` من صفحة `https://` (Mixed content)؟ أو كنت مستخدم Proxy في Vite (الفصل 14) ومش موجود في الـ Production فرجع CORS؟

مشكلة 4 — 4. عامل Login، والـ API بيقول 401

الـ Login نجح ورجّع 200، والطلب اللي بعده 401.

افتح الطلب الفاشل ← **Cookies** و فعّل show filtered out request cookies: الكوكي اتمنعت؟ السبب مكتوب (SameSite أو Secure أو Domain). أو لو بتستخدم Token: Request Headers فيها `authorization` ؟ وفي fetch لـ Origin تاني، محتاج `credentials: 'include'` عشان الكوكيز تتبعت أصلاً.

مشكلة 5 — 5. التعديل مش ظاهر

غيّرت الكود ورفعته ولسه شايف القديم.

Network ← Size: `(disk cache)` ؟ جرب Disable cache. لسه؟ `age` و `cf-cache-status: HIT` : الـ CDN ماسك القديم. لسه؟ Application ← Service workers. ولسه؟ اتأكد إنك رفعت فعلاً: `curl -s URL | grep` على الحاجة اللي غيّرتها.

مشكلة 6 — 6. بيشتغل مرة ويفشل مرة

نفس الزرار، أحياناً يشتغل وأحياناً لأ.

غالباً ترتيب: طلبين بيخلصوا بترتيب مختلف كل مرة، والكود معتمد إن واحد يخلص الأول. جرب Slow 4G في Throttling عشان تكبّر الفرق في الوقت وتخلي المشكلة تحصل كل مرة. أو حد تاني ماسك نفس الـ Port، أو الـ Token بيخلص بعد وقت.

مشكلة 7 **7. فيه طلب OPTIONS أنا ماعملتوش**

عملت POST، لقيت قبله OPTIONS، وساعات بيفشل.

ده Preflight (الفصل 14)، طبيعي مع JSON أو Authorization. لو فشل، المشكلة إن السيرفر مش بيرد على `OPTIONS` صح.
جربه بإيدك بـ `curl -X OPTIONS` مع `Origin` و `Access-Control-Request-Method`.

22.4 لما المشكلة في كودك: Breakpoints

`console.log` كويس، بس ليه حدود: لازم تعرف مقدماً عاوز تطبع إيه، وكل مرة تعدّل وتعمل Reload. الـ **Breakpoint** أقوى: بيوقف الكود عند سطر معين، **وانت تبص على كل المتغيرات** في اللحظة دي، وتمشي سطر سطر.

توقفه إزاي

`Sources ← اضغط على رقم السطر`	**Line breakpoint**: علامة زرقا على السطر. أول ما الكود يوصله، يقف. (افتح الملف بـ ⌘P.)
`debugger;`	اكتبها في كودك نفسه. لو DevTools مفتوح، الكود هيقف هنا. ومتنساش تشيلها.
`Right-click على السطر ← Add conditional breakpoint`	يقف **بس لو** الشرط صح، زي `user.id === 7`. مفيد جوه Loop بيلف 1000 مرة.
`Right-click ← Add logpoint`	مبيوقفش، بيطبع رسالة في الـ Console. يعني `console.log` من غير ما تلمس الكود.
`XHR/fetch Breakpoints (في الجنب)`	يقف أول ما أي `fetch` لـ URL فيه كلمة معينة يتبعت. تلاقي الكود اللي عمل الطلب من غير ما تعرف هو فين.
`Pause on exceptions ⏸`	يقف عند أي خطأ، قبل ما الـ Stack trace يطلع، والمتغيرات لسه موجودة. فعّل "Pause on caught exceptions" لو عاوز كمان اللي جوه `try`.

لما يقف

الصفحة هتتجمد وفوقها شريط "Paused in debugger"، والسطر متلون. وعلى الجنب:

`▶ Resume (F8)`	كمّل لحد الـ Breakpoint الجاي.
`⤼ Step over (F10)`	نفّذ السطر ده وروح للي بعده. لو فيه دالة، نفّذها كلها مرة واحدة.
`↓ Step into (F11)`	لو السطر فيه دالة، ادخل جواها.
`↑ Step out (⇧F11)`	كمّل الدالة دي واطلع للي ناداها.
`Scope`	كل المتغيرات اللي موجودة دلوقتي وقيمها: Local (جوه الدالة) و Closure و Global. وتقدر تعدّل قيمة بـ Double-click.

`Watch`	اكتب أي Expression (زي `data.users.length`) وهيتحدث مع كل خطوة.
`Call Stack`	نفس الـ Stack trace بتاع الفصل 20، بس حي: دوس على أي سطر فيه تشوف المتغيرات في اللحظة دي.

وكمان وانت واقف، الـ Console بيشتغل **في الـ Scope ده**: اكتب اسم أي متغير محلي وهيطبعه. ودي أسرع طريقة تجاوب على سؤال "الـ user ده كان undefined ليه؟": حط Breakpoint على السطر اللي قبل الخطأ، وبص على `data` اللي رجعت من الـ API.

> **الكورس هيوصل للجزء ده**
>
> كورس Jonas فيه جزء عن الـ Debugging وبيستخدم نفس الأدوات دي. الفصل ده مقدمة عشان لما توصله تبقى عارف المكان.
>
> والفكرة الأساسية: **console.log = Breakpoint لكل المتغيرات مرة واحدة، من غير ما تعدّل الكود.**

22.5 تسأل حد إزاي

ساعات هتحتاج تسأل: زميل، أو Stack Overflow، أو Discord الكورس، أو الـ AI. والسؤال الكويس نصه إجابة. ولو مشيت على الخريطة، معاك كل اللي محتاجه:

قالب سؤال كويس

```
## اللي بحاول أعمله
بجيب لستة الـ users من API وبعرضها في جدول.

## اللي حصل
الجدول فاضي، وفي الـ Console:
TypeError: Cannot read properties of undefined (reading 'map') at render (app.js:22:18)

## اللي جربته
- Network: GET /api/users بيرجع 200، والـ Response: {"data": {"users": [...]}}
- نفس الطلب بـ curl -i: نفس الرد
- الكود بيعمل data.users.map ← (هنا شكيت إن المسار غلط)

## البيئة
Chrome على macOS، Vite dev server على localhost:5173
```

لاحظ إن وانت بتكتب السؤال ده، غالباً هتلاقي الإجابة لوحدك: الرد فيه `data.data.users` مش `data.users`. ودي ظاهرة مشهورة اسمها **Rubber duck debugging**: لما تشرح المشكلة بالتفصيل (حتى لبطة بلاستيك)، بتشوفها.

> **قبل ما تلزق حاجة**
>
> شيل الـ Tokens والكوكيز والإيميلات وأي بيانات حقيقية من اللي هتلزقه، سواء لإنسان أو لـ AI. نفس الكلام بتاع Copy as cURL و HAR (الفصل 19).

ملخص الفصل

- الطريقة: لاحظ، نصّ المشكلة، جرب حاجة واحدة، اتأكد.
- الخريطة: Console ← الطلب موجود؟ ← رقم ولا أقواس؟ ← xx2؟ ← الشكل صح؟ ← كودك.
- "كودي ولا السيرفر؟": Copy as cURL. فشل = الطلب/السيرفر، نجح = المتصفح أو كودك.
- `Failed to fetch` مفيهوش تفاصيل عن قصد، والتفاصيل في Console و Network.
- Breakpoints: سطر، و`debugger`، و Conditional، و Logpoint، و fetch breakpoints، و Pause on exceptions.
- وانت واقف: Step over/into/out، و Scope، و Watch، و Call Stack، والـ Console في نفس الـ Scope.
- السؤال الكويس: اللي عاوزه، واللي حصل، واللي جربته، والبيئة. ومن غير أسرار.

صح ولا غلط؟

1. لو الطلب شغال بـ curl وفاشل في المتصفح، يبقى المشكلة أكيد في السيرفر. صح / غلط
2. الـ Logpoint بيوقف الكود زي الـ Breakpoint. صح / غلط
3. Step over بينفّذ الدالة اللي في السطر كلها من غير ما يدخل جواها. صح / غلط

أسئلة مراجعة

1. رتّب الـ 5 أسئلة بتاعة الخريطة، وقول ليه الترتيب ده.
2. Status `(failed)` و `500`: إيه الفرق في المكان اللي هتدور فيه؟
3. الموقع شغال على localhost وبايظ بعد الرفع. اذكر 3 أسباب في الشبكة.
4. إمتى تستخدم Conditional breakpoint بدل Breakpoint عادي؟
5. إيه اللي يخلي السؤال اللي بتسأله "كويس"؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل حط Breakpoint في أي ملف JS على موقع كبير، واعمل Reload، وبص على Scope و Call Stack.
2. متوسط اعمل fetch breakpoint بكلمة `api` على موقع بيعمل طلبات API، ولاقي الدالة اللي عملت الطلب.
3. صعب اختار Bug قابلك قبل كده في الكورس، واكتب له سؤال بالقالب اللي فوق. هل لقيت الإجابة وانت بتكتب؟

23

المرحلة 6 · DevTools والـ Debugging

الفصل التالت والعشرين

المشروع النهائي: صفحة بايظة وانت المحقق

هنبني صفحة صغيرة على جهازك فيها 8 أخطاء متخبية عن قصد في 6 مهام، من أنواع مختلفة: ملف مش موجود، و JSON بايظ، وشكل رد مختلف، و CORS بيخبي 404، و Body اتبعت غلط، وسيرفر مش شغال. مهمتك تلاقيهم كلهم بالأدوات اللي اتعلمتها، وتحدد كل واحد في أنهي محطة، وتثبت السبب بـ curl، وتصلّحه، وتكتب تقرير. الحلول في ملحق الإجابات.

بعد الفصل ده هتكون عارف

- [x] تجهّز مشروع وتشغّله بـ python3 -m http.server
- [x] تمشي على خريطة الفصل 22 خطوة خطوة
- [x] تستخدم Console و Network (Headers و Payload و Response) و Application
- [x] تثبت كل سبب من الـ Terminal: curl و nc و lsof و json.tool
- [x] تصلّح وتتأكد بـ Disable cache و 304
- [x] تكتب تقرير Debugging منظم

23.1 جهّز المعمل

اعمل فولدر للمشروع، وجواه 3 ملفات. اكتبهم بالظبط زي ما هم، **بالأخطاء**. متصلحش حاجة وانت بتكتب، حتى لو شفتها.

Terminal

```
user@MacBook ~ % mkdir -p ~/net-lab/data && cd ~/net-lab
user@MacBook ~ % code .
```

~/net-lab/index.html

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
  <meta charset="utf-8">
  <title>Net Lab</title>
  <script src="js/app.js" defer></script>
</head>
<body>
  <h1>المستخدمين</h1>
  <p id="status">جاري التحميل...</p>
  <ul id="users"></ul>
  <p id="quote"></p>
</body>
</html>
```

~/net-lab/app.js

```
const list = document.querySelector('#users');
const statusEl = document.querySelector('#status');

async function loadUsers() {
  const res = await fetch('data/user.json');
  const data = await res.json();
  data.users.forEach((u) => {
    list.insertAdjacentHTML('beforeend', `<li>${u.name} — ${u.email}</li>`);
  });
  statusEl.textContent = `${data.users.length} users`;
}

async function loadQuote() {
  const res = await fetch('https://example.com/quote.json');
  const q = await res.json();
  document.querySelector('#quote').textContent = q.text;
}

async function saveVisit() {
  const res = await fetch('https://httpbin.org/post', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: { page: 'home', at: Date.now() },
  });
  const echo = await res.json();
  console.log('server got:', echo.json);
}

async function ping() {
  await fetch('http://localhost:3000/health');
}

loadUsers();
loadQuote();
saveVisit();
ping();
```

~/net-lab/data/users.json

```
{
  "data": [
    {"id": 1, "name": "Sara", "email": "sara@example.com"},
    {"id": 2, "name": "Omar", "email": "omar@example.com"},
  ]
}
```

وشغّل سيرفر بسيط من نفس الفولدر (اتعرفنا عليه في الفصل 3):

تشريح الأمر

```
python3 -m http.server 8000
```

1	`python3`	Python 3. على الـ Mac جاي مع Command Line Tools، ولو مش موجود هيعرض عليك يسطبه.
2	`-m http.server`	شغّل الـ Module اللي اسمه `http.server`: سيرفر ملفات بسيط بيوزع الفولدر الحالي.
3	`8000`	الـ Port. افتح `http://localhost:8000`.

خلي الـ Terminal ده مفتوح

السيرفر ده بيطبع سطر لكل طلب بيجيله، زي `"GET /js/app.js HTTP/1.1" 404 -`. ده الـ Log السيرفر، وهو جزء من التحقيق: هيوريك اللي **وصل فعلاً** للسيرفر. افتح تابة Terminal تانية (⌘T) للأوامر.

23.2 قواعد التحقيق

افتح الصفحة في Chrome، و DevTools على Network، وفعّل **Disable cache** و**Preserve log**، واعمل Reload. وبعدين امشي على خريطة الفصل 22، ولكل خطأ تلاقيه املا الجدول ده في ملف `REPORT.md`:

الخانة	تكتب فيها
العَرَض	اللي شفته في الصفحة أو الـ Console بالنص
المحطة	DNS أو TCP أو TLS أو HTTP أو المتصفح (CORS) أو كودك
الدليل من DevTools	أنهي Panel وأنهي تابة، واللي لقيته
الدليل من الـ Terminal	أمر يثبت السبب من غير المتصفح، وناتجه
الإصلاح	التعديل بالظبط
التأكيد	إزاي اتأكدت إنه اتحل

جدول 23.1 قالب التقرير لكل خطأ

23.3 المهام

مش هقولك الأخطاء فين، بس هديك تلميحات حسب المحطة. حاول من غير التلميح الأول.

المهمة 1: الصفحة واقفة على "جاري التحميل.."

أول حاجة هتلاحظها إن مفيش أي حاجة بتحصل. ابدأ بالـ Console، وبعدين Network وفلتر `status-code:404`.

تلميح 1

تقدر تعدّيه في أول قراية

لو لقيت `net::ERR_ABORTED 404 (File not found)` على ملف JS، قارن المسار في الـ `<script>` بمكان الملف الحقيقي. وأكّد من الـ Terminal: `curl -I http://localhost:8000/js/app.js` مقابل `curl -I http://localhost:8000/app.js`، وبص على Log السيرفر.

المهمة 2: الـ users مش ظاهرين

بعد ما تصلح المهمة 1، هيظهر خطأ جديد. فيه هنا **3 أخطاء ورا بعض** في نفس الدالة، وكل ما تصلح واحد هيظهر اللي بعده. ده طبيعي جداً في الحياة الحقيقية.

تلميح 2

تقدر تعدّيه في أول قراية

الأول: `Unexpected token '<'` = جالك HTML بدل JSON (الفصل 19). بص على الـ Request URL واسم الملف. التاني: `SyntaxError` جوه JSON: افتح الملف في Response، أو من الـ Terminal: `python3 -m json.tool data/users.json` هيقولك السطر والعمود. التالت: `Cannot read properties of undefined (reading 'forEach')`: افتح Preview وقارن شكل الرد بالكود. أو حط Breakpoint على سطر `data.users.forEach` وبص على `data` في Scope.

المهمة 3: الـ quote مش ظاهرة

هتلاقي في الـ Console خطأ CORS. بس فاكر تحذير الفصل 14؟ الـ CORS error ساعات بيخبي حاجة تانية.

تلميح 3

تقدر تعدّيه في أول قراية

جرب الطلب من غير متصفح: `curl -si https://example.com/quote.json -H 'Origin: http://localhost:8000' | head -5`. بص على أول سطر، ودوّر على `access-control-allow-origin`. فيه مشكلتين هنا، مش واحدة. وإصلاحها مش في إيدك (السيرفر مش بتاعك)، فقرر: تعمل إيه في الكود عشان الصفحة متبوظش لو الطلب ده فشل؟

المهمة 4: السيرفر "مش فاهم" الـ Body

الـ Console بيطبع `server got: null`. يعني httpbin استلم الطلب ورد 200، بس الـ JSON اللي بعتناه ماوصلش.

تلميح 4

تقدر تعدّيه في أول قراية

افتح طلب `post` في Network ← **Payload** ← view source. وبص في الـ Response على `"data"` و `"json"`. وبعدين Copy as cURL، والزقه في الـ Terminal، وشوف الـ `--data-raw` فيه إيه.

المهمة 5: خطأ أحمر لطلب على localhost

آخر طلب بيفشل من غير Status رقم.

تلميح 5

تقدر تعدّيه في أول قراية

الـ Status بين أقواس، فالمشكلة قبل HTTP. اقرا اسم `net::ERR_*`، وأكّد بـ `nc -zv localhost 3000` و `lsof -nP -iTCP:3000 -sTCP:LISTEN`. القرار هنا: هل المفروض يبقى فيه سيرفر شغال، ولا الكود ده مالوش لازمة؟ وفي

الحالتين، الدالة لازم تتعامل مع الفشل.

المهمة 6: الكاش

بعد ما كل حاجة تشتغل: **اقفل** Disable cache، واعمل Reload مرتين. بص على Status و Size لملف `users.json` و `app.js`. وبعدين ضيف مستخدم تالت في `users.json`، واعمل Reload عادي. التعديل ظهر؟ وإيه الـ Status المرة دي؟

تلميح 6 — تقدر تعدّيه في أول قراية

جرب: `python3 -m http.server` بيبعت `Last-Modified`. وخد `curl -sI http://localhost:8000/data/users.json` وبعدين التاريخ، `curl -sI http://localhost:8000/data/users.json -H 'If-Modified-Since: التاريخ'`. إيه الرد؟ وإيه اللي هيحصل بعد ما تعدّل الملف؟

23.4 مهام إضافية (اختياري)

1. **Throttling**: شغّل الصفحة بعد الإصلاح على Slow 4G. ضيف `statusEl.textContent = 'جاري التحميل...'` وتأكد إنها بتبان وقت كافي. وجرب Offline: الصفحة بتعرض رسالة مفهومة؟
2. **Block**: اعمل Block لـ `httpbin.org` من Request conditions. هل الـ users لسه بيظهروا؟ لازم يظهروا: كل دالة لازم تفشل لوحدها من غير ما توقع الباقي.
3. **Override**: اعمل Override content لـ `users.json` وخليه `{"data": []}`. الصفحة بتقول "users 0" ولا بتقع؟
4. **Timing**: قيس طلب `httpbin.org/post` في تابة Timing وبـ `curl -w @timing.txt`. فين أطول مرحلة؟
5. **HAR**: صدّر HAR sanitized للجلسة كلها، وطلّع منه بـ `jq` كل الطلبات اللي Status بتاعها مش 2xx أو 3xx (لاحظ إن الطلبات اللي فشلت من غير رد Status بتاعها 0).
6. **Safari**: افتح نفس الصفحة في Safari بـ Web Inspector. نفس الأخطاء ظاهرة؟ النصوص مختلفة إزاي؟

23.5 إمتى تعرف إنك خلصت؟

الصفحة

بتعرض users 2 (أو 3 بعد المهمة 6) وعدّهم، ومفيش خطأ أحمر في الـ Console غير لو قررت تسيب طلب بيفشل ومتعامل معاه بـ `catch` (وساعتها المتصفح نفسه هيفضل يطبع سطر الشبكة).

التقرير

8 أخطاء (أو أكتر لو لقيت حاجة مش في القايمة)، لكل واحد: العرض والمحطة والدليلين والإصلاح والتأكيد.

ده شغلك الحقيقي بعد كده

المشروع ده صغير، بس الطريقة هي نفسها في أي مشروع كبير: متخمنش، امشي على المحطات، خد دليل من مكانين (المتصفح والـ Terminal)، صلّح حاجة واحدة، واتأكد. ولما تخلص الكورس وتبني مشاريعك، كل مشكلة شبكة هتقابلك هتقع في واحدة من الخانات دي. وارجع لملحق الإجابات عشان تقارن تقريرك بالحل.

ملخص المشروع

- ملف مش موجود (404 على الـ script)، ومسار JSON غلط، و JSON بايظ، وشكل رد مختلف عن الكود.
- CORS بيخبي 404 على سيرفر مش بتاعك، وعلاجه في الكود: تتعامل مع الفشل.
- Body اتبعت `[object Object]`، ويبان في Payload و Copy as cURL.
- سيرفر مش شغال: `ERR_CONNECTION_REFUSED`، وبتأكد بـ nc و lsof.
- الكاش: `Last-Modified` و `If-Modified-Since` و 304.

صح ولا غلط؟

1. `python3 -m http.server` يرد على الملفات اللي مش موجودة بصفحة HTML فيها 404. صح / غلط
2. تقدر تصلّح CORS على `example.com` من الكود بتاعك. صح / غلط
3. لو `fetch` اتبعت بـ `body: {…}` من غير `JSON.stringify`، الـ Body هيبقى `[object Object]`. صح / غلط

أسئلة مراجعة

1. ليه الخطأ في المهمة 2 الأول بيقول `Unexpected token '<'` مش 404؟
2. في المهمة 3، إيه المشكلتين اللي curl كشفهم؟
3. ليه كل دالة لازم يبقى فيها `try/catch` لوحدها؟
4. إزاي تعرف إن Log السيرفر و Network متفقين؟ وإمتى ممكن ميتفقوش؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل صلّح كل الأخطاء واكتب التقرير.
2. متوسط اعمل المهام الإضافية 1 و 2 و 3.
3. صعب ضيف بنفسك خطأين جداد في المشروع من أنواع في الكتاب (مثلاً Redirect loop أو Mixed content أو كوكي اتمنعت)، وابعته لصاحبك يحله بالتقرير.

ملحق أ

Cheat Sheet وجدول الـ Flags وجدول الرموز

ورقة مراجعة لكل أوامر الـ Terminal واختصارات DevTools والـ Console اللي في الكتاب مقسّمة حسب الموضوع، وبعدها جدول الـ Flags ومعانيها في كل أمر، وجدول لكل رمز ممكن تقابله: علامات curl -v، ورموز الـ URL، والكوكيز، وقيم Network الخاصة.

curl: read

curl: تقرا الرد

`curl URL`	الـ Body بس
`curl -i URL`	Body + الرد Headers
`curl -I URL`	طلب HEAD: Headers بس
`curl -v URL`	كل حاجة: > طالع، < جاي، * معلومات
`curl -sIL URL`	تابع كل الـ Redirects (Headers بس)
`curl -s URL | jq .`	JSON منسق

curl: send

curl: تبعت

`curl -X DELETE URL`	Method معين
`curl -H 'Name: value' URL`	Header
`curl -d 'a=1&b=2' URL`	Body فورم (POST تلقائي)
`curl --json '{"a":1}' URL`	JSON + الـ Headers بتاعته
`curl --json @file.json URL`	JSON من ملف
`curl -F 'f=@me.png' URL`	رفع ملف multipart
`curl -G --data-urlencode 'q=a b' URL`	Query متشفّر
`curl -u user URL`	Basic auth (يسألك الباسورد)

curl: control

curl: تتحكم

`curl -L URL`	اتبع الـ Redirects
`curl -o f URL / -O`	احفظ باسم / بنفس الاسم
`curl -sSf URL`	اسكت، وافشل بصوت عالي
`curl --fail-with-body URL`	افشل واطبع الـ Body
`curl -c jar -b jar URL`	احفظ وابعت كوكيز
`curl --max-time 10 URL`	حد أقصى للوقت
`curl --http1.1 / --http2`	نسخة HTTP
`curl --resolve h:443:IP URL`	DNS مؤقت للأمر ده
`curl -s -o /dev/null -w @timing.txt URL`	قيس المراحل

DNS

الأسماء

`dig example.com`	A record بالتفصيل
`dig +short example.com AAAA`	IPv6 بس
`dig example.com MX / NS / TXT`	أنواع تانية
`dig @1.1.1.1 example.com`	اسأل Resolver معين
`dig +trace example.com`	من الـ Root لحد الإجابة

⌘⇧P	Command Menu
Esc	الـ Drawer
⌘R / ⌘⇧R	Hard reload / Reload
⌘E	وقّف/شغّل تسجيل Network
⌘F (Network)	دوّر في الطلبات
⌘⌥F	دوّر في كل الملفات
⌘P	افتح ملف
⌘K	امسح الـ Console
⇧ + Hover	مين طلب الطلب ده
F8 / F10 / F11	Step into / Step over / Resume

Console — الـ Console

`await fetch(url)`	طلب سريع
`res.status / res.ok`	اتأكد من النجاح
`res.headers.get('x')`	Header من الرد
`console.table(arr)`	جدول
`console.time / timeEnd`	قيس وقت
`$0 / $_ / $$('a')`	العنصر / آخر نتيجة / كل العناصر
`copy(obj)`	انسخ JSON
`document.cookie`	الكوكيز اللي الـ JS شايفها
`localStorage.getItem('k')`	اقرا التخزين

`nslookup example.com`	البديل البسيط
`dscacheutil -q host -a name x.test`	زي ما الـ Mac شايفه (و hosts)
`sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder`	امسح كاش DNS

Network — الشبكة

`ipconfig getifaddr en0`	IP جهازك على الـ Wi-Fi
`route -n get default`	الراوتر
`curl -4 ifconfig.me`	الـ IP العام
`ping -c 4 host`	وصلنا؟ والـ RTT
`traceroute host`	الطريق راوتر راوتر
`nc -zv host 443`	Port مفتوح؟
`lsof -nP -iTCP:3000 -sTCP:LISTEN`	مين ماسك الـ Port
`python3 -m http.server 8000`	سيرفر ملفات سريع
`sudo nano /etc/hosts`	أسماء على جهازك

TLS — الشهادات

`openssl s_client -connect h:443 -servername h`	Handshake خام والسلسلة
`... | openssl x509 -noout -dates`	تاريخ الشهادة
`... | openssl x509 -noout -text | grep -A1 Alternative`	الأسماء (SAN)
`curl -v URL 2>&1 | grep '^\*'`	سطور TLS في curl

DevTools — اختصارات Chrome

⌘⌥I / ⌘⌥J / ⌘⌥C	افتح / Console / اختار عنصر

أ.1 جدول الـ Flags

نفس الحرف ممكن يبقى ليه معنى مختلف من أمر للتاني (`-v` في curl و nc، و`-n` في lsof و route). ولما تشك: `man curl` أو `curl --help all`.

الـ Flag	في الأوامر	المعنى
`-v`	curl	Verbose: كل التفاصيل
`-i`	curl	Include: Headers الرد
`-I`	curl	Head: طلب HEAD
`-X`	curl	الـ Method
`-H`	curl	Header
`-d`	curl	Data: Body (فورم)
`--data-raw`	curl	زي d- بس @ مش ملف
`--json`	curl	Accept + Content-Type + Body JSON
`-F`	curl	Form multipart
`-G`	curl	حط الـ Data في الـ Query
`-L`	curl	Location: اتبع الـ Redirects
`-o / -O`	curl	احفظ باسم / بنفس الاسم
`-s / -S`	curl	Show errors / Silent
`-f`	curl	Fail: Exit code 22 لو ≥400
`-w`	curl	Write-out: اطبع متغيرات
`-A`	curl	User-Agent
`-u`	curl	User: Basic auth
`-b / -c`	curl	ابعت كوكيز / احفظ كوكيز
`-k`	curl	Insecure: من غير تحقق (محلي بس)
`-C -`	curl	كمّل تحميل
`-4 / -6`	nc / ping / curl	بس IPv6 / IPv4
`+short`	dig	الإجابة بس
`+trace`	dig	الرحلة من الـ Root

الـ Flag	في الأوامر	المعنى
`@server`	dig	اسأل سيرفر معين
`-c N`	ping	عدد المحاولات
`-z`	nc	Zero I/O: افحص بس
`-v`	nc	Verbose
`-l`	nc	Listen: استنى اتصال
`-n`	route / lsof	أرقام بدل أسماء
`-P`	lsof	Ports أرقام
`-iTCP:3000`	lsof	الـ Port ده
`-sTCP:LISTEN`	lsof	اللي مستني اتصالات بس
`-connect`	openssl s_client	Host:Port
`-servername`	openssl s_client	SNI
`-m`	python3	شغّل Module

جدول أ.1 الـ Flags ومعانيها حسب الأمر

أ.2 جدول الرموز

الرمز	المعنى	مثال
`* / < / >`	في curl -v: طالع / جاي / معلومة	`> GET / HTTP/2`
`@`	في curl: اقرا من ملف	`-d @body.json`
`@-`	اقرا من stdin	`cat x | curl -d @- URL`
`'...'`	احمي الـ ? و & و * من zsh	`curl 'a.com/?q=1&b=2'`
`? & =`	الـ Query: بداية / فاصل / قيمة	`/search?q=git&page=2`
`#`	Fragment: مبيتبعتش للسيرفر	`/docs#install`
`%20 %26`	Percent-encoding: مسافة / &	`q=rock%20%26%20roll`
`:port`	الـ Port في الـ URL	`localhost:5173`
`:method :path`	Pseudo-headers في HTTP/2	`":path: /users"`
`; (Set-Cookie)`	بين الكوكي والـ Attributes	`sid=a1; HttpOnly`
`; (Cookie)`	بين كوكيز مختلفة	`a=1; b=2`

الرمز	المعنى	مثال
`W/"..."`	ETag ضعيف	`W/"3e-Jwo8"`
`*`	في ACAO: أي Origin	`access-control-allow-origin: *`
`q=0.8`	وزن في Accept-*	`ar;q=0.9, en;q=0.8`
`1xx..5xx`	فئة الـ Status	`404`
`(pending) (failed) (canceled)`	Status من غير رد HTTP	`(failed) net::ERR_...`
`(blocked:…)`	اتمنع: devtools / mixed-other / csp / content	`(blocked:devtools)`
`(disk cache) (memory cache)`	في Size: من الكاش	`(memory cache)`
`h2 / h3`	في Protocol: HTTP/2 / HTTP/3	`h3`
`$0 $_ $$`	Console: العنصر / آخر نتيجة / كل العناصر	`$0.click()`
`x / ! / >`	في أمثلة الكتاب: خطأ / تحذير / سطر بتكتبه	`x Uncaught TypeError`
`::1 / 127.0.0.1`	localhost بـ IPv4 / IPv6	`curl http://[::1]:8000`
`0.0.0.0`	في السيرفر: كل الـ Interfaces	`--host 0.0.0.0`
`/24`	CIDR: عدد البتات الثابتة	`192.168.1.0/24`

جدول أ.2 كل رمز في الكتاب ومعناه

ملحق ب

قاموس المصطلحات

كل مصطلح مهم في الكتاب في سطر واحد، مترتب أبجدياً. ارجعله كل ما تقابل كلمة ومش فاكر معناها.

A

ACAO
اختصار `Access-Control-Allow-Origin` : الـ Header اللي السيرفر بيقول بيه مين مسموحله يقرا الرد (CORS).

Alt-Svc
Header بيقول للمتصفح إن فيه طريقة تانية للوصول، زي `h3=":443"` لـ HTTP/3.

Anycast
نفس الـ IP متعلن من أماكن كتير، والشبكة بتوديك لأقرب واحد. بتستخدمه سيرفرات الـ DNS والـ CDNs.

API
واجهة برمجية: عناوين بترد ببيانات (غالباً JSON) بدل صفحات.

Authorization
Header فيه إثبات الهوية، زي `Bearer <token>` أو `Basic` .

B

Breakpoint
علامة على سطر بتوقف الكود عنده عشان تبص على المتغيرات.

C

CA
Certificate Authority: جهة موثوقة بتمضي على شهادات المواقع.

Cache-Control
Header بيحدد مين يكاش الرد وقد إيه: `max-age` و `no-cache` و `no-store` .

CDN
شبكة سيرفرات في مدن كتير (Edge) بتحتفظ بنسخ من المحتوى قريب من اليوزر.

Certificate
ملف فيه اسم الموقع والـ Public key ومدة الصلاحية وإمضا CA.

CHIPS
كوكيز `Partitioned` : Third-party cookie متخزنة منفصلة لكل موقع أب.

CIDR
طريقة كتابة نطاق IPs زي `192.168.1.0/24` .

Client / Server
اللي بيطلب (المتصفح أو curl) / اللي بيرد.

Console
Panel في DevTools فيه الرسايل والأخطاء ومكان تكتب فيه JS على الصفحة.

Content-Type
Header بيقول نوع الـ Body، زي `application/json` .

Cookie
تذكرة صغيرة السيرفر بيحطها بـ `Set-Cookie` والمتصفح بيرجعها مع كل طلب.

CORS
استثناء من Same-origin policy بيدّيه السيرفر بـ Headers `Access-Control-*` .

CSP
Content Security Policy: قايمة من الموقع بالأماكن المسموح للصفحة تحمّل منها أو تكلمها.

CSRF
هجوم بيخلي موقع شرير يبعت طلب لموقع تاني بكوكيز اليوزر. SameSite بيمنعه.

D

DevTools
أدوات المطورين المدمجة في المتصفح.

DNS
نظام بيترجم الأسماء زي `example.com` لـ IPs.

DOM
شجرة عناصر الصفحة اللي المتصفح بيبنيها من الـ HTML والـ JS بيعدل فيها.

Domain

اسم الموقع، ومقسم مستويات: `www` . `example` . `com` .

DOMContentLoaded

حدث: الـ HTML اتقرا كله والـ DOM جاهز (الخط الأزرق).

E

ETag

بصمة لنسخة الرد، المتصفح بيسأل بيها `If-None-Match` .

F

Fetch/XHR

الطلبات اللي الـ JS بيعملها بـ `fetch` أو `XMLHttpRequest` ، وفلتر في Network.

Fragment

الجزء بعد `#` في الـ URL، مبيتبعتش للسيرفر.

H

Handshake

تبادل رسايل أولي لفتح اتصال: TCP (SYN/SYN-ACK/ACK) أو TLS.

HAR

ملف JSON فيه جلسة Network كلها.

Header

سطر `Name: value` في الطلب أو الرد.

HOL blocking

Head-of-line blocking: طلب بطيء في الأول بيوقف اللي وراه.

Hosts file

`/etc/hosts` : أسماء وعناوين على جهازك بتتقري قبل الـ DNS.

HSTS

Header بيقول للمتصفح يستخدم HTTPS بس مع الموقع ده لمدة معينة.

HTTP

البروتوكول اللي المتصفح والسيرفر بيتكلموا بيه: طلب ورد.

HTTP/2

نسخة Binary فيها Multiplexing على اتصال واحد و HPACK.

HTTP/3

HTTP فوق QUIC (UDP)، فكل Stream مستقل.

HttpOnly

Attribute بيخبي الكوكي عن الـ JS.

HTTPS

HTTP جوه نفق TLS.

I

Idempotent

طلب لو اتكرر نفس النتيجة: GET و PUT و DELETE.

Initiator

مين اللي عمل الطلب: Parser أو Script أو Redirect.

IP address

عنوان الجهاز على الشبكة: IPv4 زي `104.20.23.154` أو IPv6.

J

JSON

صيغة نصية للبيانات شبه Objects الـ JS.

JWT

Token فيه بيانات Base64 وإمضا. بيتقري بسهولة.

L

Latency

وقت التأخير لحد أول رد.

Load

حدث: كل حاجة في الصفحة اتحملت (الخط الأحمر).

localhost

جهازك نفسه: `127.0.0.1` أو `::1` .

localStorage

تخزين Key/Value في المتصفح لكل Origin، مبيتبعتش للسيرفر.

M

Method

الفعل في الطلب: GET و POST و PUT و PATCH و DELETE و OPTIONS.

MIME type

نوع المحتوى زي `text/html` .

Mixed content

صفحة HTTPS بتحمّل حاجة من `http://` .

Multiplexing

طلبات كتير متداخلة على اتصال واحد.

N

NAT

الراوتر بيخلي أجهزة البيت كلها تطلع بـ IP عام واحد.

net::ERR_*

أخطاء Chrome لما مفيش رد HTTP، والاسم بيقول المحطة.

Network panel

Panel في DevTools بيوري كل الطلبات وتفاصيلها.

O

Origin

scheme + host + port. أساس CORS والتخزين.

Overrides

تغيير رد أو Headers من DevTools على جهازك بس.

P

Packet

حتة بيانات صغيرة بتتنقل على الشبكة لوحدها.

Payload

الـ Body أو الـ Query اللي اتبعت مع الطلب، وتابة في Network.

Port

رقم بيحدد البرنامج جوه الجهاز: 443 و 80 و 5173.

Preflight

طلب `OPTIONS` المتصفح بيعمله قبل طلب Cross-origin مش بسيط.

Preserve log

متمسحش قايمة Network أو Console مع الـ Navigation.

Protocol

مجموعة قواعد للكلام بين جهازين.

Proxy

وسيط بيستقبل الطلب ويبعته لمكان تاني.

Q

Query string

الجزء بعد `?` في الـ URL.

QUIC

بروتوكول نقل فوق UDP فيه TLS 1.3، أساس HTTP/3.

R

Redirect

رد xx3 فيه `Location` بيقول روح لعنوان تاني.

Resolver

سيرفر DNS بيدوّر بالنيابة عنك ويكاش الإجابات.

RTT

Round-trip time: وقت الرحلة رايح جاي.

S

Same-origin policy

قاعدة المتصفح: JS من Origin ميقراش ردود Origin تاني.

SameSite

Attribute بيحدد الكوكي تتبعت مع طلبات Cross-site ولا لأ: Strict و Lax و None.

SAN

Subject Alternative Name: الأسماء اللي الشهادة بتغطيها.

Secure

Attribute: الكوكي على HTTPS بس.

Service Worker

JS بيقف بين الصفحة والشبكة ويقدر يرد من كاش.

Session

حالة اليوزر متخزنة على السيرفر، والكوكي فيها رقمها.

sessionStorage

زي localStorage بس بيتمسح لما التابة تتقفل.

SNI

اسم الموقع اللي المتصفح بيبعته في ClientHello عشان السيرفر يطلع الشهادة الصح.

Stack trace

سلسلة الدوال اللي وصلت للخطأ، أولها مكانه.

Stateless

كل طلب HTTP لوحده والسيرفر مبيفتكرش اللي قبله.

Status code

رقم 3 خانات في الرد: xx2 نجاح، xx3 تحويل، xx4 غلطتك، xx5 غلطة السيرفر.

Subdomain

مستوى قبل الدومين زي `api.` .

T

TCP

بروتوكول نقل بيضمن وصول البيانات كاملة وبالترتيب.

Third-party cookie

كوكي بتاعة دومين غير الموقع اللي انت فيه.

Throttling

تقليد شبكة أبطأ في DevTools.

Timing

تابة بتقسم وقت الطلب: Queueing و DNS و Connection و SSL و Waiting و Download.

TLS

بروتوكول التشفير والتحقق من الهوية تحت HTTPS.

TTFB

Time To First Byte: لحد أول Byte من الرد. في Chrome: Waiting for server response.

TTL

مدة صلاحية: لـ DNS record، أو عدد القفزات في IP.

U

UDP

بروتوكول نقل سريع من غير ضمانات.

URL

العنوان الكامل: scheme و host و port و path و query و fragment.

User-Agent

Header بيعرّف المتصفح أو البرنامج.

V

Vary

Header بيقول للكاش الرد بيختلف حسب أنهي Headers.

W

Waterfall

عمود في Network بيرسم كل طلب على محور الوقت.

ملحق ج

مصادر للاستزادة

كل اللينكات دي اتراجعت وكانت شغالة وقت كتابة الكتاب (أكتوبر 2026)، ومعظمها مراجع رسمية بتتحدث باستمرار: MDN و Chrome for Developers و web.dev و RFCs و Cloudflare. ابدأ بـ MDN لأي موضوع، وارجع لدوكيومنتيشن Chrome الرسمية لأي حاجة في DevTools، لأن الواجهة بتتغير شوية مع كل نسخة.

ج.1 ابدأ من هنا: الصورة الكبيرة

How does the Internet work? · MDN · شرح بسيط للإنترنت والـ Packets والراوترات (الفصول 1–2).
https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/How_does_the_Internet_work

How browsers work · MDN · رحلة الصفحة من الطلب للرسم (الفصلين 2 و 16).
https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/How_browsers_work

How browsers work · web.dev · المقالة الكلاسيكية المفصلة عن جوه المتصفح.
https://web.dev/articles/howbrowserswork

Critical rendering path · web.dev · DOM و CSSOM و Render-blocking (الفصل 16).
https://web.dev/articles/critical-rendering-path

HTTP: Learn your browser's language · Julia Evans · Zine مرسوم وممتع عن HTTP (مدفوع).
https://wizardzines.com/zines/http/

ج.2 العناوين والأسماء: IP و DNS و URL

What is a URL? · MDN · أجزاء الـ URL (الفصل 6).
https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_URL

What is a domain name? · MDN · الدومينات ومستوياتها (الفصل 5).
https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_domain_name

URIs · MDN · المرجع الكامل للـ URL والـ Schemes.
https://developer.mozilla.org/en-US/docs/Web/URI

What is DNS? · Cloudflare · خطوات الـ Resolution بالرسم (الفصل 5).
https://www.cloudflare.com/learning/dns/what-is-dns/

Domain names · RFC 1035 · المواصفة الأصلية للـ DNS (للفضول).
https://www.rfc-editor.org/rfc/rfc1035

TCP · RFC 9293 · مواصفة TCP الحديثة (الفصل 4).
https://www.rfc-editor.org/rfc/rfc9293

TCP handshake · MDN · الـ Handshake الثلاثي (SYN و SYN-ACK و ACK) باختصار.
https://developer.mozilla.org/en-US/docs/Glossary/TCP_handshake

ج.3 HTTP: اللغة نفسها

MDN · Overview of HTTP · مدخل HTTP الأساسي (الفصل 7).
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview

MDN · HTTP messages · شكل الطلب والرد سطر سطر.
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages

MDN · Evolution of HTTP · من 0.9 لحد HTTP/3 (الفصلين 7 و 11).
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Evolution_of_HTTP

MDN · HTTP request methods · كل Method وخصائصه.
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods

MDN · HTTP response status codes · كل Status code (الفصل 8).
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status

MDN · HTTP headers · مرجع كل Header (الفصل 9).
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers

MDN · Using the Fetch API · fetch و res.ok والـ Body (الفصلين 7 و 20).
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

http.dev · موسوعة HTTP مرتبة بالـ Headers والـ Status codes.
https://http.dev/

RFC 9110 · HTTP Semantics · المرجع الرسمي للـ Methods والـ Status والـ Headers.
https://www.rfc-editor.org/rfc/rfc9110

RFC 9113 · HTTP/2 · مواصفة HTTP/2.
https://www.rfc-editor.org/rfc/rfc9113

RFC 9114 · HTTP/3 · مواصفة HTTP/3.
https://www.rfc-editor.org/rfc/rfc9114

RFC 9000 · QUIC · مواصفة QUIC.
https://www.rfc-editor.org/rfc/rfc9000

Cloudflare · What is HTTP/3? · QUIC و HOL blocking بالرسم (الفصل 11).
https://www.cloudflare.com/learning/performance/what-is-http3/

Cloudflare Radar · 2025 Year in Review · أرقام استخدام HTTP/2 و HTTP/3 و Post-quantum.
https://blog.cloudflare.com/radar-2025-year-in-review/

ج.4 curl والأدوات

curl man page · كل Flag في curl (الفصل 10).
https://curl.se/docs/manpage.html

Everything curl · كتاب مجاني كامل عن curl من صاحبه.
https://everything.curl.dev/

jq manual · فلاتر jq للـ JSON والـ HAR.
https://jqlang.org/manual/

httpbin.org · سيرفر تجارب بيرجعلك طلبك.
https://httpbin.org/

JSONPlaceholder · API وهمي للتجارب (الفصول 10 و 19 و 20).
https://jsonplaceholder.typicode.com/

ج.5 الأمان: TLS والكوكيز و CORS

What happens in a TLS handshake? · Cloudflare · الـ Handshake خطوة بخطوة (الفصل 12).
https://www.cloudflare.com/learning/ssl/what-happens-in-a-tls-handshake/

How HTTPS works · قصة مصورة عن HTTPS والشهادات.
https://howhttps.works/

How it works · Let's Encrypt · إزاي الشهادات المجانية بتتعمل أوتوماتيك.
https://letsencrypt.org/how-it-works/

Baseline Requirements · CA/Browser Forum · قواعد الشهادات ومنها العمر اللي بيقصر.
https://cabforum.org/working-groups/server/baseline-requirements/

TLS 1.3 · RFC 8446 · مواصفة TLS 1.3.
https://www.rfc-editor.org/rfc/rfc8446

Strict-Transport-Security · MDN · HSTS بالتفصيل.
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security

Mixed content · MDN · ليه المتصفح بيمنع http جوه https.
https://developer.mozilla.org/en-US/docs/Web/Security/Defenses/Mixed_content

badssl.com · مواقع بشهادات بايظة عن قصد للتجربة.
https://badssl.com/

Using HTTP cookies · MDN · الكوكيز والـ Attributes (الفصل 13).
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies

Set-Cookie · MDN · كل Attribute ومعناه.
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie

HTTP session · MDN · الـ Session على مستوى HTTP.
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Session

Web Storage API · MDN · localStorage و sessionStorage.
https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

HTTP State Management · RFC 6265 · المواصفة الرسمية للكوكيز.
https://www.rfc-editor.org/rfc/rfc6265

Update on plans · Privacy Sandbox · قرار جوجل عن الـ Third-party cookies وقفل Privacy Sandbox.
https://privacysandbox.google.com/blog/update-on-plans-for-privacy-sandbox-technologies

jwt.io · تفك JWT وتشوف جواه.
https://jwt.io/

Same-origin policy · MDN · القاعدة اللي CORS استثناء منها (الفصل 14).
https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

CORS · MDN · Simple requests و Preflight وكل Header.
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

Content Security Policy · MDN · الـ CSP وأخطاؤه (الفصل 20).
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP

server.proxy · Vite · إعداد Proxy التطوير.
https://vite.dev/config/server-options

mkcert · HTTPS محلي من غير تحذيرات.
https://github.com/FiloSottile/mkcert

ج.6 السرعة والكاش

HTTP caching · MDN · الكاش كله في مكان واحد (الفصل 15).
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching

Cache-Control · MDN · كل Directive.
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cache-Control

HTTP cache · web.dev · استراتيجيات عملية والـ Hash في الأسماء.
https://web.dev/articles/http-cache

Time to First Byte · web.dev · الـ TTFB وأسبابه.
https://web.dev/articles/ttfb

What is a CDN? · Cloudflare · الـ Edge والـ Origin.
https://www.cloudflare.com/learning/cdn/what-is-a-cdn/

HTTP Caching · RFC 9111 · المواصفة الرسمية للكاش.
https://www.rfc-editor.org/rfc/rfc9111

ج.7 Chrome DevTools (دوكيومنتيشن جوجل الرسمية)

Open Chrome DevTools · كل طرق الفتح (الفصل 17).
https://developer.chrome.com/docs/devtools/open

Keyboard shortcuts · كل الاختصارات بالـ Mac.
https://developer.chrome.com/docs/devtools/shortcuts

Inspect network activity · درس عملي على Network (الفصل 18).
https://developer.chrome.com/docs/devtools/network

Network features reference · المرجع الكامل: الفلاتر والأعمدة والـ Timing والـ HAR (الفصلين 18 و 19).
https://developer.chrome.com/docs/devtools/network/reference

Network panel overview · الواجهة بالصور.
https://developer.chrome.com/docs/devtools/network/overview

Request conditions · Block و Throttling لطلبات معينة.
https://developer.chrome.com/docs/devtools/request-conditions

Overrides · Override content و headers.
https://developer.chrome.com/docs/devtools/overrides

Console overview · مدخل الـ Console (الفصل 20).
https://developer.chrome.com/docs/devtools/console

Console features reference · الفلاتر والإعدادات والـ Sidebar.
https://developer.chrome.com/docs/devtools/console/reference

Log messages in the Console · console.* بالتفصيل.
https://developer.chrome.com/docs/devtools/console/log

Understand errors and warnings · Console insights وحدوده.
https://developer.chrome.com/docs/devtools/console/understand-messages

MDN · Console API · كل دوال console.
https://developer.mozilla.org/en-US/docs/Web/API/Console_API

Application panel · التخزين والـ Service Workers (الفصل 21).
https://developer.chrome.com/docs/devtools/application

View, edit, and delete cookies · جدول الكوكيز.
https://developer.chrome.com/docs/devtools/application/cookies

View and edit local storage · localStorage.
https://developer.chrome.com/docs/devtools/storage/localstorage

Debug JavaScript · درس Breakpoints عملي (الفصل 22).
https://developer.chrome.com/docs/devtools/javascript

Pause your code with breakpoints · كل أنواع الـ Breakpoints.
https://developer.chrome.com/docs/devtools/javascript/breakpoints

What's new in DevTools (Chrome 125) · مثال على تحديثات DevTools المستمرة. تابع مدونة What's new مع كل نسخة.
https://developer.chrome.com/blog/new-in-devtools-125

ج.8 Safari على الـ Mac

WebKit · Web Inspector · دوكيومنتيشن Web Inspector الرسمية.
https://webkit.org/web-inspector/

Apple · Use the developer tools in the Develop menu · تفعيل قايمة Develop (الفصل 21).
https://support.apple.com/guide/safari/use-the-developer-tools-in-the-develop-menu-sfri20948/mac

ملحق د

الإجابات والحلول

جاوب بنفسك الأول، وبعدين قارن. لكل فصل: إجابات أسئلة المراجعة، وبعدها صح ولا غلط، وبعدها حلول التحديات. أغلب التحديات تجارب عملية، فلو وصلت لنفس النتيجة بطريقة تانية فهي صح.

الفصل 1: حكاية الإنترنت

1. Circuit switching بيحجز خط كامل طول المكالمة حتى لو ساكت (هدر)، ولو حتة في الطريق وقعت المكالمة كلها بتقع. Packet switching بيقطّع الرسالة Packets كل واحدة بتمشي لوحدها وتشارك الخطوط مع غيرها، ولو طريق وقع بتلف من طريق تاني.
2. HTML (لغة الصفحة والروابط)، و URL (عنوان لكل حاجة)، و HTTP (طريقة طلبها ونقلها). ومعاهم أول متصفح وأول سيرفر.
3. لابتوبك وانت شغال Vite: المتصفح Client بيطلب من السيرفر اللي على نفس الجهاز، والسيرفر ده Server. وفي نفس الوقت اللابتوب GitHub ك Client وهو بيعمل push.
4. Protocol = قواعد متفق عليها للكلام بين جهازين: مين يبدأ، وشكل الرسالة، وتعمل إيه لو حاجة غلط. في الكتاب: IP و TCP و UDP و DNS و HTTP و TLS و QUIC.

صح ولا غلط: (1) غلط: الويب خدمة واحدة فوق الإنترنت. · (2) غلط: كل Packet ممكن تمشي في طريق. · (3) غلط: أي جهاز شغال عليه برنامج بيرد على طلبات يبقى Server، حتى اللابتوب.

التحديات:

1. مثال: YouTube و Gmail في المتصفح = ويب. WhatsApp و Zoom و Spotify App = خدمات فوق الإنترنت من غير ويب (وممكن تستخدم HTTP جواها برضه).
2. ناقص: CSS والصور وال JS والتفاعل وال Login. بس الفكرة الأساسية (صفحة فيها روابط) هي هي.
3. تلخيص: الوثيقة دي بتشرح معنى HTTP: الرسايل وال Methods وال Status codes وال Headers، بغض النظر عن نسخة النقل (1.1 أو 2 أو 3).

الفصل 2: رحلة الطلب

1. DNS، ثم TCP، ثم TLS، ثم HTTP Request، ثم (الرد و) الرسم.
2. الشبكة بتشتغل بالأرقام (IPs) مش بالأسماء. الاسم للبني آدمين، وال DNS بيترجمه لعنوان الراوترات تفهمه.
3. بيرمي ال Body (ال HTML) بدل ما يتطبع، فيفضل قدامك سطور `-v` بس.
4. كاش ال DNS (مش محتاج يسأل تاني)، وكاش المتصفح للملفات، واتصال مفتوح بيتعاد استخدامه (من غير Handshake جديد).

صح ولا غلط: (1) غلط: صفحة واحدة ممكن تعمل عشرات أو مئات الطلبات. · (2) غلط: ال DNS قبل أي اتصال. · (3) صح: `<` = جاي من السيرفر، و `>` = طالع منك.

التحديات:

1. علّم: سطور Trying/Connected (TCP)، وسطور TLS، وسطور `>` (الطلب)، وسطور `<` (الرد).
2. موقع إخباري غالباً 100–300 طلب، وموقع بسيط أقل من 20.
3. قارن بعد الكتاب: لازم تلاقي DNS و TCP و TLS و HTTP والكاش والرسم في إجابتك.

الفصل 3: IP وال Ports

1. لأن الاتنين Private IPs جوه شبكات منفصلة (كل بيت ليه NAT). الإنترنت بيشوف ال Public IP بتاع الراوتر بس.
2. نفس الجهاز، بس برنامجين مختلفين: كل Port باب لبرنامج. وكمان Origins مختلفين.
3. `2001:db8::1` .
4. `-n` أرقام IP من غير أسماء، و `-P` أرقام Ports من غير أسماء، و `-iTCP:3000` اتصالات TCP على Port 3000، و `-sTCP:LISTEN` اللي في حالة استماع بس.

صح ولا غلط: (1) غلط: كل رقم من 0 ل 255. · (2) غلط: بيشوفوا ال Public IP بتاع الراوتر. · (3) غلط: 127.0.0.1 = الجهاز نفسه بس. لازم 0.0.0.0 أو IP الجهاز.

التحديات:

1. `ipconfig getifaddr en0` و `route -n get default` و `curl -4 ifconfig.me`.
2. من غير `--bind` بيسمع على كل الـ Interfaces فالموبايل يفتحه. مع `--bind 127.0.0.1` الموبايل مش هيوصل، لأن السيرفر بيسمع على Loopback بس.
3. التاني بيقول `Address already in use`. `lsof -nP -iTCP:8000 -sTCP:LISTEN` يديك الـ PID، و `kill PID` يحرره.

الفصل 4: TCP و UDP

1. تضيع (TCP بيعيد بعتها لما مايجيش ACK)، تيجي بترتيب غلط (أرقام seq بترتبها)، تتكرر (الأرقام بتكشف المكرر).
2. الـ ACK معناه "استلمت لحد 100، ومستني 101"، يعني الرقم اللي جاي.
3. لأن الصوت المتأخر مالوش لازمة: أحسن تفوّت جزء صغير على إنك توقف المكالمة تستنى إعادة إرسال.
4. Refused: الجهاز وصل ورد إن مفيش حد على الـ Port. مستني: محدش بيرد خالص (Firewall بيرمي، أو IP غلط).
5. `\r\n` نهاية سطر في HTTP، والسطر الفاضي (التانية) معناه إن الـ Headers خلصت.

صح ولا غلط: (1) غلط: الراوتر بيبص على الـ IP بس. · (2) غلط: UDP مبيضمنش ترتيب ولا وصول. · (3) غلط: سيرفرات كتير بتمنع ping (ICMP) وهي شغالة.

التحديات:

1. الأقرب (1.1.1.1 غالباً في مصر) أقل، والبعيد (أستراليا) أعلى بكتير. السبب: المسافة الفعلية وسرعة الضوء.
2. هتلاقي قفزة كبيرة عند خروج الطلب من مصر (كابلات البحر). النجوم `* * *` = راوتر مش بيرد، مش عطل.
3. الرد `HTTP/1.0 200 OK` و Headers وبعدها HTML، وفي تابة السيرفر سطر `"GET / HTTP/1.1" 200 -`.

الفصل 5: DNS

1. ملف واحد مركزي: بيكبر جداً، وتحديثه بطيء، وفيه تعارض أسماء. DNS قسّمها شجرة موزعة كل مستوى مسؤول عن اللي تحته، وفيه كاش.
2. من غير كاش كل طلب يعدي على Root و TLD و Authoritative. الكاش بيحفظ الإجابة مدة الـ TTL، فأغلب الطلبات بترد فوراً.
3. `A` لما عندك IP مباشر. `CNAME` لما الاسم ده مجرد اسم تاني لاسم (زي `www` ← خدمة استضافة بتغير الـ IPs).
4. `dig` بيسأل سيرفر DNS مباشرة ومبيقراش hosts. استخدم `dscacheutil -q host -a name x` أو `ping` أو المتصفح.
5. `sudo` صلاحيات Admin، و `killall` ابعت إشارة لكل البرامج بالاسم ده، و `-HUP` إشارة "اقرا إعداداتك من الأول"، و `mDNSResponder` برنامج DNS بتاع الـ Mac.

صح ولا غلط: (1) غلط: بيعرف سيرفرات الـ TLDs بس. · (2) غلط. · (3) صح.

التحديات:

1. `dig +short gmail.com A` وكذلك AAAA و MX و NS. الـ MX هيشاور على سيرفرات Google.
2. المتصفح يفتح، و `dscacheutil` يرجع 127.0.0.1، و `dig` ميرجعش حاجة (أو حاجة من النت) لأنه مش بيقرا hosts.
3. أول جزء: `.` من Root server (حرف من a لـ m). التاني: `com.` من `*.gtld-servers.net`. التالت: `github.com` من سيرفرات GitHub (Authoritative) وفيه الـ A.

الفصل 6: الـ URL

1. `http` Scheme، و `127.0.0.1` Host، و `8000` Port، و `/api/users` Path، و `page=2&limit=10` Query (اتنين Parameters)، و `top` Fragment.
2. كل حرف عربي 2 Bytes في UTF-8، وكل Byte بيتكتب `%XX`. 3 حروف × 2 = 6.
3. `https://site.com/docs/img/x.png`.
4. الـ `?` zsh بيعتبرها Glob (`no matches found`)، والـ `&` بتشغّل اللي قبلها في الخلفية وتقطع الأمر.

صح ولا غلط: (1) صح: 443 هو الافتراضي لـ https. · (2) غلط: الـ Fragment مبيتبعتش. · (3) غلط: الـ Port مختلف.

التحديات:

1. هتلاقي `protocol: 'https:'` و `host` و `pathname` و `search` زي ما في الفصل.
2. مع `%26` : `q` = `rock & roll`. مع `&` عادية: `q` = `rock` وظهر Parameter غريب اسمه `roll`.

3. `const build=(base,p)=>{const u=new URL(base);for(const[k,v] of Object.entries(p))u.searchParams.set(k,v);return u.href}` .

الفصل 7: HTTP

1. خلّى سيرفر واحد على IP واحد يشيل مواقع كتير، لأن الطلب بيقول هو عاوز أنهي اسم.
2. بسطر فاضي (`\r\n\r\n`)، والـ `Content-Length` بيقوله الـ Body طوله قد إيه.
3. PUT بيستبدل الـ User كله: `PUT /api/users/3` بـ Object كامل. PATCH بيعدل جزء: `PATCH /api/users/3` بـ `{"email":"new@x.com"}` .
4. GET المفروض Safe: المتصفحات والـ Prefetch والـ Crawlers بيعملوا GET لوحدهم، فممكن يعملوا Logout أو يمسحوا من غير قصد. وكمان مفيش حماية CSRF.
5. `multipart/form-data` : معمول للملفات الـ Binary. JSON نص، فالصورة لازم تتحول Base64 وتكبر حوالي 33%.

صح ولا غلط: (1) غلط. · (2) صح: تكراره نفس النتيجة (الحاجة ممسوحة). · (3) غلط: بيرجع Response و `ok: false` .

التحديات:

1. `r.status` = 200، وفي `headers` هتلاقي User-Agent و Accept-Language و Origin... اللي المتصفح بعتها.
2. مع الـ Header: `json` فيه الـ Object. من غيره: `json: null` والنص في `data` ، والـ Content-Type `text/plain;charset=UTF-8` .
3. `async function getJSON(url){const r=await fetch(url);if(!r.ok)throw new Error('HTTP '+r.status);return r.json()}` ، وعلى 404 هترمي `HTTP 404` .

الفصل 8: الـ Status codes

1. تعرف المعنى العام لأي كود جديد من أول رقم، حتى لو أول مرة تشوفه: xx4 طلبك، xx5 السيرفر.
2. `303 See Other` (أو 302 عملياً): يخلي المتصفح يعمل GET للصفحة الجديدة. 307 بيعيد نفس الـ POST، فممكن التسجيل يتكرر.
3. 502 = Proxy أو Gateway ماعرفش ياخد رد سليم من السيرفر اللي وراه. المشكلة في السيرفر أو الإعداد، مش في الـ Frontend.
4. `-s` اسكت، و `-o /dev/null` ارمي الـ Body، و `-w '%{http_code}\n'` اطبع الكود بس وسطر جديد.

صح ولا غلط: (1) غلط: 304 = "اللي عندك لسه صح". · (2) غلط: 403 = عارفك ومش مسموحلك. الـ Login مش هيغير حاجة. · (3) غلط: لازم `-L` .

التحديات:

1. 201 و 429 و 503.
2. `http://github.com` : 301 لـ `https://github.com/` ثم 200. Google: غالباً 301 أو 302 لـ HTTPS وأحياناً لنسخة البلد.
3. `$( )` = Command substitution: نفّذ الأمر اللي جوه وحط ناتجه مكانه في النص.

الفصل 9: الـ Headers

1. إنجليزي أمريكي أولاً، ثم أي إنجليزي (0.9)، ثم عربي (0.7).
2. الـ Encoding: الملف UTF-8 والمتصفح فهمه حاجة تانية. الحل: `<meta charset="utf-8">` وكمان `Content-Type: text/html; charset=utf-8` من السيرفر.
3. `Content-Type` = نوع المحتوى الأصلي (JSON أو HTML). `Content-Encoding` = اتضغط إزاي في النقل (gzip أو br).
4. السيرفر بيبعت `text/html` أو `text/plain` مع `nosniff` ، غالباً لأن الملف مش موجود ورجعت صفحة 404. شيك على الـ Status والـ Content-Type في Network.

صح ولا غلط: (1) غلط: مجمّد على `10_15_7` . · (2) غلط: Base64 تحويل مش تشفير، أي حد يفكه. · (3) غلط: الـ Headers اللي السيرفر مش فاهمها بيتجاهلها.

التحديات:

1. المتصفح بيبعت Headers أكتر (Accept-Language و Sec-CH-UA و Sec-Fetch-*)، و curl بيبعت User-Agent و Accept بس تقريباً.
2. الصفحات النصية بتوفر غالباً 70–90% مع الضغط.
3. مثال: `strict-transport-security` = HTTPS بس لمدة طويلة، و `content-security-policy` = المصادر المسموحة، و `x-frame-options` = متتحطش في iframe عند حد تاني.

الفصل 10: معمل curl

1. لما نفس الطلب (نفس الـ Headers والـ Body) ينجح في curl ويفشل في المتصفح: يبقى السبب حاجة المتصفح بيضيفها (CORS أو كوكيز أو كاش أو Extension).
2. الـ Body، و `Content-Type: application/json`، و `Accept: application/json` (ومعاهم POST تلقائي). عشان كده لـ PATCH لازم `-X PATCH`.
3. `-d` بيحط القيمة زي ما هي (المسافة هتبوظ الـ URL)، و `--data-urlencode` بيشفّرها (`a%20b`).
4. TTFB ناقص TLS ≈ 1.8 ثانية: السيرفر بطيء في تجهيز الرد. الشبكة سليمة.
5. عشان zsh يبدّل `$TOKEN` بقيمته. جوه Single quotes بتتبعت حرفياً `$TOKEN`.

صح ولا غلط: (1) صح. · (2) غلط: `-I` HEAD، و `-i` يضمّن الـ Headers مع GET. · (3) غلط: تراكمي من البداية. وقت TLS = connect – appconnect.

التحديات:

1. 200 GET، و 201 POST، و 200 PATCH، و 200 DELETE.
2. الموقع القريب TLS و TTFB أقل. البعيد كل مرحلة فيها RTT بتكبر.
3. `curl -s -c jar -b jar -L 'https://httpbin.org/cookies/set?name=sara'` ثم `curl -s -b jar https://httpbin.org/cookies` يرجع `"name": "sara"`.

الفصل 11: نسخ HTTP

1. زي طابور شباك واحد وأول واحد معاه معاملة طويلة. HTTP/2 خلى كل الطلبات تمشي متداخلة على نفس الاتصال (Streams)، فالصغير ميستناش الكبير.
2. لأن TCP تحتيه بيوقف كل الـ Streams لو Packet واحدة ضاعت، وده بيحصل كتير على الموبايل.
3. "فيه HTTP/3 على UDP Port 443 لنفس الـ Host، افتكر ده يوم".
4. HTTP/2 و 3 بيفرضوا الأسماء small letters، و curl بيطبعها زي ما جت.

صح ولا غلط: (1) غلط: الـ Status codes زي ما هي. · (2) صح: QUIC فوق UDP. · (3) غلط: الطبيعي، المتصفح اتعلم من `alt-svc`.

التحديات:

1. مواقع Google و Cloudflare غالباً h3، ومواقع كتير h2، وطلبات قليلة http/1.1.
2. مع 1.1: `HTTP/1.1 200 OK` بـ Reason phrase، والـ Headers ممكن بحروف كبيرة. مع 2: `HTTP/2 200` وأسماء صغيرة.
3. أيوه ممكن: كل Domain تالت اتصال منفصل وممكن يبقى 1.1، والمتصفح بيأخر الطلبات الأقل أولوية (Priority) عن قصد.

الفصل 12: HTTPS و TLS

1. في الكافيه الراوتر ممكن يقول "أنا bank.com" ويشفّر معاك هو. التشفير هيبقى سليم بس للحرامي. الشهادة بتثبت إنك بتكلم الحقيقي.
2. SNI بيقول اسم الموقع في أول رسالة، عشان سيرفر واحد شايل آلاف المواقع على نفس الـ IP يطلع الشهادة الصح. Cloudflare فيها ملايين المواقع على IPs مشتركة.
3. Cloudflare TLS Issuing ECC CA 3 مضى على example.com، و SSL.com TLS Transit ECC CA R2 مضى عليه، و SSL.com TLS ECC Root CA 2022 مضى على ده، والـ Root ده (أو اللي فوقه) في الـ Root store.
4. Chrome يمنعك ومفيش زرار تعدي.
5. الشهادة المسروقة بتبقى خطر لمدة أقل، والتغييرات بتنتشر أسرع. الأثر: التجديد لازم يبقى أوتوماتيك.

صح ولا غلط: (1) غلط. · (2) صح: مستوى واحد. · (3) غلط: بيلغي التحقق خالص.

التحديات:

1. هتلاقي غالباً Let's Encrypt أو Google Trust Services أو DigiCert، والعمر 90 يوم أو أقل من 200.
2. `echo | openssl s_client -connect github.com:443 -servername github.com 2>/dev/null | head -15` وكذلك google.com.
3. expired: `NET::ERR_CERT_DATE_INVALID` و curl (60). wrong.host: `ERR_CERT_COMMON_NAME_INVALID`. self-signed: `ERR_CERT_AUTHORITY_INVALID`.

الفصل 13: الكوكيز والـ Sessions

1. عشان محدش يقدر يغيرها ويبقى حد تاني، ومفيش بيانات حساسة عند المتصفح. الرقم العشوائي صعب يتخمن.

2. موقع شرير يبعت فورم POST لموقعك والمتصفح يبعت كوكيز اليوزر. مع Lax، الكوكي مبتتبعتش مع POST أو fetch جايين من موقع تاني.
3. في Set-Cookie بتفصل الكوكي عن الـ Attributes بتاعتها (كوكي واحدة). في Cookie بتفصل كوكيز مختلفة.
4. التخزين مربوط بالـ Origin، والـ Port جزء منه.
5. الـ Session: تمسح السطر من السيرفر فتنتهي فوراً. الـ JWT صالح لحد الـ exp إلا لو عملت Blocklist.

صح ولا غلط: (1) صح. · (2) غلط. · (3) غلط: Base64 بيتقري، الإمضا بس اللي بيمنع التعديل.

التحديات:

1. الأعمدة: Domain، و Include subdomains (TRUE/FALSE)، و Path، و Secure، و Expires (Session = 0)، و Name، و Value.
2. `JSON.parse(atob(token.split('.')[1]))`، وهتلاقي `sub` و `exp` وغيرهم. (لو فيه `-` أو `_` بدّلهم بـ `+` و `/`).
3. غالباً دومينات إعلانات و Analytics، وكتير منهم `SameSite=None; Secure` لأنهم محتاجين يتبعتوا Cross-site.

الفصل 14: CORS

1. الصفحة الشريرة تعمل fetch للبنك بكوكيزك. Same-origin policy بتخلي الرد ميتقريش من الـ JS بتاع الصفحة دي.
2. لأن الـ JS ممكن يكون هو الشرير. صاحب البيانات هو اللي يقرر مين يقراها.
3. Method زي PUT أو DELETE، و Content-Type زي application/json، و Header زي Authorization.
4. صفحة الخطأ 500 غالباً من غير Headers الـ CORS، فالمتصفح يقول CORS والسبب الحقيقي السيرفر.
5. المتصفح بيكلم نفس الـ Origin (`localhost:5173/api`)، و Vite بيبعت الطلب للـ API من غير متصفح، فمفيش CORS.

صح ولا غلط: (1) غلط. · (2) صح. · (3) غلط: الرد Opaque ومتقدرش تقراه.

التحديات:

1. اشتغل: jsonplaceholder بيرد بـ `access-control-allow-origin` بالـ Origin بتاعك.
2. مفيش Preflight (GET بسيط)، والخطأ: No 'Access-Control-Allow-Origin' header is present. الرد وصل والمتصفح حجزه.
3. httpbin هيرد بـ allow-methods فيها DELETE و allow-headers: authorization. وفي Network هتلاقي سطر Type بتاعه `preflight` قبل DELETE.

الفصل 15: الكاش والـ CDN

1. Fresh = جوه `max-age`، يتستخدم من غير سؤال. Stale = خلص، فيتسأل السيرفر (Revalidate) ولو اتغير يجيب الجديد.
2. أول رد فيه `ETag: "a1"` والمتصفح يحفظه. بعدين يسأل بـ `If-None-Match: "a1"`، ولو ماتغيرش السيرفر يرد 304 من غير Body.
3. الـ HTML فيه أسماء الملفات الجديدة فلازم يبقى أحدث حاجة. الـ JS اسمه فيه Hash، فأي تغيير = اسم جديد.
4. 100 ثانية.
5. `(disk cache)` = مفيش طلب خالص. `304` = طلب اتبعت والسيرفر قال "زي ما هي".

صح ولا غلط: (1) غلط: يعني اسأل الأول. · (2) غلط: من غير Body. · (3) صح.

التحديات:

1. غالباً حاجات كتير من `memory cache` (لو نفس التابة) أو `disk cache`.
2. ملفات الـ CDN الثابتة وأغلب الـ APIs اللي فيها ETag هترجع 304.
3. `age` بيزيد و `cf-cache-status: HIT`. مع `?v=123` الـ URL جديد فمفتاح الكاش جديد: أول مرة MISS و age صغير أو مش موجود.

الفصل 16: رسم الصفحة

1. Parse HTML ← DOM، و CSS ← CSSOM، ودمجهم Render tree، و Layout يحسب الأماكن، و Paint/Composite يرسم.
2. عشان ميرسمش الصفحة من غير Styles وبعدين تنط (FOUC).
3. Parser = لقاه في الـ HTML وهو بيقراه. Script = كود JS عمل الطلب (fetch مثلاً) بسطر معين.
4. Analytics: `async` (مستقل ومش مهم ترتيبه). التطبيق: `defer` أو module (محتاج الـ DOM والترتيب).

صح ولا غلط: (1) غلط. · (2) صح. · (3) غلط: DOMContentLoaded الأول.

التحديات:

1. مع Slow 4G الرقمين بيكبروا جداً، و Load أكتر.
2. عادي و defer: a ثم b دايماً. async: اللي يوصل الأول.
3. مثال: الصفحة ← main.js و chunk.js و api/user.

الفصل 17: جولة DevTools

1. كان مفيش طريقة تشوف الأخطاء والطلبات وتعدل الـ HTML/CSS مباشرة. Firebug جمّع ده في مكان واحد.
2. افتح Network ودوس Esc يطلعلك Console في الـ Drawer.
3. الرسايل والدوكيومنتيشن ونتايج البحث بالإنجليزي، فهتلاقي الأسماء زي ما هي. وكمان Console insights محتاج إنجليزي.
4. غالباً من Extension. جرب Incognito أو Profile من غير Extensions، ولو اختفى، مش بتاعك.

صح ولا غلط: (1) صح. · (2) غلط: الـ DOM الحالي بعد الـ JS. · (3) غلط: طول ما DevTools مفتوح بس.

التحديات:

1. ⌥⌘I و J و C، ثم ⋮ ← Dock side، ثم Network ← Disable cache.
2. "Capture full size screenshot" و "Disable JavaScript". في مواقع React مثلاً الصفحة هتبقى فاضية تقريباً.
3. في Profile عادي هتلاقي طلبات ورسايل زيادة من الـ Extensions.

الفصل 18: Network: القايمة

1. الـ POST بيعمل Redirect والصفحة الجديدة بتمسح القايمة. فعّل Preserve log.
2. `method:POST -status-code:201`.
3. transferred = اللي اتنقل على الشبكة (مضغوط ومن غير الكاش). resources = الحجم الحقيقي بعد فك الضغط.
4. المتصفح استخدم اتصال مفتوح قبل كده لنفس الـ Origin.
5. يأكدلك إن الطلب راح للـ IP اللي حطيته في hosts فعلاً.

صح ولا غلط: (1) غلط: الـ `-` عكس، فيوريك كل حاجة ما عدا google.com. · (2) غلط: مفيش رد HTTP خالص. · (3) صح.

التحديات:

1. غالباً صور أو فيديو أو JS كبير، والـ Domains ممكن تبقى 20 أو أكتر.
2. Slow 4G بيضرب Finish و Load في 3 أو 5 أو أكتر.
3. المفروض الأرقام تقرب: DNS ≈ namelookup، و Initial ≈ namelookup – connect ≈ connection، و SSL ≈ connect – appconnect، و Waiting ≈ starttransfer – appconnect.

الفصل 19: Network: التفاصيل

1. مش الـ Headers الحقيقية. أسباب: جه من الكاش، أو اتمنع (CORS أو Extension)، أو لسه مكملش.
2. Payload ← view source. السبب: نسيت `JSON.stringify`.
3. Preview منسق، و Response خام. Preview مش بيعرض JSON بايظ أو HTML جه مكان JSON.
4. أي حد يقدر يعمل طلبات بحسابك بالـ Token والكوكيز لحد ما يخلصوا.
5. Block request domain في Request conditions، واعمل Reload، وشوف الصفحة شغالة.

صح ولا غلط: (1) غلط: بيشيل Cookie و Set-Cookie و Authorization بس. · (2) غلط: عندك انت بس. · (3) صح.

التحديات:

1. Headers فيها POST و 201، و Payload فيه title، و Preview فيه id: 101.
2. نفس 201 ونفس الشكل، والـ fetch المعدل يظهر طلب جديد في Network.
3. `jq -r '.log.entries[].request.url' file.har`.

الفصل 20: الـ Console

1. الحاجة اللي قبل `.name`، مش name.
2. NAME_NOT_RESOLVED = الـ DNS، اختبره بـ `dig`. TCP = CONNECTION_REFUSED ومفيش حد على الـ Port، اختبره بـ `nc -zv` و `lsof`.
3. CORS: السيرفر التاني مدّاش إذن. CSP: الموقع اللي انت فيه حاطط قايمة منعت الطلب.
4. الـ Console بيحتفظ بـ Reference وبيقرا القيم لما تفتح الـ Object.
5. `const u = await (await fetch('https://jsonplaceholder.typicode.com/users/1')).json(); u.email`.

صح ولا غلط: (1) غلط. · (2) صح. · (3) غلط.

التحديات:

1. `console.table(users)` و `console.time/timeEnd` حوالين fetch و `console.group`.
2. الأول ERR_NAME_NOT_RESOLVED (DNS). التاني ERR_CONNECTION_REFUSED (TCP)، مفيش حد على 3999). التالت 404 (HTTP) ومن غير Error في الكود.
3. الـ Stack هيقول السطر اللي فيه `.x.y`، وتصلحه بـ Optional chaining `?.` أو تصلح البيانات.

الفصل 21: Application و Safari

1. Domain أو Path مش مطابق، أو SameSite منعها (Cross-site)، أو Secure على HTTP، أو الطلب fetch من غير `credentials: 'include'`.
2. لما الموقع معلق في حالة غريبة ومش عارف أنهي كوكي أو تخزين أو Service Worker السبب.
3. يرد بنسخة قديمة من الكاش بتاعه. فعّل Update on reload أو Bypass for network أو Unregister.
4. WebKit مختلف في الكوكيز والخصوصية، وكل متصفحات الـ iPhone جواها WebKit.

صح ولا غلط: (1) غلط: بتظهر وعليها علامة HttpOnly. · (2) غلط: الموقع ده بس. · (3) غلط.

التحديات:

1. غالباً HttpOnly و Secure و SameSite Lax.
2. غالباً اتعملك Logout. و Clear site data يرجّع الموقع كأنه أول مرة.
3. ممكن تلاقي كوكيز أقل في Safari (Third-party ممنوعة).

الفصل 22: خطة الـ Debugging

1. Console ← الطلب موجود؟ ← رقم ولا أقواس؟ ← 2xx؟ ← الشكل صح؟ ← كودك. كل سؤال بيشيل نص الاحتمالات ومن الأرخص للأغلى.
2. `(failed)` : قبل HTTP، فدوّر بـ dig و nc و curl -v. `500` : السيرفر رد، فدوّر في Response و Logs السيرفر.
3. URL لسه localhost، أو Mixed content (http جوه https)، أو Proxy التطوير مش موجود فرجع CORS.
4. جوه Loop أو دالة بتتنادى كتير ومحتاج حالة معينة بس.
5. اللي عاوزه، واللي حصل بالنص، واللي جربته ونتيجته، والبيئة، ومن غير أسرار.

صح ولا غلط: (1) غلط: المتصفح أو الطلب نفسه. · (2) غلط: بيطبع بس. · (3) صح.

التحديات:

1. هتلاقي Local و Closure و Global، والـ Call Stack.
2. XHR/fetch Breakpoints ← + ← `api`، والكود يقف على سطر الـ fetch.
3. غالباً آه: كتابة القالب بتكشف الحاجة اللي ناقصة.

الفصل 23: المشروع النهائي

1. الملف `data/user.json` مش موجود، فالسيرفر رد 404 بصفحة HTML، والكود عمل `res.json()` من غير ما يشيك `res.ok`. أول حرف `<`.
2. الملف نفسه 404، ومفيش `access-control-allow-origin`. يعني لو حتى اتسمح، الملف مش موجود.
3. عشان فشل طلب واحد ميوقفش الباقي، ويبقى فيه رسالة مفهومة.
4. كل طلب لـ localhost:8000 يقابله سطر في Log السيرفر، إلا لو جه من الكاش أو اتمنع قبل ما يتبعت.

صح ولا غلط: (1) صح. · (2) غلط: الإذن من السيرفر بتاع example.com. · (3) صح.

التحديات:

1. الحلول: (1) `src="app.js"` بدل `js/app.js` (أو انقل الملف). (2) `data/users.json`. (3) شيل الفاصلة اللي بعد آخر Object (json.tool بيقول: Illegal trailing comma). (4) `data.data.forEach` و `data.data.length`. (5) loadQuote: شيلها أو حطها في try/catch واعرض "مفيش quote". (6) `body: JSON.stringify({...})`، وبعدها `echo.json` هيبقى فيه البيانات. (7) ping: شيلها أو try/catch (ERR_CONNECTION_REFUSED). (8) الكاش: Reload عادي يرجع 304 بسبب Last-Modified، وبعد التعديل 200 بالجديد.
2. Slow 4G تبان رسالة التحميل، والـ Block لـ httpbin ميأثرش على الـ users، والـ Override بـ `[]` يظهر 0.
3. مثال سهل: كوكي اتمنعت، أو Mixed content على صفحة https.

دلوقتي الشبكة مش سحر

كل طلب بقى ليه رحلة انت عارف محطاتها: اسم يتترجم، واتصال يتفتح، ونفق يتقفل، وطلب ورد. ولما حاجة تبوظ، معاك طريقة: Console، ثم Network، ثم curl، ثم كودك. افتح DevTools في كل مشروع من أول يوم، وارجع للخريطة في الفصل 22 وللـ Cheat Sheet كل ما تحتاج.

```
$ curl -sI https://you.dev | head -1 → HTTP/2 200
```